

MENTIFY

AI Enhanced Digital Learning Platform

By:

M. Holins – SE/2021/002

S. Dilakshan – SE/2021/003

J. Jenushan – SE/2021/035

M. R. Zaiena – SE/2021/040

A report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

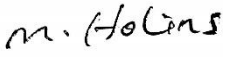
2026

DECLARATION

I hereby certify that this project and the all the artifacts associated with it is my own work and it has not been submitted before nor is currently being submitted for any other degree programme.

Full name of the student: M. Holins

Student No: SE/2021/002

Signature of the student: 

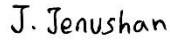
Full name of the student: S. Dilakshan

Student No: SE/2021/003

Signature of the student: 

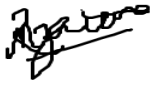
Full name of the student: J. Jenushan

Student No: SE/2021/035

Signature of the student: 

Full name of the student: M. R. Zaiena

Student No: SE/2021/040

Signature of the student: 

ACKNOWLEDGEMENTS

The development team would like to express sincere gratitude to TIT Education and Mr. G. Hajananan, Director, for providing the opportunity to design this system and for offering invaluable domain expertise, feedback, and collaboration throughout the project.

We extend our appreciation to the Software Engineering Teaching Unit of the Faculty of Science, University of Kelaniya, for providing the academic framework and supervision that guided the completion of this project.

We also acknowledge the collective contribution of each team member, Mohanathas Holins, Sivanathan Dilakshan, Jeyaruban Jenushan, and Zaiena Rifkie whose dedication and collaborative effort made this final design report possible.

ABSTRACT

Mentify is a comprehensive, AI-enhanced learning management system designed for the education provider TiT Education to modernize and streamline digital education delivery. The platform integrates core academic functionalities such as course management, class scheduling, and student engagement with advanced intelligent features to improve both teaching efficiency and learning outcomes.

Mentify enables students to attend live classes, access structured learning materials, and participate in interactive academic activities. The system further enhances learning through personalized feedback generation, enabling students to identify knowledge gaps and improve performance in a targeted manner. It also incorporates intelligent chatbot assistants that provide real-time academic support, answer frequently asked questions, and guide users through platform features. Collaborative learning is fostered through integrated discussion forums, where students and instructors can engage in knowledge sharing and peer interaction.

Beyond standard LMS capabilities, Mentify introduces enhanced features such as adaptive learning pathways, performance analytics dashboards, predictive insights for student progress, and content recommendation systems. These additions enable educators to make data-driven decisions while offering students a more personalized and engaging learning experience.

Overall, Mentify represents a scalable, intelligent, and user-centric system that leverages artificial intelligence and modern software architecture to redefine digital education for TiT Education.

TABLE OF CONTENTS

DECLARATION 2
ACKNOWLEDGEMENTS 3
ABSTRACT 4
TABLE OF CONTENTS 5
LIST OF FIGURES 10
LIST OF TABLES 13
ABBREVIATIONS 16
1 INTRODUCTION 18
1.1 Cient Details 18
1.2 Problem Statement 18
1.3 Proposed Solution Overview 19
1.4 Stakeholders 20
1.5 Client Agreement Letter 21
2 SOFTWARE REQUIREMENT SPECIFICATION 25
2.1 Introduction 25
2.1.1 Purpose 25
2.1.2 Scope 25
2.1.3 Document Overview 26
2.2 Project Scope and Stakeholders 27
2.2.1 Project Scope 27
2.2.2 Stakeholders 27
2.3 Functional Requirements 29
2.3.1 Student Functional Requirements 29
2.3.2 Teacher Functional Requirements 34
2.3.3 Administrator Functional Requirements 42
2.3.4 Support Team Functional Requirements 45
2.3.5 AI System Functional Requirements 48
2.4 Non-Functional Requirements 50
2.4.1 Performance 50
2.4.2 Reliability 50
2.4.3 Usability 51

2.4.4 Security 52
2.4.5 Scalability 53
2.4.6 Maintainability 53
2.4.7 Portability 53
2.5 Business Rules, Constraints and Assumptions 55
2.5.1 Business Rules 55
2.5.2 Constraints 56
2.5.3 Assumptions 56
2.6 Use Case Diagrams and Descriptions 58
2.6.1 Student Use Case Diagrams 58
2.6.2 Teacher Use Case Diagram 59
2.6.3 Administrator Use Case Diagram 60
2.6.4 Support Team Use Case Diagram 61
2.6.5 Use Case Descriptions 62
2.7 Activity Diagrams 66
2.7.1 Student Login (JWT-Based) 67
2.7.2 Student Registration 68
2.7.3 Course Enrolment 69
2.7.4 Attempt Online Quiz 70
2.7.5 Teacher Creates and Publishes Quiz with AI Assistance 71
2.7.6 Teacher Grades Assignment with AI Assistance 72
2.7.7 Upload Learning Materials 73
2.7.8 Admin Creates User Account 74
2.7.9 Support Team Handles Ticket 75
2.7.10 AI Help Chatbot 76
2.8 External Interface Requirements 77
2.8.1 User Interface Requirements 77
2.8.2 Hardware Interface Requirements 77
2.8.3 Software Interface Requirements 77
2.8.4 Communication Interface Requirements 78
2.9 Data Requirements 79
2.9.1 User Data 79

2.9.2 Course Data 79
2.9.3 Assessment Data 79
2.9.4 Communication Data 79
2.9.5 Analytics Data 79
2.9.6 Support Data 80
2.9.7 Data Security and Retention 80
2.10 Alternative Solutions Considered 81
2.10.1 Adoption of an Existing Learning Management System 81
2.10.2 Low-Code / No-Code Development Platforms 81
2.10.3 Custom System Development (Selected Approach) 81
2.11 Feasibility Study 82
2.11.1 Technical Feasibility 82
2.11.2 Economic Feasibility 82
2.11.3 Operational Feasibility 82
2.11.4 Schedule Feasibility 83
2.12 Risk Analysis 84
3 SYSTEM DESIGN SPECIFICATION DOCUMENT 87
3.1 Architectural Design 87
3.1.1 High-Level Architecture 87
3.1.2 Cloud-Native Architecture 90
3.1.3 Justification for Architecture Choice 92
3.1.4 Scalability and Maintainability Considerations 93
3.2 Database Design 95
3.2.1 ER Diagram 95
3.2.2 Normalisation Analysis 101
3.3 Class Diagram 107
3.3.1 User Management Service 107
3.3.2 Course Management Service 108
3.3.3 Assignment Service 109
3.3.4 Analytics Service 110
3.3.5 Communication Service 111
3.3.6 AI Chat Service 112

3.3.7 Inter-Service Class Diagram 113
3.4 Sequence Diagrams 114
3.4.1 User Login and Authentication 115
3.4.2 Administrator Manages Users with RBAC 120
3.4.3 Teacher Uploads and Organises Learning Materials 125
3.4.4 AI-Assisted Quiz Generation 130
3.4.5 Student Attempts Online Quiz with Auto-Grading 135
3.4.6 Teacher Grades Assignment with AI Assistance 140
3.4.7 Real-time Messaging between Student and Teacher 144
3.4.8 Teacher Posts Class-wide Announcement 149
3.4.9 Student Interacts with AI Help Chatbot 153
3.4.10 Student Views Performance Dashboard with Analytics 157
3.4.11 Student Takes Personal Notes Within Platform 161
3.5 UI/UX Design 165
3.5.1 Design Overview 165
3.5.2 Screen Mock-ups for Key User Interfaces 166
3.5.3 User Flow and Navigation Diagrams 177
3.5.4 Accessibility Considerations 182
3.6 Security Design 185
3.6.1 Authentication Mechanism 185
3.6.2 Authorisation Levels 187
3.6.3 Data Protection Approach 188
3.6.4 Input Validation and Sanitisation Strategy 189
3.6.5 Encryption Standards for Data at Rest and in Transit 191
3.7 Deployment Design 194
3.7.1 Hosting Plan 194
3.7.2 Hardware Requirements 195
3.7.3 Network Diagram 196
3.7.4 Environment Specification 198
3.7.5 CI/CD Pipeline Overview 199
4 PROTOYPE DEVELOPMENT AND CLIENT VALIDATION 203
4.1 UI/UX Design 203

4.1.1 Students' View 204
4.1.2 Teachers' View 212
4.2 Client Involvement 219
4.2.1 Client Demonstration 219
4.2.2 Client Feedback 220
4.3 Ethical Considerations 220
4.3.1 Data Privacy 220
4.3.2 AI-Driven features 220
4.3.3 Academic Integrity 221
4.4 Reflection on Stakeholder Communication 221
4.5 Formal Sign Off from the Client for the Development 222
5 CONCLUSION 223
5.1 Degree of Objectives Met 223
5.2 Usability, Accessibility, Reliability and Friendliness 224
5.3 Limitations and Drawbacks 225
5.4 Timeline 227
5.5 Future modifications, improvements and extensions possible 227
APPENDIX A – DOCUMENT REVIEW 229
APPENDIX B – UI PROTOTYPES 230
APPENDIX C – REFERENCES FROM THE SDS DOCUMENT 231

LIST OF FIGURES

Figure 2.1: Student Use Case Diagram 58
Figure 2.2: Teacher Use Case Diagram 59
Figure 2.3: Admin Use Case Diagram 60
Figure 2.4: Support Team Use Case Diagram 61
Figure 2.5: Activity Diagram - Student Login 67
Figure 2.6: Activity Diagram - Student Registration 68
Figure 2.7: Activity Diagram - Course Enrolment 69
Figure 2.8: Activity Diagram - Atempt Quiz 70
Figure 2.9: Activity Diagram - Publish Quiz 71
Figure 2.10: Activity Diagram - Grade Quiz 72
Figure 2.11: Activity Diagram - Upload Materials 73
Figure 2.12: Activity Diagram - Create User Account 74
Figure 2.13: Activity Diagram - Ticket Handling 75
Figure 2.14: Activity Diagram: Chatbot 76
Figure 3.1 - High-Level System Architecture Diagram 89
Figure 3.2: ER - User Service 95
Figure 3.3: ER - Course Management Service 96
Figure 3.4: ER - Assignment Service 97
Figure 3.5: ER - Analytics Service 98
Figure 3.6: ER - Communication Service 99
Figure 3.7: ER - AI Entities 100
Figure 3.8: Class Diagram - User Management Service 107
Figure 3.9: Class Diagram — Course Management Service 108
Figure 3.10: Class Diagram — Assignment Service 109
Figure 3.11: Class Diagram — Analytics Service 110
Figure 3.12: Class Diagram — Communication Service 111
Figure 3.13: Class Diagram — AI Chat Service 112
Figure 3.14: Inter-Service Class Diagram 113
Figure 3.15: Sequence Diagram — Administrator Manages Users with RBAC 124
Figure 3.16: Use Case Details — Teacher Uploads and Organises Learning Materials 125

Figure 3.17: Actors and Components — Teacher Uploads and Organises Learning Materials. 126
Figure 3.18: Sequence Diagram — Teacher Uploads and Organises Learning Materials 129
Figure 3.19: Sequence Diagram — AI-Assisted Quiz Generation 134
Figure 3.20: Sequence Diagram — Student Attempts Online Quiz with Auto-Grading..... 139
Figure 3.21: Sequence Diagram — Teacher Grades Assignment with AI Assistance 143
Figure 3.22: Sequence Diagram — Real-time Messaging between Student and Teacher 148
Figure 3.23: Sequence Diagram — Teacher Posts Class-wide Announcement 152
Figure 3.24: Sequence Diagram — Student Interacts with AI Help Chatbot 156
Figure 3.25: Sequence Diagram — Student Views Performance Dashboard with Analytics 160
Figure 3.26: Sequence Diagram — Student Takes Personal Notes Within Platform..... 164
Figure 3.27: Student Dashboard — Mentify..... 167
Figure 3.28: Quiz and Assessment View — Mentify 168
Figure 3.29: Collaborative Chat..... 169
Figure 3.30: Lesson and Video View 170
Figure 3.31: Course Detail Page 171
Figure 3.32: Assignment Submission 172
Figure 3.33: Quiz Generator 173
Figure 3.34: AI Learning Assistant..... 174
Figure 3.35: Cognitive Gallery Explorer 175
Figure 3.36: User Management..... 176
Figure 3.37: User Login Flow Diagram..... 178
Figure 3.38: Course Enrolment Flow Diagram..... 179
Figure 3.39: Course Management Flow..... 180
Figure 3.40: Forum and Group Messaging 181
Figure 3.41: Security Architecture Diagram..... 185
Figure 3.42: System Network and Deployment Architecture Diagram..... 197
Figure 4.1: Student Registration Page 204
Figure 4.2: Student Dashboard 205
Figure 4.3: Assignment Details..... 206
Figure 4.4: Browse Courses 207
Figure 4.5: Course Content 208

Figure 4.6: Discussion Forum 209
Figure 4.7: All Quizzes 210
Figure 4.8: Quiz View 211
Figure 4.9: Course Management 212
Figure 4.10: Courses View 213
Figure 4.11: Quiz Analytics 214
Figure 4.12: Quiz View 215
Figure 4.13: Student Management 216
Figure 4.14: Assignment Management 217
Figure 4.15: Teacher Dashboard 218

LIST OF TABLES

2.1: Functional Requirements - Students 33
2.2: Functional Requirements - Teachers 41
2.3: Functional Requirements - Administrators 44
2.4: Functional Requirements - Support Team 47
2.5: Functional Requirements - AI System 49
2.6: Non Functional Requirements - Performance 50
2.7: Non-Functional Requirements - Reliability 51
2.8: Non-Functional Requirements - Usability 52
2.9: Non-Functional Requirements - Security 52
2.10: Non-Functional Requirements - Scalability 53
2.11: Non-Functional Requirements - Maintainability 53
2.12: Non-Functional Requirements - Portability 54
2.13: Business Rules 56
2.14: Constraints 56
2.15: Assumptions 57
2.16: Use Case Description: UC-S001 63
2.17: Use Case Description: UC-S002 64
2.18: Use Case Description: UC-S003 65
2.19: Risk Analysis 86
3.1: Microservice Decomposition and Data Store Mapping 88
3.2: Cloud-Native Infrastructure Components 91
3.3: Architectural Style Comparison 92
3.4: User Table Functional Dependency Analysis 101
3.5: User Sub-type Tables - Normalisation Summary 102
3.6: Course Management Service — Normalisation Summary 103
3.7: Assignment Service — Normalisation Summary 104
3.8: Analytics Service — Normalisation Summary 105
3.9: Communication Service — Normalisation Summary 106
3.10 : Summary of Major Use Cases and Corresponding Sequence Diagrams 115
3.11: Use Case Details — User Login and Authentication 116

3.12: Actors and Components — User Login and Authentication 116
3.13: Sequence Diagram — User Login and Authentication 119
3.14: Use Case Details — Administrator Manages Users with RBAC 120
3.15: Actors and Components — Administrator Manages Users with RBAC 121
3.16 : Use Case Details — AI-Assisted Quiz Generation 130
3.17: Actors and Components — AI-Assisted Quiz Generation 131
3.18 : Use Case Details — Student Attempts Online Quiz with Auto-Grading 135
3.19 : Actors and Components — Student Attempts Online Quiz with Auto-Grading 136
3.20: Use Case Details — Teacher Grades Assignment with AI Assistance 140
3.21: Actors and Components — Teacher Grades Assignment with AI Assistance 141
3.22: Use Case Details — Real-time Messaging between Student and Teacher 144
3.23:Actors and Components — Real-time Messaging between Student and Teacher 145
3.24: Use Case Details — Teacher Posts Class-wide Announcement 149
3.25: Actors and Components — Teacher Posts Class-wide Announcement 150
3.26: Use Case Details — Student Interacts with AI Help Chatbot 153
3.27: Actors and Components — Student Interacts with AI Help Chatbot 154
3.28: Use Case Details — Student Views Performance Dashboard with Analytics 157
3.29: Actors and Components — Student Views Performance Dashboard with Analytics 158
3.30: Use Case Details — Student Takes Personal Notes Within Platform 161
3.31: Actors and Components — Student Takes Personal Notes Within Platform 162
3.32: Summary of Key User Interface Screens 166
3.33: Summary of User Flow Diagrams 177
3.34: WCAG 2.1 Accessibility Compliance Summary 184
3.35: Authentication Token Summary 187
3.36: Role-Based Access Control Definitions 188
3.37: Data Sensitivity Classification and Controls 189
3.38: Input Validation and Sanitisation Mechanisms 191
3.39: Encryption Standards Summary 193
3.40: Hosting Plan Summary 195
3.41: Cloud Hardware / Instance Specifications 196
3.42: Environment Specification Matrix 199

3.43: CI/CD Pipeline Stages 201
3.44: External Integration Summary 201

ABBREVIATIONS

AI	Artificial Intelligence, a technology that enables automated decision making, content generation, and personalisation.
API	Application Programming Interface, a set of protocols used for building and integrating application software.
AWS	Amazon Web Services, a cloud computing platform that provides scalable infrastructure, storage, and various online services for application deployment.
CRUD	Create, Read, Update, Delete, the four fundamental operations of persistent data storage.
FAQ	Frequently Asked Questions, a collection of commonly asked questions and their corresponding answers.
HTTPS	Hypertext Transfer Protocol Secure, an encrypted version of HTTP used for secure data transmission.
JWT	JSON Web Token, a compact and URL-safe token used for secure authentication between client and server.
LMS	Learning Management System, a software platform used for delivering, managing, and tracking educational content.
MCQ	Multiple Choice Question, a question format consisting of predefined answer options.
MFA	Multi-Factor Authentication, a security method that requires two or more verification factors during login.
MVC	Model-View-Controller, a software architectural pattern that separates application logic, user interface, and data handling.
NFR	Non-Functional Requirement, a requirement that specifies system quality attributes such as performance, security, and usability.
PDF	Portable Document Format, a file format used to present documents independently of software or hardware.
RBAC	Role-Based Access Control, a method of restricting system access based on user roles.
SMTP	Simple Mail Transfer Protocol, a standard protocol used for sending emails.

SQL	Structured Query Language, a standard programming language used for managing and querying relational databases.
SRS	Software Requirement Specification, a document that describes the functional and non-functional requirements of a system.
SSO	Single Sign-On, an authentication mechanism that allows users to log in once and access multiple systems.
TLS	Transport Layer Security, a cryptographic protocol used to secure communications over a network.
UI	User Interface, the visual elements through which a user interacts with the system.
UX	User Experience, the overall experience a user has when interacting with a system, focusing on usability, accessibility, and satisfaction.

1 INTRODUCTION

1.1 Cient Details

Client Name	G. Hajananan
Designamtion	Director / Owner
Organisation	TIT Education
Type of Organisation	Education Provider
Contact Information	077 015 8446
Location	Jaffna, Sri Lanka

1.2 Problem Statement

Current educational systems face several limitations that hinder effective learning, engagement, and collaboration. The learning management platforms utilized by TIT Education are predominantly static and lack interactivity, making it difficult to address the diverse learning needs of students. These systems generally adopt a one-size-fits-all approach rather than supporting student-centred learning, thereby limiting opportunities for personalized educational experiences.

Furthermore, educators rely heavily on manual processes for essential academic tasks such as content delivery, assessment management, grading, and feedback provision. These manual workflows are time-consuming, reduce operational efficiency, and limit the ability of educators to focus on improving teaching quality and student support.

In addition, communication between students and educators is highly fragmented, occurring across multiple disconnected platforms such as emails, messaging applications, and third-party tools. This lack of a centralized communication environment restricts seamless collaboration and reduces the effectiveness of interactions beyond formal classroom settings.

As a result of these challenges, student engagement levels decline, learning outcomes become inconsistent, and educators face difficulties in tracking, evaluating, and analyzing student performance in a comprehensive manner. Consequently, there is a strong need for an intelligent, centralized educational platform that integrates modern technologies—particularly AI-driven

personalization—to enhance learning effectiveness, streamline educational processes, and foster improved collaboration between all stakeholders.

1.3 Proposed Solution Overview

Mentify is proposed as an intelligent, centralized learning management system designed to address the limitations of existing educational platforms used by TIT Education. The system leverages modern software architecture and artificial intelligence to create a dynamic, student-centered learning environment that enhances engagement, improves efficiency, and enables data-driven decision-making.

At its foundation, Mentify provides a unified platform where all academic activities—such as course delivery, assessments, communication, and collaboration—are seamlessly integrated. Students can attend live or recorded classes, access structured learning materials, and participate in interactive learning experiences within a single system. This eliminates the need for multiple external tools and ensures a consistent and streamlined user experience.

A key component of Mentify is its AI-powered assessment and feedback mechanism. The platform supports both automatically graded quizzes and teacher-evaluated assessments, enabling scalability while maintaining the quality of evaluation. The AI engine provides instant feedback, performance insights, and personalized recommendations, helping students identify weaknesses and improve their learning outcomes effectively.

To further enhance personalization, Mentify incorporates adaptive learning features that tailor content and assessments based on individual student performance and learning patterns. This shifts the traditional one-size-fits-all approach toward a more customized and student-centric model.

The system also introduces integrated communication tools, including real-time chatbots, discussion forums, and messaging features. These tools create a centralized communication environment that promotes continuous interaction between students and educators, fostering collaboration and active learning beyond the classroom.

In addition, Mentify provides comprehensive analytics dashboards for educators and administrators. These dashboards offer insights into student progress, engagement levels, and performance trends, enabling timely interventions and informed academic decisions.

Overall, Mentify serves as a scalable, efficient, and intelligent solution that transforms traditional educational systems into a modern digital learning ecosystem, addressing current challenges while preparing TIT Education for future advancements in education technology.

1.4 Stakeholders

Stakeholder	Role	Interest/Responsibility
TiT Education	Client/Project Sponsor	Approves requirements, provides domain expertise
Students	Primary End Users	Learning, assessments, communication, progress tracking
Teachers	Primary End Users	Course creation, content delivery, student evaluation
System Administrators	Internal Operators	Manage users, roles, courses, system configuration, and overall platform health
Support Team	Internal Operators	Resolve user issues, manage support tickets, and maintain the knowledge base
Design and Development Team	System Developers	Design, develop, test, and deploy the platform
Academic Supervisor	Academic Oversight	Guidance and evaluation of deliverables

1.5 Client Agreement Letter

CLIENT AGREEMENT LETTER

Date: 14/03/2026

To Whom It May Concern,

This letter confirms that **Mr. G. Hajananan** representing **TIT Education**, have agreed to collaborate with the following students from the **Faculty of Science, University of Kelaniya, Sri Lanka** for the purpose of developing a software-based system as part of their academic System Design Project (SENG 31242).

Student Developers

- **SE/2021/002 – Mohanathas Holins**
- **SE/2021/003 – Sivanathan Dilakshan**
- **SE/2021/035 – Jeyaruban Jenushan**
- **SE/2021/040 – Zaina Rifkie**

1. Project Title

MENTIFY - AI Enhanced Digital Learning Platform

2. Background and Problem Statement

Current educational systems face significant challenges that hinder effective learning and collaboration. The learning management platforms currently used by TIT Education are largely static and non-interactive, limiting their ability to adapt to the diverse needs of students.

Most existing systems follow a one-size-fits-all approach rather than focusing on student-centered learning, which restricts personalized educational experiences. Educators often rely heavily on manual processes for tasks such as content delivery, assessments, grading, and feedback. These manual workflows require considerable time and effort and reduce the efficiency of the teaching process.

Additionally, communication between students and educators is decentralized, occurring across multiple platforms such as messaging applications, emails, and external tools. This lack of a centralized communication environment makes collaboration difficult and prevents effective interaction beyond the classroom.

G.Hajananan
Director
TIT Education
Jaffna.
Tel. 0770158446

© Software Engineering Teaching Unit
2026/03/14

Due to these limitations, student engagement is reduced, learning outcomes become inconsistent, and educators lack the ability to effectively track, evaluate, and analyze student performance. Therefore, there is a clear need for a smart, centralized educational platform that integrates modern technologies, including AI-driven personalization, to improve learning effectiveness and collaboration.

3. Purpose of the Proposed System

The proposed system is expected to:

- **Provide AI-Powered Personalized Learning**
  Deliver personalized learning experiences by analyzing individual student behavior, progress, and learning preferences. The system will dynamically adapt content, recommendations, and learning paths tailored to each learner's needs.

- **Automate Assessment and Content Management**
  Reduce manual workloads by implementing intelligent automated grading systems, instant feedback generation, and streamlined course management tools that enable educators to manage learning materials, assignments, and evaluations efficiently.

- **Provide a Centralized Learning and Collaboration Platform**
  Create a unified digital environment where students and educators can interact through real-time messaging, discussion forums, collaborative tools, and analytics-based performance monitoring, enabling continuous learning beyond the physical classroom.

4. Scope of Collaboration

We agree to:

- Provide necessary information related to current processes
- Participate in requirement discussions and clarification meetings
- Review and validate the Software Requirement Specification (SRS)
- Provide feedback on the proposed design and prototype

The student developers agree to:

- Maintain confidentiality of all proprietary or sensitive information disclosed by the client
- Use client-provided information solely for the purposes of this academic project
- Provide regular progress updates and notify the client of any material changes to the project scope



© *Software Engineering Teaching Unit*
2026/03/14

5. Nature of the Project

This project is undertaken solely for academic purposes. We understand and agree that:

- **To the maximum extent permitted by applicable law, the university of Kelaniya and the student developers expressly disclaim all warranties, whether express, implied, statutory or otherwise, including any implied warranties of merchantability, fitness for a particular purpose, or non-infringement. The university and student developers shall have no liability whatsoever for any direct, indirect, incidental, special, consequential or punitive damages, data loss, or financial impact arising from or related to the use or inability to use this software.**

6. Confidentiality (If Applicable)

- **"Confidential Information"** means any non-public data, documents, trade secrets, or business information disclosed by the client in connection with this project. The student developers and the University shall: (a) use Confidential Information solely for the purposes of this academic project; (b) not disclose it to any third party without the client's prior written consent; and (c) apply reasonable measures to protect it from unauthorised access. These obligations shall survive for a period of two (2) years following the completion or termination of the project. Confidential Information does not include information that is or becomes publicly known through no breach of this agreement, or that was already in the receiving party's possession prior to disclosure. Upon written request, all Confidential Information shall be returned or securely destroyed.

7. Intellectual Property

- All intellectual property rights in the software, code, designs, documentation, and other deliverables created by the student developers as part of this project ("Project IP") shall vest in and remain the property of the student developers and/or the University of Kelaniya. The client is granted a non-exclusive, non-transferable, royalty-free licence to use the Project IP solely for internal evaluation and testing purposes. The client shall not reproduce, distribute, sublicense, modify, or commercially exploit the Project IP without the prior written consent of the University. The client acknowledges that any data, logos, trademarks, or pre-existing materials provided by the client remain the client's exclusive property.

G.Rajandran
Director
TIT Education
Jaffna.
Tel. 0770158446

© *Software Engineering Teaching Unit*
2026/03/14

8. Term and Termination

- This agreement shall commence on the date of signing and shall remain in effect until the final submission of the project deliverables to the Faculty of Science, University of Kelaniya, or upon written notice of termination by either party. Either party may terminate this agreement by providing fourteen (14) days' written notice. Upon termination, the student developers shall return or destroy any Confidential Information in their possession, and the licence granted under Section 7 shall cease immediately.

I/We hereby confirm that the above project request is genuine and that we require the proposed solution as described.

Client Name: G. Hajananan

Designation: Owner

Organization: TiT Education

Contact Information: 0770158446

Signature: 

Official Seal: G.Hajananan
Director
TiT Education
Jaffna.
Tel: 0770158446

Date of Signing: 14-03-2026

© *Software Engineering Teaching Unit*
2026/03/14

2 SOFTWARE REQUIREMENT SPECIFICATION

2.1 Introduction

The Mentify Digital Learning Platform is a comprehensive Learning Management System (LMS) designed and developed for TIT Education, an educational institution that provides tailored tuition services for students. This document presents the complete Software Requirement Specification (SRS) for the proposed system, capturing all relevant functional and non-functional requirements, use cases, activity flows, system constraints, and feasibility considerations. Furthermore, this document has been prepared in accordance with the guidelines of the SENG 31242 module and adheres to the IEEE standard for software requirement documentation.

2.1.1 Purpose

The purpose of this document is to provide a detailed and structured specification of the requirements for the Mentify Digital Learning Platform. It is intended to serve as the primary reference for the development team, the client (TIT Education), project supervisors, and all stakeholders involved in the design, development, testing, and deployment of the system. This document defines the system's functional capabilities, non-functional quality attributes, user interactions, business rules, and technical constraints. The intended audience of this document includes software engineers, UI/UX designers, quality assurance engineers, the project supervisor, and the client representative at TIT Education.

2.1.2 Scope

System Name: Mentify – AI Enhanced Digital Learning Platform The Mentify platform is a web-based Learning Management System (LMS) designed to deliver AI-powered personalised learning experiences for students. The system enables students to enrol in courses, access multimedia learning content, complete assessments, communicate with teachers and peers, and track their academic progress through visual dashboards. Page 6 of 63 Teachers are provided with functionalities to create and manage courses, upload learning materials, design quizzes and assignments, evaluate student submissions with AI-assisted grading, and monitor student performance through analytical tools. Administrators oversee the overall operation of the platform, including user management, role-based access control, course oversight, and system health monitoring. A dedicated Support Team manages user support tickets, maintains a knowledge base, and escalates technical issues when required. An integrated AI subsystem supports content

summarisation, quiz generation, automated grading, personalised course recommendations, and provides a 24/7 AI-powered help chatbot to enhance the learning experience.

Explicit Exclusions: The system shall not include live video conferencing or real-time virtual classroom functionality in the initial release. Payment processing and financial management modules are excluded from the current scope. Offline access to course content will not be supported in the first version of the system. Integration with external Student Information Systems (SIS) of other institutions is not considered within the scope of this implementation.

2.1.3 Document Overview

This document is organised into multiple chapters that collectively define the Software Requirement Specification (SRS) for the Mentify Digital Learning Platform. Chapter 1 provides the introductory information, including the purpose, scope, definitions, and overall document structure. Chapter 2 describes the project scope in detail and identifies the key stakeholders involved in the system. Chapter 3 presents the functional requirements of the system, while Chapter 4 specifies the non functional requirements aligned with recognised quality attributes. Chapter 5 outlines the applicable business rules, system constraints, and underlying assumptions. Chapters 6 and 7 include the use case diagrams, detailed use case descriptions, and activity diagrams that illustrate system interactions and workflows. Chapter 8 describes the external interface requirements, including user, hardware, software, and communication interfaces. Chapter 9 defines the system's data requirements. Chapter 10 discusses alternative solutions considered during the system design phase. Chapter 11 presents the feasibility study covering technical, economic, operational, and schedule feasibility aspects. Chapter 12 provides a detailed risk analysis. Finally, Chapter 13 lists all references used in the preparation of this document.

2.2 Project Scope and Stakeholders

2.2.1 Project Scope

The Mentify Digital Learning Platform aims to transform TIT Education's existing manual and static educational processes into a modern, AI-driven, student-centred digital learning environment. The platform shall serve five categories of users: Students, Teachers, Administrators, Support Team members, and an integrated AI System. The system shall deliver AI-powered personalised learning experiences by analysing individual student behaviour, progress, and preferences to dynamically adapt content, recommendations, and learning paths tailored to each learner's needs. It shall automate assessment, feedback, and content management processes by replacing manual workflows with intelligent automated grading, instant feedback generation, and streamlined course management tools for educators. Furthermore, the platform shall provide a centralised collaboration and communication environment that connects students and educators through real-time messaging, discussion forums, and analytics-driven performance insights, thereby extending the learning experience beyond the traditional classroom setting. Although the system is initially developed for TIT Education, it is designed with scalability considerations to support future expansion to the broader student community.

2.2.2 Stakeholders

Stakeholder	Role	Interest/Responsibility
TiT Education	Client/Project Sponsor	Approves requirements, provides domain expertise
Students	Primary End Users	Learning, assessments, communication, progress tracking
Teachers	Primary End Users	Course creation, content delivery, student evaluation
System Administrators	Internal Operators	Manage users, roles, courses, system configuration, and overall platform health

Support Team	Internal Operators	Resolve user issues, manage support tickets, and maintain the knowledge base
Design and Development Team	System Developers	Design, develop, test, and deploy the platform
Academic Supervisor	Academic Oversight	Guidance and evaluation of deliverables

2.3 Functional Requirements

This chapter specifies the functional requirements of the Mentify Digital Learning Platform. These requirements describe the expected system behaviours and operations for Students, Teachers, Administrators, Support Team members, and the integrated AI subsystem.

2.3.1 Student Functional Requirements

ID	Description	Priority	Source	Status
FR-S001	The system shall allow a new student to register by entering their name, email, grade, and password, or by using Google authentication.	High	Student, Admin	Proposed
FR-S002	The system shall allow students to log in using JWT-based secure authentication.	High	Student	Proposed
FR-S003	The system shall allow students to view and edit their profile including photo and email.	Medium	Student	Proposed
FR-S004	The system shall allow students to change their account password.	Medium	Student	Proposed
FR-S005	The system shall allow students to log out with secure session termination.	High	Student	Proposed
FR-S006	The system shall allow students or	Medium	Student, Parent	Proposed

	parents to view and update parental contact information.			
FR-S007	The system shall allow students to view all available courses with filtering by grade level.	High	Student	Proposed
FR-S008	The system shall allow students to enrol in a course.	High	Student	Proposed
FR-S009	The system shall allow students to view a list of all their enrolled courses.	High	Student	Proposed
FR-S010	The system shall allow students to drop (unenrol from) a course.	Medium	Student	Proposed
FR-S011	The system shall provide AI-based course recommendations by analysing the student's learning history.	Medium	Student, AI System	Proposed
FR-S012	The system shall display a course dashboard with an overview of modules, lessons, progress, and announcements.	High	Student	Proposed
FR-S013	The system shall allow students to download study materials uploaded by teachers.	High	Student	Proposed

FR-S014	The system shall allow students to take and save personal notes within the platform.	Low	Student	Proposed
FR-S015	The system shall allow students to search topics by keyword or chapter across their courses.	Medium	Student	Proposed
FR-S016	The system shall allow students to mark individual lessons as completed.	Medium	Student	Proposed
FR-S017	The system shall provide an AI learning assistant that can summarise content, answer questions, and explain difficult topics.	Medium	Student, AI System	Proposed
FR-S018	The system shall allow students to view lesson videos and multimedia content.	High	Student	Proposed
FR-S019	The system shall allow students to bookmark lessons for quick access.	Low	Student	Proposed
FR-S020	The system shall display all assignments and their due dates to students.	High	Student	Proposed

FR-S021	The system shall allow students to upload and submit completed assignments.	High	Student	Proposed
FR-S022	The system shall allow students to attempt online quizzes including MCQ and short answer formats.	High	Student	Proposed
FR-S023	The system shall display quiz results including marks and correct solutions after completion.	High	Student	Proposed
FR-S024	The system shall allow students to view all their grades and scores across assignments and quizzes.	High	Student	Proposed
FR-S025	The system shall allow students to view feedback provided by the teacher or generated by the AI.	Medium	Student, Teacher, AI System	Proposed
FR-S026	The system shall allow students to chat with classmates in group chats within a course.	Medium	Student	Proposed
FR-S027	The system shall allow students to send private messages to	Medium	Student, Teacher	Proposed

	teachers.			
FR-S028	The system shall allow students to participate in topic based discussion forums within a course.	Medium	Student	Proposed
FR-S029	The system shall provide a 24/7 AI powered help chatbot for study questions, platform issues, and course content.	Medium	Student, AI System	Proposed
FR-S030	The system shall deliver notifications to students about new assignments, announcements, grades, and messages.	High	Student	Proposed
FR-S031	The system shall display visual progress reports with charts showing completion, scores, and learning trends.	Medium	Student	Proposed
FR-S032	The system shall allow students to set personal learning goals.	Low	Student	Proposed
FR-S033	The system shall allow students to view their attendance records.	Medium	Student	Proposed

2.1: Functional Requirements - Students

2.3.2 Teacher Functional Requirements

ID	Description	Priority	Source	Status
FR-T001	The system shall allow a new teacher to register by entering personal details, subject specialisation, and credentials.	High	Teacher, Admin	Proposed
FR-T002	The system shall allow teachers to log in using JWT-based secure authentication.	High	Teacher	Proposed
FR-T003	The system shall allow teachers to view and edit their profile information.	Medium	Teacher	Proposed
FR-T004	The system shall allow teachers to reset their password via a "forgot	Medium	Teacher	Proposed

	password" mechanism.			
FR-T005	The system shall allow teachers to log out with secure session termination.	High	Teacher	Proposed
FR-T006	The system shall allow teachers to create a new course with name, description, and grade level.	High	Teacher	Proposed
FR-T007	The system shall allow teachers to edit or delete an existing course.	High	Teacher	Proposed
FR-T008	The system shall allow teachers to view the list of students enrolled in each of their courses.	High	Teacher	Proposed
FR-T009	The system shall allow teachers to approve or reject	Medium	Teacher	Proposed

	student enrolment requests.			
FR-T010	The system shall allow teachers to update course contents and materials.	High	Teacher	Proposed
FR-T011	The system shall allow teachers to set scheduled time slots for class sessions.	Medium	Teacher	Proposed
FR-T012	The system shall allow teachers to manage and update their availability schedule.	Medium	Teacher	Proposed
FR-T013	The system shall allow teachers to upload learning materials including PDFs, videos, and slides.	High	Teacher	Proposed
FR-T014	The system shall allow teachers to organise lessons into modules, chapters, or units.	High	Teacher	Proposed

FR-T015	The system shall allow teachers to edit or delete individual lessons.	Medium	Teacher	Proposed
FR-T016	The system shall allow teachers to schedule lesson release dates.	Medium	Teacher	Proposed
FR-T017	The system shall allow teachers to view student engagement data including lesson views and study time.	Medium	Teacher	Proposed
FR-T018	The system shall allow teachers to add subtitles or transcripts to video lessons.	Low	Teacher	Proposed
FR-T019	The system shall allow teachers to set prerequisite lessons.	Medium	Teacher	Proposed
FR-T020	The system shall allow teachers to post course	Medium	Teacher	Proposed

	announcements to enrolled students.			
FR-T021	The system shall allow teachers to create assignments with instructions, file requirements, and deadlines.	High	Teacher	Proposed
FR-T022	The system shall allow teachers to view all submitted assignments.	High	Teacher	Proposed
FR-T023	The system shall	High	Teacher	Proposed
	allow teachers to grade student assignments.			
FR-T024	The system shall allow teachers to create online quizzes with questions, answer choices, and scoring rules.	High	Teacher	Proposed
FR-T025	The system shall provide AI auto generation of quiz questions	Medium	Teacher, AI System	Proposed

	based on course content.			
FR-T026	The system shall allow teachers to publish quiz results to students.	High	Teacher	Proposed
FR-T027	The system shall provide AI-assisted grading for assignments and quizzes.	Medium	Teacher, AI System	Proposed
FR-T028	The system shall allow teachers to set assignment deadlines and submission rules.	High	Teacher	Proposed
FR-T029	The system shall allow teachers to return graded assignments with written feedback.	High	Teacher	Proposed
FR-T030	The system shall allow teachers to chat with individual	Medium	Teacher	Proposed

	students or groups.			
FR-T031	The system shall allow teachers to post class-wide announcements.	Medium	Teacher	Proposed
FR-T032	The system shall allow teachers to moderate discussion forums.	Medium	Teacher	Proposed
FR-T033	The system shall allow teachers to send bulk notifications to all students in a course.	Medium	Teacher	Proposed
FR-T034	The system shall display a student performance dashboard with scores, completion rates, and progress.	High	Teacher	Proposed
FR-T035	The system shall	Medium	Teacher	Proposed

	allow teachers to manually trigger notifications and alerts to students.			
FR-T036	The system shall provide AI generated insights identifying student weaknesses.	Medium	Teacher, AI System	Proposed
FR-T037	The system shall allow teachers to export student performance reports.	Medium	Teacher	Proposed
FR-T038	The system shall allow teachers to view attendance reports.	Medium	Teacher	Proposed

2.2: Functional Requirements - Teachers

2.3.3 Administrator Functional Requirements

ID	Description	Priority	Source	Status
FR-A001	The system shall allow the admin to create and edit teacher and student accounts.	High	Admin	Proposed
FR-A002	The system shall allow the admin to assign or modify user roles (Admin, Teacher, Student).	High	Admin	Proposed
FR-A003	The system shall allow the admin to deactivate or archive user accounts.	Medium	Admin	Proposed
FR-A004	The system shall allow the admin to configure Single Sign-On (SSO).	Medium	Admin	Proposed
FR-A005	The system shall allow the admin to configure Multi	Medium	Admin	Proposed

	Factor Authentication (MFA).			
FR-A006	The system shall allow the admin to configure Role Based Access Control (RBAC).	High	Admin	Proposed
FR-A007	The system shall allow the admin to create custom roles beyond the default three.	Low	Admin	Proposed
FR-A008	The system shall allow the admin to view all courses and content on the platform.	High	Admin	Proposed
FR-A009	The system shall allow the admin to audit course content for compliance with institutional standards.	Medium	Admin	Proposed

FR-A010	The system shall allow the admin to publish or unpublish courses.	High	Admin	Proposed
FR-A011	The system shall allow the admin to manage course categories and tags.	Medium	Admin	Proposed
FR-A012	The system shall allow the admin to assign a teacher to a course.	High	Admin	Proposed
FR-A013	The system shall allow the admin to generate comprehensive usage reports.	Medium	Admin	Proposed
FR-A014	The system shall allow the admin to analyse system engagement metrics.	Medium	Admin	Proposed

2.3: Functional Requirements - Administrators

2.3.4 Support Team Functional Requirements

ID	Description	Priority	Source	Status
FR-SP001	The system shall allow the support team to view and filter support tickets by status, priority, or type.	High	Support Team	Proposed
FR-SP002	The system shall allow the support team to assist users with login and password issues.	High	Support Team	Proposed
FR-SP003	The system shall allow the support team to guide users through platform features.	Medium	Support Team	Proposed
FR-SP004	The system shall allow the support team to	High	Support Team	Proposed

	troubleshoot technical glitches.			
FR-SP005	The system shall allow the support team to provide course enrolment support.	Medium	Support Team	Proposed
FR-SP006	The system shall allow the support team to create and edit FAQ articles.	Medium	Support Team	Proposed
FR-SP007	The system shall allow the support team to organise knowledge base categories.	Medium	Support Team	Proposed
FR-SP008	The system shall allow the support team to identify gaps	Low	Support Team	Proposed

	in knowledge base content.			
FR-SP009	The system shall allow the support team to escalate complex issues to admins or developers.	High	Support Team	Proposed
FR-SP010	The system shall allow the support team to collaborate with teachers on student issues.	Medium	Support Team	Proposed
FR-SP011	The system shall provide the support team with read-only access to relevant admin tools.	Medium	Support Team	Proposed

2.4: Functional Requirements - Support Team

2.3.5 AI System Functional Requirements

ID	Description	Priority	Source	Status
FR-AI001	The AI system shall analyse student learning history and provide personalised course recommendations.	Medium	AI System, Student	Proposed
FR-AI002	The AI system shall summarise lesson content and explain difficult topics on demand.	Medium	AI System, Student	Proposed
FR-AI003	The AI system shall provide a 24/7 chatbot for answering study questions, platform issues, and course content queries.	Medium	AI System, Student	Proposed
FR-AI004	The AI system shall auto-generate quiz questions based on course content.	Medium	AI System, Teacher	Proposed

FR-AI005	The AI system shall assist in grading assignments and quizzes, particularly objective-type questions.	Medium	AI System, Teacher	Proposed
FR-AI006	The AI system shall analyse student performance data and identify specific areas of weakness.	Medium	AI System, Teacher	Proposed
FR-AI007	The AI system shall generate feedback on student submissions.	Medium	AI System, Teacher, Student	Proposed

2.5: Functional Requirements - AI System

2.4 Non-Functional Requirements

Non-functional requirements define the quality attributes, performance expectations, operational constraints, and system characteristics that the Mentify Digital Learning Platform must satisfy.

2.4.1 Performance

ID	Category	Description	Acceptance Criterion
NFR-P001	Performance	The system shall respond to user interactions promptly under normal load conditions.	95% of page load requests shall complete within 2 seconds under normal load (up to 500 concurrent users).
NFR-P002	Performance	The system shall handle search queries efficiently.	95% of search queries shall return results within 2 seconds.
NFR-P003	Performance	The system shall process quiz submissions without delay.	Quiz submission and result calculation shall complete within 3 seconds.
NFR-P004	Performance	The system shall support concurrent usage during peak hours.	The system shall support a minimum of 1,000 concurrent users without response time degradation exceeding 20%.

2.6: Non Functional Requirements - Performance

2.4.2 Reliability

ID	Category	Description	Acceptance Criterion
NFR-R001	Reliability	The system shall maintain high availability.	The system shall achieve 99.5% uptime measured monthly, excluding scheduled maintenance.
NFR-R002	Reliability	The system shall recover from	Full recovery shall occur within 30 minutes with no

		failures with minimal data loss.	more than 5 minutes of transactional data loss.
NFR-R003	Reliability	The system shall perform automated backups.	Automated database backups shall occur every 24 hours and be stored in a geographically separate location.

2.7: Non-Functional Requirements - Reliability

2.4.3 Usability

ID	Category	Description	Acceptance Criterion
NFR-U001	Usability	The system shall be easy to learn for new users.	A new student shall enrol in a course and access lesson content within 5 minutes of first login without assistance.
NFR-U002	Usability	The system shall provide a responsive user interface.	The UI shall function correctly on screens ranging from 320px to 1920px.
NFR-U003	Usability	The system shall meet accessibility standards.	The system shall comply with WCAG 2.1 Level AA guidelines.

NFR-U004	Usability	The system shall support multiple languages.	The system shall support English and Tamil as primary interface languages.

2.8: Non-Functional Requirements - Usability

2.4.4 Security

ID	Category	Description	Acceptance Criterion
NFR-SE001	Security	The system shall authenticate users securely.	Authentication shall use JWT tokens with a maximum expiry of 24 hours and refresh token support.
NFR-SE002	Security	The system shall encrypt sensitive data.	Data in transit shall use TLS 1.2 or higher; passwords shall be hashed using bcrypt (minimum cost factor 10).
NFR-SE003	Security	The system shall enforce role-based access control.	Users shall not access functionality beyond assigned permissions, verified via RBAC middleware.
NFR-SE004	Security	The system shall protect against common web vulnerabilities.	The system shall pass OWASP Top 10 assessment with no critical or high-severity findings.
NFR-SE005	Security	The system shall support Multi Factor Authentication.	MFA shall be available for all roles and mandatory for admin accounts.

2.9: Non-Functional Requirements - Security

2.4.5 Scalability

ID	Category	Description	Acceptance Criterion
NFR-SC001	Scalability	The system shall scale horizontally to accommodate growth.	The architecture shall support a 300% user base increase without redesign.
NFR-SC002	Scalability	The system shall handle growing data volumes.	The database shall manage up to 1 million course records and 10 million activity records with performance degradation not exceeding 10%.

2.10: Non-Functional Requirements - Scalability

2.4.6 Maintainability

ID	Category	Description	Acceptance Criterion
NFR-M001	Maintainability	The system shall follow modular architecture principles.	The system shall adopt a microservice architecture enabling independent module deployment.
NFR-M002	Maintainability	The system shall maintain comprehensive code documentation.	All APIs shall include Swagger/OpenAPI documentation; code coverage shall exceed 80%.
NFR-M003	Maintainability	The system shall support continuous integration and deployment.	Automated CI/CD pipelines shall enable deployment within 30 minutes.

2.11: Non-Functional Requirements - Maintainability

2.4.7 Portability

ID	Category	Description	Acceptance Criterion

NFR-PO001	Portability	The system shall be accessible across major web browsers.	The system shall function on the latest two versions of Chrome, Firefox, Safari, and Edge.
NFR-PO002	Portability	The system shall be deployable across different hosting environments.	The system shall be containerised using Docker and deployable on AWS, Azure, or Kubernetes-compatible environments.

2.12: Non-Functional Requirements - Portability

2.5 Business Rules, Constraints and Assumptions

This chapter defines the governing business policies, operational limitations, and underlying assumptions that influence the design, development, and implementation of the Mentify Digital Learning Platform.

2.5.1 Business Rules

ID	Business Rule
BR-001	A student may only enrol in courses that match their registered grade level unless explicitly approved by a teacher or administrator.
BR-002	Each course must have at least one assigned teacher before it can be published and made visible to students.
BR-003	Assignment submissions shall not be accepted after the deadline unless the teacher has enabled a late submission policy for that assignment.
BR-004	A student must complete all prerequisite lessons before accessing lessons configured with prerequisites by the teacher.
BR-005	Only users with the Administrator role may create, modify, or deactivate user accounts. Teachers shall not manage accounts of other teachers.
BR-006	Support team members shall have read-only access to administrative tools and shall not be permitted to modify user accounts or system configurations.
BR-007	AI-generated quiz questions must be reviewed and approved by the teacher before being published to students.
BR-008	All communication within the platform, including group chats, private messages, and discussion forums, shall comply with TIT Education's code of conduct and be subject to moderation.
BR-009	Notifications shall be delivered in real time where possible, with fallback

	delivery via email if the user is not actively logged into the system.

2.13: Business Rules

2.5.2 Constraints

ID	Constraint
C-001	The system must be developed within the academic timeline defined by the SENG 31242 module.
C-002	The development team consists of four members (Holins, Dilakshan, Jenushan, Zaina), which limits the feature scope deliverable in the initial release.
C-003	The system shall be accessible via standard web browsers and shall not require native mobile applications in the initial release.
C-004	AI functionalities are dependent on the availability, performance, and cost of third-party AI/ML service providers.
C-005	The system must comply with Sri Lankan data protection regulations and institutional policies of TIT Education.
C-006	Initial deployment must align with the existing infrastructure capabilities of TIT Education.

2.14: Constraints

2.5.3 Assumptions

ID	Assumption
A-001	It is assumed that all students and teachers have access to internet enabled devices such as laptops, tablets, or smartphones.
A-002	It is assumed that TIT Education will provide the required domain expertise, user feedback, and initial course content.
A-003	It is assumed that users possess basic digital literacy required to navigate a web-based application.
A-004	It is assumed that TIT Education will designate at least one staff member to function as the system administrator.

A-005	It is assumed that the institution will provide or procure the necessary cloud hosting infrastructure for deployment.
A-0066	It is assumed that continuous internet connectivity is required for all system operations.

2.15: Assumptions

2.6 Use Case Diagrams and Descriptions

The use case diagrams illustrate the interactions between the primary actors and the Mentify Digital Learning Platform. The following use case diagrams are included in this chapter

2.6.1 Student Use Case Diagrams

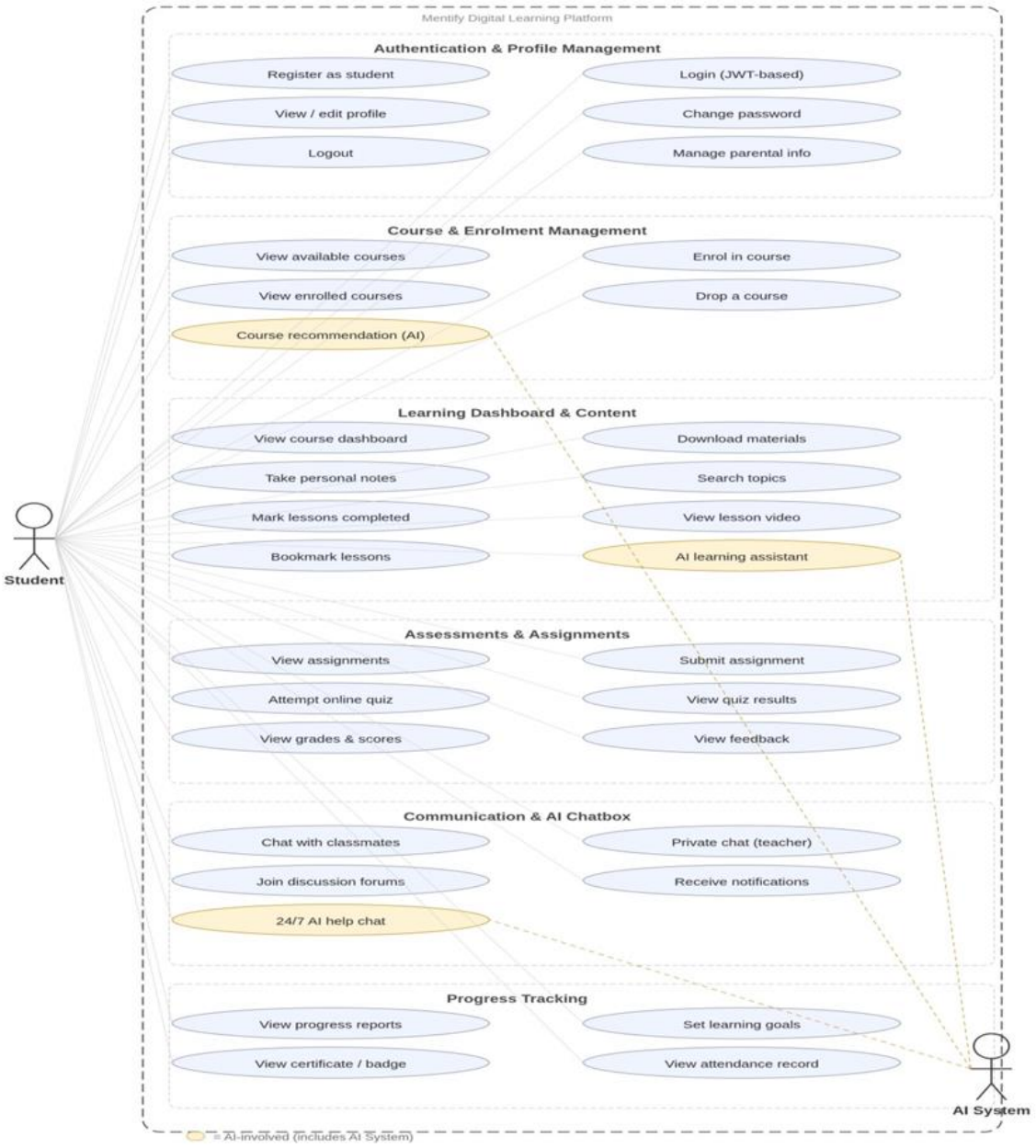


Figure 2.1: Student Use Case Diagram

2.6.2 Teacher Use Case Diagram

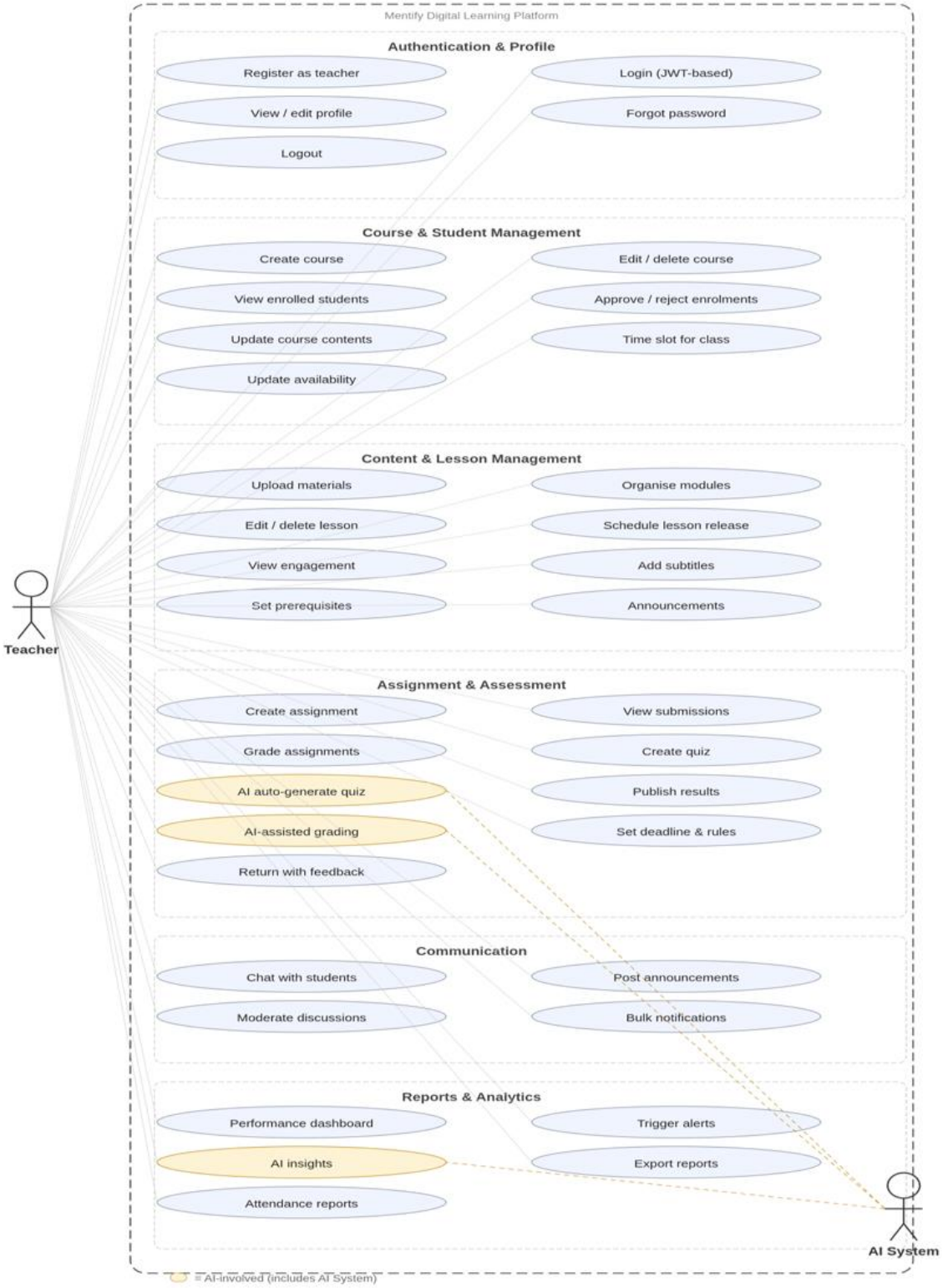


Figure 2.2: Teacher Use Case Diagram

2.6.3 Administrator Use Case Diagram

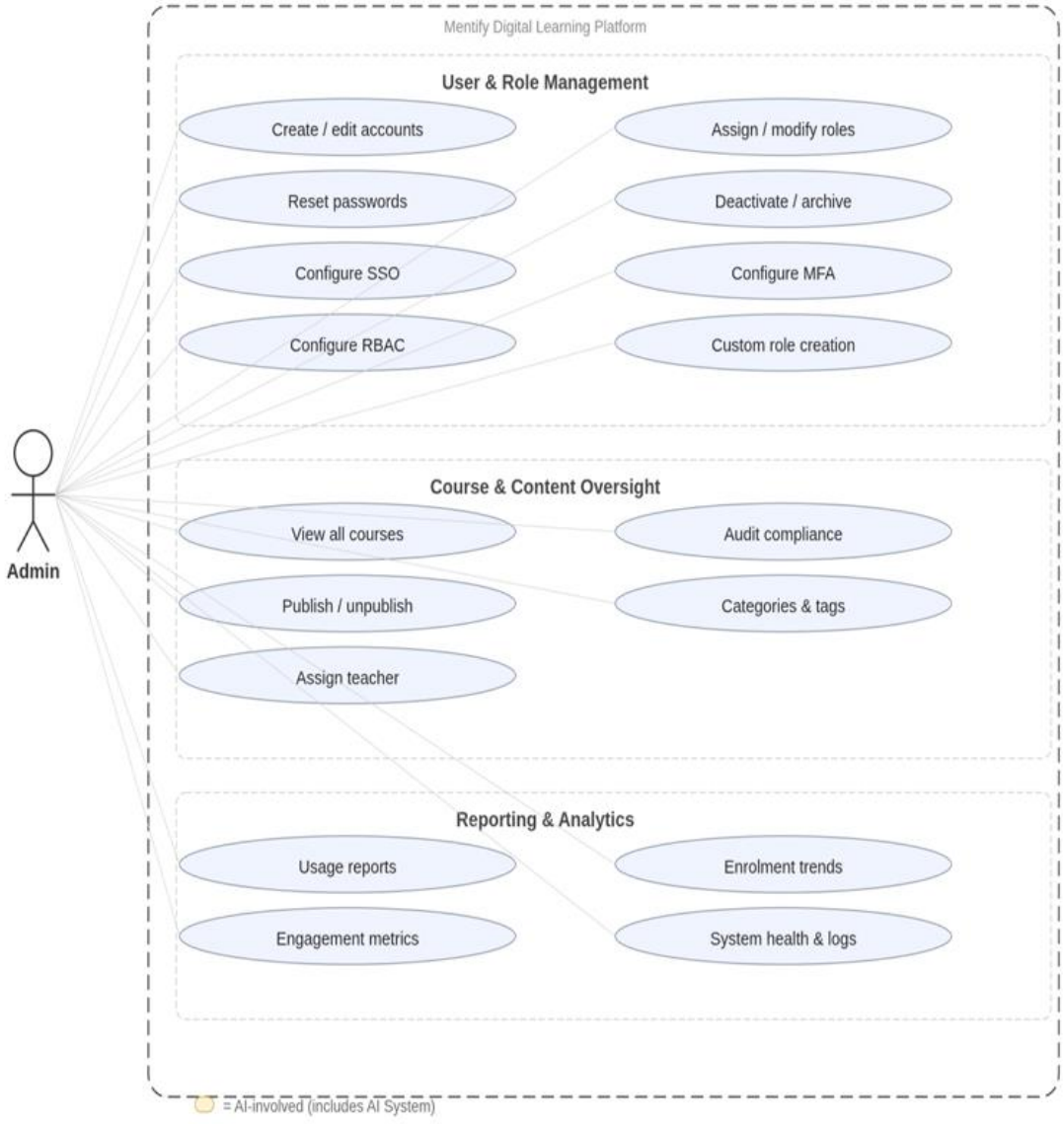


Figure 2.3: Admin Use Case Diagram

2.6.4 Support Team Use Case Diagram

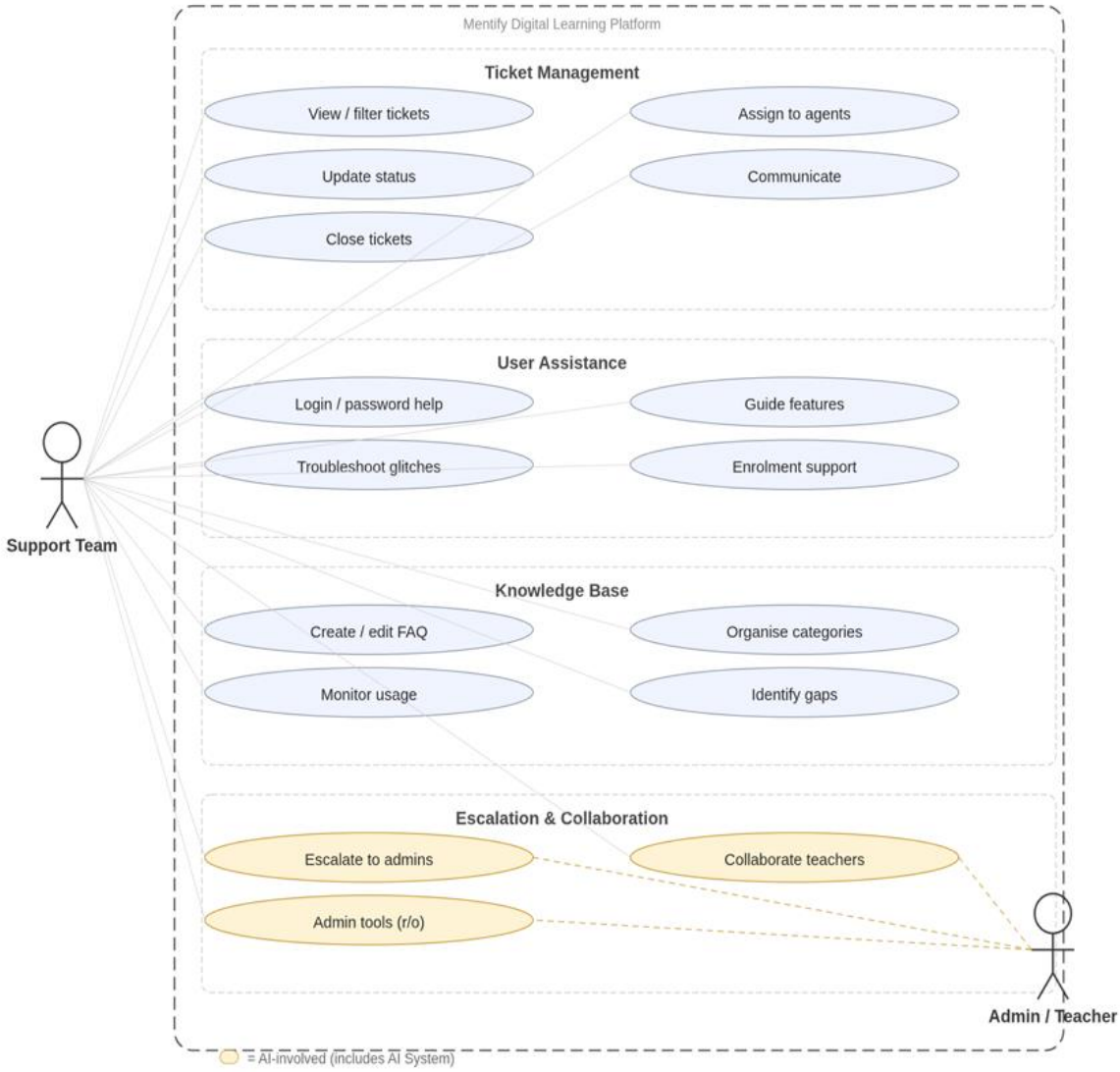


Figure 2.4: Support Team Use Case Diagram

2.6.5 Use Case Descriptions

The following use case descriptions present the major system interactions for the primary actors of the Mentify Digital Learning Platform.

Field	Description
Use Case ID	UC-S001
Use Case Name	Register as Student
Primary Actor	Student
Description	A new student creates an account on the platform by providing personal details or using third-party authentication.
Preconditions	The student does not already have an account on the platform. The registration page is accessible.
Postconditions	A new student account is created and the student is redirected to the login page or dashboard.
Main Flow	1. The student navigates to the registration page. 2. The system displays the registration form. 3. The student enters their name, email, grade level, and password. 4. The student submits the form. 5. The system validates the input data. 6. The system creates the account and sends a verification email. 7. The student verifies their email.

	8. The system activates the account and redirects the student to the login page.
Alternative Flow	3a. The student selects “Register with Google.” 3b. The system redirects the student to the third-party authentication provider. 3c. Upon successful authentication, the system creates the account using the received profile information.
Exception Flow	5a. If the email is already registered, the system displays the message: “An account with this email already exists.” 5b. If required fields are missing, the system highlights the missing fields and prompts the student to complete them.
Priority	High

2.16: Use Case Description: UC-S001

Field	Description
Use Case ID	UC-S002
Use Case Name	Login (JWT-based)
Primary Actor	Student
Description	The student logs into the system using email and password with JWT-based authentication.

Preconditions	The student has a registered and verified account.
Postconditions	The student is authenticated and a JWT token is issued for the session.
Main Flow	1. The student navigates to the login page. 2. The student enters their email and password. 3. The system validates the credentials. 4. The system generates a JWT access token and refresh token. 5. The system redirects the student to their dashboard.
Alternative Flow	2a. The student selects "Login with Google." 2b. The system authenticates the student via the third-party provider and issues a JWT token upon success.
Exception Flow	3a. If the credentials are invalid, the system displays the message: "Invalid email or password." 3b. If the account is deactivated, the system displays the message: "Your account has been deactivated. Please contact support."
Priority	High

2.17: Use Case Description: UC-S002

Field	Description
Use Case ID	UC-S008
Use Case Name	Enrol in Course
Primary Actor	Student
Description	The student selects a course from the available courses list and enrols in it to gain access to the course content.

Preconditions	The student is logged in. The course is published and matches the student's grade level.
Postconditions	The student is enrolled in the course and it appears in their enrolled courses list.
Main Flow	1. The student navigates to the available courses page. 2. The student selects a course. 3. The system displays the course details. 4. The student clicks "Enrol." 5. The system processes the enrolment request. 6. If auto-approval is enabled, the system confirms enrolment immediately. 7. The course appears in the student's enrolled courses list.
Alternative Flow	6a. If teacher approval is required, the system sends the enrolment request to the teacher and displays the message: "Your enrolment is pending teacher approval."
Exception Flow	5a. If the student is already enrolled, the system displays the message: "You are already enrolled in this course."
Priority	High

2.18: Use Case Description: UC-S003

2.7 Activity Diagrams

Activity diagrams illustrate the dynamic workflows and operational processes of the Mentify Digital Learning Platform. These diagrams represent the sequence of actions, decision points, system validations, and outcomes involved in key user interactions and system operations. Each activity diagram provides a visual representation of system behaviour for major functional scenarios involving Students, Teachers, Administrators, Support Team members, and the AI subsystem.

2.7.1 Student Login (JWT-Based)

This activity diagram illustrates the student login workflow. The process begins when the student navigates to the login page and enters their email and password. The system validates the credentials. If the credentials are invalid, an error message is displayed with options to retry authentication or initiate the password recovery process. If the credentials are valid, the system verifies whether the account is active. If the account is deactivated, an appropriate message is displayed. If the account is active, the system generates a JWT access token and refresh token, establishes a session, and redirects the student to the dashboard.

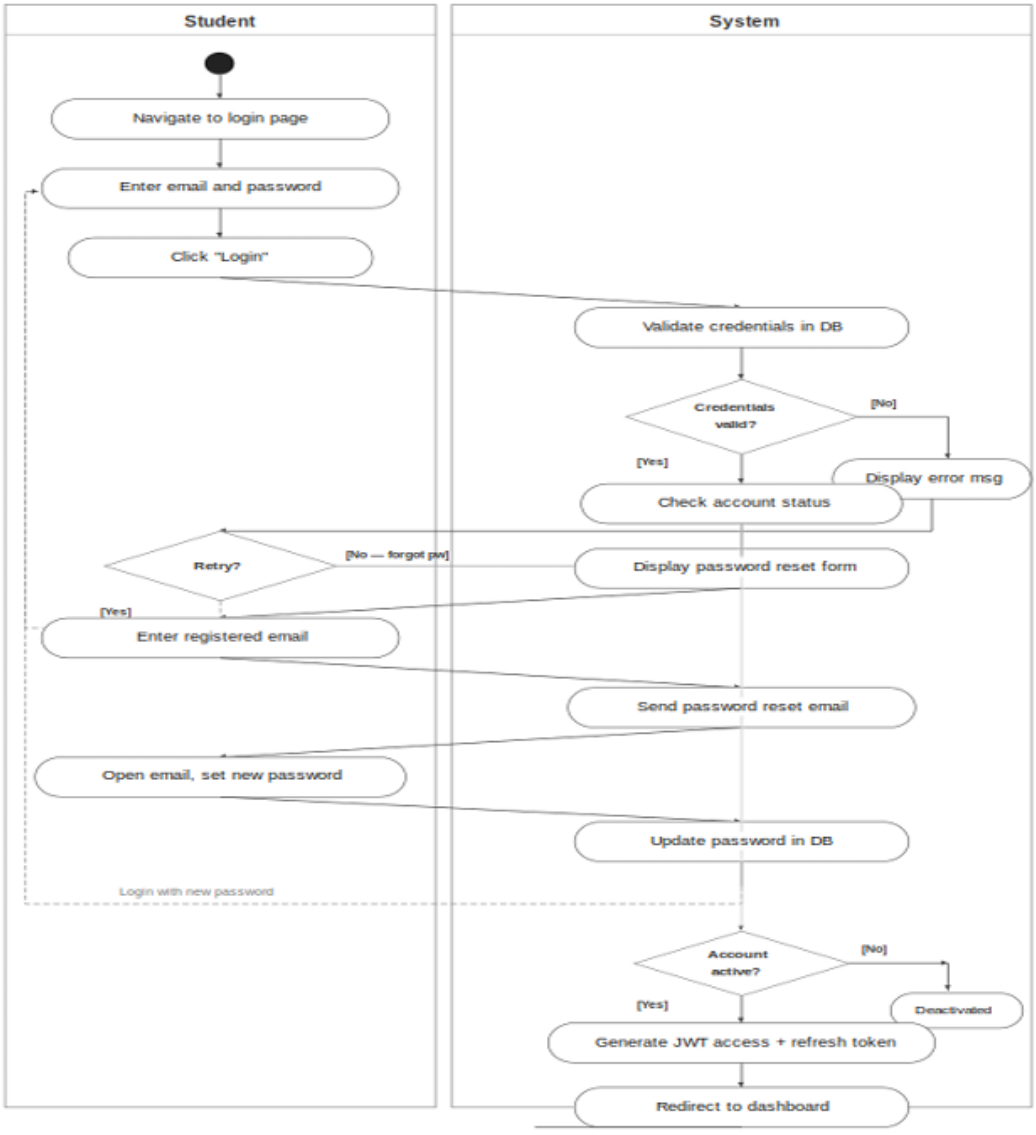


Figure 2.5: Activity Diagram - Student Login

2.7.2 Student Registration

This activity diagram illustrates the student registration process. The student navigates to the registration page and selects either standard registration or third-party authentication. In the standard flow, the system validates the entered details, creates the account, and sends a verification email. The account becomes active upon successful email verification. In the third party authentication flow, the system redirects the user to the external provider, retrieves the user profile upon successful authentication, and creates the account. Both flows converge at the account activation stage.

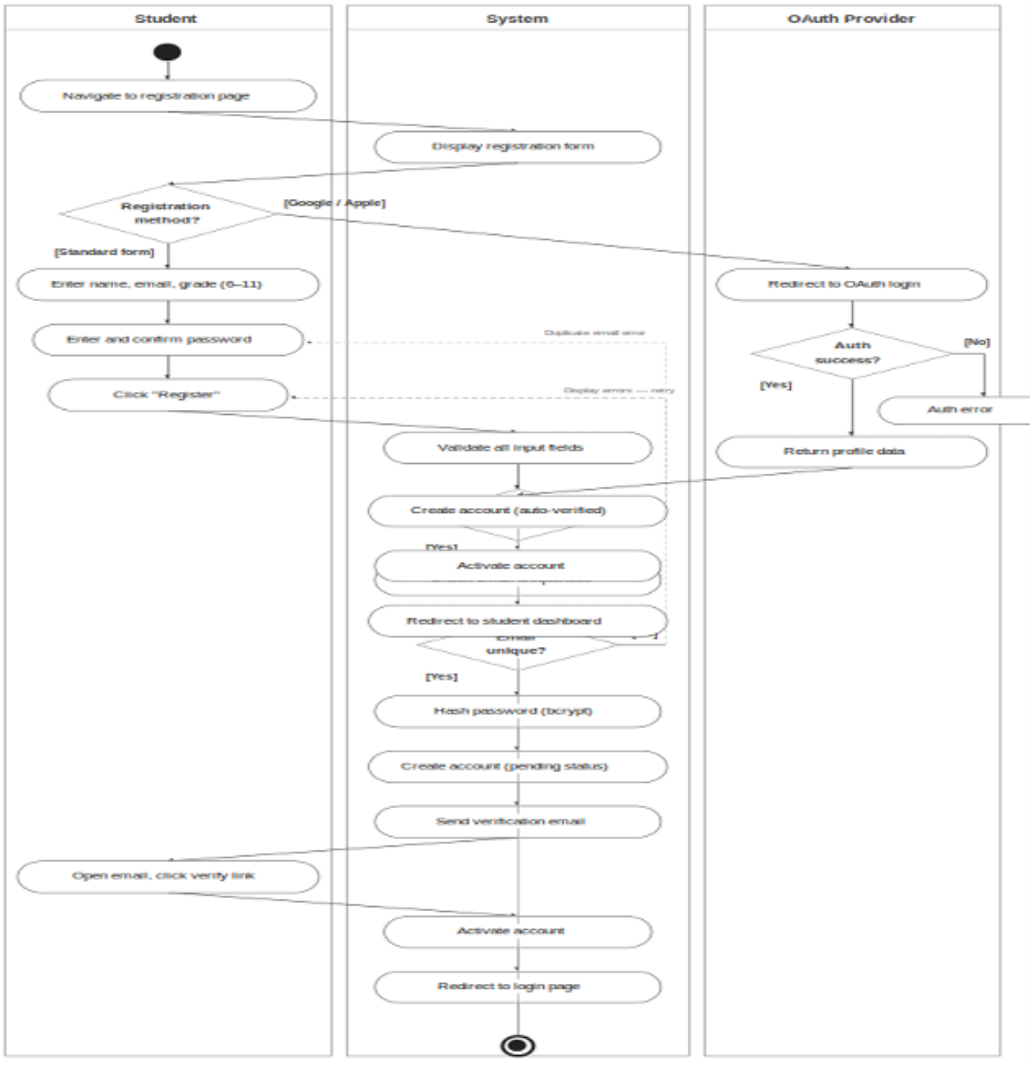


Figure 2.6: Activity Diagram - Student Registration

2.7.3 Course Enrolment

This activity diagram describes the course enrolment workflow. The student browses available courses, filters them based on grade level, selects a course, and initiates enrolment. The system verifies eligibility. If the student is not eligible, an error message is displayed. If eligible, the system determines whether enrolment requires teacher approval. In the case of auto-approval, enrolment is confirmed immediately. If teacher approval is required, the request is forwarded to the teacher. Upon approval, enrolment is confirmed; otherwise, the student is notified of rejection.

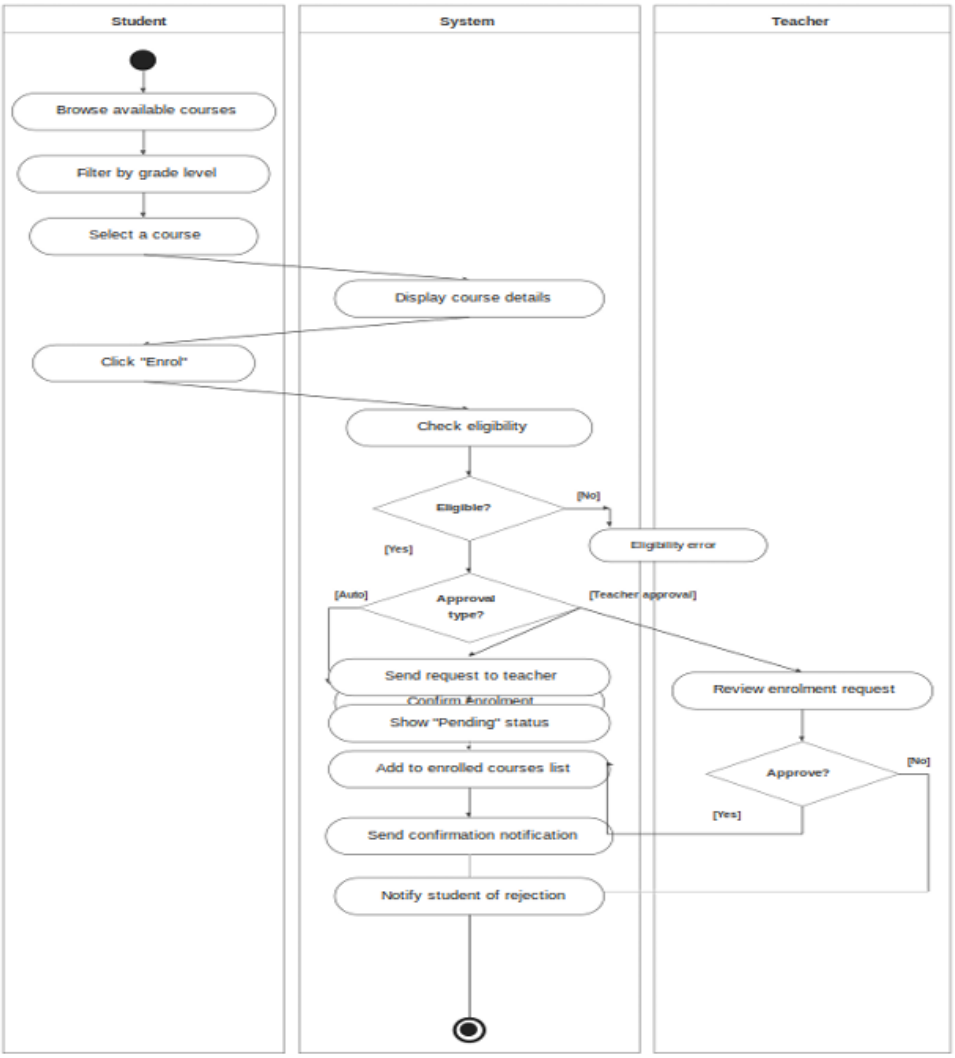


Figure 2.7: Activity Diagram - Course Enrolment

2.7.4 Attempt Online Quiz

This activity diagram illustrates the quiz-taking process. The student selects a quiz and the system verifies the availability of remaining attempts. If no attempts remain, an error message is displayed. If attempts are available, the student initiates the quiz and the system starts the timer. Questions are presented sequentially or collectively, and the student submits responses. If the timer expires, the system automatically submits the quiz. Otherwise, the student submits manually. The system then evaluates objective questions and displays the results.

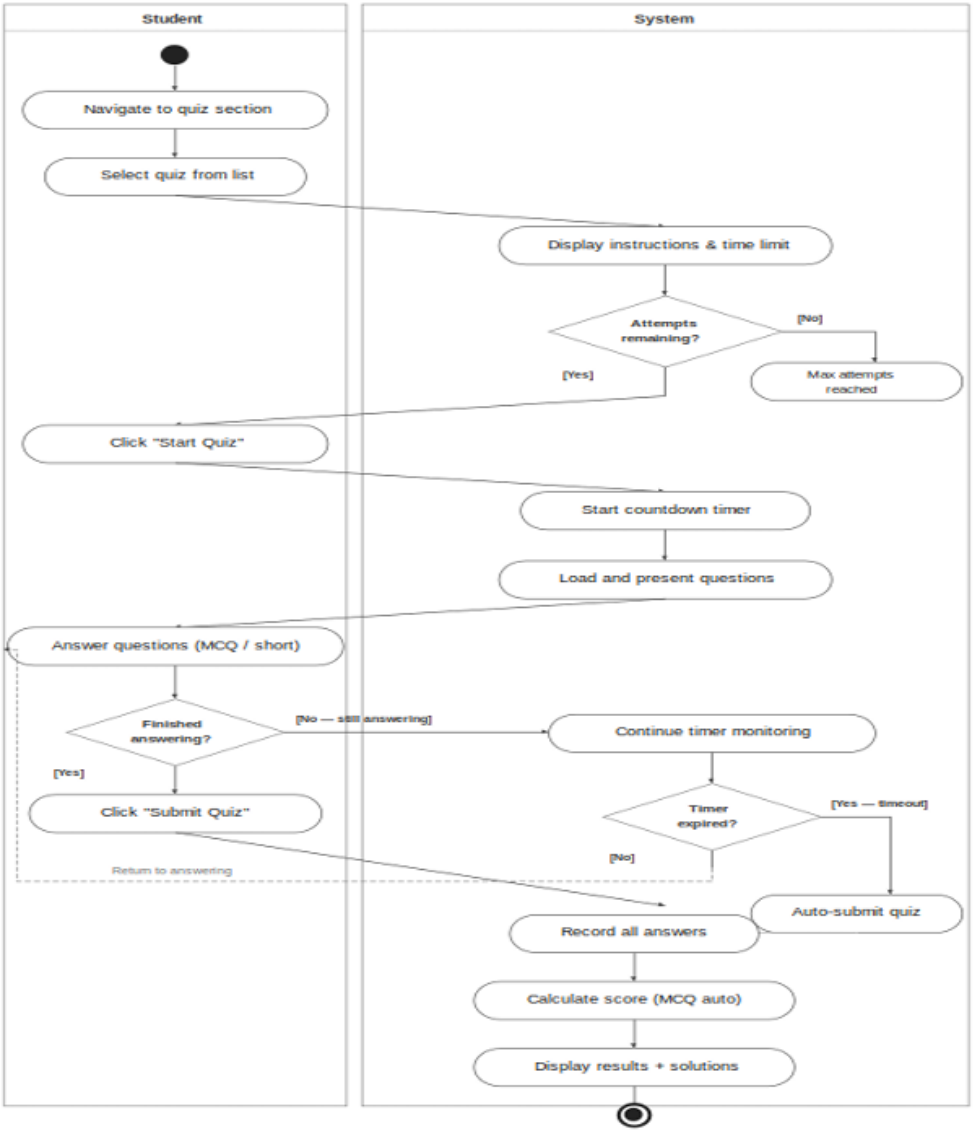


Figure 2.8: Activity Diagram - Atempt Quiz

2.7.5 Teacher Creates and Publishes Quiz with AI Assistance

This activity diagram represents the quiz creation workflow. The teacher navigates to the assessment section, initiates quiz creation, and enters required details. The teacher may add questions manually or utilise AI-generated suggestions. In the AI flow, the system analyses course content and generates suggested questions, which the teacher reviews and modifies. Both flows converge when scoring rules are defined. The quiz is saved in draft status and subsequently published.

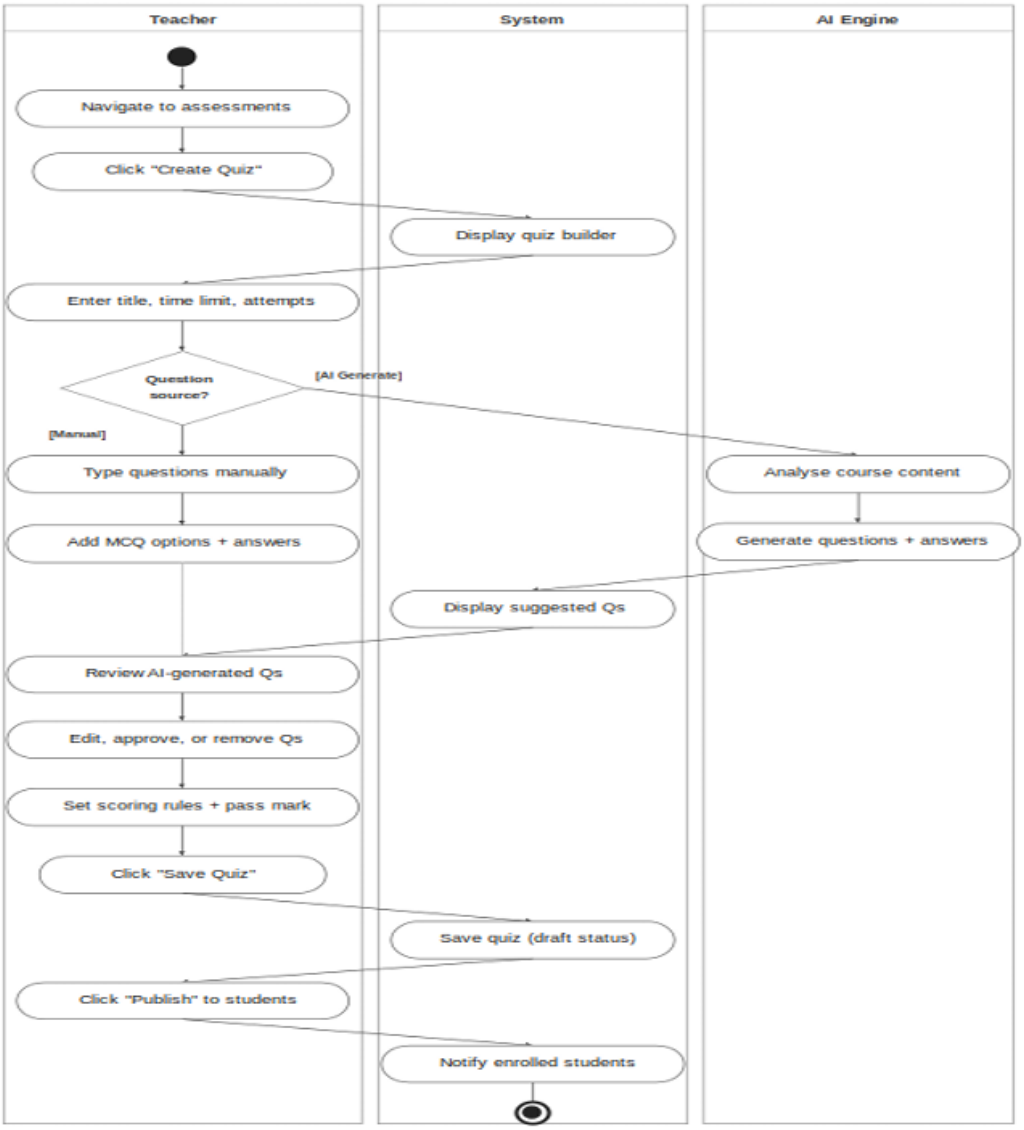


Figure 2.9: Activity Diagram - Publish Quiz

2.7.6 Teacher Grades Assignment with AI Assistance

This activity diagram shows the assignment grading process. The teacher accesses submitted assignments and reviews the content. The teacher may use AI assistance to obtain suggested scores. Whether graded manually or with AI support, the process converges when the teacher finalises the score and feedback. The system records the grade and notifies the student.

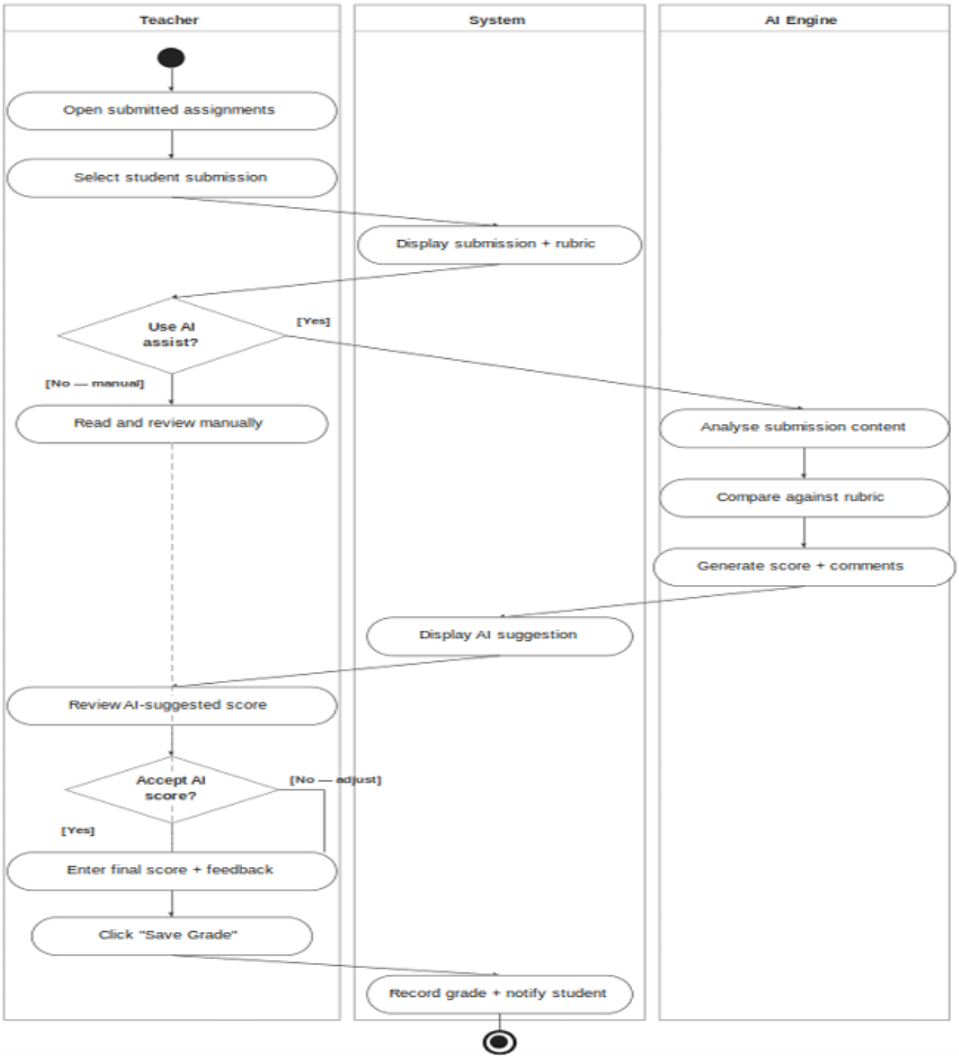


Figure 2.10: Activity Diagram - Grade Quiz

2.7.7 Upload Learning Materials

This activity diagram illustrates the content upload workflow. The teacher navigates to the course content area, selects the target module, and uploads learning materials. The system validates file type and size. If validation fails, an error is displayed. If successful, the content is uploaded and confirmation is provided.

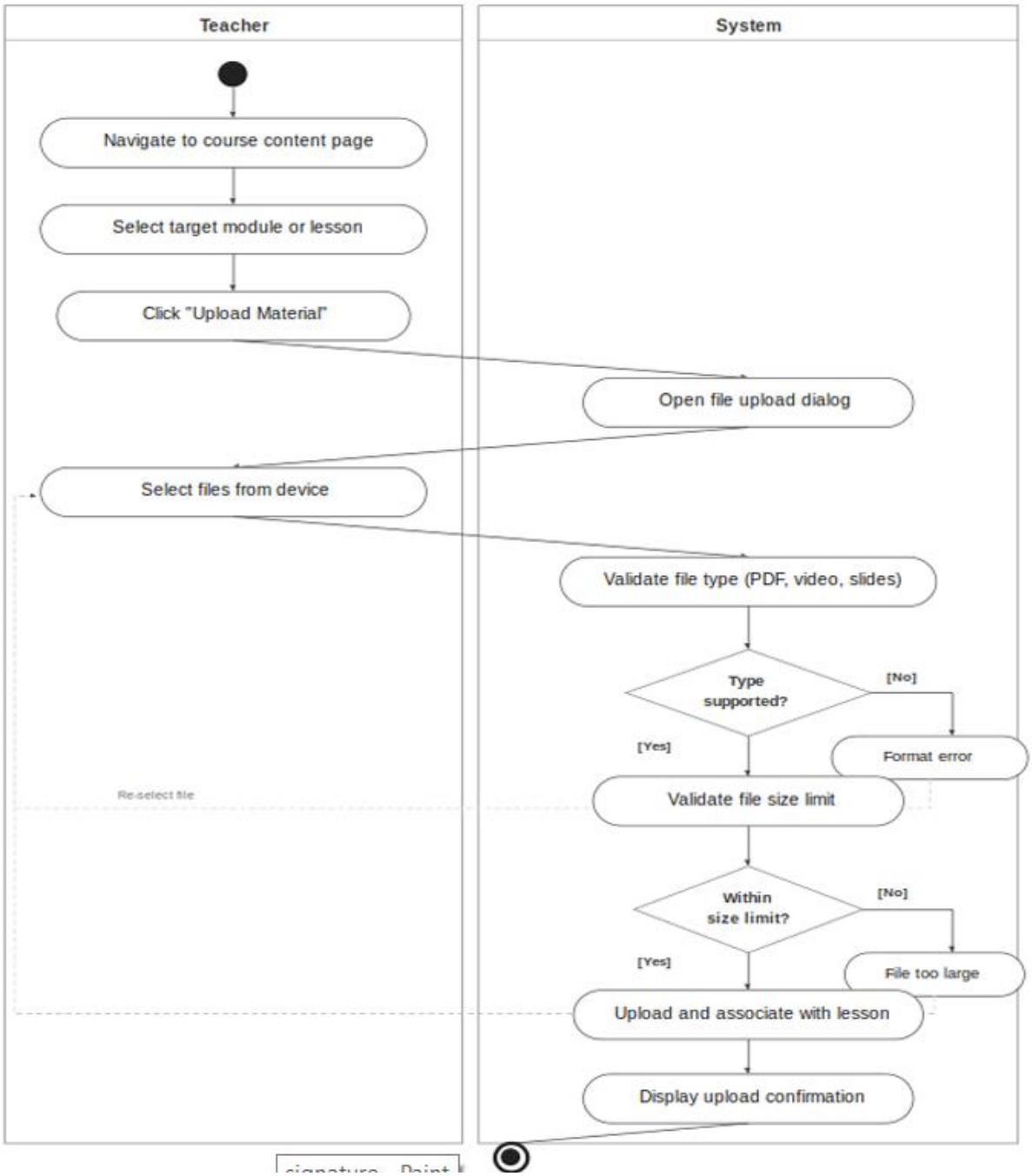


Figure 2.11: Activity Diagram - Upload Materials

2.7.8 Admin Creates User Account

This activity diagram shows the administrative user account creation process. The administrator navigates to the user management module and enters required details. The system validates input and checks for duplicate records. If validation fails, errors are displayed. If validation succeeds, the account is created and a welcome email is sent.

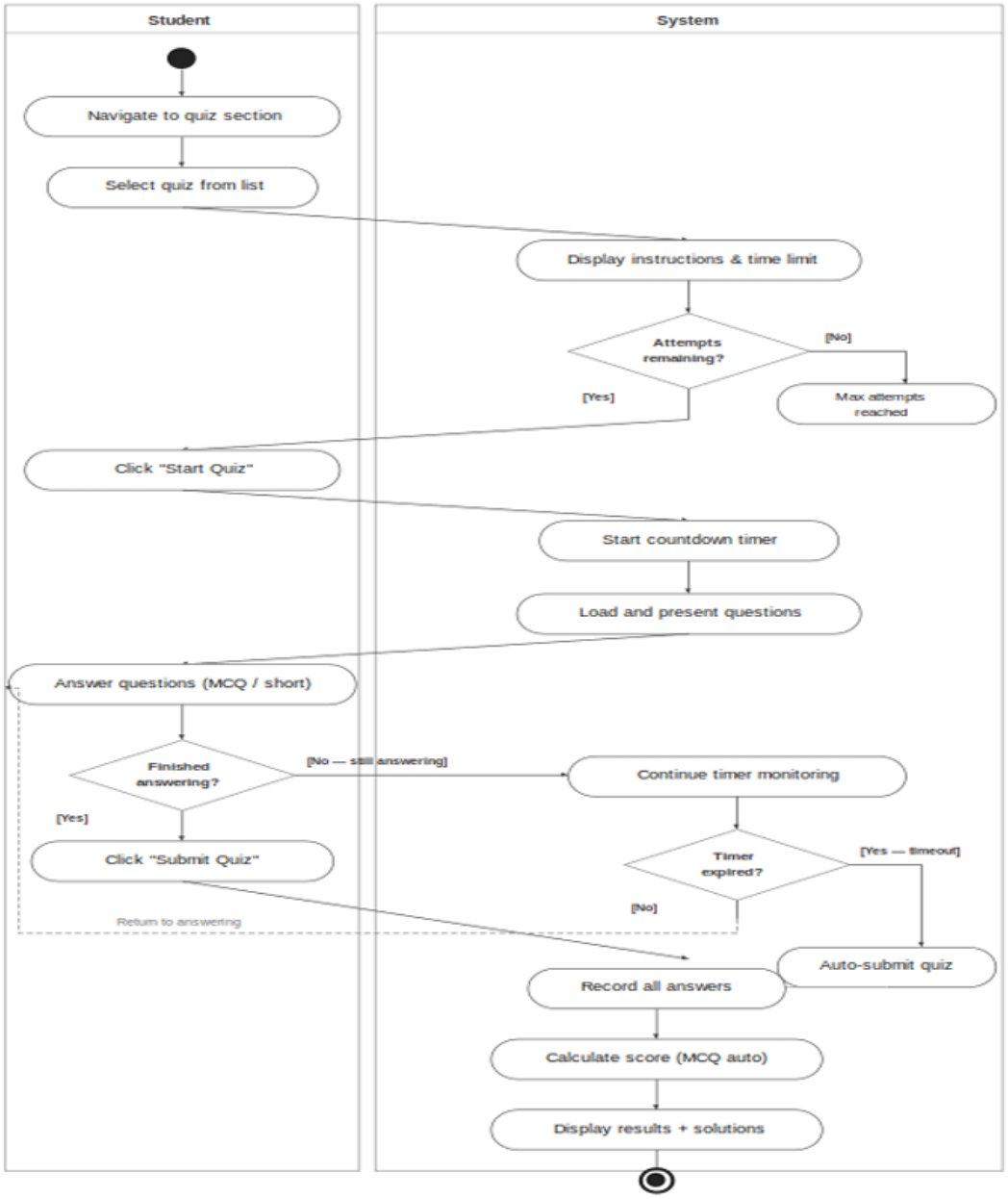


Figure 2.12: Activity Diagram - Create User Account

2.7.9 Support Team Handles Ticket

This activity diagram describes the support ticket lifecycle. The support agent reviews the issue and determines whether it can be resolved. If resolved, the ticket is updated and closed. If unresolved, the agent escalates the issue to the administrator or development team. The system updates the ticket status and sends notifications.

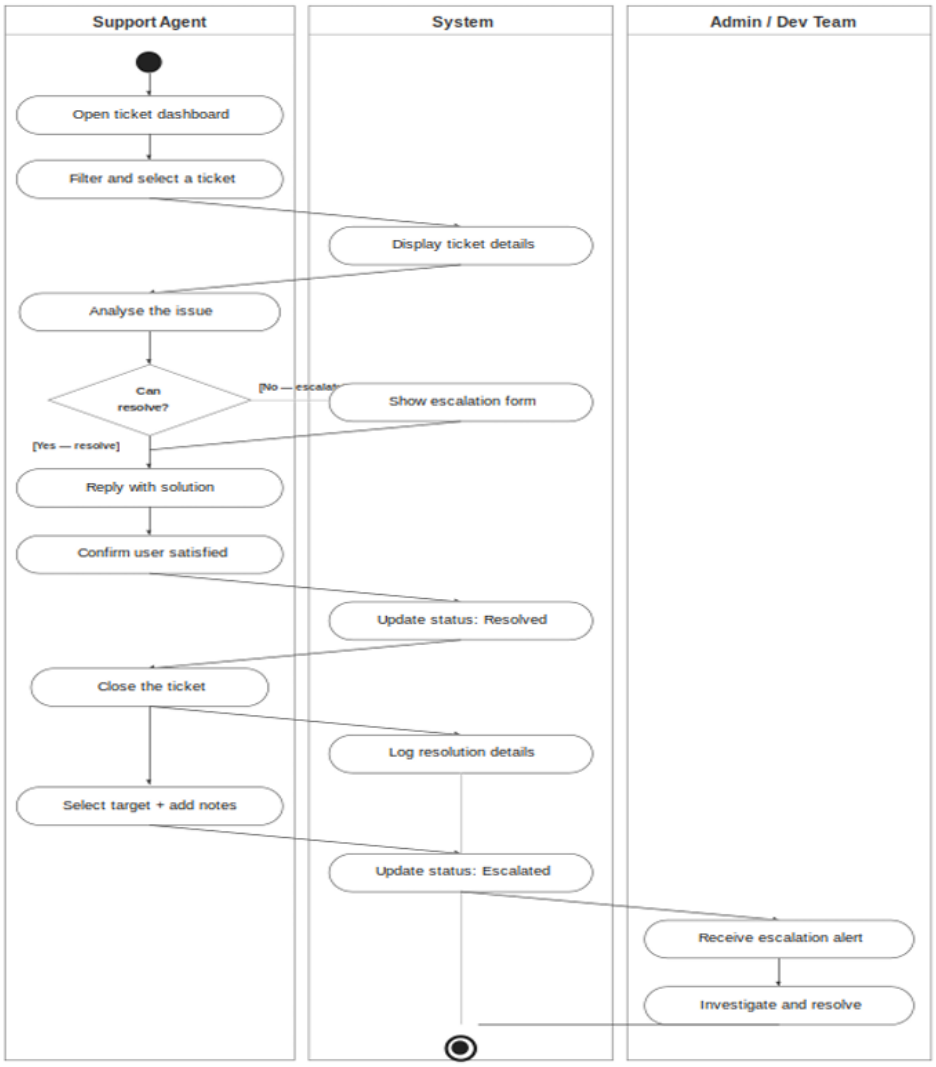


Figure 2.13: Activity Diagram - Ticket Handling

2.7.10 AI Help Chatbot

This activity diagram represents the AI chatbot interaction process. The student submits a query through the chatbot interface. The AI system analyses the query and categorises it as a study question, platform issue, or content request. Based on classification, the system generates an appropriate response. If the issue remains unresolved, the chatbot offers to create a support ticket.

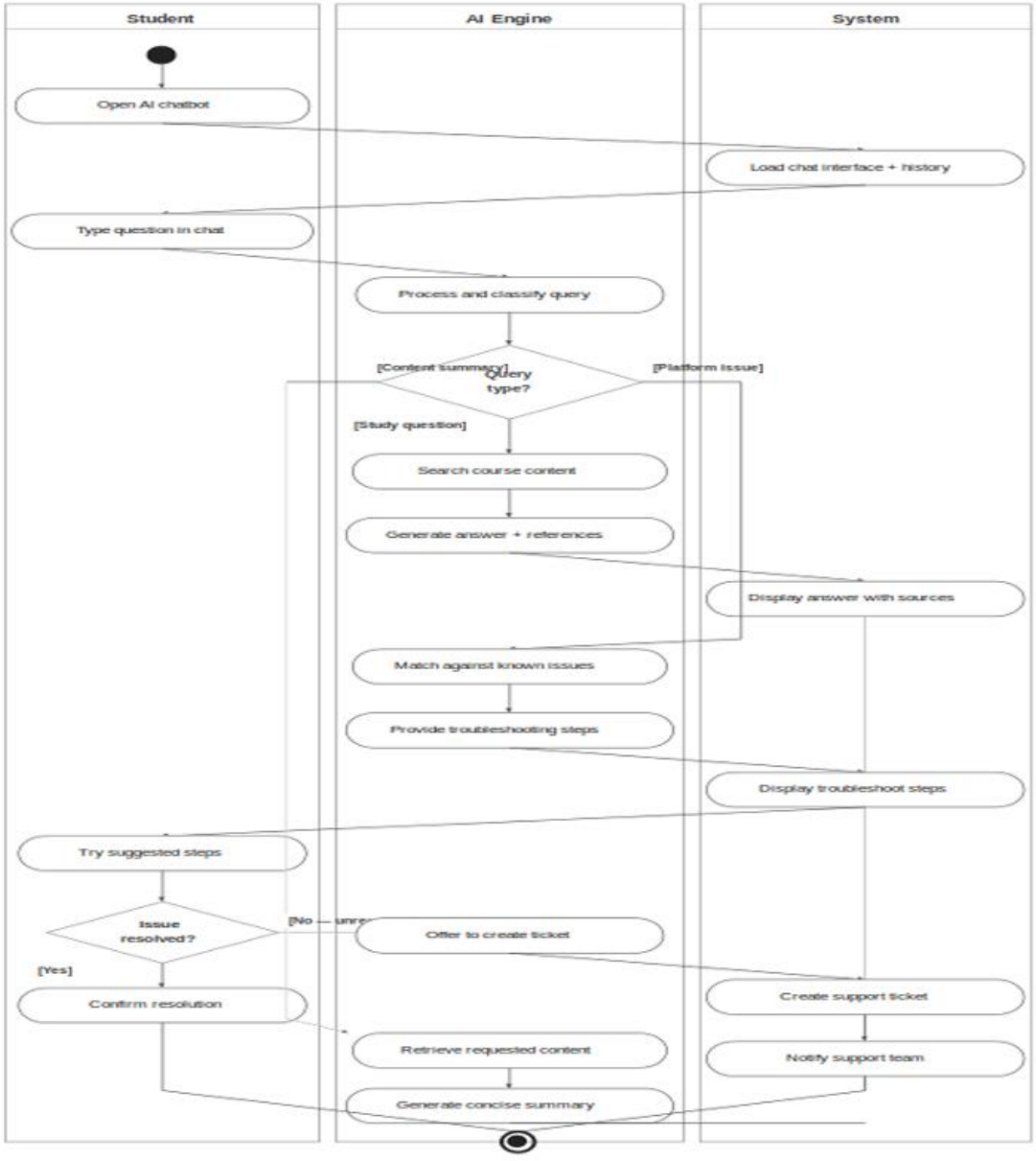


Figure 2.14: Activity Diagram: Chatbot

2.8 External Interface Requirements

This chapter describes the external interfaces of the Mentify Digital Learning Platform, including user interface characteristics, hardware interactions, software integrations, and communication protocols required for system operation.

2.8.1 User Interface Requirements

The system shall provide a modern, responsive, web-based user interface that is intuitive, accessible, and aligned with contemporary usability standards. The interface shall adopt a clean and minimalist design with consistent colour theming that reflects the branding guidelines of TIT Education. The system interface shall be fully responsive and capable of adapting seamlessly across desktop, tablet, and mobile devices. Accessibility requirements shall comply with WCAG 2.1 Level AA standards, including appropriate colour contrast ratios, keyboard navigation support, screen reader compatibility, and alternative text for all non-text content. The dashboard shall utilise visual elements such as progress indicators, charts, and card-based layouts to present information in a clear and structured manner. All input forms shall implement client-side validation with descriptive and user-friendly error messages. Navigation shall be role based, ensuring that users only view features and menu options relevant to their assigned roles.

2.8.2 Hardware Interface Requirements

The Mentify Digital Learning Platform is designed as a cloud-hosted web application and therefore does not require direct integration with specialised hardware components. However, the system shall support interaction through standard input devices, including keyboards, mice, and touch-enabled interfaces. The platform shall be compatible with display devices ranging from small mobile screens (approximately 320 pixels in width) to high-resolution displays up to 4K resolution. The system shall support file uploads from locally connected storage devices available on the user's machine.

2.8.3 Software Interface Requirements

The system shall integrate with external software services and application programming interfaces to support authentication, AI-powered functionality, storage, messaging, and data management. Third-party authentication shall be supported through integration with Google OAuth 2.0. AI driven functionalities, including content summarisation, automated quiz generation, grading assistance, course recommendations, and chatbot interaction, shall be implemented through

integration with AI service providers such as OpenAI or equivalent platforms. The system shall interface with email delivery services via SMTP-based providers such as SendGrid or AWS Simple Email Service to facilitate transactional communications, including account verification, password reset notifications, and system alerts. Cloud storage services such as Amazon S3 or Google Cloud Storage shall be utilised for storing multimedia learning materials, assignment submissions, and related content. The platform shall interface with a relational database management system, such as PostgreSQL, for structured data storage, and may utilise a NoSQL database such as MongoDB for storing chat logs and user activity data. Additionally, the system shall integrate with a push notification service to enable real-time in-app notifications and mobile push messaging.

2.8.4 Communication Interface Requirements

All communication between client and server components shall be conducted over secure HTTPS connections using TLS version 1.2 or higher. The system shall expose RESTful application programming interfaces that use JSON as the primary data exchange format. Real-time communication features, including messaging and notifications, shall be implemented using the WebSocket protocol to ensure low-latency data transmission. The system APIs shall conform to REST architectural principles and utilise appropriate HTTP status codes for all responses. File transfer operations shall support multipart/form-data encoding for efficient handling of uploaded content. The system shall also implement rate-limiting mechanisms across API endpoints to prevent misuse, ensure service stability, and enhance overall security.

2.9 Data Requirements

This chapter describes the types of data that shall be managed, processed, stored, and protected by the Mentify Digital Learning Platform. The system shall handle multiple categories of data essential for learning delivery, assessment, communication, analytics, and support operations.

2.9.1 User Data

The system shall store and manage user-related data, including personal identification information such as name, email address, grade level, and assigned role. Authentication-related data shall include securely hashed passwords and issued JWT tokens. Additional profile information may include profile photographs, contact details, and parental contact information where applicable.

2.9.2 Course Data

The system shall maintain course-related data, including course metadata such as course name, description, grade level, category, and associated tags. It shall store structured content organisation, including modules and lessons, along with associated learning materials such as documents, multimedia files, and videos. The system shall also maintain prerequisite configurations and scheduled release timelines for lessons and modules.

2.9.3 Assessment Data

The platform shall manage assessment-related data, including assignment specifications such as instructions, submission deadlines, and submission rules. It shall store quiz configurations, including question sets, answer options, scoring logic, and time constraints. Student submission data shall include uploaded files, textual responses, and submission timestamps. The system shall also record grades, performance scores, and feedback provided by teachers or generated through AI assistance.

2.9.4 Communication Data

The system shall store communication-related data, including group and private chat messages, discussion forum content, announcements, and system-generated notifications. This data shall support collaborative learning and information dissemination across the platform.

2.9.5 Analytics Data

The system shall collect and maintain analytics-related data, including student engagement metrics such as lesson views, time spent on learning activities, completion rates, and attendance records.

Performance analytics shall include score trends, learning progress indicators, and system usage logs used for monitoring platform effectiveness.

2.9.6 Support Data

Support-related data shall include support ticket information such as ticket status, priority level, issue category, and resolution notes. The system shall also maintain knowledge base content and escalation history records to support efficient issue management.

2.9.7 Data Security and Retention

All personal and sensitive data shall be encrypted at rest using AES-256 encryption standards. Data retention policies shall comply with the institutional policies of TIT Education as well as applicable data protection regulations. Automated database backups shall be performed at intervals of 24 hours and retained for a minimum period of 30 days to ensure data recoverability and system reliability.

2.10 Alternative Solutions Considered

During the system planning and requirement analysis phase, several alternative implementation approaches were evaluated to determine the most suitable solution for developing the Mentify Digital Learning Platform.

2.10.1 Adoption of an Existing Learning Management System

One potential approach considered was the adoption of an existing Learning Management System such as Moodle, Canvas, or Google Classroom. These platforms provide pre-built functionalities for course management, assessment delivery, and communication features. However, they lack the advanced AI-driven personalisation capabilities required by TIT Education. Furthermore, these platforms offer limited flexibility for customisation to align with the institution's specific workflows and pedagogical strategies. Proprietary platforms also involve recurring licensing costs. Although Moodle is open-source, it would require extensive custom development to support the desired AI-based functionality and tailored system features.

2.10.2 Low-Code / No-Code Development Platforms

Another alternative involved the use of low-code or no-code development platforms such as Bubble or AppSheet. This approach could potentially accelerate the development timeline and reduce initial implementation complexity. However, such platforms impose constraints on system performance, scalability, and the integration of advanced AI capabilities. Additionally, reliance on proprietary development environments may introduce long-term challenges in maintainability, system portability, and vendor dependency.

2.10.3 Custom System Development (Selected Approach)

The selected approach for the Mentify Digital Learning Platform is full custom system development. This approach enables complete control over system functionality, user experience design, AI integration, and architectural scalability. Although custom development requires greater initial development effort, it provides the flexibility needed to address TIT Education's requirement for a student-centred, AI-powered digital learning environment. Moreover, a custom-built solution allows future expansion and scalability beyond the institutional context.

2.11 Feasibility Study

This chapter evaluates the feasibility of developing and implementing the Mentify Digital Learning Platform across technical, economic, operational, and schedule dimensions.

2.11.1 Technical Feasibility

The proposed system is considered technically feasible. The selected technology stack, comprising React.js for the frontend, Spring Boot for the backend, PostgreSQL for database management, and cloud hosting through AWS or an equivalent provider, is mature, well documented, and widely supported within the software development ecosystem. AI-driven functionalities can be effectively implemented through integration with established AI service providers such as OpenAI. Core architectural components including JWT-based authentication, Role-Based Access Control (RBAC), and WebSocket-based real-time communication are supported by robust frameworks and libraries. Furthermore, the development team possesses the required competencies in web application development, database management, and API integration, ensuring that the technical requirements of the system can be successfully addressed.

2.11.2 Economic Feasibility

The project demonstrates strong economic feasibility as it is being developed within an academic context, thereby significantly reducing labour-related costs. Infrastructure expenses can be managed effectively through the utilisation of cloud provider free tiers and educational support programmes such as AWS Educate and Google Cloud for Education. Primary anticipated costs include cloud hosting services, AI API usage, and domain registration. These expenses are expected to remain within a manageable budget. From the perspective of TIT Education, the implementation of this system eliminates the need for costly commercial Learning Management System subscriptions and reduces administrative workload through automation, resulting in long-term cost savings.

2.11.3 Operational Feasibility

Operational feasibility is supported by the positive reception of the system concept among the staff of TIT Education, who have indicated readiness to adopt the proposed platform. The system is designed with a strong emphasis on usability, requiring only basic digital literacy from its users. The availability of training materials and an integrated knowledge base will facilitate user onboarding and system adoption. Additionally, the inclusion of an AI-powered chatbot capable of

providing continuous assistance reduces the operational workload on the support team. The system is intended to operate alongside existing manual processes during the transition phase, enabling gradual adoption without significant operational disruption.

2.11.4 Schedule Feasibility

The development of the Mentify Digital Learning Platform aligns with the academic schedule defined by the SENG 31242 module. The project is structured into phased development stages, prioritising essential system components such as authentication, course management, content delivery, and assessment functionalities for the initial release. Advanced features, including AI-driven personalisation, analytics dashboards, and comprehensive reporting capabilities, are planned for subsequent iterations. The development team, consisting of four members, has distributed responsibilities to support parallel development activities. The adoption of Agile development practices, including two-week sprint cycles, facilitates continuous progress monitoring and supports timely project completion.

2.12 Risk Analysis

This chapter identifies potential risks associated with the development, deployment, and operation of the Mentify Digital Learning Platform. Each risk is evaluated based on its likelihood and impact, and appropriate mitigation strategies are defined to minimise potential adverse effects.

Risk ID	Description	Likelihood	Impact	Risk Level	Mitigation Strategy
R-001	AI API service outages or rate limiting may disrupt AI features such as summarisation, quiz generation, and chatbot functionality.	Medium High		High	Implement graceful degradation so that the platform remains functional without AI features. Cache AI responses where feasible and evaluate multiple AI providers for fallback support.
R-002	Scope creep due to evolving client requirements may delay the project timeline.	High	High	High	Establish a formal change request process. Prioritise features using the MoSCoW methodology and maintain a fixed scope for each sprint.
R-003	Team member unavailability due to academic or personal commitments.		Medium Medium Medium		Cross-train team members across modules. Maintain comprehensive documentation and utilise version control with frequent commits.

R-004	Data breach or security vulnerability may expose sensitive student information.	Low	High	High	Conduct regular security audits, implement encryption for data at rest and in transit, follow
					OWASP security guidelines, and enforce multi-factor authentication for administrative accounts.
R-005	Poor user adoption by teachers and students unfamiliar with digital learning platforms.		Medium Medium Medium		Design an intuitive user interface, provide onboarding tutorials, maintain a comprehensive knowledge base, and conduct structured user training sessions.
R-006	Performance degradation under high concurrent user load during peak usage periods.	Medium	High	High	Implement load balancing and horizontal scaling. Conduct stress testing prior to deployment. Utilise content delivery networks for static resources and optimise database queries using indexing.

R-007	Failure of third-party services such as authentication providers, email services, or cloud storage.	Low		Medium Medium	Implement fallback authentication mechanisms, use redundant email service providers, and design retry mechanisms for failed operations.
R-008	Insufficient testing coverage leading to undetected defects in production.		Medium Medium Medium		Maintain code coverage above 80%, implement automated unit, integration, and end-to-end testing, and conduct user acceptance testing with TIT Education prior to deployment.

2.19: Risk Analysis

3 SYSTEM DESIGN SPECIFICATION DOCUMENT

3.1 Architectural Design

This chapter presents the architectural design of the system, describing the high-level structure, the cloud-native deployment model, the rationale behind key architectural decisions, and the mechanisms employed to ensure scalability and maintainability. The system is built upon a microservices architecture and is deployed on Amazon Web Services (AWS) using Kubernetes-based container orchestration.

3.1.1 High-Level Architecture

The system adopts a Microservices Architecture pattern, wherein the application is decomposed into a collection of small, independently deployable services, each responsible for a distinct business capability. This architectural style was selected over monolithic or layered alternatives due to its superior alignment with the system's scalability, resilience, and continuous delivery requirements.

3.1.1.1 Microservices Decomposition

The system is decomposed into the following six core microservices, each developed using the Spring Boot framework and deployed as independent containerised units within an AWS Elastic Kubernetes Service (EKS) cluster:

Microservice	Responsibility	Data Store
User Service	Authentication, registration, and profile management of all system users.	PostgreSQL (User DB)
Assessment Service	Creation, distribution, and grading of assessments and evaluations.	PostgreSQL (Assessment DB)
Course Service	Management of course content, enrolment, and learning materials.	PostgreSQL (Course DB) + S3
Communication Service	Real-time messaging and notification delivery between users.	MongoDB (Communication DB)

Analytics Service	Collection, aggregation, and reporting of usage and performance metrics.	MongoDB (Analytics DB)
AI Service	Integration with external AI providers for intelligent features.	MongoDB (AI DB)

3.1: Microservice Decomposition and Data Store Mapping

Each microservice is granted ownership of its own dedicated data store, adhering to the Database-per Service pattern. This ensures loose coupling between services and eliminates shared database dependencies, which are a common source of tight coupling in traditional monolithic systems.

3.1.1.2 Client Layer

The system exposes its functionality to two categories of clients: a Mobile Application and a Web Application. Both clients communicate exclusively through the API Gateway, which serves as the single entry point into the backend infrastructure. This design enforces a consistent security perimeter and simplifies client-side integration.

3.1.1.3 Request Routing and Infrastructure

Incoming client requests are received by an AWS Application Load Balancer (ALB), which distributes traffic to the API Gateway. The API Gateway performs request routing, authentication token validation, and rate limiting before forwarding requests to the appropriate microservice within the EKS cluster. A Redis cache is integrated with the API Gateway layer to serve frequently accessed data without incurring the overhead of repeated database queries.

3.1.1.4 Asynchronous Event-Driven Communication

Inter-service communication that does not require an immediate response is facilitated through an Apache Kafka Event Bus. Microservices publish events to Kafka topics upon completing significant operations (e.g., user enrolment, assessment submission), and subscribing services consume these events asynchronously. This decouples services temporally and prevents cascading failures.

3.1.1.5 Architecture Diagram

The overall system architecture is illustrated in Figure 2 below. The diagram depicts all major system components, their interconnections, and the external services integrated with the platform.

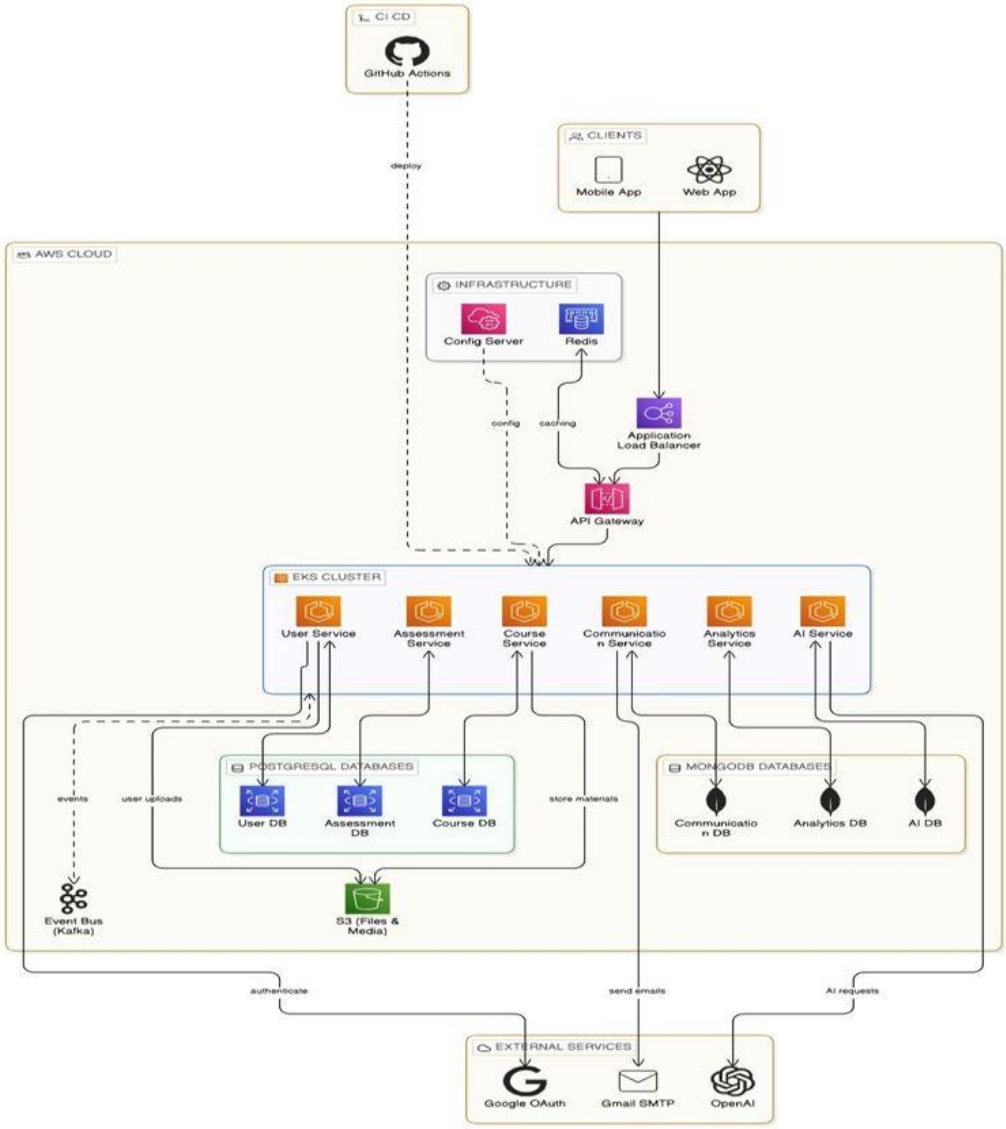


Figure 3.1 - High-Level System Architecture Diagram

3.1.2 Cloud-Native Architecture

The system is designed and deployed as a cloud-native application on Amazon Web Services (AWS). Cloud-native architecture principles guide every aspect of the system's infrastructure, including containerisation, orchestration, managed services, and infrastructure-as-code automation.

3.1.2.1 Containerisation and Orchestration

Each microservice is packaged as a Docker container image. Container images are stored in Amazon Elastic Container Registry (ECR) and deployed to an AWS Elastic Kubernetes Service (EKS) cluster. Kubernetes provides automated deployment, self-healing, horizontal scaling, and rolling update capabilities for all microservices.

3.1.2.2 Managed Storage Services

File and media assets — including course materials, user uploads, and other binary content — are stored in Amazon S3. S3 provides highly durable, scalable object storage with lifecycle management capabilities. Relational data is persisted in Amazon RDS-managed PostgreSQL instances, while document-oriented data is maintained in MongoDB Atlas, providing managed NoSQL storage for semi-structured datasets

3.1.2.3 CI/CD Pipeline

A fully automated CI/CD pipeline is implemented using GitHub Actions. Upon each code commit to the main branch, the pipeline executes automated test suites, builds Docker container images, pushes the images to ECR, and triggers a rolling deployment to the EKS cluster. This pipeline eliminates manual deployment steps and reduces the risk of human error in production releases.

3.1.2.4 External Service Integrations

The platform integrates with three external services to augment its capabilities:

- Google OAuth is utilised for user authentication, delegating identity verification to a trusted third-party provider and eliminating the need for the system to manage user credentials directly.
- Gmail SMTP is employed for transactional email delivery, including registration confirmations, assessment notifications, and system alerts. Component
- OpenAI API is consumed by the AI Service to provide intelligent features such as automated feedback generation and content recommendations.

Component	AWS Technology	Purpose
Container Orchestration	AWS / Technology Purpose AWS EKS (Kubernetes)	Deployment, scaling, and self healing of microservices
API Entry Point	AWS ALB + API Gateway	Traffic distribution and request routing
Caching Layer	Redis	Low-latency data retrieval and session caching
Config Management	Spring Cloud Config Server	Externalised configuration management
Relational Databases	PostgreSQL (AWS RDS)	Structured data for User, Assessment, and Course services
Document Databases	MongoDB Atlas	Semi-structured data for Communication, Analytics, and AI services
Object Storage	Amazon S3	File and media asset storage
Message Broker	Apache Kafka	Asynchronous event-driven inter service communication
CI/CD Automation	GitHub Actions	Automated build, test, and deployment pipeline

3.2: Cloud-Native Infrastructure Components

3.1.3 Justification for Architecture Choice

The selection of a microservices architecture was driven by the functional and non-functional requirements of the system. The following rationale is presented in comparison with the principal architectural alternatives considered during the design phase.

3.1.3.1 Comparison with Alternative Architectures

Criterion	Monolithic	Layered	Microservices
Independent Deployability	Not supported	Partially supported	Fully supported
Technology Heterogeneity	Not supported	Limited	Fully supported
Fault Isolation	Poor	Moderate	Strong
Horizontal Scalability	Coarse-grained	Coarse-grained	Fine-grained per service
Team Autonomy	Low	Moderate	High

3.3: Architectural Style Comparison

3.1.3.2 Key Justifications

The following factors informed the final architectural decision:

- Independent Scalability: Individual services — particularly the Assessment Service and AI Service — are anticipated to experience high and bursty load. Microservices enable each of these components to be scaled independently without affecting unrelated services.
- Independent Deployability: Each service can be updated, tested, and deployed in isolation, enabling faster release cycles and reducing the blast radius of defects.
- Technology Optimisation: The polyglot persistence model allows each service to use the data storage technology best suited to its data access patterns. Transactional relational data is stored in PostgreSQL, while document-oriented data is stored in MongoDB.
- Fault Tolerance: Failure in one microservice does not propagate to others, provided circuit breaking patterns are applied. This is a significant improvement over monolithic systems, where a single component failure can bring down the entire application.

- Spring Boot Ecosystem: The Spring Boot framework provides first-class support for microservices patterns including Spring Cloud Gateway, Spring Cloud Config, and Spring Boot Actuator, which align directly with the architectural requirements.

3.1.4 Scalability and Maintainability Considerations

Scalability and maintainability are first-class design concerns embedded throughout the architecture. The following subsections document the specific mechanisms and design decisions that address these quality attributes.

3.1.4.1 Scalability

- Horizontal Pod Autoscaling (HPA): Kubernetes HPA is configured for each microservice deployment. CPU and memory utilisation metrics are monitored, and additional pod replicas are automatically provisioned when thresholds are exceeded.
- Application Load Balancer: The AWS ALB distributes incoming traffic across all healthy pod instances, preventing any single replica from becoming a bottleneck.
- Redis Caching: Frequently accessed data — such as user session tokens and course metadata — is served from the Redis cache, reducing load on the underlying databases and improving response latency.
- Kafka Message Broker: By decoupling producers from consumers through Kafka, the system is able to absorb traffic spikes without requiring synchronous inter-service communication. Consumers process messages at their own rate, preventing overload.
- S3 for Media Storage: Amazon S3 is used for binary asset storage, offloading large file I/O from the application tier and providing effectively unlimited storage capacity with no operational overhead.

3.1.4.2 Maintainability

- Service Isolation: Each microservice has a well-defined API boundary and owns its data store. Changes to one service's internal implementation do not require modifications to other services, provided the API contract is preserved.
- Externalised Configuration: The Spring Cloud Config Server stores all environment-specific configuration externally. Configuration changes can be applied without redeploying application code.

- Automated CI/CD: The GitHub Actions pipeline enforces automated testing at every code change, reducing the likelihood of regressions reaching production. Automated deployments ensure consistency across environments.
- Containerisation: Docker containers ensure that all services run in identical, reproducible environments across development, staging, and production, eliminating environment-specific defects.
- Observability: Spring Boot Actuator endpoints expose health checks and metrics for each microservice. These endpoints are consumed by Kubernetes liveness and readiness probes, enabling automated recovery from unhealthy states.

The combination of these mechanisms ensures that the system is capable of scaling to meet growing user demand while remaining straightforward to operate, modify, and extend over its operational lifetime.

3.2 Database Design

3.2.1 ER Diagram

The following subsections present the ER diagram for each microservice. Entities depicted with a dashed border in the diagrams represent entities owned by a different service and are shown for contextual clarity only. Relationships are annotated with their cardinality (1, M, or N).

3.2.1.1 User Service

The User Service manages all user-related data across the four system roles: Student, Teacher, Administrator, and SupportAgent. A generalisation hierarchy (IS-A relationship) is employed, whereby all four specialised entity types inherit the common attributes of the base User entity. The User entity serves as the base type and encapsulates authentication and identification data. The following attributes are defined for User: id (primary key), firstName, lastName, email, passwordHash, role, contactNumber, gradeLevel, isActive, and createdAt. The Student sub-type extends User with enrollmentDate and gradeLevel attributes. The Teacher sub type extends User with specialization and hireDate. The Administrator sub-type extends User with accessLevel. The SupportAgent sub-type extends User with adminReadAccess.

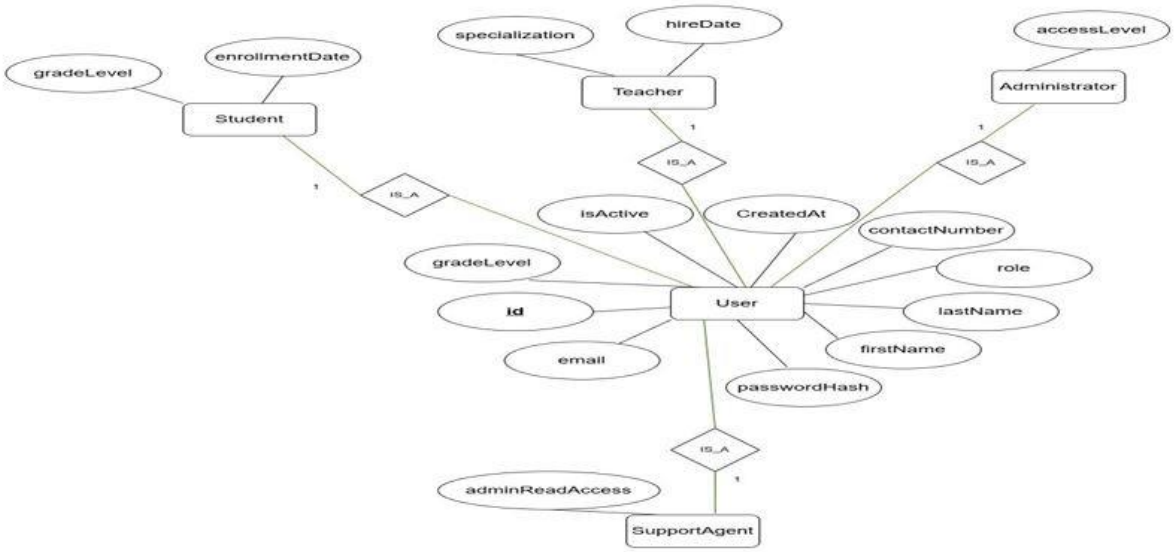


Figure 3.2: ER - User Service

3.2.1.2 Course Management Service

The Course Management Service is responsible for the complete lifecycle of educational content, from course categorisation through to the delivery of individual lessons and learning materials. The central entity of this service is the Course. A CourseCategory groups one or more courses through a Belongs To relationship (N:1). Each Course Contains one or more Modules (1:N). Each Module in turn Contains one or more Lessons (1:N) and Has Materials, which are represented by the LearningMaterial entity. Course delivery is mediated by a Teacher, reflected through the Taught By relationship between Module and Teacher. Student access to a course is tracked through the Enrollment entity, which represents the makes Enroll relationship between Student and Course. Additionally, the Personal Note entity captures student-authored annotations, associated with a course through a Contains relationship. The key entities and their attributes are as follows. Course: id, title, description, gradeLevel, isPublished, createdAt, updatedAt. Module: id, title, sequenceOrder. Lesson: id, title, content, releaseDate, sequenceOrder, isVisible. LearningMaterial: id, title, type, fileUrl. Enrollment: id, status, enrolled_at. PersonalNote: id, content, createdAt, updatedAt.

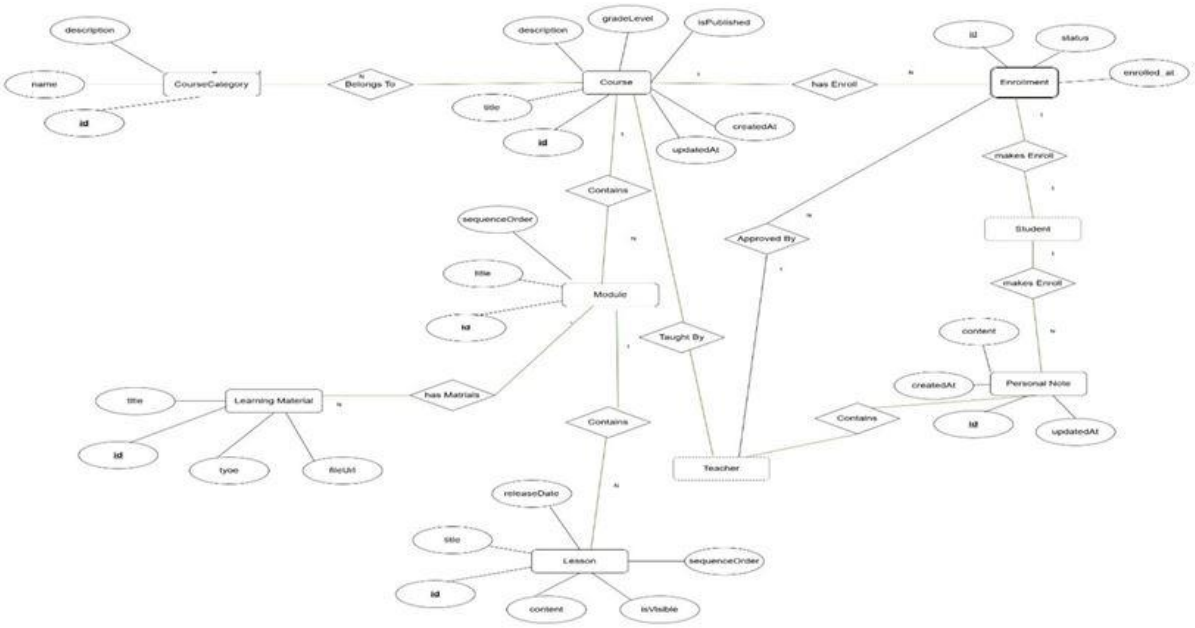


Figure 3.3: ER - Course Management Service

3.2.1.3 Assignment Service

The Assignment Service governs the creation, submission, grading, and feedback workflow for student assignments, as well as the administration of quizzes and the recording of student responses. The Assignment entity represents a task assigned to a course module, with attributes including id, title, description, dueDate, maxScore, submissionFormat, status, and lateSubmissionAllowed. Students submit work via the AssignmentSubmission entity, which records id, submittedAt, status, fileUrl, isLate, and submitted. Grades are recorded through the Grade entity (id, score, gradedAt) associated with a Teacher via the grades relationship. Teacher-authored written Feedback (id, comment, createdAt) is linked to the submission workflow. The Quiz entity (id, title, timeLimit, isPublished) Contains one or more Question entities (id, questionText, questionType, points), each of which Has Options represented by AnswerOption (id, optionText, isCorrect). A student's quiz attempt is captured by QuizAttempt (id, startedAt, completedAt, score), which records individual responses through studentAnswer (id, isCorrect).

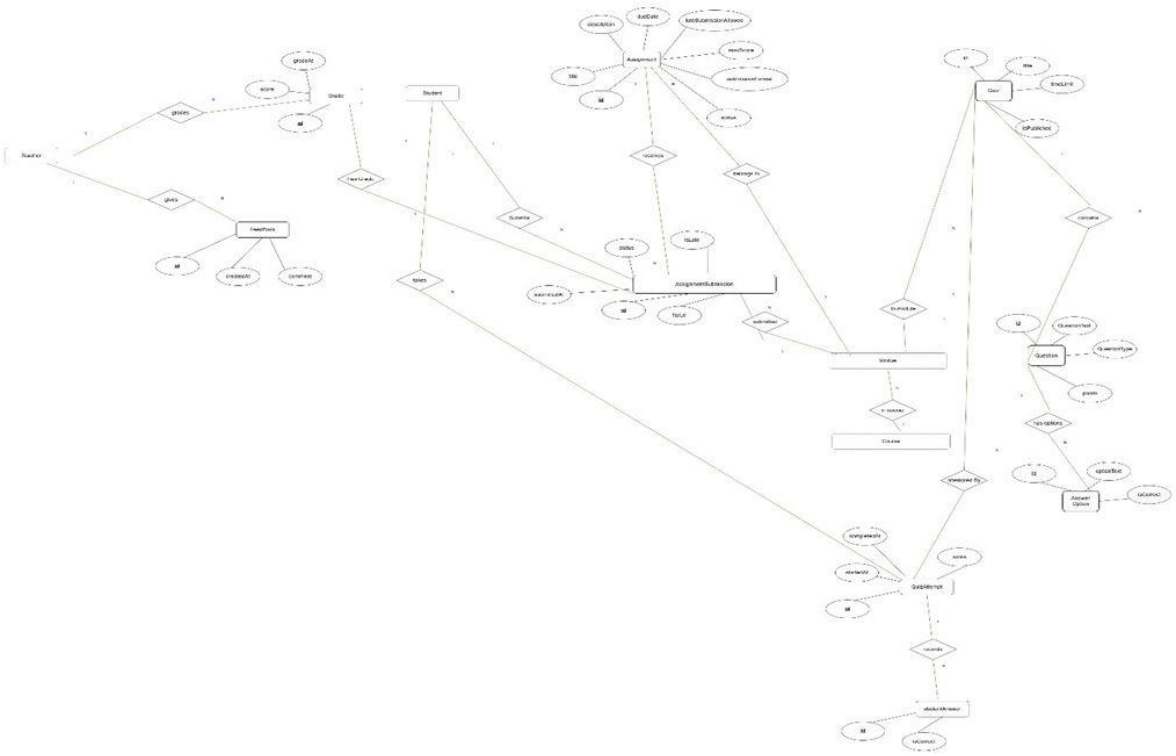


Figure 3.4: ER - Assignment Service

3.2.1.4 Analytics Service

The Analytics Service collects, aggregates, and reports on learner engagement and academic performance data. Four principal entities are defined within this service boundary. AttendanceRecord (id, attendanceAt, isPresent) is associated with a Student through a has relationship (1:N). ProgressReport (id, completionPercent, generatedAt) is generated by a Student and reported by a Course. PerformanceAnalytics (id, avgScore, passRate, generatedAt) is calculated for a Course and tracks individual Lessons. EngagementMetric (id, timeSpent, recordedAt) is tracked by a Lesson.

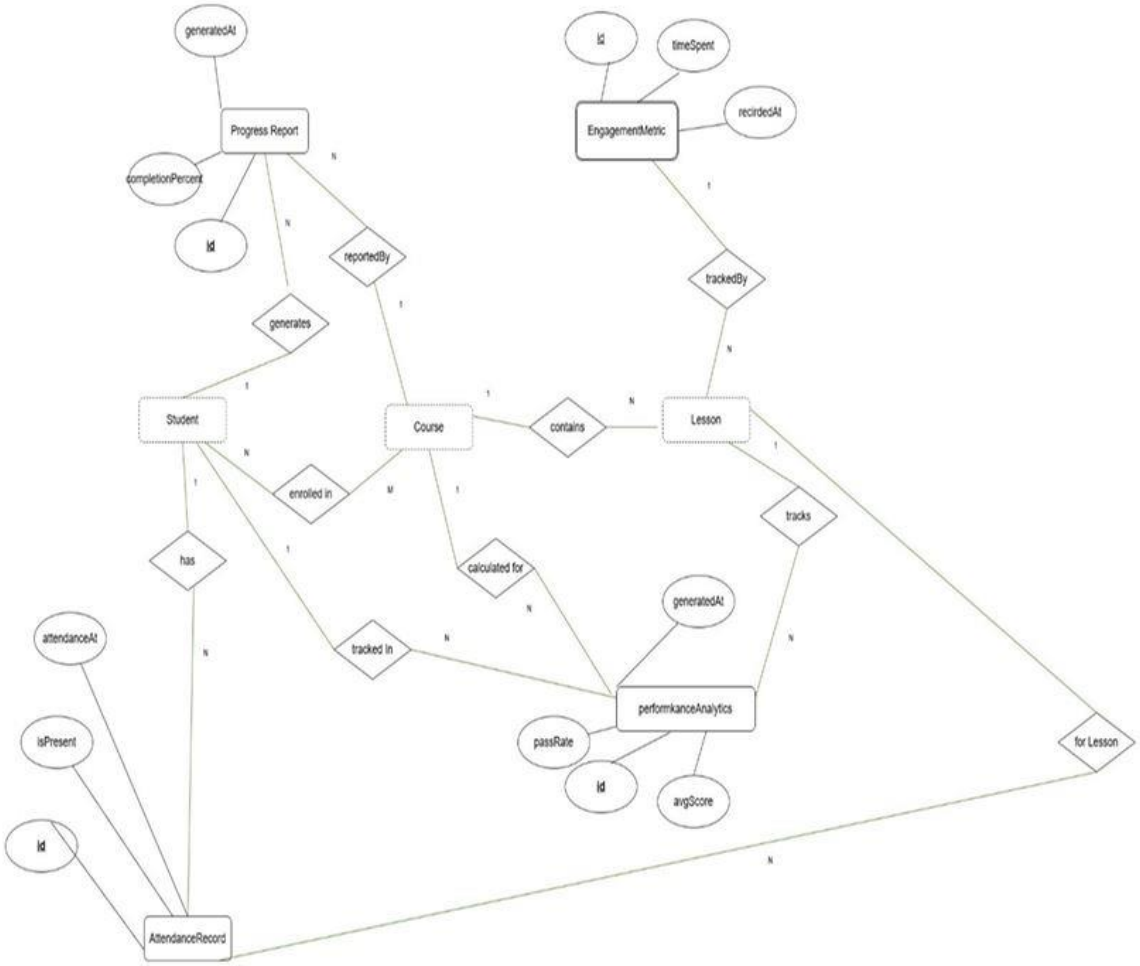


Figure 3.5: ER - Analytics Service

3.2.1.5 Communication Service

The Communication Service provides all messaging, announcement, notification, and discussion functionality within the LMS. Six primary entities are defined. Announcement (id, title, content, publishedAt, updatedAt) is authored by a user via the AuthoredBy relationship and published in a Course. DiscussionForum (id, title, createdAt) belongs to a Course (N:1). GroupChat (id, name, content, createdAt) is created and used by students and teachers. GroupChatMember (id, joinedAt) represents members of a group chat. PrivateMessage (id, content, sentAt, isRead) is exchanged between individual users. Notification (id, type, message, createdAt, isRead) is received by Students, Administrators, and SupportAgents.

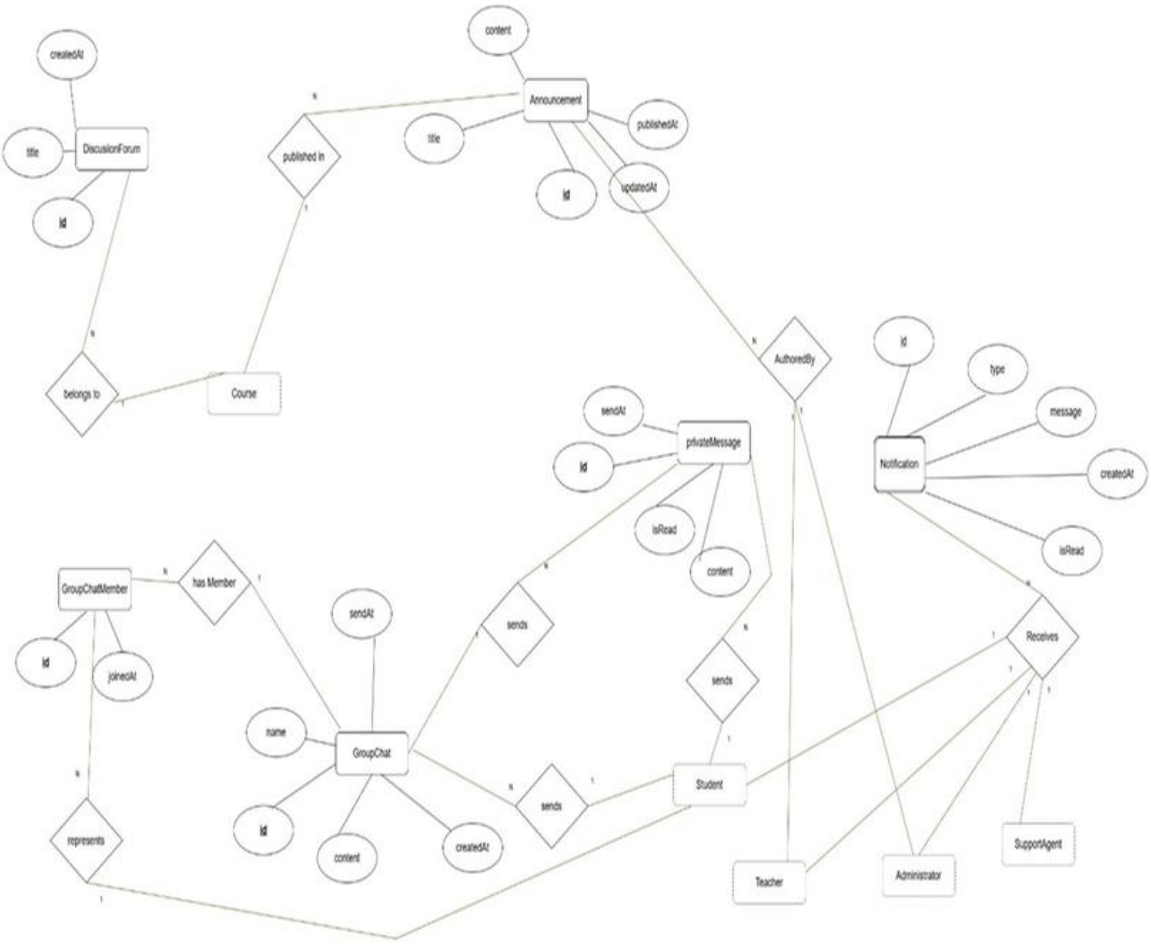


Figure 3.6: ER - Communication Service

3.2.1.6 AI Integration Entities

A dedicated set of entities supports AI-driven functionality within the LMS. These entities span multiple services but are grouped here as a logical unit for clarity. AIRecommendation (id, content, createdAt) is received by a Student (1:M). AIInsightReport (id, reportType, insights, generatedAt) is generated by a Teacher (1:M) and analyzed across student data. AIGeneratedContent (id, contentType, answer, pendingApproval, approvalAt) is approved by an AIInsightReport and recommends a Course. This entity generates either a Quiz or a LearningMaterial. AIChatSession (id, status, startedAt, endedAt) is initiated by a Student (1:M) and contains multiple AIChatMessage entities (id, role, content, sentAt).

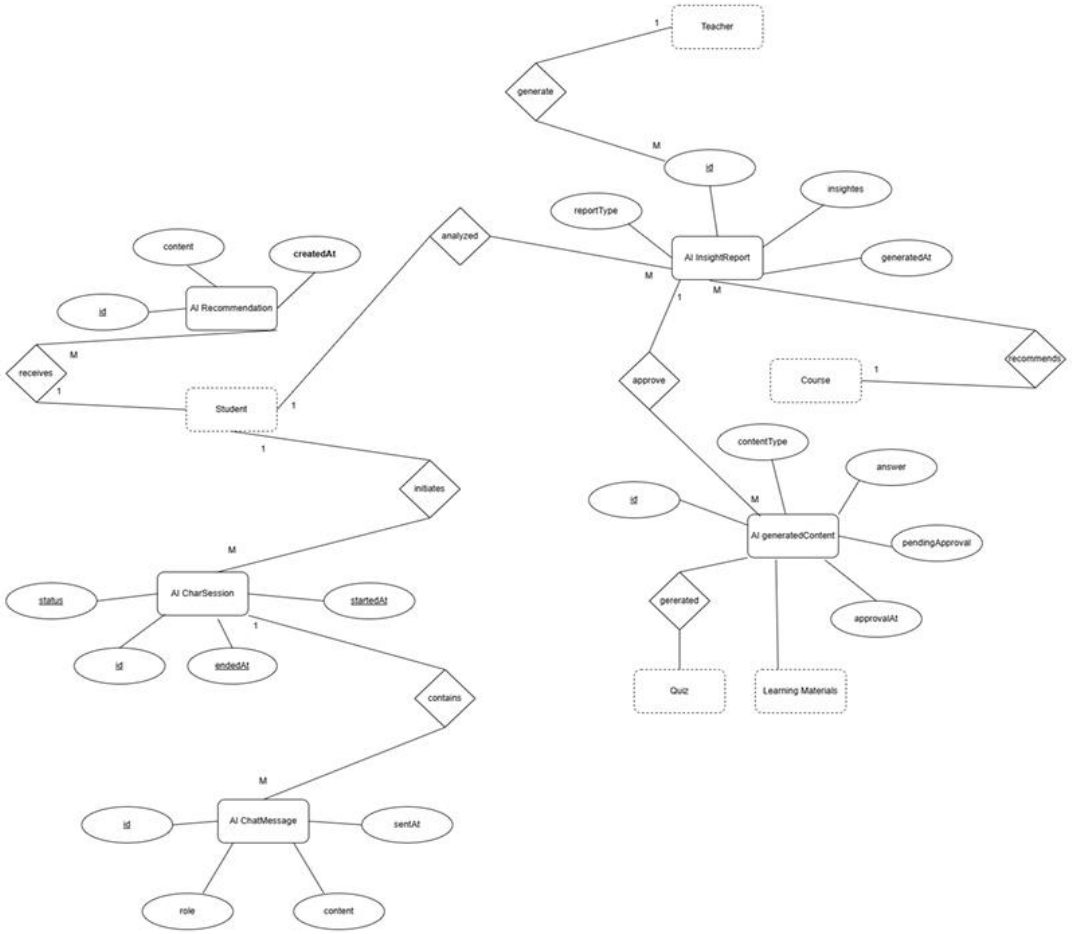


Figure 3.7: ER - AI Entities

3.2.2 Normalisation Analysis

All relational schemas derived from the ER diagrams presented in Chapter 3 are verified to conform to Third Normal Form (3NF). The following subsections summarise the normalisation analysis for each service, demonstrating compliance with the definitions of First Normal Form (1NF), Second Normal Form (2NF), and 3NF.

3.2.2.1 Definitions

First Normal Form (1NF) requires that all attribute values are atomic (i.e., indivisible) and that each column contains values of a single type. Second Normal Form (2NF) requires that 1NF is satisfied and that every non-key attribute is fully functionally dependent on the entire primary key (applicable only to relations with composite keys). Third Normal Form (3NF) requires that 2NF is satisfied and that no non-key attribute is transitively dependent on the primary key through another non-key attribute.

3.2.2.2 User Table

The User table holds a single-attribute primary key (id). All non-key attributes — firstName, lastName, email, passwordHash, role, contactNumber, gradeLevel, isActive, createdAt — are directly and solely dependent on id. No multi-valued or composite attributes exist. No transitive dependencies are present. The table is therefore in 3NF.

Attribute	Dependency	Normal Form
id (PK)		1NF, 2NF, 3NF
firstName, lastName	id → firstName, lastName	3NF
email	id → email (unique)	3NF
passwordHash	id → passwordHash	3NF
role	id → role	3NF
contactNumber	id → contactNumber	3NF
isActive, createdAt	id → isActive, createdAt	3NF

3.4: User Table Functional Dependency Analysis

3.2.2.3 Specialisation Sub-type Tables

Each sub-type (Student, Teacher, Administrator, SupportAgent) is stored in a separate table whose primary key is a foreign key referencing User(id). The IS-A relationship is implemented using the table-per-subtype pattern. Each sub-type table contains only those attributes specific to that role. Consequently, no partial or transitive dependencies exist in any sub-type table, and all are in 3NF.

Sub-type Table	Additional Attributes	3NF Status
Student	enrollmentDate, gradeLevel	Compliant
Teacher	specialization, hireDate	Compliant
Administrator	accessLevel	Compliant
Support Agent	adminReadAccess	Compliant

3.5: User Sub-type Tables - Normalisation Summary

3.2.2.4 Course Table

The Course table uses id as its primary key. All attributes – title, description, gradeLevel, isPublished, createdAt, updatedAt – are directly dependent on id. The category association is resolved through a foreign key (categoryId), avoiding the repetition of category name and description within the Course table. Accordingly, the Course table is in 3NF.

3.2.2.5 Module Table

The Module table uses id as its primary key and holds courseId as a foreign key. The attributes title and sequenceOrder are directly dependent on Module.id, and there is no transitive dependency via courseId. The table is in 3NF.

3.2.2.6 Lesson Table

The Lesson table uses id as its primary key and references Module through moduleId. Attributes title, content, releaseDate, sequenceOrder, and isVisible are fully functionally dependent on Lesson.id. No derived or transitively dependent attributes are present. The table is in 3NF.

3.2.2.7 Enrollment Table

The Enrollment table (id, studentId, courseId, status, enrolled_at) captures the many-to-many relationship between Student and Course. The primary key is id (a surrogate key). The combination (studentId, courseId) is constrained to be unique. The attributes status and enrolled_at depend only on this enrollment record and not on any non-key attribute. The table is in 3NF.

Table	Key	3NF Compliance Note
CourseCategory	id	name, description → id only
Course	id	Foreign key: categoryId; no transitives
Module	id	Foreign key: courseId; no transitives
Lesson	id	Foreign key: moduleId; no transitives
Learning Material	id	Foreign key: moduleId; no transitives
Enrollment	id	Unique: (studentId, courseId); status, enrolled_at → id
Personal Note	id	Foreign key: studentId, courseId; no transitives

3.6: Course Management Service — Normalisation Summary

3.2.2.8 Assignment Table

The Assignment table uses id as its primary key. All descriptive attributes — title, description, dueDate, maxScore, submissionFormat, status, lateSubmissionAllowed — are directly dependent on id. The relationship to a module is captured by a foreign key (moduleId), and the relationship to a course is captured by courseId. No transitive dependencies exist. The table is in 3NF.

3.2.2.9 Assignment Submission Table

AssignmentSubmission (id, assignmentId, studentId, submittedAt, status, fileUrl, isLate, submitted) references Assignment and Student via foreign keys. All non-key attributes describe properties of the submission itself, and none is transitively dependent on another non-key attribute. The table is in 3NF.

3.2.2.10 Quiz, Question, and AnswerOption Tables

Quiz (id, title, timeLimit, isPublished) is in 3NF as all attributes depend solely on id. Question (id, quizId, questionText, questionType, points) references Quiz through quizId; all attributes depend on Question.id. AnswerOption (id, questionId, optionText, isCorrect) references Question; attributes depend on AnswerOption.id. QuizAttempt (id, quizId, studentId, startedAt, completedAt, score) and StudentAnswer (id, attemptId, questionId, isCorrect) are similarly in 3NF.

Table	Primary Key	3NF Compliance
Assignment	id	Compliant — no transitives
AssignmentSubmission	id	Compliant — no transitives
Grade	id	Compliant — score, gradedAt → id
Feedback	id	Compliant — comment, createdAt → id
Quiz	id	Compliant — no transitives
Question	id	Compliant — FK: quizId
AnswerOption	id	Compliant — FK: questionId
QuizAttempt	id	Compliant — FK: quizId, studentId
StudentAnswer	id	Compliant — FK: attemptId, questionId

3.7: Assignment Service — Normalisation Summary

3.2.2.11 AttendanceRecord Table

AttendanceRecord (id, studentId, attendanceAt, isPresent) has all non-key attributes directly dependent on id. The foreign key studentId is used to resolve the has relationship. The table is in 3NF.

3.2.2.12 ProgressReport Table

ProgressReport (id, studentId, courseId, completionPercent, generatedAt) uses a surrogate key. The attributes completionPercent and generatedAt are facts about the report and are not transitively dependent through studentId or courseId. The table is in 3NF.

3.2.2.13 PerformanceAnalytics Table

PerformanceAnalytics (id, courseId, lessonId, avgScore, passRate, generatedAt) stores aggregated metrics per course and lesson. All metrics are dependent on the record's primary key id, and no non key attribute is derived from another non-key attribute within the same table. The table is in 3NF.

3.2.2.14 EngagementMetric Table

EngagementMetric (id, lessonId, timeSpent, recordedAt) records a single engagement observation. Both timeSpent and recordedAt depend directly on id. The table is in 3NF.

Table	Foreign Keys	3NF Compliance
AttendanceRecord	studentId	Compliant
ProgressReport	studentId, courseId	Compliant
PerformanceAnalytics	courseId, lessonId	Compliant
EngagementMetric	lessonId	Compliant

3.8: Analytics Service — Normalisation Summary

3.2.2.15 Announcement Table

Announcement (id, title, content, publishedAt, updatedAt) uses id as its primary key. The authoring user and the target course are referenced through foreign keys (authorId, courseId). All descriptive attributes depend solely on id. The table is in 3NF.

3.2.2.16 DiscussionForum Table

DiscussionForum (id, title, createdAt, courseId) associates a forum with a course via courseId. No attribute depends on courseId independently of id. The table is in 3NF.

3.2.2.17 GroupChat and GroupChatMember Tables

GroupChat (id, name, content, createdAt) stores group-level properties. GroupChatMember (id, groupChatId, userId, joinedAt) represents the membership many-to-many relationship using a surrogate primary key; joinedAt is a fact about the membership record itself. Both tables are in 3NF.

3.2.2.18 PrivateMessage and Notification Tables

PrivateMessage (id, senderId, receiverId, content, sentAt, isRead) uses id as its primary key. All attributes depend directly on id. No transitive dependencies exist. Notification (id, userId, type, message, createdAt, isRead) stores a single notification per record; all attributes depend solely on id. Both tables are in 3NF.

Table	Foreign Keys	3NF Compliance
Announcement	authorId, courseId	Compliant
DiscussionForum	courseId	Compliant
GroupChat	creatorId	Compliant
GroupChatMember	groupChatId, userId	Compliant

3.9: Communication Service — Normalisation Summary

3.2.2.19 AIGeneratedContent Table

AIGeneratedContent (id, insightReportId, courseId, contentType, answer, pendingApproval, approvalAt) represents AI-produced content pending teacher review. All attributes depend on id. The foreign keys insightReportId and courseId do not create transitive dependencies because content specific details (contentType, answer, approvalAt) pertain to the content record itself. The table is in 3NF.

3.2.2.20 AIChatSession and AIChatMessage Tables

AIChatSession (id, studentId, status, startedAt, endedAt) records a session's lifecycle. All attributes are direct facts about the session and depend solely on id. AIChatMessage (id, sessionId, role, content, sentAt) records individual messages within a session; the sessionId foreign key does not introduce transitive dependencies. Both tables are in 3NF.

3.3 Class Diagram

3.3.1 User Management Service

The User Management Service implements the user identity and authentication domain of the LMS. An abstract base class User defines the shared identity contract, which is specialised into four concrete sub-types — Student, Teacher, Administrator, and SupportAgent — through generalisation relationships. This hierarchy follows the table-per-subtype pattern at the persistence layer.

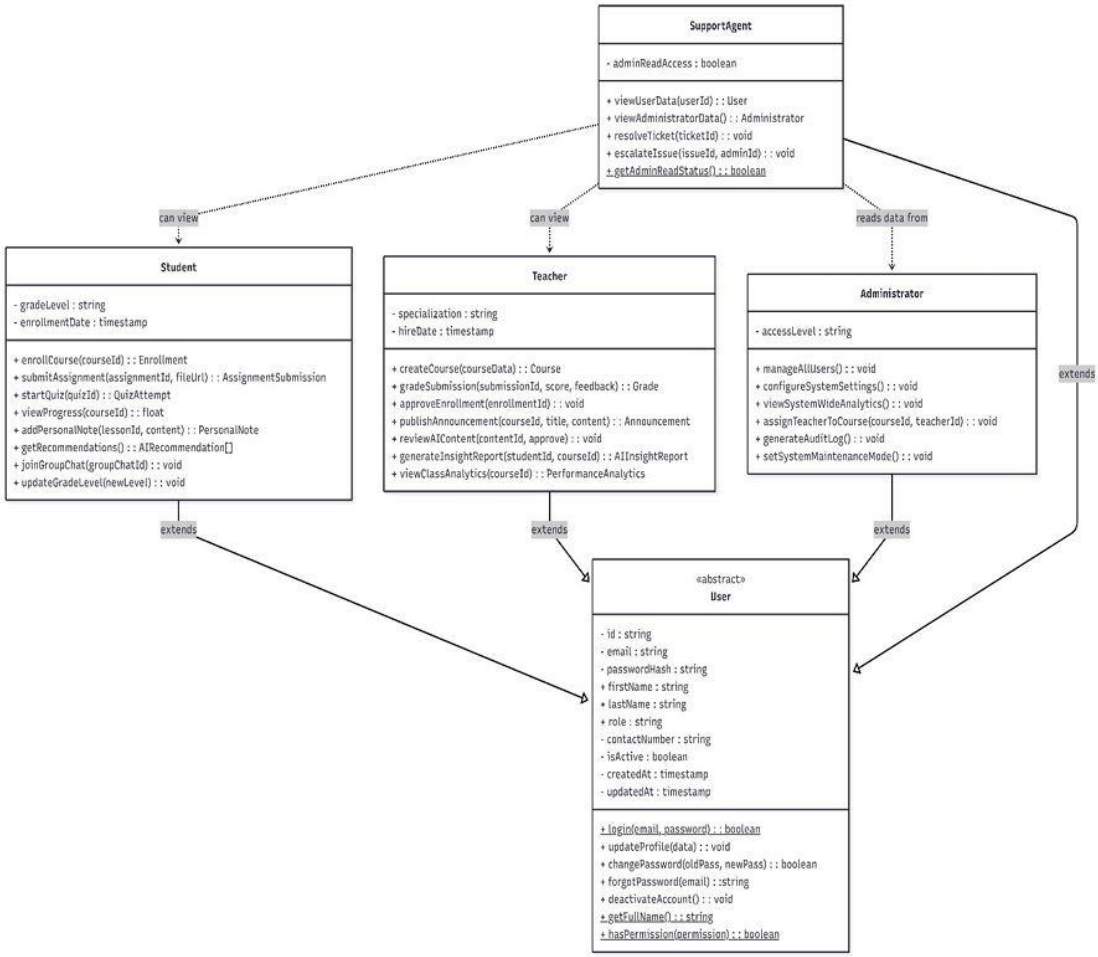


Figure 3.8: Class Diagram - User Management Service

3.3.2 Course Management Service

The Course Management Service governs the creation, organisation, and delivery of educational content. The Course class is the central entity, which contains Modules through a composition relationship. Modules in turn contain Lessons. Learning materials are attached to Lessons. Student access is managed through the Enrollment class, and personal note-taking is supported through PersonalNote. Course categorisation is provided by CourseCategory.

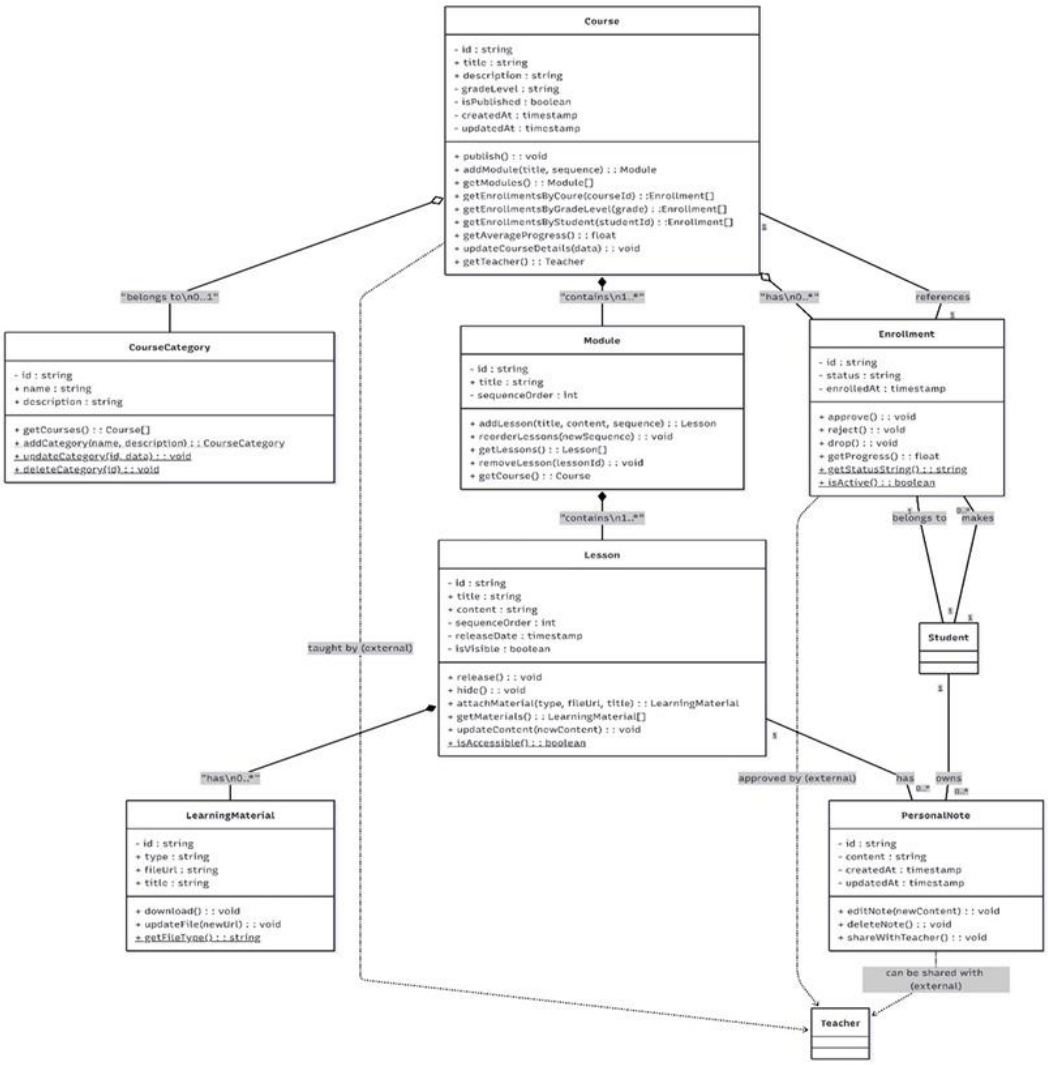


Figure 3.9: Class Diagram — Course Management Service

3.3.3 Assignment Service

The Assignment Service manages the full lifecycle of student assessments: assignment creation, submission, grading, and feedback, as well as quiz management including question authoring, quiz attempts, and answer recording. The two principal entities are Assignment and Quiz, each supported by a family of related classes.

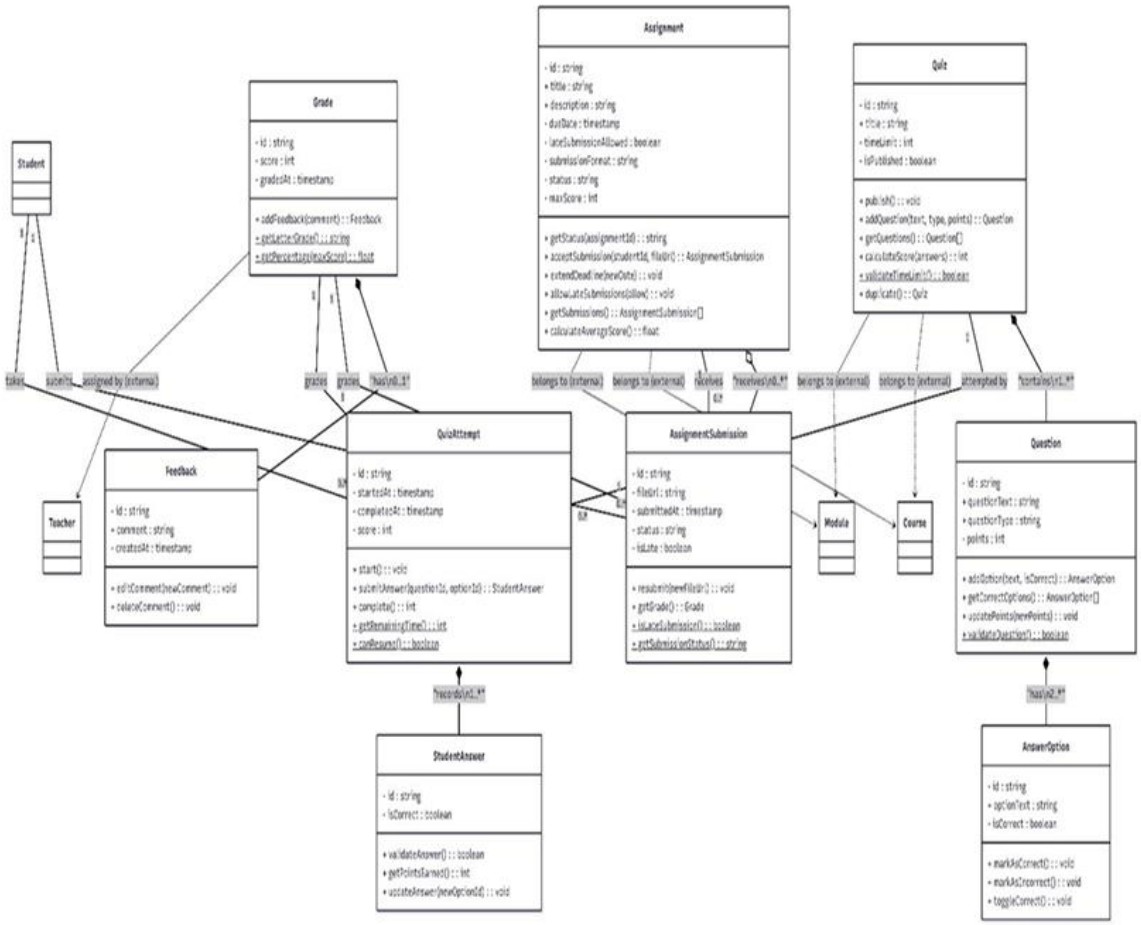


Figure 3.10: Class Diagram — Assignment Service

3.3.4 Analytics Service

The Analytics Service aggregates data from across the LMS to produce attendance records, progress reports, performance analytics, and engagement metrics. It depends on the Student, Course, and Lesson entities owned by other services, referenced through service-level identifiers.

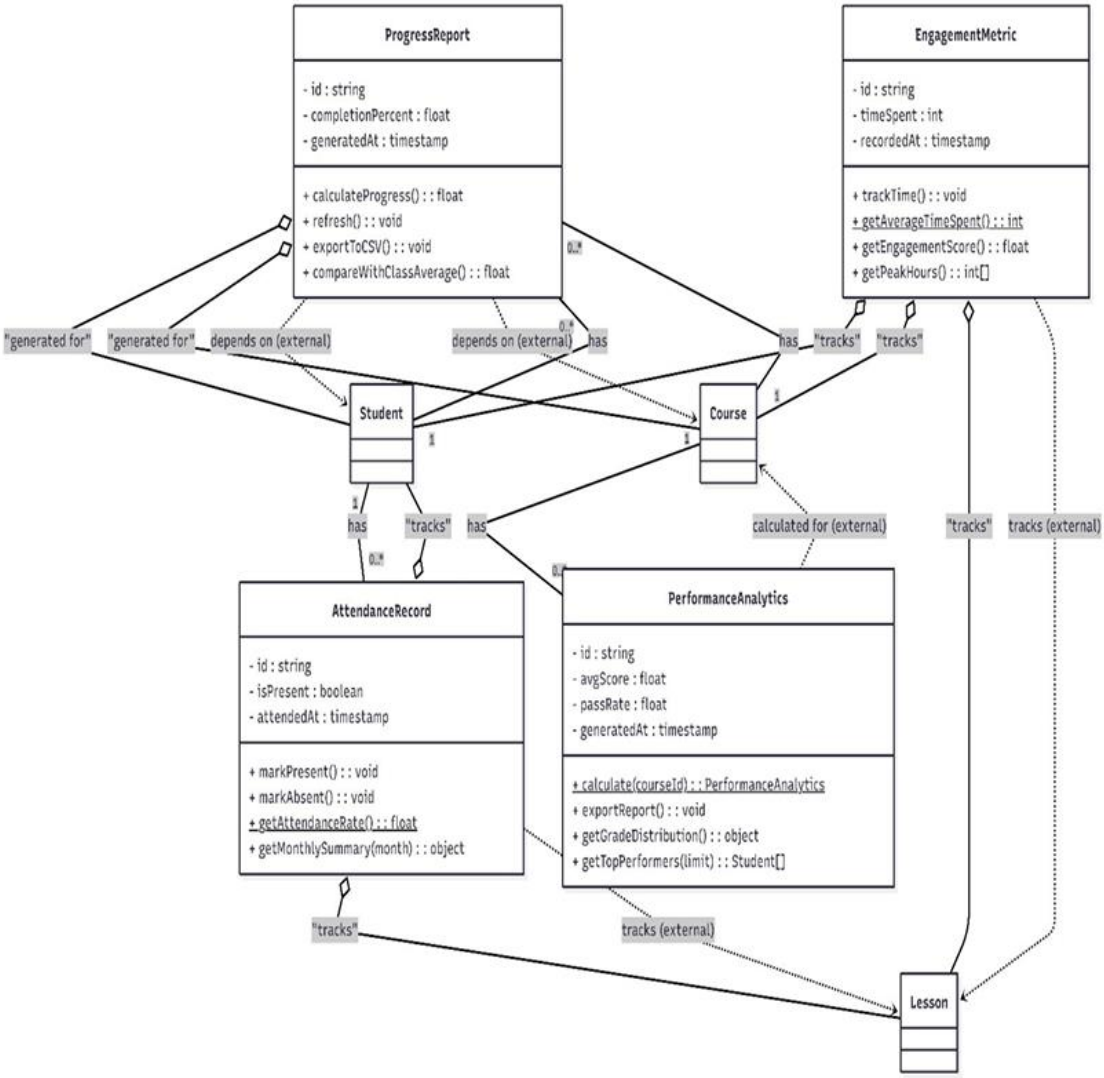


Figure 3.11: Class Diagram — Analytics Service

3.3.5 Communication Service

The Communication Service provides messaging, announcement, notification, and discussion capabilities. An abstract Message class establishes a common messaging contract, from which PrivateMessage is derived. GroupChat manages multi-user conversations, with GroupChatMember representing individual memberships. DiscussionForum supports course-level threaded discussions. Announcement provides broadcast communication within courses, and Notification handles system generated alerts.

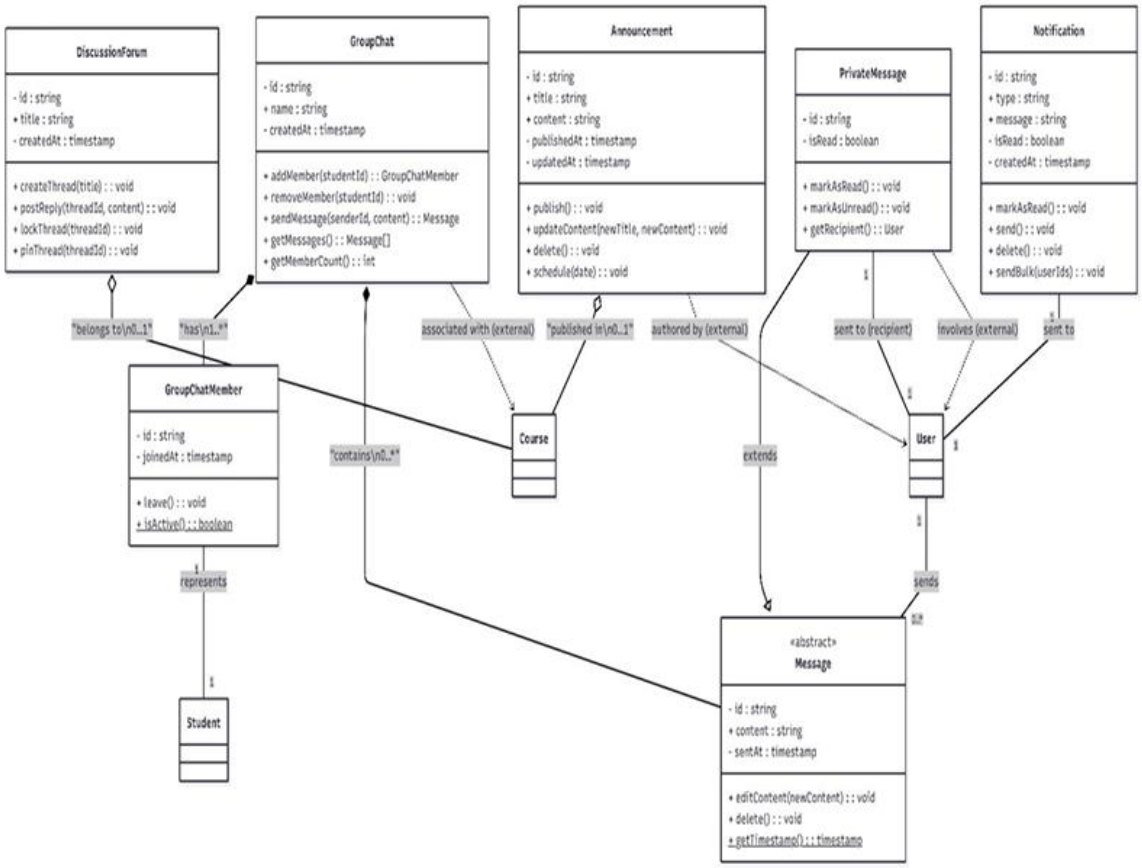


Figure 3.12: Class Diagram — Communication Service

3.3.6 AI Chat Service

The AI Chat Service provides AI-powered conversational, recommendation, and content-generation capabilities to the LMS. Four primary classes are defined: AIChatSession, AIChatMessage, AIRecommendation, AIInsightReport, and AIGeneratedContent. External service entities — Student, Teacher, Course, Quiz, and LearningMaterial — are referenced through cross-service identifiers.

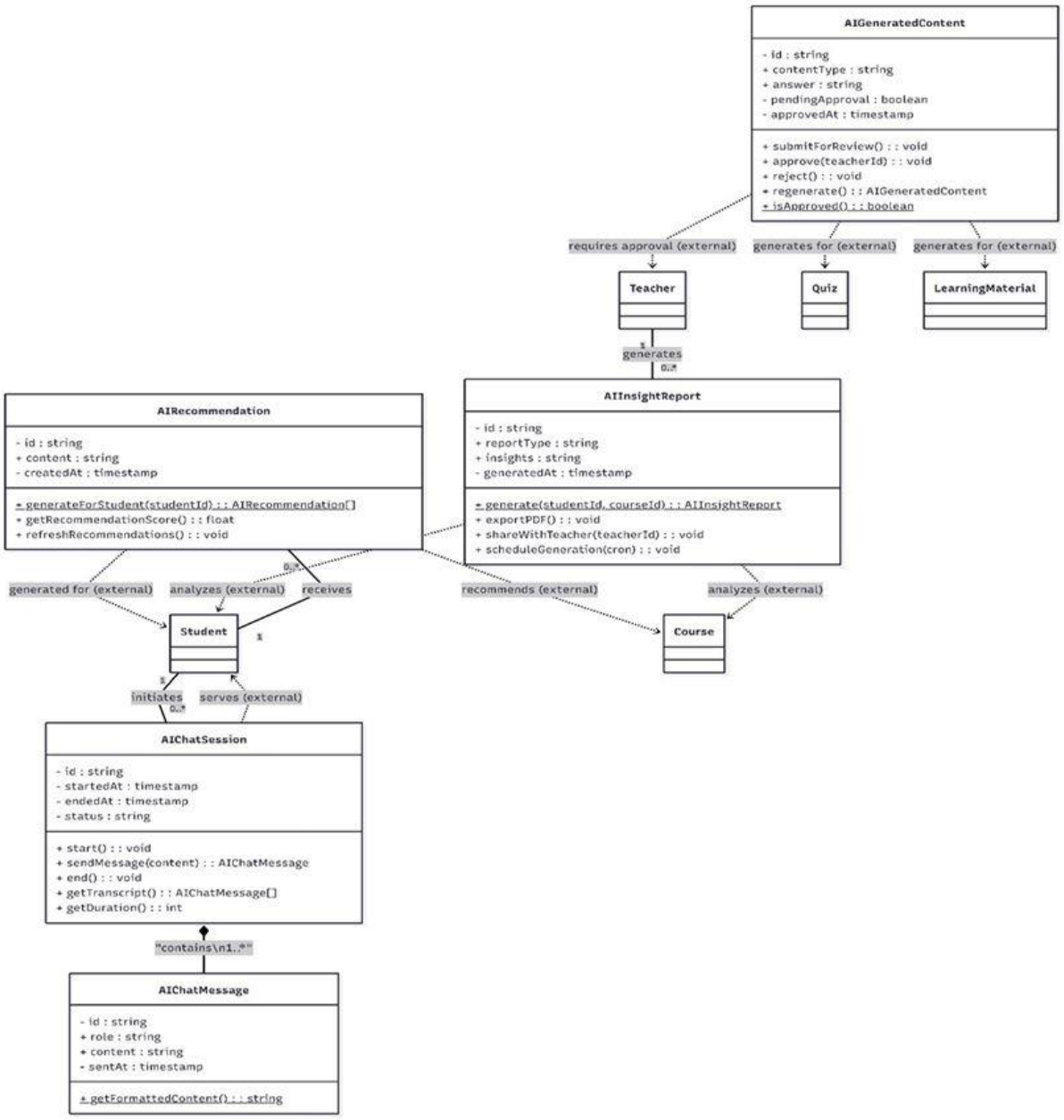


Figure 3.13: Class Diagram — AI Chat Service

3.3.7 Inter-Service Class Diagram

The inter-service class diagram illustrates the high-level composition and communication relationships between the six microservices of the LMS. Each service is represented as a stereotyped class (<>) containing its principal controllers, service objects, and domain entities. A shared DeliveryServiceAPI interface is implemented by multiple services to standardise content scheduling and delivery operations.

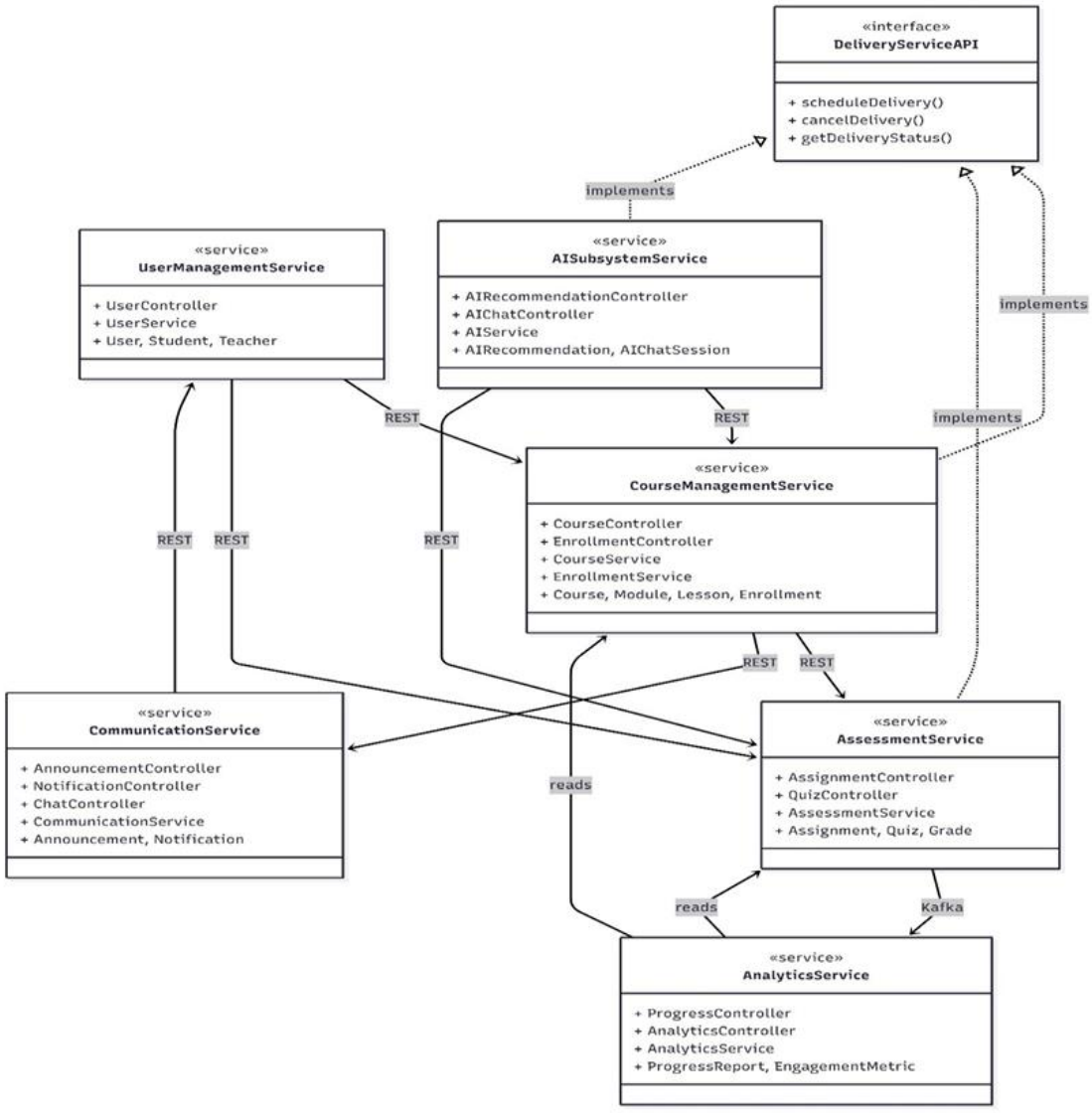


Figure 3.14: Inter-Service Class Diagram

3.4 Sequence Diagrams

This chapter presents the Unified Modelling Language (UML) sequence diagrams for all major use cases identified within the system. Each sequence diagram models the chronological flow of messages and interactions between system participants — encompassing end users, frontend components, backend microservices, databases, caching layers, external service integrations, and messaging infrastructure. The diagrams are constructed in accordance with the UML 2.5 sequence diagram notation standard and collectively cover the complete set of primary workflows exercisable by the Student, Teacher, and Administrator roles.

Each use case section is structured to include: a descriptive overview, a formal use case details table, a tabular summary of actors and system components, pre-conditions and post-conditions, a numbered main success flow with alternative and exception flows where applicable, and the corresponding UML sequence diagram embedded within the document body.

The table below provides a consolidated reference of all eleven use cases documented in this chapter, together with their unique identifier, primary actor, and the principal system components involved in fulfilling each interaction.

Ref.	Use Case Name	Primary Actor	Key System Components
4.1	User Login and Authentication	Student / Teacher / Admin	Browser, React, Spring Boot (Auth), PostgreSQL
4.2	Administrator Manages Users with RBAC	Administrator	React Admin, Spring Boot, PostgreSQL, Redis, Email Service
4.3	Teacher Uploads and Organises Learning Materials	Teacher	React, Spring Boot, Amazon S3, CloudFront/CDN, PostgreSQL
4.4	AI-Assisted Quiz Generation	Teacher	React, Spring Boot, OpenAI API, PostgreSQL

4.5	Student Attempts Online Quiz with Auto-Grading	Student	React, Spring Boot, AI Provider (OpenAI), PostgreSQL
4.6	Teacher Grades Assignment with AI Assistance	Teacher	React, Spring Boot, Amazon S3, AI Provider, PostgreSQL
4.7	Real-time Messaging between Student and Teacher	Student & Teacher	WebSocket Server, REST API, MongoDB, Redis, Email Service
4.8	Teacher Posts Class-wide Announcement	Teacher	Frontend, Backend, Database, Email Service, WebSocket
4.9	Student Interacts with AI Help Chatbot	Student	React, Spring Boot, Vector DB (Pinecone/PGVector), AI Provider, MongoDB
4.10	Student Views Performance Dashboard with Analytics	Student	React Dashboard, Spring Boot, AI Analytics Engine, PostgreSQL, Redis
4.11	Student Takes Personal Notes Within Platform	Student	React Frontend, Spring Boot Backend, PostgreSQL

3.10 : Summary of Major Use Cases and Corresponding Sequence Diagrams

3.4.1 User Login and Authentication

This use case describes the credential-based authentication workflow by which a registered user gains access to the platform. The authentication sequence involves the browser, the React frontend, the Spring Boot Authentication Service, and the PostgreSQL User database. Upon successful authentication, a JSON Web Token (JWT) is generated and returned to the client. The use case additionally covers the authenticated retrieval of a protected resource — the enrolled course list — demonstrating the token validation mechanism on subsequent API calls. This use case is a prerequisite for all other use cases in the system.

3.4.1.1 Use Case Details

Use Case ID	UC-01
Use Case Name	User Login and Authentication
Primary Actor	Student / Teacher / Administrator
Supporting Actors	Browser, React (Frontend), Spring Boot (Backend — Authentication Service), PostgreSQL
Trigger	A registered user navigates to the platform login page and submits their email address and password credentials.

3.11: Use Case Details — User Login and Authentication

3.4.1.2 Actors and System Components

Participant	Type	Responsibility
Student / Teacher / Admin	Primary Actor	Initiates the login action by submitting credentials via the browser.
Browser	Client Runtime	Hosts the React application and handles HTTPS communication with the backend.
React (Frontend)	Frontend Component	Renders the login form, handles form submission, stores the JWT, and redirects the user to the appropriate dashboard.
Spring Boot Backend	Application Server	Validates submitted credentials, performs BCrypt password verification, generates the signed JWT, and serves protected resource responses.
PostgreSQL	Relational Database	Persists user account records including hashed passwords and role assignments. Responds to credential lookup queries.

3.12: Actors and Components — User Login and Authentication

3.4.1.3 Pre-conditions

- The user possesses a valid, active account registered in the PostgreSQL User database.
- The user's device is connected to the network and can reach the platform URL via HTTPS.
- The Spring Boot Authentication Service and PostgreSQL instance are operational.
- The JWT signing keys are loaded and accessible by the Authentication Service.

3.4.1.4 Main Success Flow

1. The user navigates to the platform login page and enters their email address and password into the login form rendered by the React frontend in the browser.
2. The browser submits the login form. The React frontend sends a POST /api/auth/login HTTP request carrying the email and password credentials over HTTPS to the Spring Boot backend.
3. The Spring Boot backend receives the request and issues a SELECT query to PostgreSQL: SELECT user WHERE email = ?, to retrieve the user record and BCrypt-hashed password associated with the submitted email address.
4. PostgreSQL returns the user record including the stored BCrypt hash. The backend verifies the submitted plaintext password against the stored hash using the BCrypt algorithm.
5. Upon successful password verification, the Spring Boot backend generates a signed JWT encoding the user's unique identifier, assigned role, and a 15-minute expiry timestamp, using the RS256 algorithm.
6. The backend returns an HTTP 200 OK response to the React frontend, containing the JWT access token and the user's role.
7. The React frontend stores the JWT securely in an HttpOnly cookie or in-memory store. The user is redirected to the role-appropriate dashboard (e.g., Student Dashboard, Instructor Panel, or Admin Console).
8. The user requests a protected resource (e.g., their enrolled course list). The React frontend attaches the JWT in the Authorization: Bearer <token> header and sends GET /api/courses to the backend.
9. The Spring Boot backend intercepts the request, validates the JWT signature and checks the token expiry timestamp. Upon successful validation, the backend fetches the user's courses from PostgreSQL.

10. PostgreSQL returns the course records. The backend returns HTTP 200 OK with the course list to the React frontend.
11. The React frontend renders the dashboard with the returned course data. The user is fully authenticated and authorised to access role-appropriate resources.

3.4.1.5 Alternative and Exception Flows

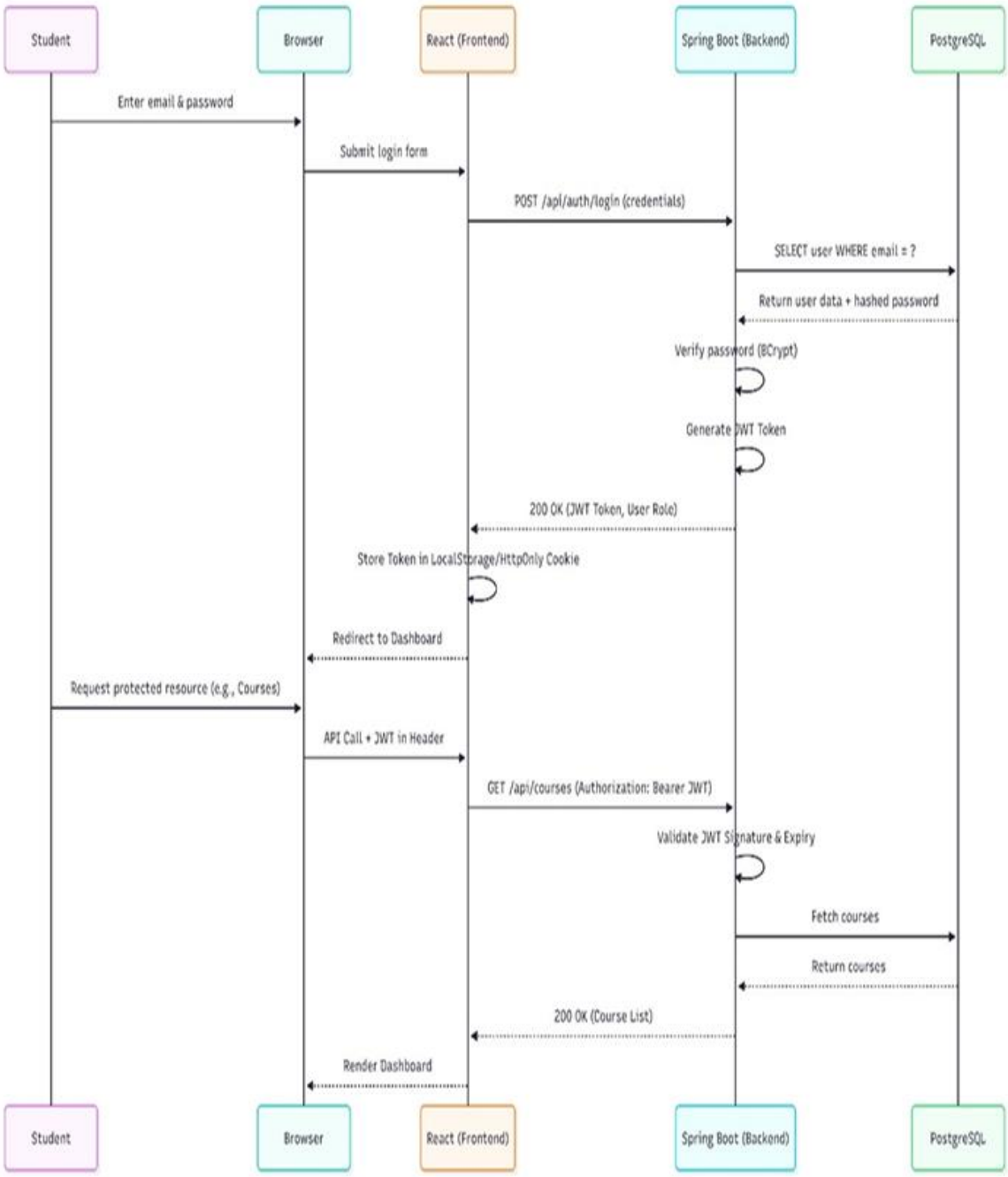
If the submitted email address does not exist in the PostgreSQL database, the backend returns HTTP 401 Unauthorized with a generic error message ('Invalid email or password'). If the email exists but the BCrypt password verification fails, an identical HTTP 401 response is returned to prevent email enumeration. If five consecutive failed login attempts are recorded within a 10-minute window, the user account is temporarily locked and an HTTP 423 Locked response is returned. If the JWT token presented in a subsequent request has expired, the backend returns HTTP 401 Unauthorized, and the client must re-authenticate or submit a valid refresh token to obtain a new access token.

3.4.1.6 Post-conditions

- A valid JWT access token is held by the authenticated client and included in all subsequent API requests.
- The user is redirected to and presented with their role-appropriate dashboard.
- All subsequent requests to protected endpoints are authorised through JWT signature and expiry validation.

3.4.1.7 Sequence Diagram

Figure below illustrates the complete interaction sequence for the use case described above.



3.13: Sequence Diagram — User Login and Authentication

3.4.2 Administrator Manages Users with RBAC

This use case models the complete set of administrative user-management operations available to a system administrator. The interaction encompasses four distinct sub-workflows: (1) creation of a new user account with automatic email notification, (2) editing of role-based permissions for the Teacher role and invalidating the permission cache, (3) deactivation of a user account with immediate session termination, and (4) assignment of a teacher to a course with notification dispatch. The sequence spans the React Admin Panel, Spring Boot Admin API, PostgreSQL database, Redis cache, and the Email Service. Role-based access control (RBAC) ensures that all administrative actions are restricted exclusively to users holding the Administrator role.

3.4.2.1 Use Case Details

Use Case ID	UC-02
Use Case Name	Administrator Manages Users with RBAC
Primary Actor	Administrator
Supporting Actors	React Admin Panel, Spring Boot (Admin API), PostgreSQL, Redis Cache, Email Service
Trigger	The administrator opens the Admin Panel to perform user management, role configuration, or course-assignment operations.

3.14: Use Case Details — Administrator Manages Users with RBAC

3.4.2.2 Actors and System Components

Participant	Type	Responsibility
Administrator	Primary Actor	Performs all user management, role configuration, deactivation, and assignment operations via the React Admin Panel.
React Admin Panel	Frontend Component	Renders the user management interface, role-permission grid, and form modals. Submits all administrative actions to the Spring Boot API.
Spring Boot API	Application Server	Processes all administrative requests, enforces business rules (e.g., email uniqueness), applies RBAC updates, and triggers external notifications.
PostgreSQL	Relational Database	Persists user records, role definitions, role-permission mappings, and course-assignment relationships. Source of truth for all user and RBAC data.
Redis Cache	Cache Layer	Caches the RBAC permission matrix for low-latency permission checks. Cache entries are invalidated when role-permission mappings are updated.
Email Service	External Integration	Dispatches transactional emails for account creation welcome messages and teacher course-assignment notifications.

3.15: Actors and Components — Administrator Manages Users with RBAC

3.4.2.3 Pre-conditions

- The administrator is authenticated and holds the Administrator role.
- The Admin Panel is accessible via HTTPS.
- The PostgreSQL database and Redis cache are operational.
- The Email Service is configured and reachable.

3.4.2.4 Main Success Flow

1. The administrator opens the Admin Panel. The React Admin Panel issues GET /api/admin/users to the Spring Boot API, which executes SELECT users with roles against PostgreSQL and returns the user list. The admin panel displays the user table.
2. The administrator clicks 'Create User' and the frontend opens a form modal. The administrator enters the new user's details (name, email, role) and submits the form.
3. The React Admin Panel sends POST /api/admin/users to the Spring Boot API. The backend queries PostgreSQL to check whether the submitted email address is already registered.
4. If the email address is unique: the backend generates a temporary password, inserts the new user record into PostgreSQL, and dispatches a welcome email containing login credentials via the Email Service. A success confirmation is returned to the Admin Panel.
5. If the email address already exists: the backend returns HTTP 409 Conflict. The Admin Panel displays an 'Email already registered' error message.
6. The administrator navigates to the Roles and Permissions section. GET /api/admin/roles is issued. The backend fetches the role-permission mapping from PostgreSQL and returns the RBAC permission matrix. The Admin Panel renders the permission grid.
7. The administrator edits the Teacher role to add the export_grades permission and submits the change. PUT /api/admin/roles/teacher is issued.
8. The backend updates the role_permissions table in PostgreSQL. The cached RBAC permission matrix in Redis is invalidated to ensure that subsequent permission checks reflect the updated configuration. The backend returns an 'Update successful' response.
9. The administrator selects a user account and initiates deactivation. PUT /api/admin/users/123/deactivate is issued to the backend.
10. The backend sets the user's status to deactivated in PostgreSQL, records the deactivation timestamp, and blacklists all active session tokens for that user in Redis, terminating all active sessions immediately. A 'User deactivated' confirmation is returned.

11. The administrator assigns a teacher to a course by submitting POST /api/admin/course-assignment. The backend links the teacher to the course in PostgreSQL and dispatches a notification email to the teacher via the Email Service. An 'Assignment complete' response is returned to the Admin Panel.

3.4.2.5 Alternative and Exception Flows

If an email address conflict is detected during user creation (Step 4), a 409 Conflict HTTP response is returned and no user record is persisted. If the Email Service is unavailable during welcome email dispatch, the user record is still created but the email dispatch failure is logged for retry. If Redis is unavailable during cache invalidation, the update to PostgreSQL is committed and the cache invalidation is retried asynchronously to ensure eventual consistency.

3.4.2.6 Post-conditions

- New user accounts are created and persisted in PostgreSQL; welcome emails are dispatched to registered addresses.
- The Teacher role's permission mapping is updated in PostgreSQL and the Redis permission cache is invalidated.
- Deactivated user accounts have all active session tokens blacklisted in Redis and all sessions terminated.
- The teacher-to-course assignment is recorded in PostgreSQL and the teacher is notified via email.

3.4.2.7 Sequence Diagram

Figure below illustrates the complete interaction sequence for the use case described above.

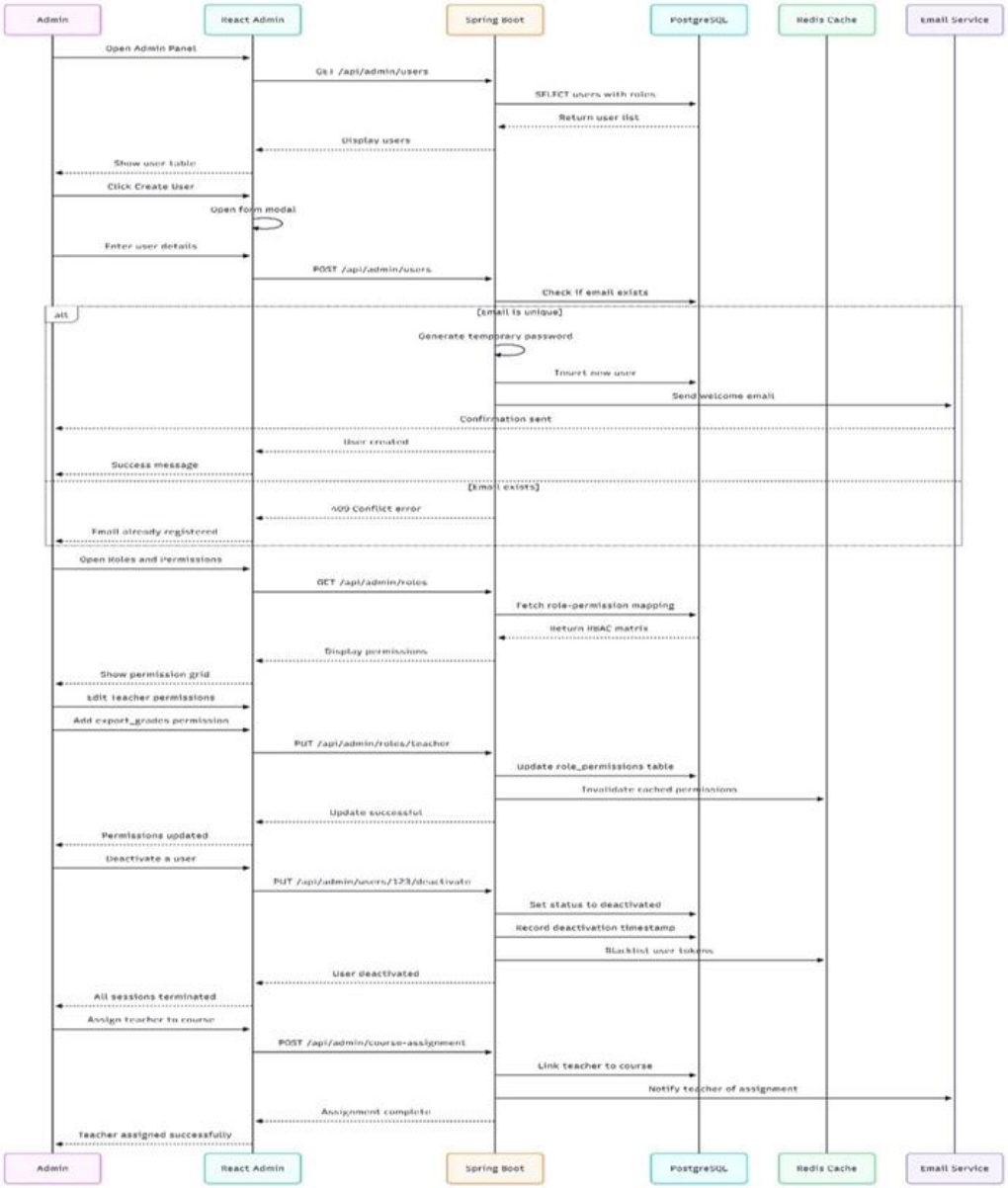


Figure 3.15: Sequence Diagram — Administrator Manages Users with RBAC

3.4.3 Teacher Uploads and Organises Learning Materials

This use case describes the full workflow by which a teacher uploads learning materials — including PDF documents, video files, and presentation slides — to the platform, organises them within a hierarchical course module structure, links them to individual lessons, and schedules their release for student access. Uploaded files are streamed directly to Amazon S3 and subjected to a virus and malware scan before metadata is persisted in PostgreSQL. Course content is subsequently distributed to enrolled students via the CloudFront content delivery network (CDN). Lesson scheduling is handled by a server-side Quartz/Cron job that publishes lessons automatically at the specified release date and time.

3.4.3.1 Use Case Details

Use Case ID	UC-03
Use Case Name	Teacher Uploads and Organises Learning Materials
Primary Actor	Teacher
Supporting Actors	React (Course Builder), Spring Boot, Amazon S3, CloudFront/CDN, PostgreSQL
Trigger	The teacher navigates to the Course Builder interface to upload learning materials or organise the course module structure.

Figure 3.16: Use Case Details — Teacher Uploads and Organises Learning Materials

3.4.3.2 Actors and System Components

Participant	Type	Responsibility
Teacher	Primary Actor	Initiates all content upload, module creation, lesson linking, and scheduling actions via the Course Builder interface.

React (Course Builder)	Frontend Component	Renders the drag-and-drop module editor and file upload interface. Submits multipart uploads and REST API calls to the backend.
Spring Boot Backend	Application Server	Handles file upload requests, orchestrates S3 streaming, performs malware scanning, persists metadata, and schedules lesson release jobs.
Amazon S3	Object Storage	Stores all uploaded file assets (PDFs, videos, slides) as durable objects. Returns the file URL and metadata upon successful upload.
CloudFront / CDN	Content Delivery Network	Distributes stored course materials to students with low latency. Acts as the access layer over S3 for student-facing content delivery.
PostgreSQL	Relational Database	Persists course structure (modules, lessons), material metadata (URL, type, size, uploader), and lesson release schedules.

Figure 3.17: Actors and Components — Teacher Uploads and Organises Learning Materials

3.4.3.3 Pre-conditions

- The teacher is authenticated and holds the Instructor role.
- The course to which materials will be uploaded has been previously created in the system.
- Amazon S3 and CloudFront are operational and the teacher's IAM credentials allow upload access.
- The Quartz scheduler service is running within the Spring Boot backend.

3.4.3.4 Main Success Flow

1. The teacher navigates to the Course Builder. The React frontend issues GET /api/courses/:id/structure to the Spring Boot backend. The backend queries PostgreSQL for the module and lesson tree of the specified course and returns the hierarchical structure. The Course Builder renders the drag-and-drop interface.
2. The teacher clicks 'Upload Material' and the Course Builder opens a file picker. The teacher selects one or more files (PDF, video, or presentation slides).
3. The React frontend submits a POST /api/materials/upload multipart/form-data request to the Spring Boot backend, carrying the selected file content.
4. The Spring Boot backend streams the file content directly to Amazon S3. S3 returns the file URL and metadata upon successful storage.
5. The backend performs an inline virus and malware scan on the uploaded file content to detect any malicious payloads prior to availability.
6. Upon a clean scan result, the backend inserts a material record into PostgreSQL, persisting the S3 URL, file type, file size, and uploader ID. The backend returns HTTP 200 OK to the frontend with the Material ID and a CloudFront preview URL.
7. The teacher clicks 'Create New Module' and enters the module name ('Week 1'). The frontend submits POST /api/modules with the module name, description, and display order.
8. The backend inserts the module record into PostgreSQL and returns HTTP 201 Created with the generated module_id.
9. The teacher drags the uploaded material into the new module and clicks 'Add Lesson'. The frontend submits POST /api/lessons carrying the moduleId, materialId, and lesson title.
10. The backend inserts the lesson record into PostgreSQL and links the material to the lesson. The backend returns HTTP 201 Created. The Course Builder confirms 'Lesson added to module'.
11. The teacher sets a release date for the lesson (e.g., '2026-04-25'). The frontend submits PUT /api/lessons/:id/schedule carrying the releaseDate.
12. The backend updates the release_date field in the PostgreSQL lessons table and registers a Quartz/Cron job to automatically change the lesson status to 'published' at the specified date and time. HTTP 200 OK is returned. The Course Builder confirms 'Lesson scheduled successfully'.

3.4.3.5 Alternative and Exception Flows

If the virus scan in Step 5 detects a malicious file, the backend deletes the file from S3, discards the upload, and returns HTTP 422 Unprocessable Entity to the frontend with an appropriate error message. No material record is persisted in PostgreSQL. If the S3 upload fails due to a network or service interruption (Step 4), the backend returns HTTP 503 Service Unavailable and the frontend prompts the teacher to retry the upload. Unsupported file types (outside the permitted MIME type whitelist) are rejected at the API Gateway level before reaching the backend.

3.4.3.6 Post-conditions

- Uploaded material files are durably stored in Amazon S3 and accessible via CloudFront CDN URLs.
- Material metadata (file URL, type, size, and uploader) is persisted in the PostgreSQL materials table.
- A new module record is created in PostgreSQL and associated with the specified course.
- A lesson record is created, linked to the uploaded material, and associated with the new module.
- A Quartz/Cron job is scheduled to automatically publish the lesson at the specified release date.

3.4.3.7 Sequence Diagram

Figure below illustrates the complete interaction sequence for the use case described above.

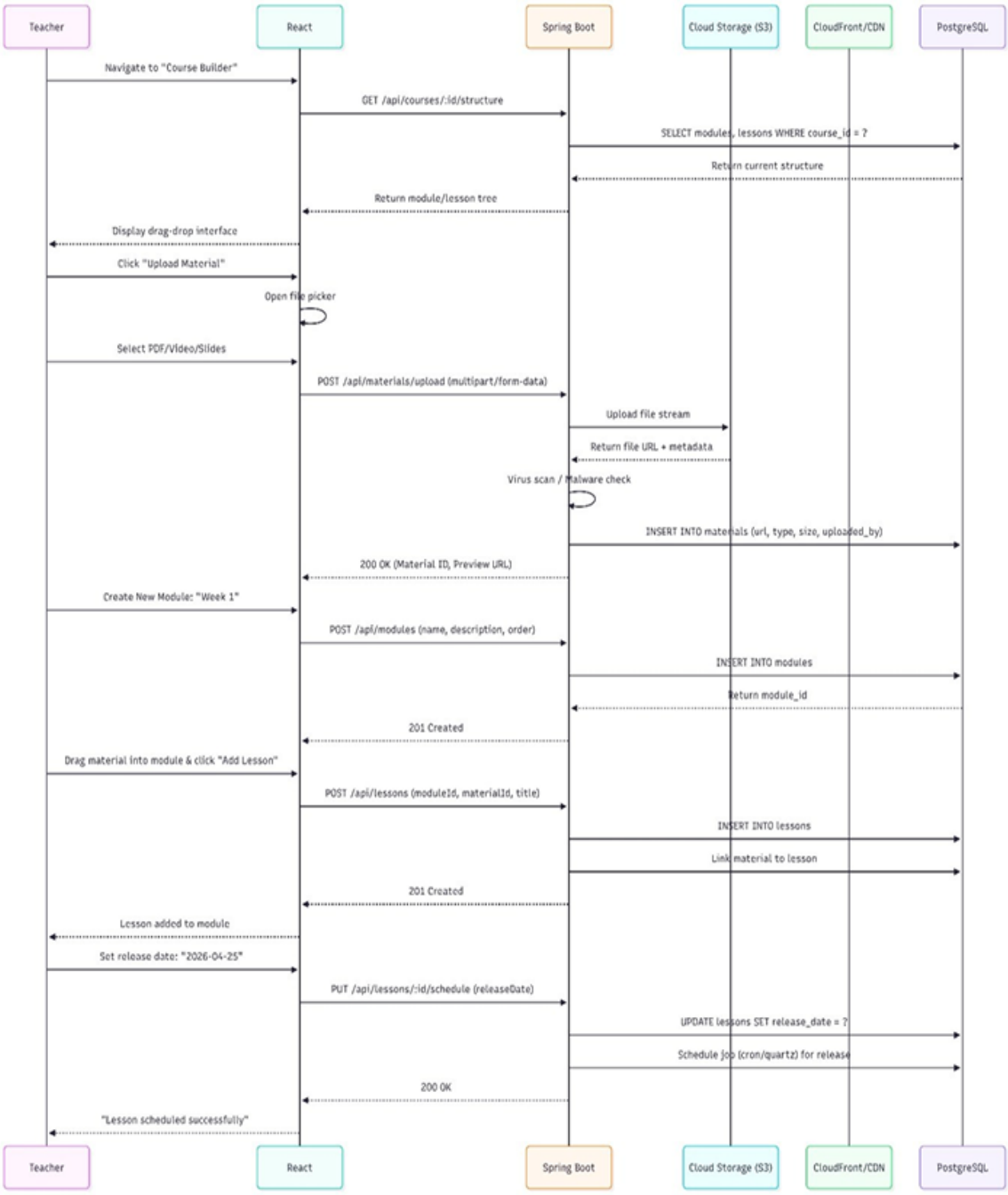


Figure 3.18: Sequence Diagram — Teacher Uploads and Organises Learning Materials

3.4.4 AI-Assisted Quiz Generation

This use case models the AI-assisted workflow by which a teacher generates a multiple-choice question (MCQ) quiz from existing course material stored in the system. The Spring Boot AI Service retrieves relevant course content from PostgreSQL, constructs a structured prompt that includes the content and the desired question count, and submits the prompt to the OpenAI completions API. The AI-generated response — containing questions, answer options, and correct answers — is parsed, validated, and stored in PostgreSQL as a draft quiz. The teacher is presented with the draft for review and may edit individual questions before publishing the quiz to enrolled students.

3.4.4.1 Use Case Details

Use Case ID	UC-04
Use Case Name	AI-Assisted Quiz Generation
Primary Actor	Teacher
Supporting Actors	React (Frontend), Spring Boot (AI Service), AI Provider (OpenAI), PostgreSQL
Trigger	The teacher selects a course and topic, then clicks the 'Generate Quiz' button within the course management interface.

3.16 : Use Case Details — AI-Assisted Quiz Generation

3.4.4.2 Actors and System Components

Participant	Type	Responsibility
Teacher	Primary Actor	Initiates quiz generation by selecting a course and topic, reviews the draft quiz, optionally edits questions, and publishes the final quiz.
React (Frontend)	Frontend Component	Renders the quiz generation interface, displays the draft quiz for review, and submits publish and edit actions to the backend.
Spring Boot AI Service	Application Server	Retrieves course materials from PostgreSQL, constructs the AI prompt, communicates with the OpenAI API, and parses and validates the AI response.
AI Provider (OpenAI)	External AI Service	Processes the structured prompt and returns a JSON payload containing generated quiz questions, answer options, and correct answers.
PostgreSQL	Relational Database	Stores course materials used to generate quiz content. Persists the generated quiz in draft status and updates its status upon publishing.

3.17: Actors and Components — AI-Assisted Quiz Generation

3.4.4.3 Pre-conditions

- The teacher is authenticated and holds the Instructor role.
- The selected course has at least one associated material record in PostgreSQL.
- The OpenAI API integration is configured with a valid API key and is operational.
- The desired question count and topic are specified by the teacher before submission.

3.4.4.4 Main Success Flow

1. The teacher selects a course from the course list and enters a topic for the quiz. The teacher clicks 'Generate Quiz'. The React frontend submits POST /api/ai/generate-quiz with the courseId and topic to the Spring Boot AI Service.

2. The Spring Boot AI Service queries PostgreSQL using SELECT course_materials WHERE course_id = ? to retrieve all materials associated with the specified course, including PDF text content.
3. PostgreSQL returns the course materials. The AI Service processes the retrieved content and constructs a structured prompt that combines the material content, the specified topic, and the desired question count.
4. The AI Service submits the prompt to the OpenAI completions API via POST /v1/completions, requesting the generation of MCQ questions in a structured JSON format.
5. The OpenAI API returns a JSON payload containing the generated questions, four answer options per question, and the correct answer identifier for each question.
6. The Spring Boot AI Service parses the JSON response and validates its structural integrity, checking that each question includes at least two answer options and a valid correct answer reference.
7. The AI Service inserts the validated quiz into PostgreSQL with status set to 'draft', associating the quiz with the specified course and teacher.
8. The AI Service returns HTTP 200 OK with the list of generated questions to the React frontend. The frontend renders the Quiz Draft interface, displaying each question and its answer options for teacher review.
9. The teacher reviews each question, optionally edits or removes individual questions, and clicks 'Publish Quiz'.
10. The React frontend submits PUT /api/quizzes/:id/publish to the backend. The backend executes UPDATE quizzes SET status = 'published' in PostgreSQL, making the quiz available to enrolled students.
11. The backend returns HTTP 200 OK (Published). The frontend displays a success message confirming that the quiz has been published.

3.4.4.5 Alternative and Exception Flows

If the OpenAI API returns a malformed or structurally invalid response in Step 5, the Spring Boot AI Service logs the error and returns HTTP 502 Bad Gateway to the frontend, prompting the teacher to retry the generation. If PostgreSQL contains insufficient course material text for the specified course (e.g., materials consist solely of video files with no extractable text), the AI

Service returns HTTP 422 Unprocessable Entity with an appropriate error message advising the teacher to select a course with text-based materials.

3.4.4.6 Post-conditions

- A draft quiz record containing AI-generated questions and answer options is persisted in PostgreSQL with status 'draft'.
- The teacher has reviewed and optionally modified the generated questions.
- The quiz status is updated to 'published' in PostgreSQL, making it available to enrolled students.

3.4.4.7 Sequence Diagram

Figure 4 below illustrates the complete interaction sequence for the use case described above.

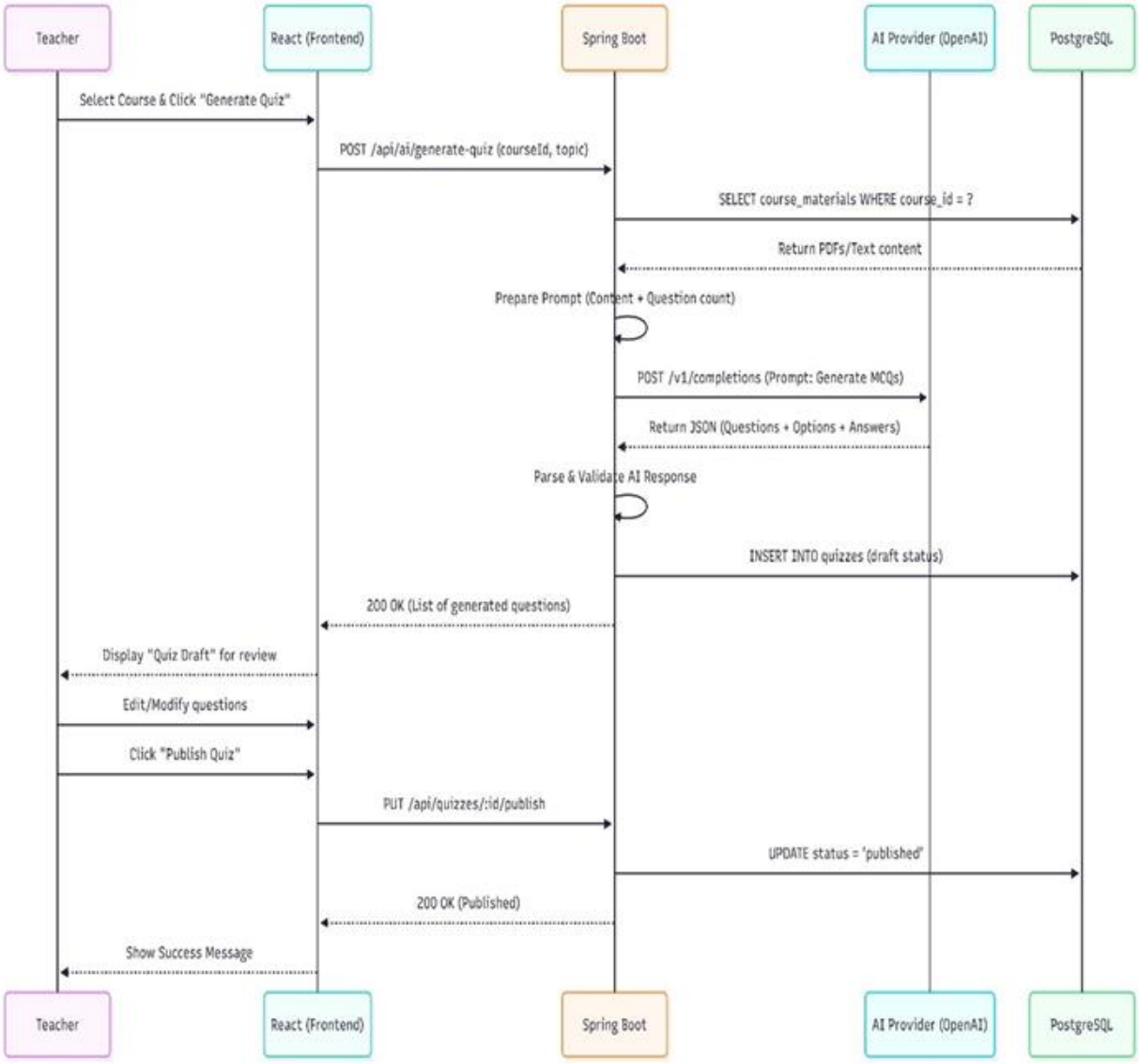


Figure 3.19: Sequence Diagram — AI-Assisted Quiz Generation

3.4.5 Student Attempts Online Quiz with Auto-Grading

This use case describes the end-to-end lifecycle of a student's quiz attempt, from question retrieval through to results presentation. Questions are retrieved from PostgreSQL with correct answers withheld from the response to prevent client-side exposure. Upon submission, the Assessment Service applies a dual grading strategy: objective questions (MCQ and True/False) are graded automatically by the backend through direct answer comparison, while subjective questions (text and essay responses) are evaluated by the OpenAI AI Provider using a grading prompt and rubric. All grades and feedback are persisted in PostgreSQL, and the complete results — including score, percentage, feedback, and correct answers — are returned to the student upon completion.

3.4.5.1 Use Case Details

Use Case ID	UC-05
Use Case Name	Student Attempts Online Quiz with Auto-Grading
Primary Actor	Student
Supporting Actors	React (Frontend), Spring Boot (Assessment Service), AI Provider (OpenAI), PostgreSQL
Trigger	The student navigates to a published quiz and clicks the 'Start Quiz' button.

3.18 : Use Case Details — Student Attempts Online Quiz with Auto-Grading

3.4.5.2 Actors and System Components

Participant	Type	Responsibility
Student	Primary Actor	Initiates the quiz attempt, selects and submits answers, and views the results upon completion.
React (Frontend)	Frontend Component	Renders the quiz interface, manages the attempt session, submits student answers, and displays results with explanations.

Spring Boot Assessment Service	Application Server	Retrieves questions, generates the attempt session ID, receives and persists student responses, applies auto-grading logic, and coordinates AI grading for subjective questions.
AI Provider (OpenAI)	External AI Service	Evaluates student text and essay responses against the model answer using an AI grading prompt. Returns a suggested score and textual feedback.
PostgreSQL	Relational Database	Stores quiz questions and correct answers, persists student attempt sessions, student responses, AI-generated grades, and final scores.

3.19 : Actors and Components — Student Attempts Online Quiz with Auto-Grading

3.4.5.3 Pre-conditions

- The student is authenticated and holds the Student role.
- The quiz is in 'published' status and the student is enrolled in the associated course.
- The OpenAI AI Provider is configured and operational for subjective question grading.
- The quiz contains at least one question.

3.4.5.4 Main Success Flow

12. The student clicks 'Start Quiz'. The React frontend sends GET /api/quizzes/:id/attempt to the Spring Boot Assessment Service.
13. The Assessment Service queries PostgreSQL using SELECT questions WHERE quiz_id = ?, retrieving all questions associated with the quiz. Correct answer fields are excluded from the query result set to prevent exposure to the client.
14. The Assessment Service generates a unique Attempt Session ID for this attempt, which is used to associate all submitted answers with this specific attempt.
15. The backend returns HTTP 200 OK with the list of questions (without correct answers) and the Attempt Session ID. The React frontend renders the quiz interface.

16. The student reads each question, selects or types their answers, and clicks 'Submit'. The frontend submits POST /api/quizzes/submit carrying the array of submitted answers and the Attempt Session ID.
17. The Assessment Service persists the student's responses to PostgreSQL, associating each response with the student ID, quiz ID, and Attempt Session ID.
18. For each objective question (MCQ or True/False), the backend compares the student's submitted answer against the stored correct answer in a loop, computing a binary correct/incorrect outcome for each item.
19. For each subjective question (text or essay response), the backend sends the student's submitted answer, the model answer, and a structured grading prompt to the OpenAI AI Provider via POST /v1/completions.
20. The OpenAI AI Provider evaluates the response against the model answer and returns a suggested numerical score and a textual feedback comment for each subjective question.
21. The backend inserts the grade and feedback records into the PostgreSQL grades table. The total score and percentage are calculated across all questions.
22. The backend returns HTTP 200 OK with the complete results payload — including the total score, percentage, per-question feedback, and correct answers — to the React frontend.
23. The React frontend renders the results view, displaying the student's performance breakdown, feedback for each question, and the correct answers with explanations.

3.4.5.5 Alternative and Exception Flows

If the OpenAI AI Provider is unavailable during subjective question grading (Step 8), the backend flags the subjective questions as 'pending grading' in PostgreSQL, returns the objective question results immediately, and schedules a retry of AI grading asynchronously. The student is notified that subjective results will be available once grading is complete. If the Attempt Session ID is invalid or the session has expired, the backend returns HTTP 400 Bad Request and the student is prompted to restart the quiz attempt.

3.4.5.6 Post-conditions

- The student's responses are persisted in PostgreSQL, associated with the Attempt Session ID.
- Objective questions are graded and scores are stored in PostgreSQL.
- Subjective questions are evaluated by the AI Provider, with suggested scores and feedback persisted in PostgreSQL.
- The total score, percentage, per-question feedback, and correct answers are displayed to the student.

3.4.5.7 Sequence Diagram

Figure 5 below illustrates the complete interaction sequence for the use case described above.

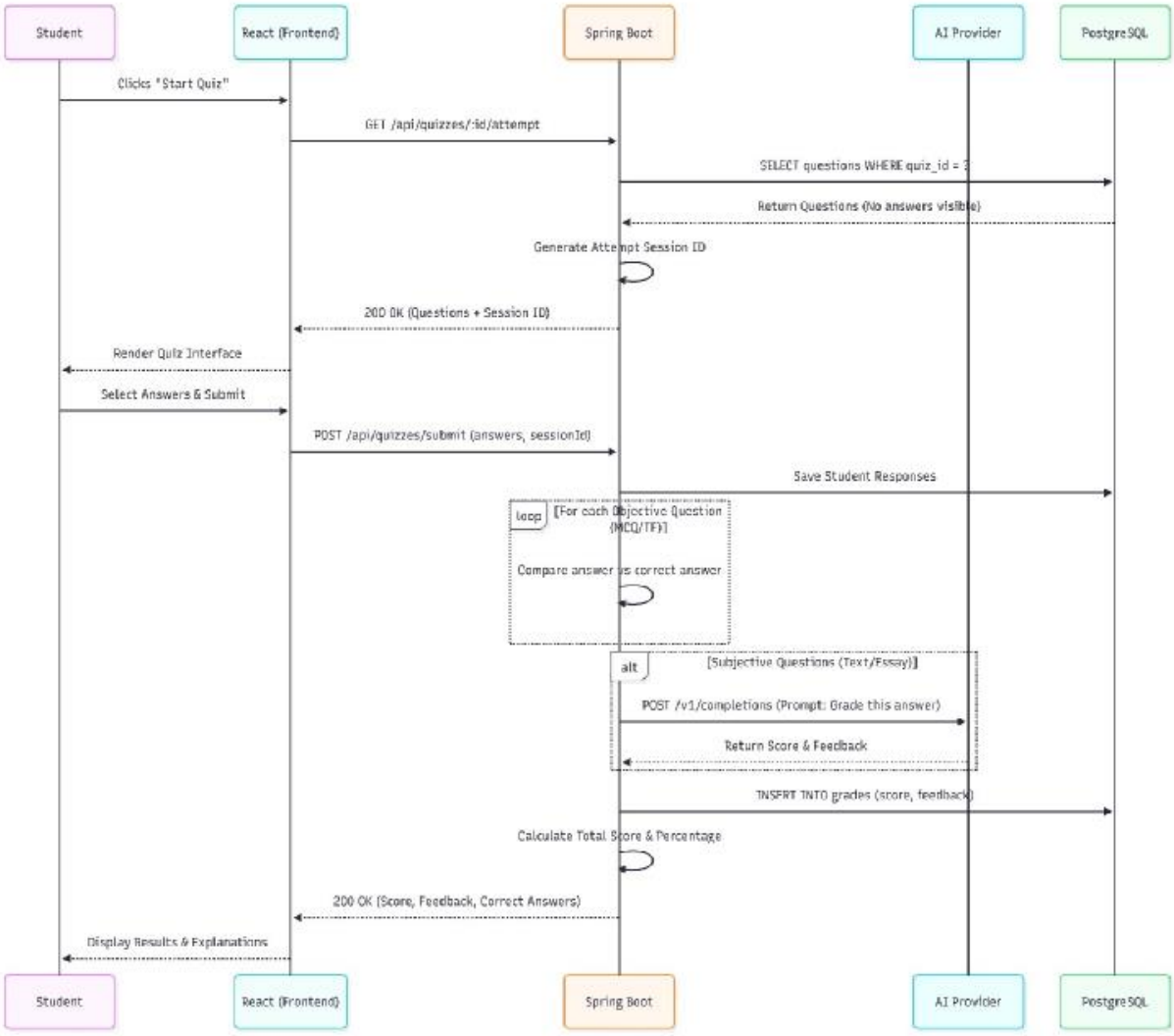


Figure 3.20: Sequence Diagram — Student Attempts Online Quiz with Auto-Grading

3.4.6 Teacher Grades Assignment with AI Assistance

This use case models the AI-assisted grading workflow for file-based assignment submissions. The Spring Boot AI Service downloads the student's submitted PDF from Amazon S3, retrieves the assignment rubric and model answer from PostgreSQL, and sends both alongside a grading prompt to the OpenAI AI Provider for evaluation. The AI Provider returns a suggested score and textual feedback. The suggested grade is stored as a draft in PostgreSQL and presented to the teacher for review. The teacher may accept or modify the suggested grade and feedback before releasing it. Upon release, the grade status is updated to 'released' in PostgreSQL and the Notification Service dispatches a notification to the student.

3.4.6.1 Use Case Details

Use Case ID	UC-06
Use Case Name	Teacher Grades Assignment with AI Assistance
Primary Actor	Teacher
Supporting Actors	React (Frontend), Spring Boot (AI Service), Amazon S3 (Cloud Storage), AI Provider (OpenAI), PostgreSQL
Trigger	The teacher navigates to the Submissions list for an assignment and clicks 'Grade with AI' on a specific student submission.

3.20: Use Case Details — Teacher Grades Assignment with AI Assistance

3.4.6.2 Actors and System Components

Participant	Type	Responsibility
Teacher	Primary Actor	Reviews the assignment submission list, initiates AI grading, reviews and optionally modifies the suggested grade, and releases the final grade.

React (Frontend)	Frontend Component	Renders the submission list, displays the AI-suggested grade and feedback for review, and submits grade finalisation actions.
Spring Boot AI Service	Application Server	Orchestrates the grading workflow: downloads the submission from S3, retrieves the rubric, sends the grading request to the AI Provider, and persists the draft grade.
Amazon S3	Object Storage	Stores student-submitted assignment files. Returns the file stream upon download request from the backend.
AI Provider (OpenAI)	External AI Service	Evaluates the student's submitted file against the assignment rubric and model answer. Returns a suggested numerical score and detailed textual feedback.
PostgreSQL	Relational Database	Stores assignment rubrics, model answers, student submission records, draft grades, and finalised grade records with release status.

3.21: Actors and Components — Teacher Grades Assignment with AI Assistance

3.4.6.3 Pre-conditions

- The teacher is authenticated and holds the Instructor role.
- The assignment has received at least one student submission.
- The student's submitted file is stored in Amazon S3.
- An assignment rubric and model answer are configured in PostgreSQL for the assignment.
- The OpenAI AI Provider integration is configured and operational.

3.4.6.4 Main Success Flow

24. The teacher navigates to the Submissions section for an assignment. The React frontend sends GET /api/assignments/:id/submissions to the Spring Boot backend.

25. The backend fetches the list of student submissions from PostgreSQL and returns it. The React frontend displays the submissions list with student names and submission statuses.
26. The teacher selects a specific student submission and clicks 'Grade with AI'. The frontend submits POST /api/ai/grade-submission carrying the submissionId.
27. The Spring Boot AI Service downloads the student's submitted PDF file from Amazon S3 using the stored file reference. Amazon S3 returns the file stream.
28. The AI Service queries PostgreSQL using SELECT assignment_rubric, model_answer to retrieve the assessment criteria and model answer for the assignment.
29. The AI Service constructs a structured grading prompt combining the submitted file content, the assignment rubric, and the model answer. The prompt is submitted to the OpenAI AI Provider via POST.
30. The OpenAI AI Provider evaluates the submission against the rubric and model answer and returns a suggested score (e.g., 85/100) and detailed textual feedback.
31. The AI Service stores the suggested grade and feedback as a draft entry in the PostgreSQL grades table with status 'draft'. The backend returns HTTP 200 OK with the suggested grade and feedback to the React frontend.
32. The React frontend displays the AI suggestion to the teacher, presenting options to accept or edit the suggested grade and feedback.
33. The teacher reviews the suggestion, optionally modifies the feedback text or adjusts the numerical grade, and clicks 'Release Grade'.
34. The React frontend submits POST /api/grades/finalize to the Spring Boot backend, carrying the finalised grade and feedback.
35. The backend updates the grade record in PostgreSQL, setting the status to 'released'. The Notification Service is triggered to dispatch a notification to the student informing them that their grade is available.
36. The backend returns HTTP 200 OK. The React frontend confirms 'Grade released to student'.

3.4.6.5 Alternative and Exception Flows

If the AI Provider returns an unusable or structurally invalid response (Step 7), the backend logs the error and returns HTTP 502 Bad Gateway. The teacher is prompted to grade the submission manually. If the submission file cannot be downloaded from S3 due to a service error (Step 4), the backend returns HTTP 503 Service Unavailable and the teacher is advised to retry.

3.4.6.6 Post-conditions

- The AI Provider's suggested grade and feedback are stored as a draft record in PostgreSQL.
- The teacher has reviewed and confirmed or modified the suggested grade.
- The finalised grade is stored in PostgreSQL with status 'released'.
- The student is notified of the released grade via the Notification Service.

3.4.6.7 Sequence Diagram

Figure below illustrates the complete interaction sequence for the use case described above.

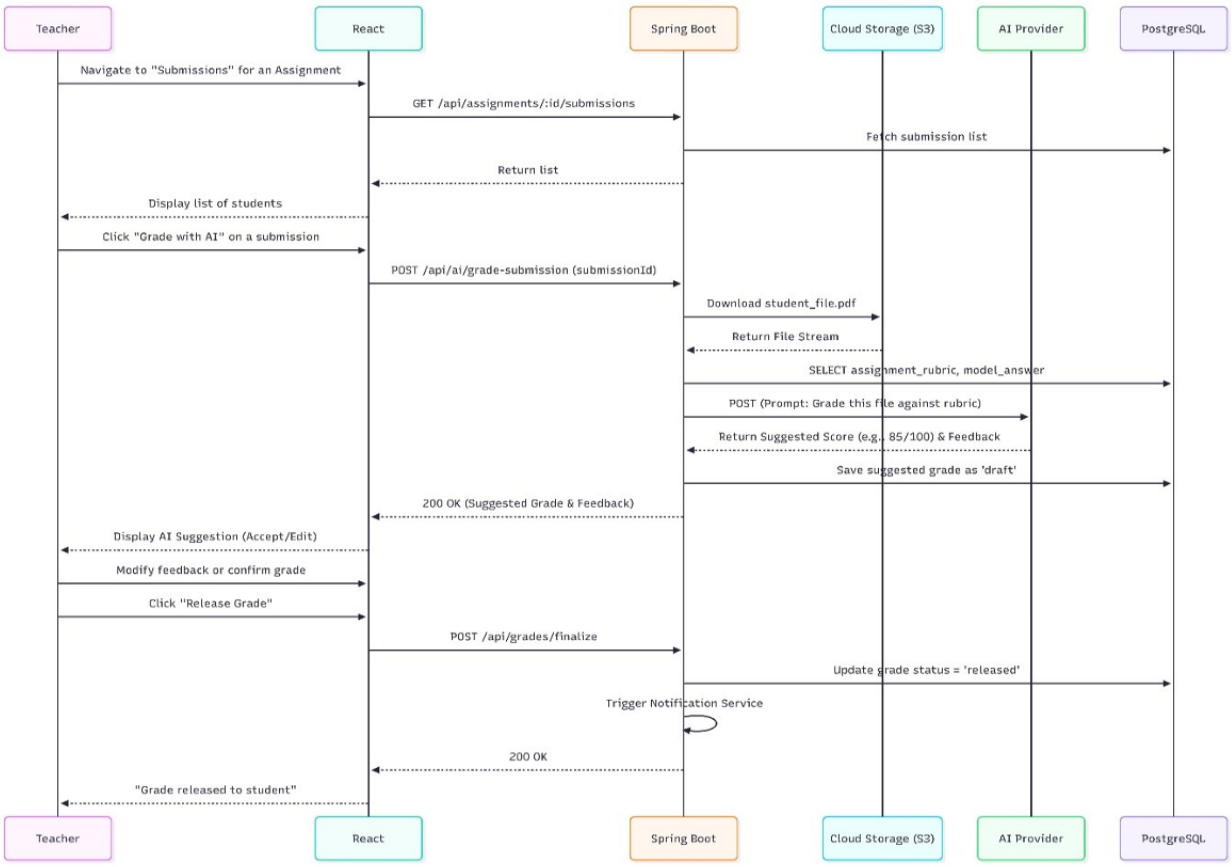


Figure 3.21: Sequence Diagram — Teacher Grades Assignment with AI Assistance

3.4.7 Real-time Messaging between Student and Teacher

This use case models real-time bidirectional messaging between a student and a teacher using WebSocket-based communication. Messages are transmitted over a persistent WebSocket connection for real-time delivery and persisted in MongoDB for durability. The sequence covers three scenarios: (1) real-time exchange when both participants are online, (2) offline message queuing and email digest notification when the teacher is unavailable, and (3) file attachment upload with integrated virus scanning. Unread message counts are maintained in Redis for efficient retrieval. The use case leverages the Communication Service microservice for all REST-based conversation management operations.

3.4.7.1 Use Case Details

Use Case ID	UC-07
Use Case Name	Real-time Messaging between Student and Teacher
Primary Actor	Student and Teacher
Supporting Actors	Student Frontend, Teacher Frontend, WebSocket Server, REST API (Communication Service), Redis Cache, MongoDB, Email Service
Trigger	The student opens a chat session with a teacher from the messaging interface.

3.22: Use Case Details — Real-time Messaging between Student and Teacher

3.4.7.2 Actors and System Components

Participant	Type	Responsibility
Student	Primary Actor	Initiates the chat session, types and sends messages, and sends file attachments.
Teacher	Supporting Actor	Receives and responds to messages in real time when online, or retrieves queued messages upon next login.

Student Frontend	Frontend Component	Renders the student's chat interface, initiates conversation retrieval, and sends messages and attachments via WebSocket and REST.
Teacher Frontend	Frontend Component	Displays real-time incoming messages, reply interface, and unread message counts.
WebSocket Server	Real-time Messaging	Maintains persistent WebSocket connections with online clients. Routes messages between student and teacher in real time.
REST API (Comm. Svc.)	Application Server	Manages conversation retrieval, offline message queuing, file upload, virus scanning, and unread message fetching via REST endpoints.
Redis Cache	Cache Layer	Stores and tracks unread message counts per user for fast retrieval. Queues offline messages pending delivery.
MongoDB	Document Database	Persists all messages, conversation records, and file attachment references as documents.
Email Service	External Integration	Dispatches email digest notifications to offline users informing them of pending unread messages.

3.23:Actors and Components — Real-time Messaging between Student and Teacher

3.4.7.3 Pre-conditions

- Both the student and teacher are registered, authenticated users.
- The WebSocket Server is operational and accepts connections from authenticated clients.
- The MongoDB Communication database and Redis cache are operational.
- The Email Service is configured for offline notification delivery.

3.4.7.4 Main Success Flow

37. The student opens the chat interface and selects a teacher to message. The Student Frontend sends GET /api/conversations to the REST API (Communication Service).
38. The Communication Service fetches the list of recent conversations from MongoDB and returns the conversation list. The Student Frontend renders the chat interface.
39. The student types a message and sends it. The Student Frontend transmits the message to the WebSocket Server via a WebSocket send event.
40. The WebSocket Server persists the message as a document in MongoDB and updates the teacher's unread message count in Redis.
41. If the teacher is online: the WebSocket Server pushes the message in real time to the Teacher Frontend. The Teacher Frontend displays the message with a notification indicator. The teacher types a reply and sends it via WebSocket. The WebSocket Server delivers the reply to the Student Frontend, which displays the teacher's response.
42. If the teacher is offline: the REST API queues the message in Redis as an offline message. The Email Service dispatches an email digest notification to the teacher's registered email address with a summary of unread messages.
43. When the teacher later logs in, the Teacher Frontend sends GET /api/unread-messages to the REST API. The Communication Service fetches the queued offline messages from MongoDB and returns them. The Teacher Frontend displays the unread messages.
44. The student sends a file attachment. The Student Frontend submits POST /api/messages/upload to the REST API.
45. The REST API performs an automated virus scan on the uploaded file content. Upon a clean scan result, the file is stored and a file URL is returned to the Student Frontend.
46. The Student Frontend sends a message containing the file attachment URL and metadata via WebSocket. The WebSocket Server delivers the attachment message to the Teacher Frontend.
47. The Teacher Frontend displays the message with a downloadable file link.

3.4.7.5 Alternative and Exception Flows

If the WebSocket connection drops during an active session, the frontend automatically attempts reconnection with exponential backoff. Messages sent during a reconnection window are queued locally in the client and retransmitted upon successful reconnection. If the virus scan in Step 9 detects a malicious file, the upload is rejected, the file is not stored, and HTTP 422 Unprocessable Entity is returned to the Student Frontend with an appropriate warning message.

3.4.7.6 Post-conditions

- All exchanged messages are persisted in MongoDB with timestamps and sender metadata.
- Unread message counts are tracked and updated in Redis.
- Online messages are delivered in real time via WebSocket. Offline messages are queued and delivered upon the teacher's next login.
- File attachments are scanned, stored, and delivered as downloadable links within the chat interface.

3.4.7.7 Sequence Diagram

Figure 7 below illustrates the complete interaction sequence for the use case described above.

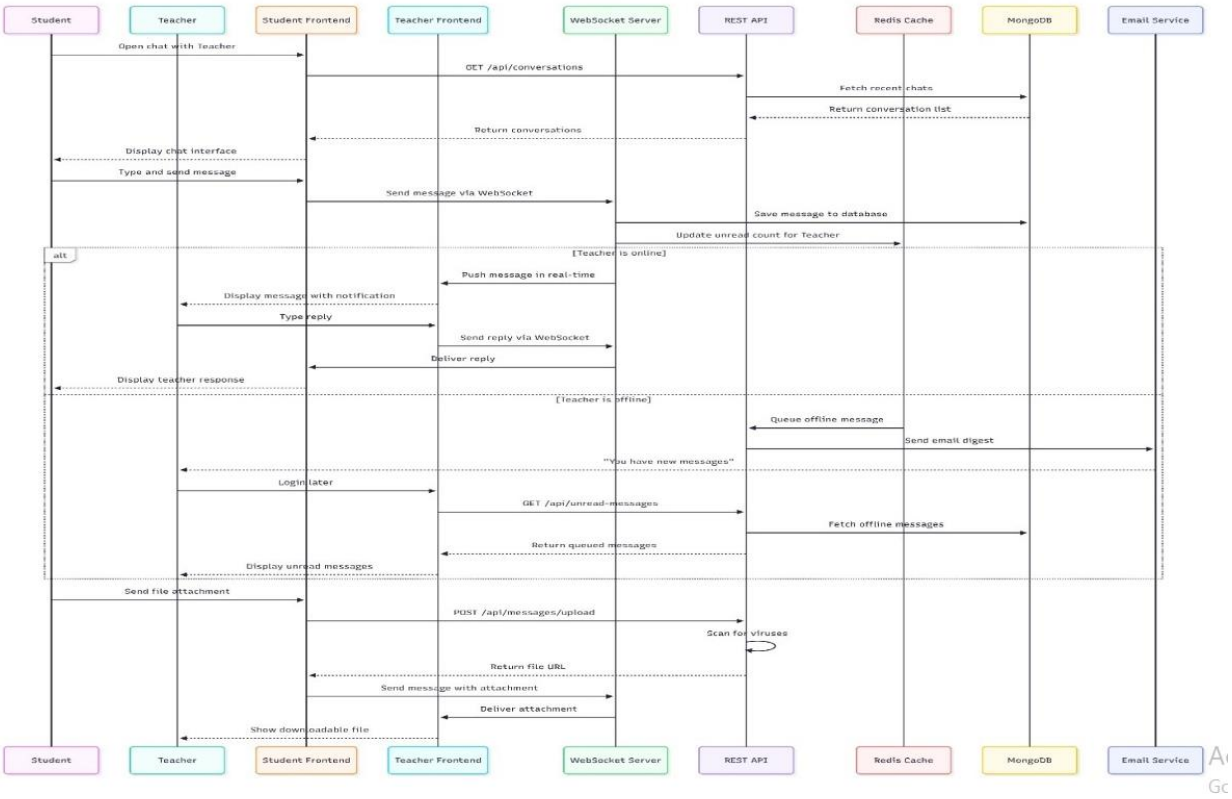


Figure 3.22: Sequence Diagram — Real-time Messaging between Student and Teacher

3.4.8 Teacher Posts Class-wide Announcement

This use case describes the workflow by which a teacher publishes a class-wide announcement to all students enrolled in a specific course. The announcement is persisted in the database and delivered through two parallel notification channels: transactional email notifications sent via the Email Service to all enrolled students, and real-time in-app push notifications broadcast to all students with active WebSocket sessions. Notification read-receipt tracking is implemented to record which students have viewed the announcement. The student-side interaction — including receiving the banner notification, clicking to read the full text, and the server-side read-receipt recording — is also captured within this use case.

3.4.8.1 Use Case Details

Use Case ID	UC-08
Use Case Name	Teacher Posts Class-wide Announcement
Primary Actor	Teacher
Supporting Actors	React (Frontend), Spring Boot (Backend), Database, Email Service, WebSocket Server, Students
Trigger	The teacher opens the course management interface and clicks 'Post Announcement'.

3.24: Use Case Details — Teacher Posts Class-wide Announcement

3.4.8.2 Actors and System Components

Participant	Type	Responsibility
Teacher	Primary Actor	Authors the announcement, configures its priority and email option, and submits it for distribution.
React (Frontend)	Frontend Component	Renders the announcement editor modal, submits the announcement to the backend, and displays the confirmation response.

Spring Boot Backend	Application Server	Persists the announcement, retrieves the enrolled student list, orchestrates parallel email and WebSocket notifications, and records read receipts.
Database	Relational Database	Persists announcement records, enrolled student associations, and per-student read-receipt records.
Email Service	External Integration	Dispatches individual announcement email notifications to each enrolled student.
WebSocket Server	Real-time Messaging	Broadcasts in-app popup notifications to all students with active WebSocket sessions.
Student	Recipient	Receives the announcement via email and/or in-app notification, and reads the full announcement content.

3.25: Actors and Components — Teacher Posts Class-wide Announcement

3.4.8.3 Pre-conditions

- The teacher is authenticated and holds the Instructor role.
- The course to which the announcement is directed has at least one enrolled student.
- The Email Service and WebSocket Server are operational.
- The database is operational and accessible.

3.4.8.4 Main Success Flow

48. The teacher opens the course management interface and clicks 'Post Announcement'. The React frontend opens an announcement editor modal.
49. The teacher types the announcement title and content, selects a priority level ('High'), checks the 'Send email to all' option, and clicks 'Post'.
50. The React frontend submits POST /api/announcements to the Spring Boot backend, carrying the title, content, priority, and email notification flag.

51. The backend inserts the announcement record into the database with a timestamp and the teacher's ID.
52. The backend queries the database to retrieve the complete list of students enrolled in the course.
53. In parallel, the backend executes two notification channels simultaneously: (a) Email notifications — for each enrolled student, an announcement email is dispatched via the Email Service; and (b) In-app notifications — for each student with an active WebSocket session, the announcement is broadcast via the WebSocket Server as a real-time popup notification.
54. The backend stores notification read-receipt records in the database, initialising each student's read status to 'unread'.
55. The backend returns an 'Announcement posted' confirmation to the React frontend, which displays 'Announcement sent to [n] students'.
56. An online student's client receives the WebSocket broadcast and displays an announcement banner notification in the student's interface.
57. The student clicks the banner to read the full announcement. The Student Frontend sends GET /api/announcements/101 to the backend.
58. The backend marks the announcement as 'read' for this specific student in the database and returns the full announcement content.
59. The Student Frontend renders and displays the complete announcement text.

3.4.8.5 Alternative and Exception Flows

If the Email Service is unavailable during Step 6a, the backend logs the failure and queues the email dispatch for retry without failing the announcement creation. Announcement creation and in-app WebSocket notifications proceed independently. If no students are currently online (all WebSocket sessions are inactive), Step 6b produces no immediate notifications; however, announcement banners are displayed to each student upon their next login via a pending notification query.

3.4.8.6 Post-conditions

- The announcement record is persisted in the database with the teacher's authorship and priority metadata.

- Announcement email notifications are dispatched to all enrolled students.
- In-app WebSocket notifications are broadcast to all currently online students.
- Per-student notification read-receipt records are initialised in the database.
- Students who click the announcement banner have their read status updated to 'read' in the database.

3.4.8.7 Sequence Diagram

Figure 8 below illustrates the complete interaction sequence for the use case described above.

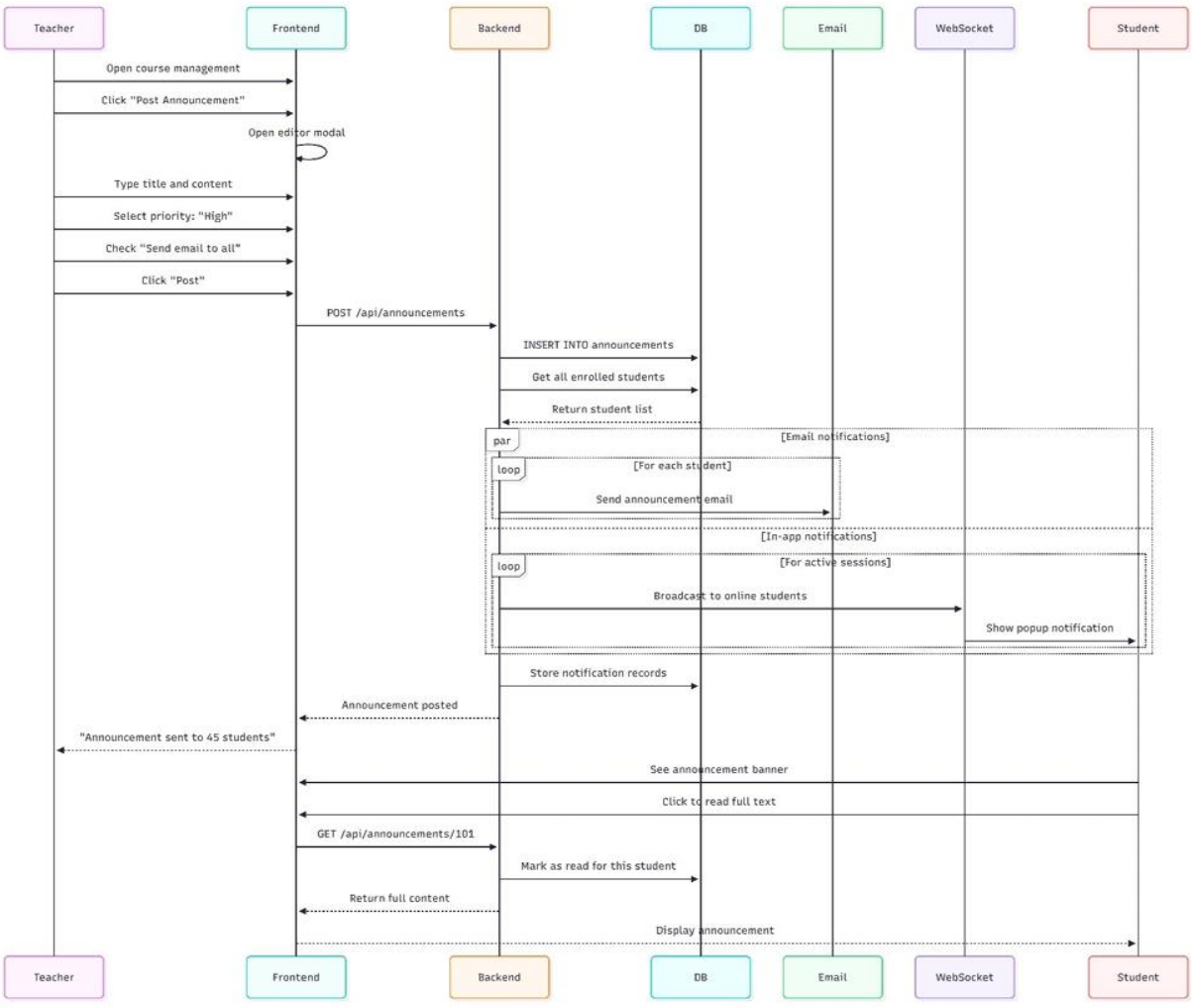


Figure 3.23: Sequence Diagram — Teacher Posts Class-wide Announcement

3.4.9 Student Interacts with AI Help Chatbot

This use case models a student's multi-turn interaction with the AI Help Chatbot, which is implemented using a Retrieval-Augmented Generation (RAG) architecture. Upon receiving a student query, the Spring Boot AI Service performs a semantic similarity search against the Vector Database, retrieving the most contextually relevant text chunks from the student's enrolled course materials. These retrieved chunks are provided as grounding context in the prompt sent to the OpenAI AI Provider, ensuring that chatbot responses are accurate, relevant, and constrained to the content of the enrolled courses. AI responses are streamed back to the client using Server-Sent Events (SSE) to produce a real-time typing effect. Conversation history is maintained across turns to support follow-up queries, and the session log is optionally persisted in MongoDB.

3.4.9.1 Use Case Details

Use Case ID	UC-09
Use Case Name	Student Interacts with AI Help Chatbot
Primary Actor	Student
Supporting Actors	React (Frontend), Spring Boot (AI Service), Vector Database (Pinecone/PGVector), AI Provider (OpenAI), MongoDB
Trigger	The student opens the AI Help Chatbot interface and submits a learning-related query.

3.26: Use Case Details — Student Interacts with AI Help Chatbot

3.4.9.2 Actors and System Components

Participant	Type	Responsibility
Student	Primary Actor	Initiates the chatbot session, submits learning queries, and requests follow-up simplifications or clarifications.
React (Frontend)	Frontend Component	Renders the chatbot interface, displays streaming response text with a typing

		animation, and maintains the conversation history for multi-turn context.
Spring Boot AI Service	Application Server	Authenticates the user, fetches enrolled courses, constructs the RAG prompt, communicates with the Vector Database and AI Provider, and streams the response.
Vector Database	Vector Search Engine	Stores vectorised embeddings of course material text chunks. Executes semantic similarity searches against student queries filtered by enrolled course IDs.
AI Provider (OpenAI)	External AI Service	Processes the RAG prompt (system instruction + retrieved context + user query) and generates a contextually grounded, streamed answer.
MongoDB	Document Database	Optionally persists the complete chat session log for conversation history retrieval and analytics.

3.27: Actors and Components — Student Interacts with AI Help Chatbot

3.4.9.3 Pre-conditions

- The student is authenticated with the Student role and is enrolled in at least one course.
- Course materials for the student's enrolled courses have been vectorised and indexed in the Vector Database.
- The OpenAI AI Provider is configured and operational.
- The Vector Database is operational and accessible.

3.4.9.4 Main Success Flow

60. The student opens the AI Help Chatbot and types a query (e.g., 'Explain Chapter 3'). The React frontend sends the query to the Spring Boot AI Service via WebSocket or POST /api/chat

(query).The Spring Boot AI Service authenticates the student and fetches the list of courses in which the student is enrolled.

61. The AI Service executes a semantic similarity search against the Vector Database using the student's query text as the search vector, filtered by the enrolled course IDs: similarity_search('Explain Chapter 3', filters: {courseId}).
62. The Vector Database returns the most semantically relevant text chunks from the indexed course materials as context for the query.
63. The AI Service constructs a structured RAG prompt comprising: a system instruction constraining the model to use only the provided context, the retrieved text chunks as grounding context, and the student's original query.
64. The AI Service submits the prompt to the OpenAI AI Provider via POST. The AI Provider processes the prompt and generates an answer, streaming the response back in chunks.
65. The AI Service relays the streaming response chunks to the React frontend via Server-Sent Events (SSE). The React frontend displays the incoming text with a real-time typing animation.
66. The student reads the response and submits a follow-up query ('I don't understand, simplify it'). The React frontend sends POST /api/chat with the follow-up query and the full conversation history.
67. The AI Service constructs a new prompt incorporating the conversation history and the follow-up instruction (e.g., 'Explain like I am 10'), and submits it to the AI Provider.
68. The AI Provider returns a simplified answer. The AI Service returns the simplified response to the React frontend, which renders the simplified explanation.
69. Optionally, the complete chat session log — including all turns, queries, and responses — is saved to MongoDB for conversation history retrieval and analytics purposes.

3.4.9.5 Alternative and Exception Flows

If the Vector Database returns no relevant context for the student's query (e.g., the topic falls outside the indexed course materials), the AI Service constructs a prompt instructing the AI Provider to inform the student that the topic is not covered in the enrolled course content, rather than generating an answer from general knowledge. If the OpenAI AI Provider is unavailable, the

AI Service returns HTTP 503 Service Unavailable and the student is informed that the chatbot is temporarily unavailable.

3.4.9.6 Post-conditions

- The student receives a contextually grounded, streamed AI-generated response based on their enrolled course materials.
- Follow-up queries are processed with full conversation history context, enabling coherent multi-turn dialogue.
- The chat session log is optionally persisted in MongoDB for session history retrieval.

3.4.9.7 Sequence Diagram

Figure 9 below illustrates the complete interaction sequence for the use case described above.

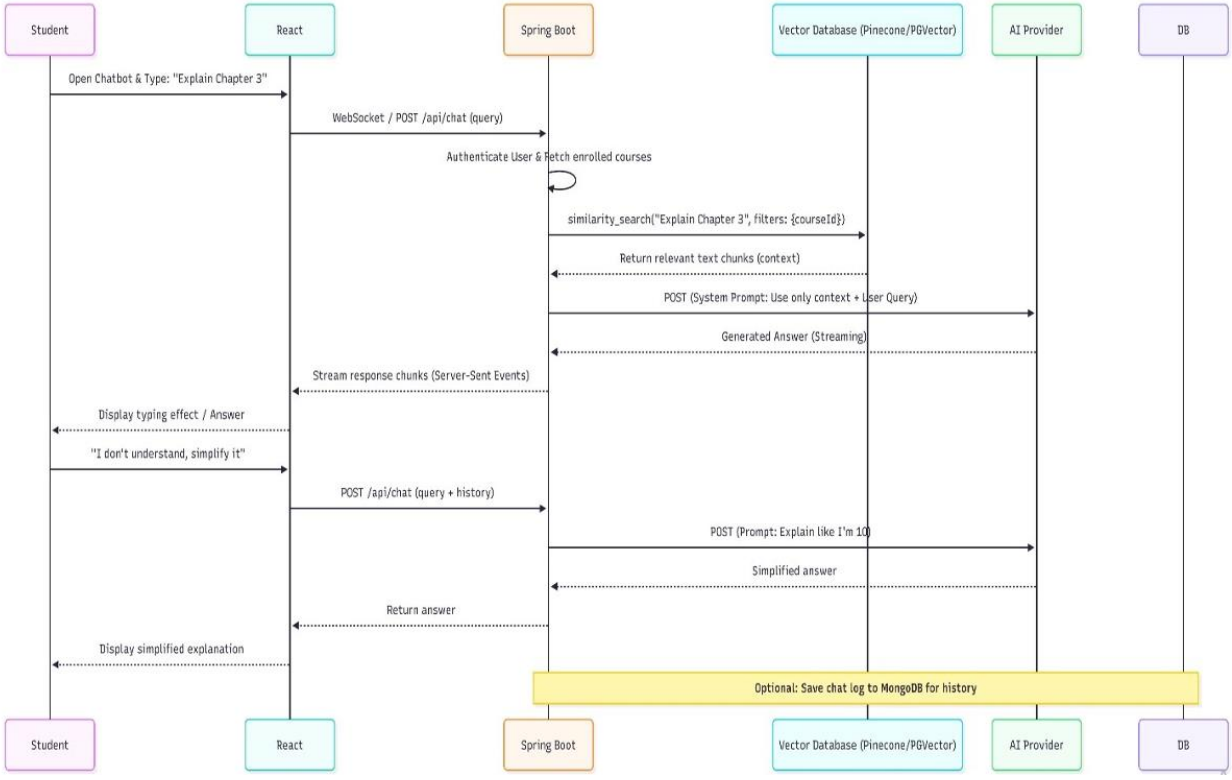


Figure 3.24: Sequence Diagram — Student Interacts with AI Help Chatbot

3.4.10 Student Views Performance Dashboard with Analytics

This use case models the retrieval, aggregation, analysis, and visualisation of a student's personalised learning performance metrics. The Analytics Service first checks the Redis cache for a pre-computed dashboard dataset. On a cache miss, it queries PostgreSQL for three categories of data: assessment grades and scores, course completion progress, and lesson engagement metrics. The aggregated data is forwarded to the AI Analytics Engine, which calculates score averages, generates historical trend lines, identifies performance patterns, and produces personalised recommendations. The computed analytics dataset is cached in Redis for one hour before being returned to the React Dashboard for chart rendering. The student may additionally filter analytics by subject area and export a downloadable PDF performance report.

3.4.10.1 Use Case Details

Use Case ID	UC-10
Use Case Name	Student Views Performance Dashboard with Analytics
Primary Actor	Student
Supporting Actors	React Dashboard, Spring Boot (Analytics Service), AI Analytics Engine, PostgreSQL, Redis Cache
Trigger	The student clicks 'My Performance' from the navigation menu to open the personalised performance dashboard.

3.28: Use Case Details — Student Views Performance Dashboard with Analytics

3.4.10.2 Actors and System Components

Participant	Type	Responsibility
Student	Primary Actor	Navigates to the performance dashboard, filters analytics by subject, and exports the PDF report.
React Dashboard	Frontend Component	Renders interactive charts and performance indicators, handles filter submissions, and triggers the PDF download.

Spring Boot Analytics Service	Application Server	Checks the Redis cache, queries PostgreSQL for performance data, forwards aggregated data to the AI Analytics Engine, caches the result, and generates the PDF export.
AI Analytics Engine	Internal Service	Calculates performance averages, generates trend lines, analyses performance patterns, and produces personalised learning recommendations.
PostgreSQL	Relational Database	Stores student assessment scores, course completion records, and lesson engagement event data.
Redis Cache	Cache Layer	Caches pre-computed dashboard datasets with a one-hour TTL to reduce database load and improve dashboard response time.

3.29: Actors and Components — Student Views Performance Dashboard with Analytics

3.4.10.3 Pre-conditions

- The student is authenticated and holds the Student role.
- The student has completed at least one assessment or lesson interaction that generates recordable performance data.
- PostgreSQL and Redis are operational.
- The AI Analytics Engine is running and accessible by the Analytics Service.

3.4.10.4 Main Success Flow

70. The student clicks 'My Performance'. The React Dashboard sends GET /api/analytics/dashboard to the Spring Boot Analytics Service.
71. The Analytics Service checks the Redis cache for an existing pre-computed dashboard dataset for this student.
72. If a cache entry is present (cache hit): the cached analytics dataset is returned directly to the React Dashboard, bypassing all database and AI processing steps. The React Dashboard renders the charts.

73. If no cache entry is present (cache miss): the Analytics Service queries PostgreSQL for: (a) the student's assessment grades and scores; (b) course completion progress data; and (c) lesson view counts and engagement time metrics.
74. The aggregated dataset is forwarded to the AI Analytics Engine. The engine calculates score averages, generates historical trend lines, and analyses performance patterns.
75. The AI Analytics Engine returns insights and personalised learning recommendations to the Analytics Service.
76. The Analytics Service stores the complete analytics dataset in Redis with a one-hour time-to-live (TTL).
77. The Analytics Service returns the complete analytics dataset to the React Dashboard. The dashboard renders all charts, trend lines, performance indicators, and AI-generated recommendations.
78. The student applies a subject filter (e.g., 'Mathematics'). The React Dashboard submits GET /api/analytics/filter?subject=Math to the Analytics Service.
79. The Analytics Service queries filtered data from PostgreSQL for the specified subject and returns the filtered results to the React Dashboard, which re-renders the charts with the filtered dataset.
80. The student clicks 'Export Report'. The React Dashboard submits GET /api/analytics/export/pdf to the Analytics Service.
81. The Analytics Service generates a structured PDF performance report containing the student's grade history, trend analysis, and recommendations.
82. The Analytics Service returns the PDF file to the React Dashboard, which triggers a browser download, saving the report to the student's device.

3.4.10.5 Alternative and Exception Flows

If the AI Analytics Engine is unavailable (Step 5), the Analytics Service returns the raw aggregated data without AI-generated insights or recommendations. The dashboard is rendered with the available data, and a note is displayed informing the student that personalised insights are temporarily unavailable. If no performance data exists for the student (newly enrolled with no

activity), the Analytics Service returns an empty dataset and the React Dashboard displays an appropriate empty state message.

3.4.10.6 Post-conditions

- The student's performance dashboard is rendered with charts showing grades, completion rates, lesson engagement, and trend analysis.
- The computed analytics dataset is cached in Redis with a one-hour TTL.
- Filtered analytics results are presented upon subject selection.
- A downloadable PDF performance report is generated and saved to the student's device upon request.

3.4.10.7 Sequence Diagram

Figure 10 below illustrates the complete interaction sequence for the use case described above.

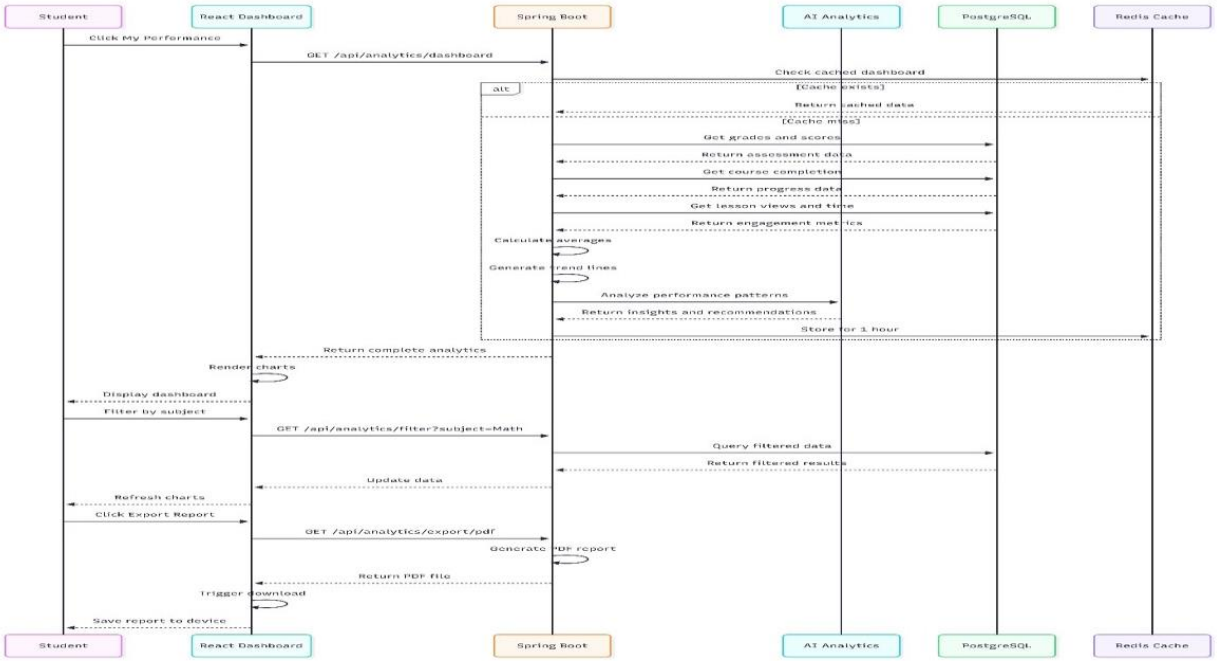


Figure 3.25: Sequence Diagram — Student Views Performance Dashboard with Analytics

3.4.11 Student Takes Personal Notes Within Platform

This use case describes the complete lifecycle of the personal note-taking feature within the platform's video lesson interface. The feature enables students to create timestamped notes during video playback — associating each note with a specific position in the lesson — and to retrieve, organise, navigate, edit, and delete their notes. Note timestamps are interactive: clicking a timestamp in the note list causes the video player to seek directly to the associated position in the lesson, allowing the student to immediately revisit the content at which the note was recorded. Note deletion is implemented as a soft delete, preserving the record in the database while removing it from the student's view. All note data is persisted in the student_notes table in PostgreSQL.

3.4.11.1 Use Case Details

Use Case ID	UC-11
Use Case Name	Student Takes Personal Notes Within Platform
Primary Actor	Student
Supporting Actors	React (Frontend), Spring Boot (Backend), PostgreSQL (Database)
Trigger	The student clicks the 'Take Notes' button while watching a video lesson.

3.30: Use Case Details — Student Takes Personal Notes Within Platform

3.4.11.2 Actors and System Components

Participant	Type	Responsibility
Student	Primary Actor	Creates, retrieves, edits, and deletes personal notes while interacting with the video lesson interface.
React (Frontend)	Frontend Component	Renders the notes sidebar, manages the note creation form, displays organised notes with clickable timestamps, and controls the video player seek action.

Spring Boot Backend	Application Server	Processes all note CRUD operations, handles the video seek-position lookup, and enforces per-student data isolation.
PostgreSQL	Relational Database	Persists all note records in the student_notes table, including content, lessonId, video timestamp, and soft-delete flags.

3.31: Actors and Components — Student Takes Personal Notes Within Platform

3.4.11.3 Pre-conditions

- The student is authenticated and holds the Student role.
- The student is enrolled in the course and is actively viewing a video lesson.
- The Spring Boot backend and PostgreSQL database are operational.

3.4.11.4 Main Success Flow

83. The student is watching a video lesson and clicks the 'Take Notes' button. The React frontend opens a notes sidebar adjacent to the video player.
84. The student types note content (e.g., 'Key concept: JWT tokens...') and clicks 'Save Note'. The React frontend submits POST /api/notes to the Spring Boot backend, carrying the note content, the current lessonId, and the current video playback timestamp.
85. The backend inserts a new record into the student_notes table in PostgreSQL, associating the note with the authenticated student's ID, the lessonId, and the captured timestamp. A 'Note saved' confirmation is returned.
86. The React frontend displays a confirmation message: 'Note saved at 02:35'.
87. Later, the student opens the 'My Notes' section. The React frontend sends GET /api/notes?lessonId=789 to the backend.
88. The backend queries PostgreSQL: SELECT notes WHERE student_id = X, retrieving all non-deleted notes for the authenticated student associated with the specified lesson, grouped by lesson. The results are returned with note content and timestamps.
89. The React frontend displays the organised notes list. Each note entry displays the note content alongside a clickable timestamp link.

90. The student clicks the timestamp link '02:35'. The React frontend sends GET /api/lessons/789/seek?time=155 to the backend.
91. The backend returns the video seek position. The React frontend instructs the video player to seek to the 02:35 position and resume playback from that point.
92. The student clicks 'Edit Note' on a note. The React frontend presents an editable text field. The student modifies the content and saves the changes by submitting PUT /api/notes/456 to the backend.
93. The backend updates the note content in PostgreSQL and records the updated_at timestamp. A 'Note updated' confirmation is returned. The React frontend displays 'Changes saved'.
94. The student clicks 'Delete Note'. The React frontend submits DELETE /api/notes/456 to the backend.
95. The backend performs a soft delete on the note record in PostgreSQL — setting a deleted_at timestamp and a deleted flag — without physically removing the record. A 'Note removed' confirmation is returned. The React frontend removes the note from the displayed list.

3.4.11.5 Alternative and Exception Flows

If the student attempts to save a note with empty content, the React frontend enforces client-side validation and prevents the POST /api/notes request from being submitted, displaying an inline validation message. If the requested lessonId in Step 5 does not correspond to any notes for the authenticated student, PostgreSQL returns an empty result set and the React frontend displays an empty-state message: 'No notes found for this lesson'.

3.4.11.6 Post-conditions

- A new note record is persisted in the student_notes table with the note content, lessonId, and video timestamp.
- Existing notes are retrievable, grouped by lesson, with clickable timestamp navigation.
- Clicking a note timestamp causes the video player to seek to the corresponding playback position.
- Updated note content is persisted with a new updated_at timestamp in PostgreSQL.
- Deleted notes are soft-deleted in PostgreSQL with a deleted_at timestamp; the physical record is preserved but excluded from the student's view.

3.4.11.7 Sequence Diagram

Figure 11 below illustrates the complete interaction sequence for the use case described above.

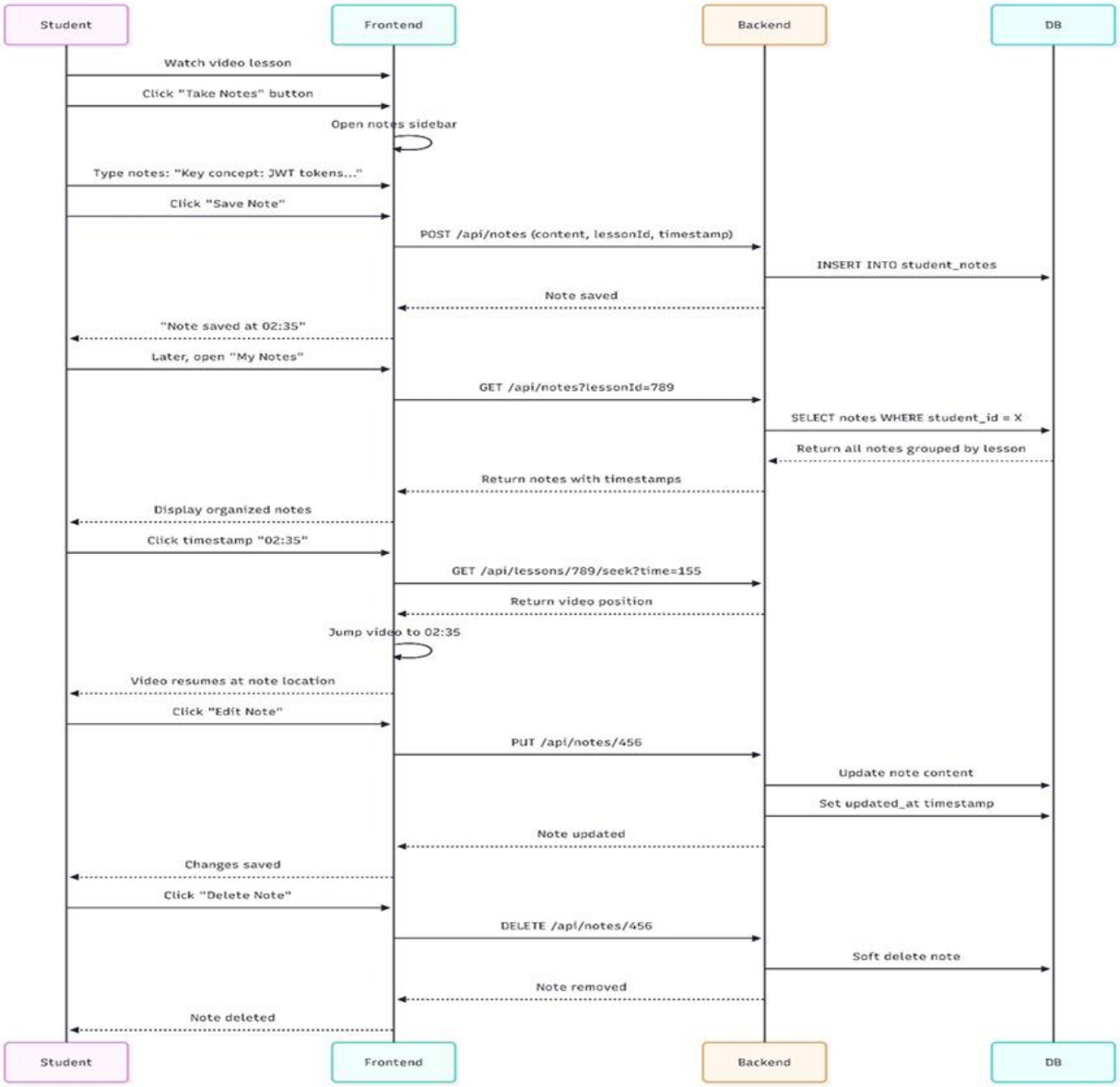


Figure 3.26: Sequence Diagram — Student Takes Personal Notes Within Platform

3.5 UI/UX Design

This chapter presents the user interface and user experience (UI/UX) design specifications for the Mentify AI academic platform. The design artefacts documented herein include screen mock-ups for all key user interfaces, user flow and navigation diagrams for primary system interactions, and a detailed accessibility compliance strategy aligned with the Web Content Accessibility Guidelines (WCAG) 2.1 standard.

3.5.1 Design Overview

The Mentify AI platform is designed to serve three primary user groups: students, teachers (faculty), and administrators. The interface adheres to a clean, minimal aesthetic with a consistent colour palette, typography, and component library applied across all screens. The design system employs a predominantly white and light-grey background with indigo-blue (#2E4A82) and purple (#6C3FC5) as primary accent colours, ensuring visual hierarchy and clear call-to-action affordances. The platform is structured around two distinct branded experiences: Mentify AI (student and faculty portal) and Cognitive Gallery (course discovery and learning explorer). Both experiences share a unified navigation paradigm consisting of a fixed left-hand sidebar navigation and a top header bar, enabling predictable and efficient wayfinding.

The following table provides a summary of all key screens documented in this chapter.

No.	Screen	Description
1	Student Dashboard	Displays course progress, AI recommendations, upcoming deadlines, and recent performance metrics.
2	Quiz / Assessment View	Presents multiple-choice questions with a timer, AI hint panel, and session summary sidebar.
3	Collaborative Chat (Grade Channel)	Group messaging interface with AI-generated insights, linked tools, and a resource desk panel.
4	Lesson / Video View	Video player with real-time AI summary, suggested questions, and module navigation.
5	Course Detail Page	Course overview, curriculum outline, pricing, and an AI learning insight panel.
6	Assignment Submission	Assignment instructions, file upload, integrity check, and AI writing support.

7	Quiz Generator (Faculty)	Faculty tool to configure and generate AI-synthesised multiple-choice questions.
8	AI Learning Assistant	Conversational AI tutor with contextual learning tips and past inquiry history.
9	Cognitive Gallery Explorer	AI-curated course recommendations, trending courses, and skill boosters
10	User Management (Admin)	Administrative panel for managing faculty, student, and support accounts with AI role auditor.

3.32: Summary of Key User Interface Screens

3.5.2 Screen Mock-ups for Key User Interfaces

The following subsections present high-fidelity screen mock-ups for each key interface within the Mentify AI platform. Each mock-up is accompanied by a descriptive annotation identifying its purpose, constituent UI components, and intended user interactions.

3.5.2.1 Student Dashboard

The Student Dashboard serves as the primary landing screen upon successful authentication for student users. It consolidates personalised academic information into a single, scannable view. The left sidebar provides persistent navigation to My Courses, AI Insights, Faculty Hub, and Settings. The central panel displays course progress cards for enrolled subjects (Mathematics, Science, History), each presenting the completion percentage and current grade. An AI Recommendation banner, driven by the platform's learning analytics engine, surfaces contextual study suggestions based on detected study patterns. The right panel surfaces upcoming deadlines and recent performance metrics, enabling students to monitor academic progress at a glance. A "New AI Session" call-to-action button is prominently positioned in the sidebar footer.

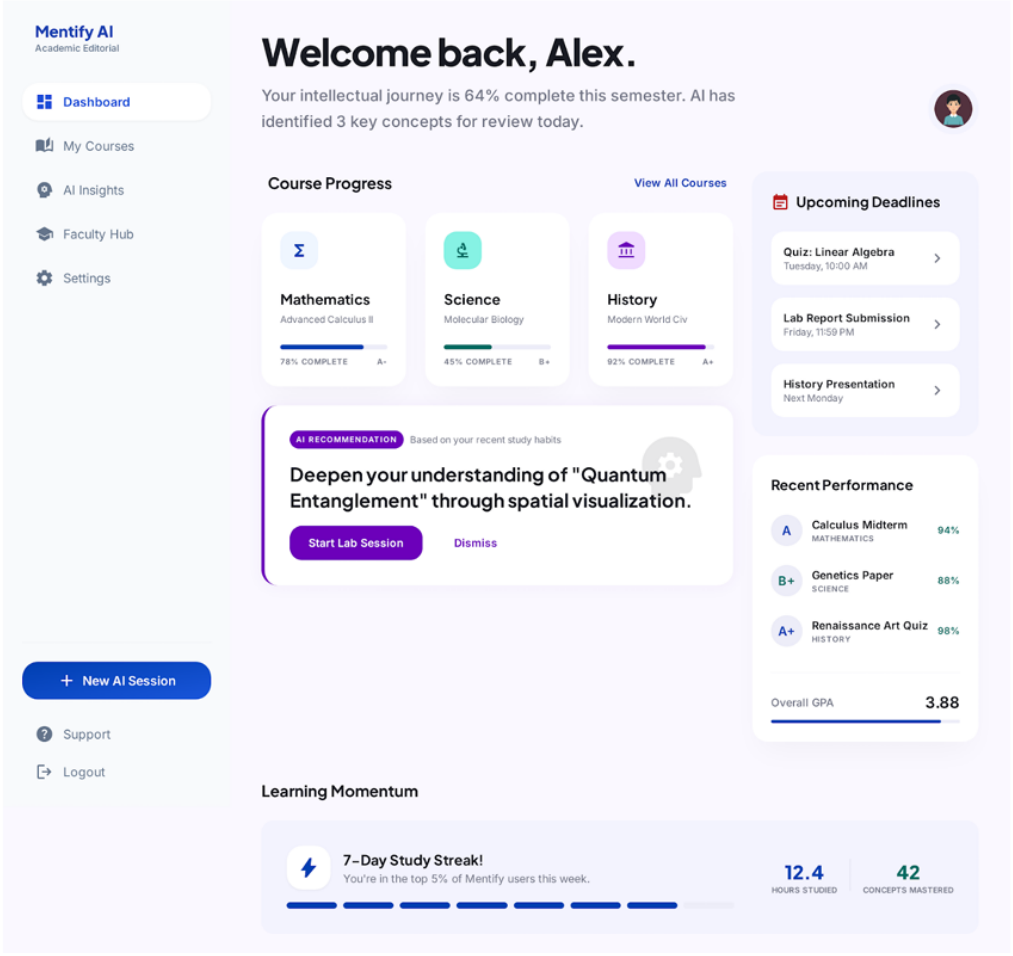


Figure 3.27: Student Dashboard — Mentify

3.5.2.2 Quiz and Assessment View

The Quiz and Assessment View presents students with interactive multiple-choice assessments within a timed, focused environment. A countdown timer and progress indicator (e.g., "Question 4 of 10") are displayed in the top navigation bar to support time awareness. Each question is rendered with four labelled answer options. The right panel provides an AI-powered hint functionality through the "Teacher Enabled Hint" feature, which offers Socratic guidance without directly revealing the answer. A session summary widget tracks correct responses and average time per question in real time. The "Submit Answer" and "Previous" navigation controls are positioned at the bottom of the screen for consistency.

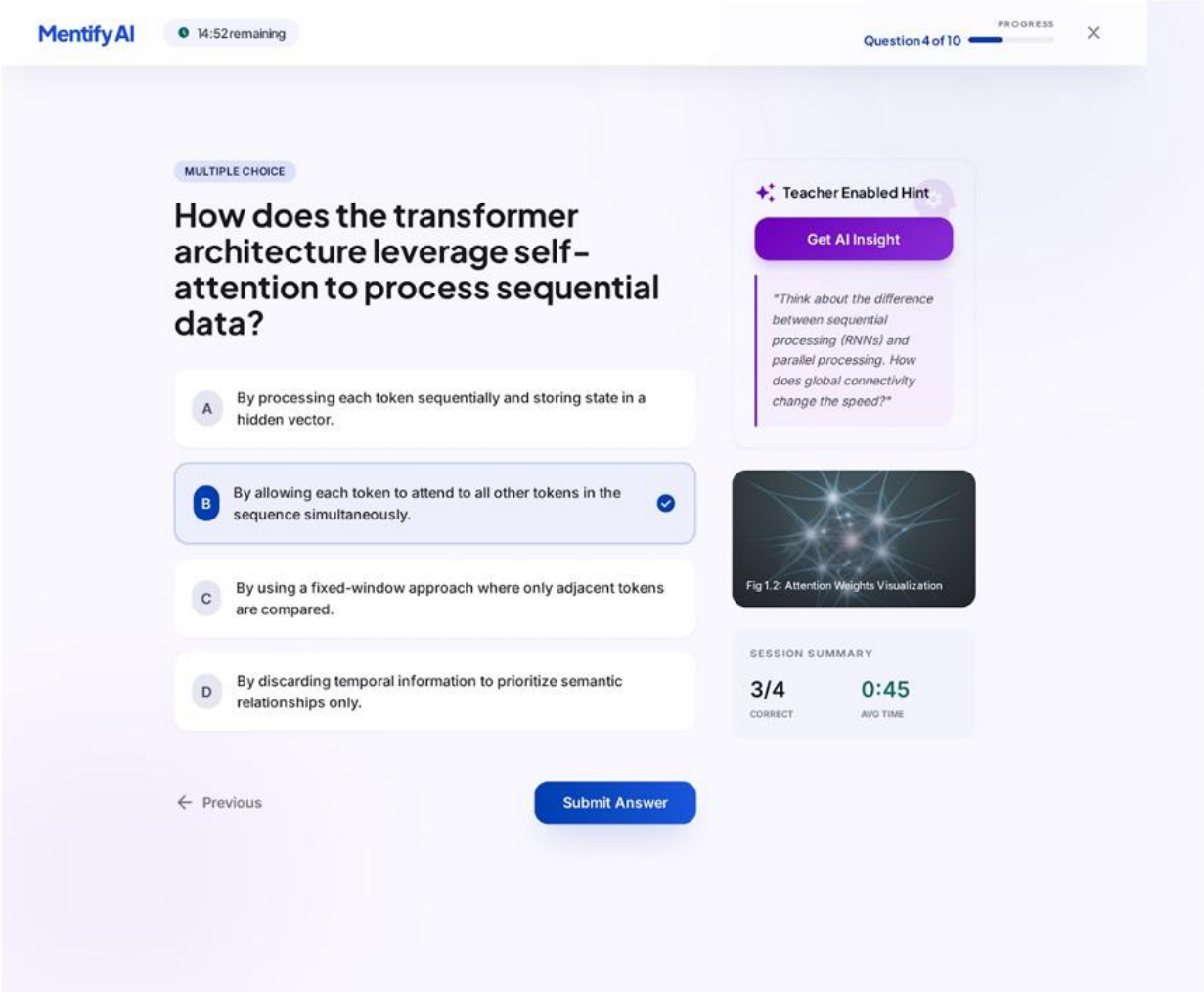


Figure 3.28: Quiz and Assessment View — Mentify

3.5.2.3 Collaborative Chat – Grade Channel

The Collaborative Chat interface enables peer-to-peer and group communication within grade-specific channels. The left sidebar provides access to Channels, Study Groups, Direct Messages, and Resources. The central panel renders a threaded message view with user avatars, timestamps, and emoji reaction support. A distinguishing feature is the integrated Mentify Insight widget, which monitors conversation context and proactively injects AI-generated study guidance, interactive tool links (e.g., Solar Scale Calculator), and lesson references directly into the chat thread. The right-hand Desk panel surfaces week's resources, assignment deadlines, a contextual practice quiz, and an online classmates list.

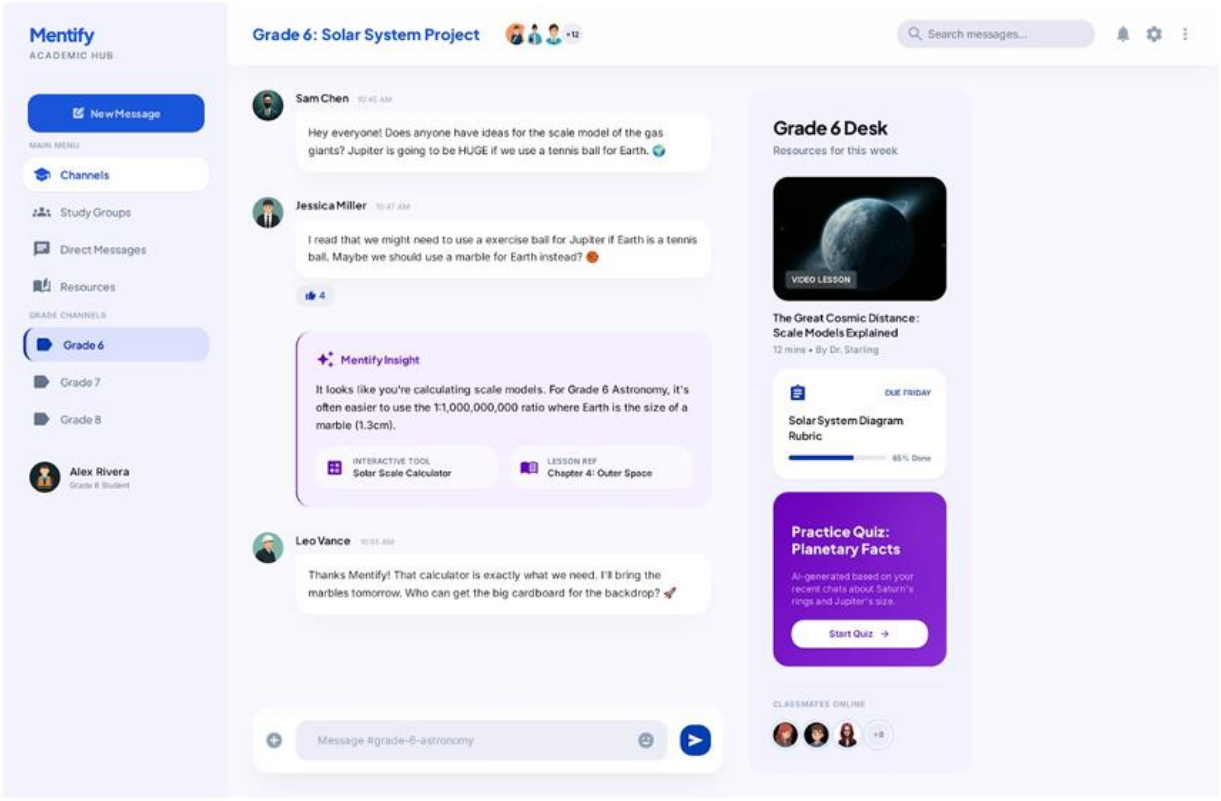


Figure 3.29: Collaborative Chat

3.5.2.4 Lesson and Video View

The Lesson and Video View provides an immersive content consumption experience for structured course materials. A full-width video player with a scrubbing timeline is positioned prominently in the upper content area. The lesson title, instructor details, difficulty level, and estimated duration are displayed beneath the player. Supplementary content includes a Key Formula panel and a Visual Insight sandbox link, enabling students to engage with interactive concept visualisations. A module navigation list at the bottom of the page tracks lesson completion status. An AI Assistant panel anchored to the right edge delivers real-time content summaries and suggested questions aligned with the currently playing segment.

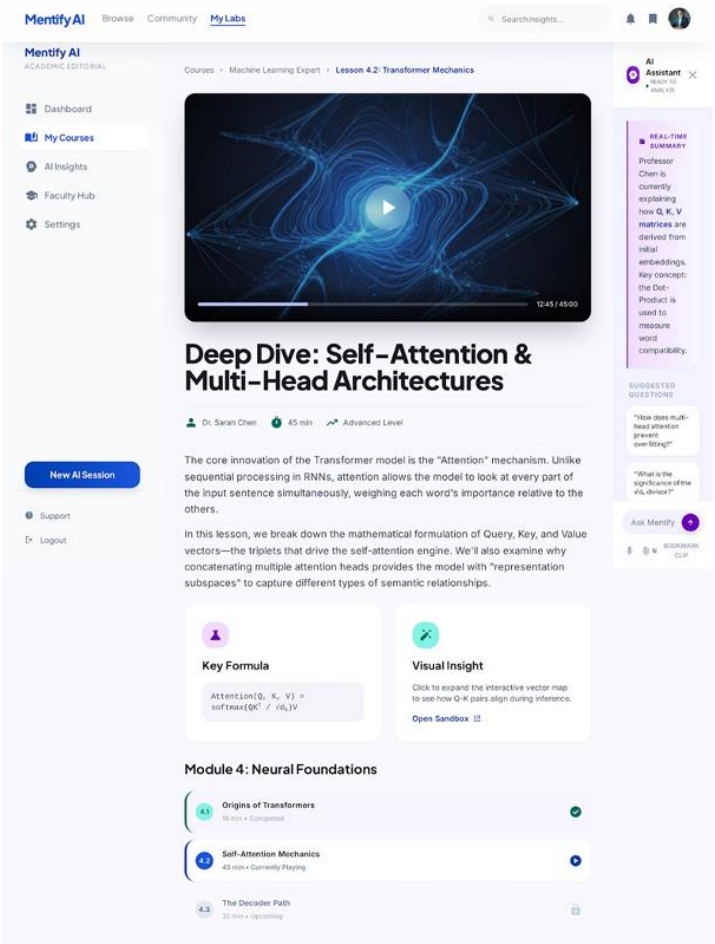


Figure 3.30: Lesson and Video View

3.5.2.5 Course Detail Page – Cognitive Gallery

The Course Detail Page within the Cognitive Gallery provides a structured overview of an individual course offering. The page displays the course title, instructor profile, average rating, and enrolment count. A Course Overview section describes the academic scope and learning outcomes. Four thematic content pillars are surfaced as icon-based feature cards. Pricing information, enrolment controls, and a 30-day money-back guarantee are presented in a sticky right-hand panel. An AI Learning Insight card provides a personalised mastery probability assessment based on the student's existing knowledge profile. The Course Curriculum accordion lists all modules and individual lessons with durations, and a Requirements and Target Audience section is provided to assist students in self-selection.

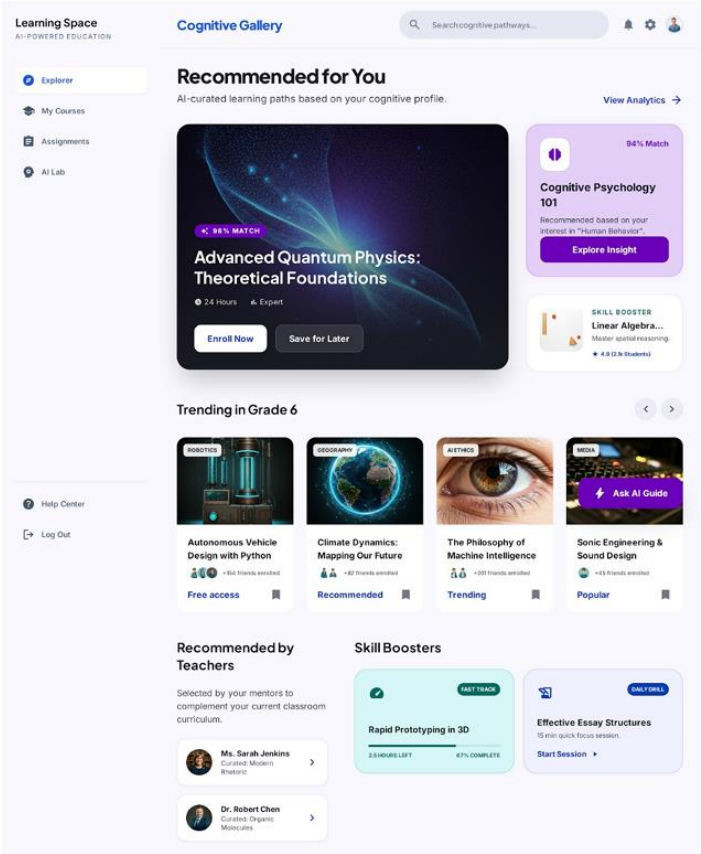


Figure 3.31: Course Detail Page

3.5.2.6 Assignment Submission

The Assignment Submission interface presents the full context of an active assignment alongside the tools required for submission. Assignment metadata including the title, due date, point value, and AI Assisted Grading status are displayed in the header. The Instructions panel provides a structured description of submission requirements. An AI Writing Support widget surfaces targeted feedback derived from prior drafts, linked to a Reference Gallery. The file upload area supports drag-and-drop and cloud drive integration (Google Drive, OneDrive). Uploaded files are listed with file size and upload timestamp. An Integrity Check panel requires the student to confirm original authorship and consent to plagiarism screening. The Submit Assignment button and Student Portfolio archival notice are positioned prominently in the right-hand action panel.

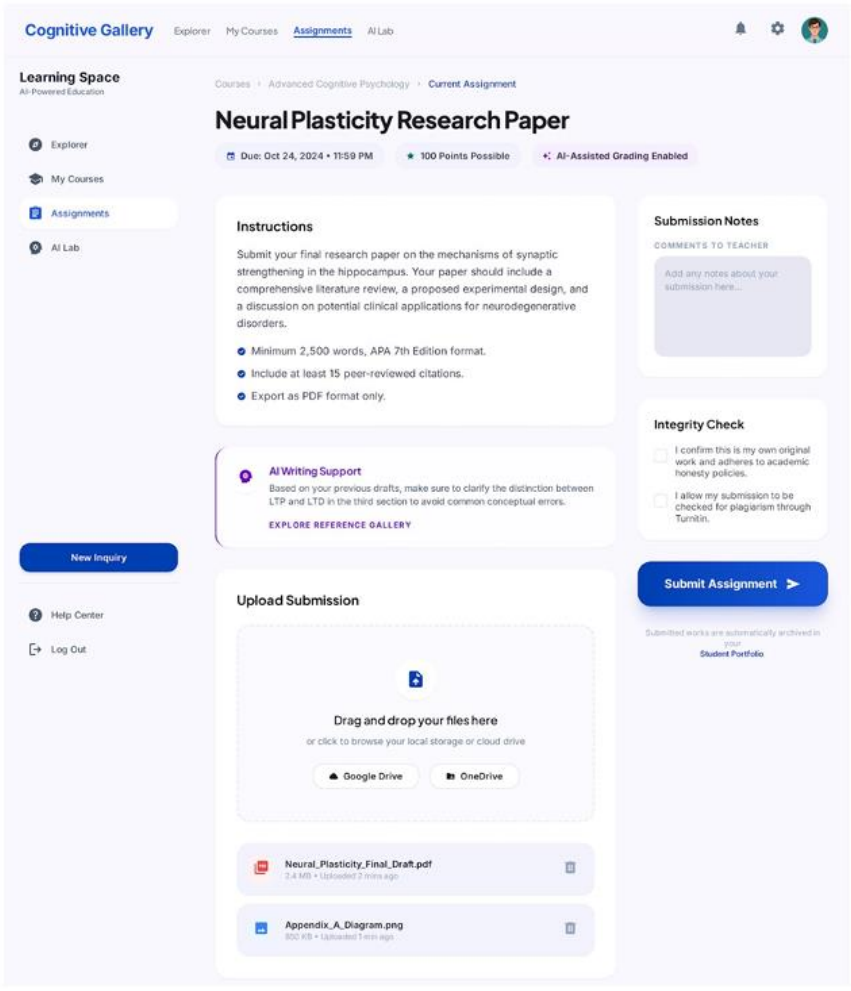


Figure 3.32: Assignment Submission

3.5.2.7 Quiz Generator

The Quiz Generator is a faculty-facing tool accessible through the Faculty Hub. It enables instructors to transform academic source materials into structured multiple-choice assessments using AI-driven question synthesis. The left configuration panel allows the selection of a target course, entry of content focus material (lecture notes or textbook excerpts), and specification of difficulty level and question count. Upon generation, a Question Preview panel renders the AI-suggested questions with labelled answer options and correct answer indicators. An Editorial Precision feature description communicates the platform's commitment to preserving academic nuance. Faculty may discard or publish the draft directly to the associated course. A reviewer indicator shows the number of instructors currently reviewing the draft.

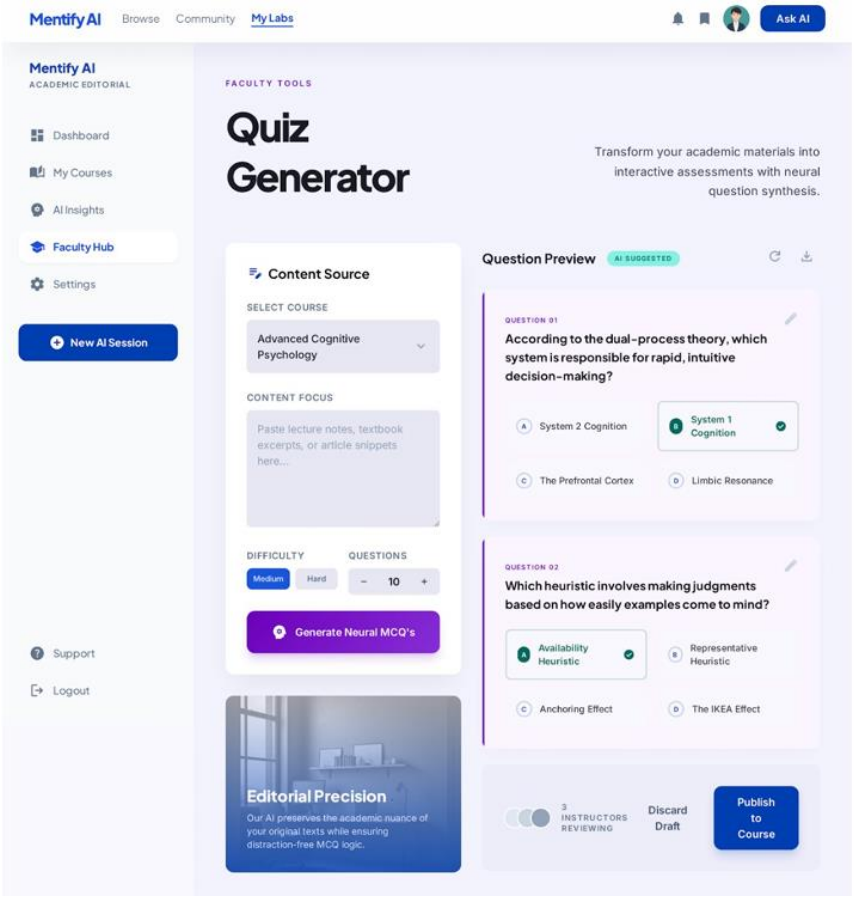


Figure 3.33: Quiz Generator

3.5.2.8 AI Learning Assistant

The AI Learning Assistant provides a conversational tutoring interface within the Cognitive Gallery platform. The left panel displays a list of past inquiry sessions, each labelled with the topic and message count for easy retrieval. The central chat panel greets the user by name and presents a contextual opening prompt derived from prior session analysis. A Learning Tip panel surfaces targeted remediation suggestions based on identified knowledge gaps. The student's query is rendered in a highlighted message bubble, and the assistant's response is structured with bold concept labels, bullet point steps, and an embedded Python code snippet. Quick-action suggestion chips at the bottom of the input area allow rapid follow-up prompts. A disclaimer regarding the verification of AI-generated content is displayed beneath the input field. A Pro upgrade prompt is surfaced for users on the free tier.

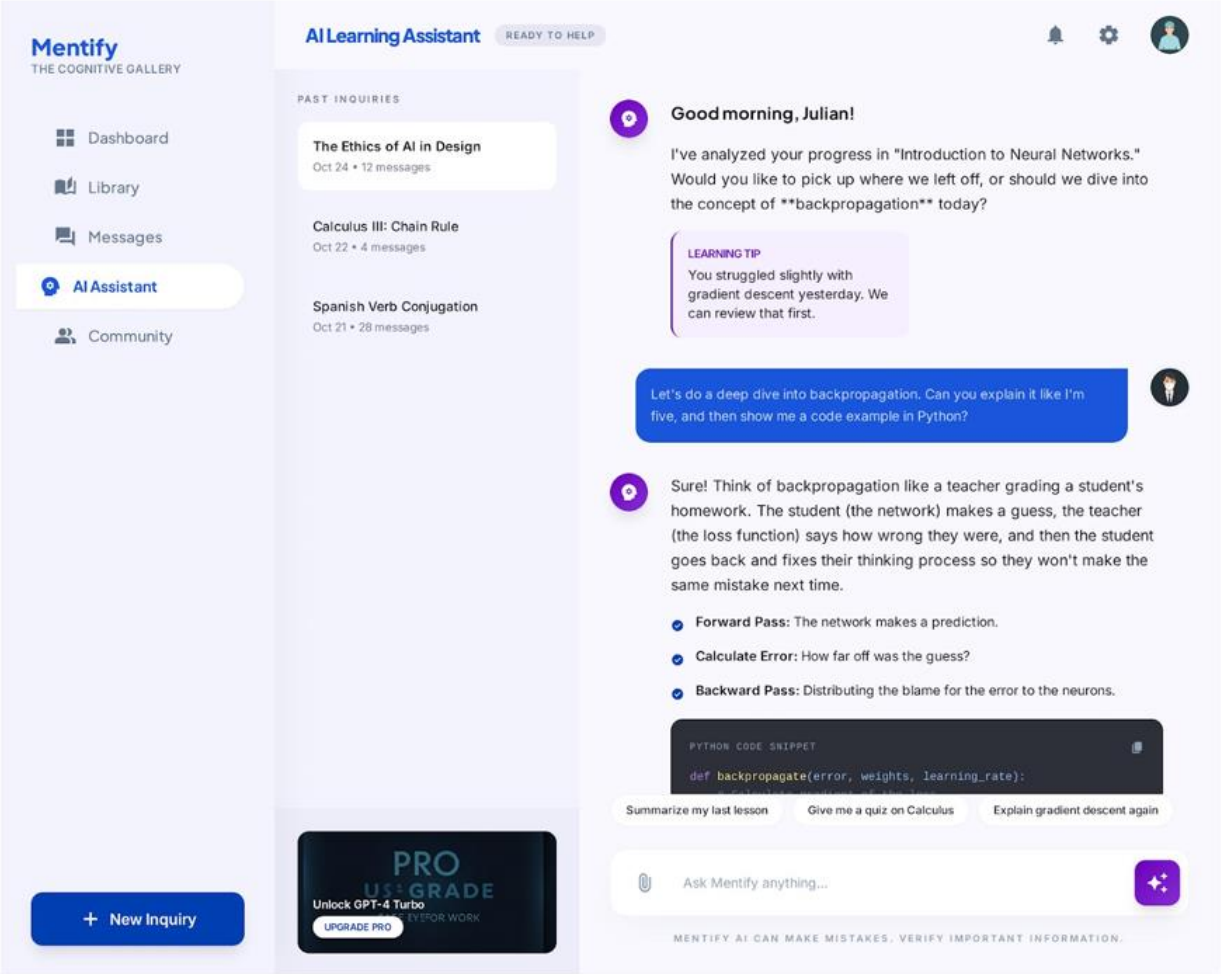


Figure 3.34: AI Learning Assistant

3.5.2.9 Cognitive Gallery Explorer

The Cognitive Gallery Explorer serves as the primary course discovery interface. An AI-curated hero banner presents the highest-match course recommendation based on the student's cognitive profile, displaying the match percentage, course duration, and difficulty level with enrolment and save controls. A secondary recommendation panel surfaces an additional match with a "Explore Insight" call to action. A Trending in Grade carousel displays topic-tagged course cards enriched with social proof indicators (number of enrolled friends). Recommended by Teachers and Skill Boosters sections further personalise the discovery experience by surfacing mentor-curated and efficiency-focused learning modules.

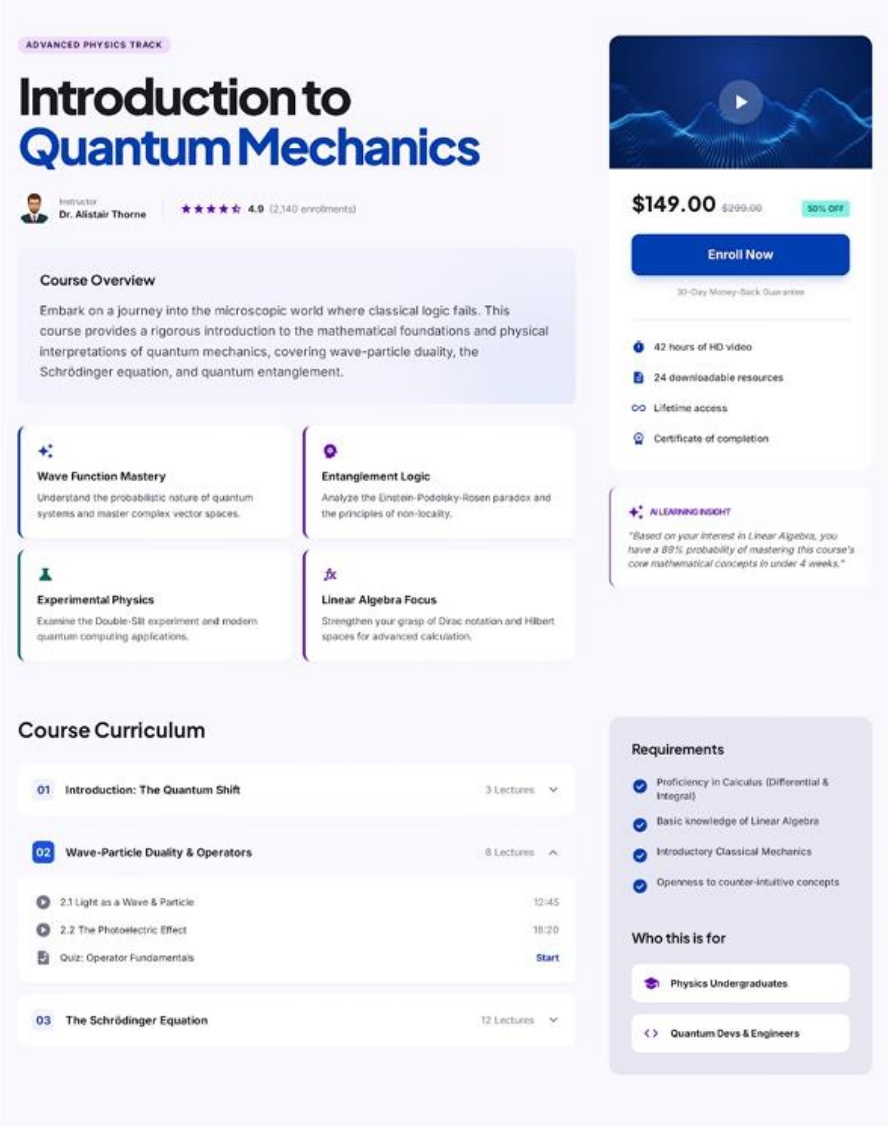


Figure 3.35: Cognitive Gallery Explorer

3.5.2.10 User Management – Administration

The User Management screen is an administrator-facing interface accessible through the Faculty Hub. It provides a comprehensive overview of all registered platform users — faculty, students, and support staff — within a paginated, filterable data table. Summary metrics at the top of the page display total faculty count, active users, AI Insight Score, and pending approvals. Each table row presents the user's name, email, role badge, active status indicator, and grade or department. Action controls allow administrators to edit records or access additional options. An AI Role Auditor widget surfaces data driven promotion recommendations with accept or dismiss controls. A System Health panel and a Pending Tasks list provide operational visibility. Access to Security Policy, API Docs, and Audit Logs is provided in the footer, restricted to Internal Admin Access Only.

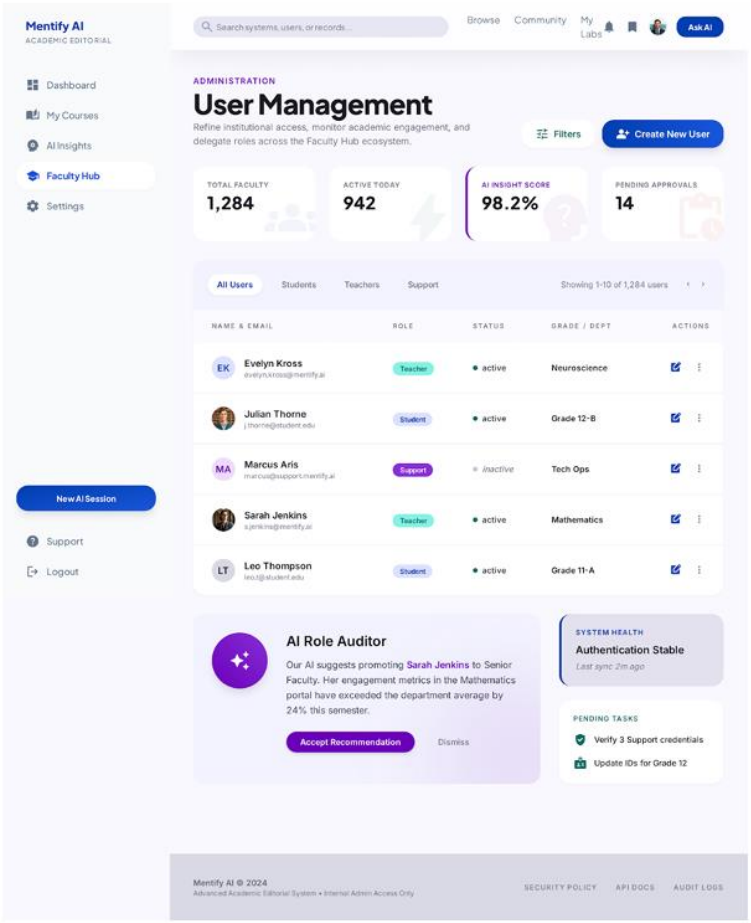


Figure 3.36: User Management

3.5.3 User Flow and Navigation Diagrams

This section presents the user flow diagrams for the four primary system interactions identified within the Mentify AI platform. Each diagram models the sequential steps, decision points, and system responses that govern a user's path through a given process. These diagrams inform both the front-end navigation architecture and the back-end logic required to support each flow. The following table provides a summary of the documented user flows.

User Flow	Description
User Login	The user navigates to the login page and enters credentials. If an account does not exist, a registration prompt is displayed. If credentials are incorrect, the forgot-password flow is triggered via email verification. Upon successful authentication, the user is redirected to the dashboard.
Course Enrolment	The student navigates to the Explore page, applies grade and subject filters, selects a relevant course, and submits an enrolment request. Upon teacher approval, a payment portal is opened. Payment completion finalises enrolment.
Course Management	The teacher opens the dashboard and accesses My Courses. The teacher may choose to add a new course or edit an existing one. After completing required fields and submitting for review, an admin approves or rejects the request. Notifications are dispatched accordingly.
Forum	The student logs in and navigates to the Forums section. If the student is already a participant, messages may be sent and read immediately. Non-participants must request access; upon approval, participation is granted. Denied requests trigger an email notification.

3.33: Summary of User Flow Diagrams

3.5.3.1 User Login Flow

The User Login Flow governs the authentication process for all platform users. The process commences at the Login Page where the user submits their credentials. The system first verifies whether an account exists for the provided username. If no account is found, the user is directed to register. If an account exists, the system validates the username and password combination. Upon successful validation, the user is logged in and redirected to the dashboard. If credentials are incorrect, a prompt is issued to initiate the Forgot Password process. The user enters their registered email address, which is validated against database records. If a match is found, a password reset link is dispatched; otherwise, the user is instructed to contact support for account recovery. Upon resetting the password, the login process is reinitiated.

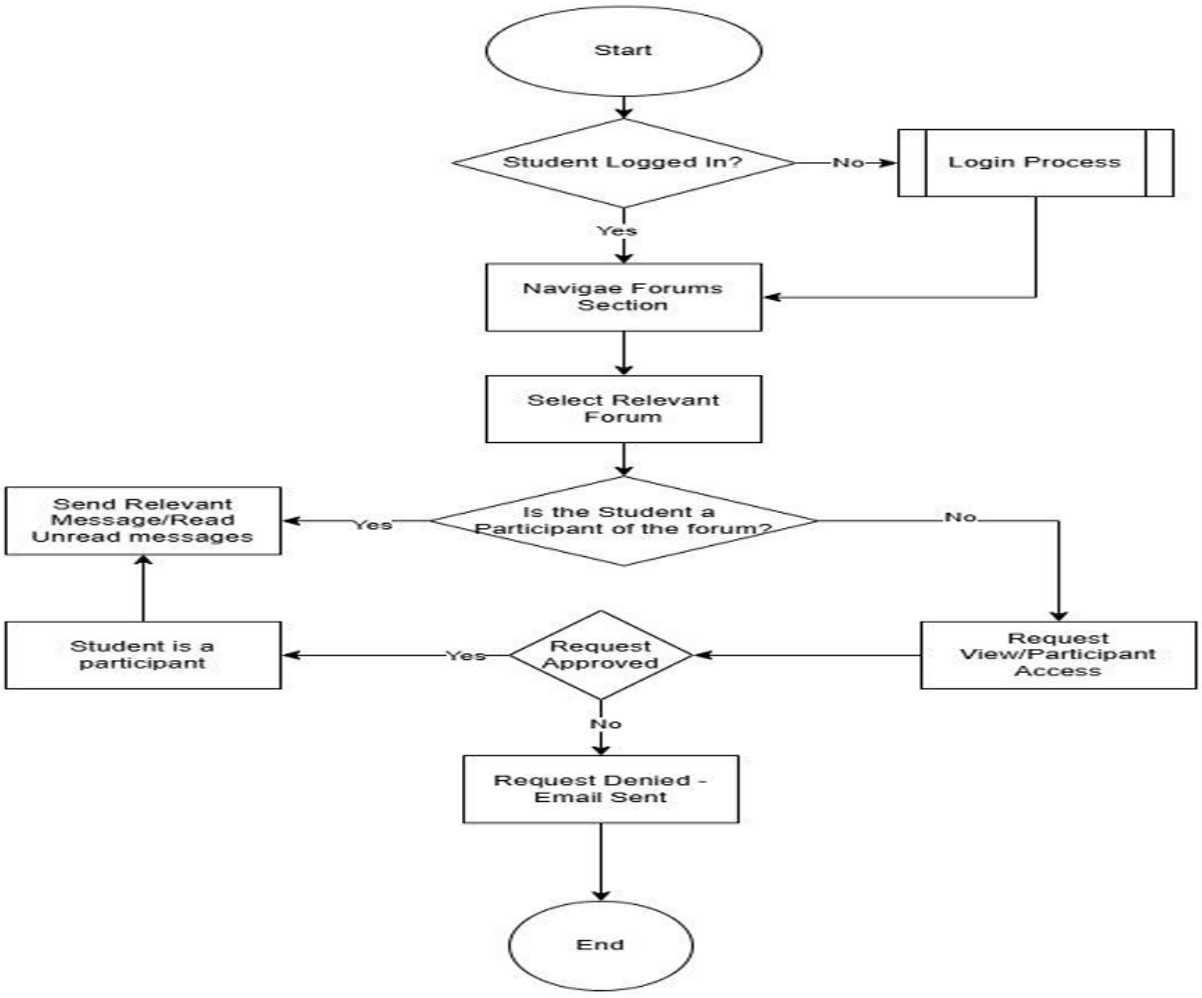


Figure 3.37: User Login Flow Diagram

3.5.3.2 Course Enrolment Flow

The Course Enrolment Flow captures the end-to-end process by which a student discovers and formally enrols in a course. The student navigates to the Explore Page and refines the course listing by applying grade-level and subject-area filters. Relevant courses are browsed, and the student selects a desired course and initiates an enrolment request by clicking the Enrol button. The enrolment request is transmitted to the assigned teacher for review. If the request is approved, a payment portal is presented to the student, and enrolment is confirmed upon payment completion. If the request is rejected, the student is notified by email and the flow terminates.

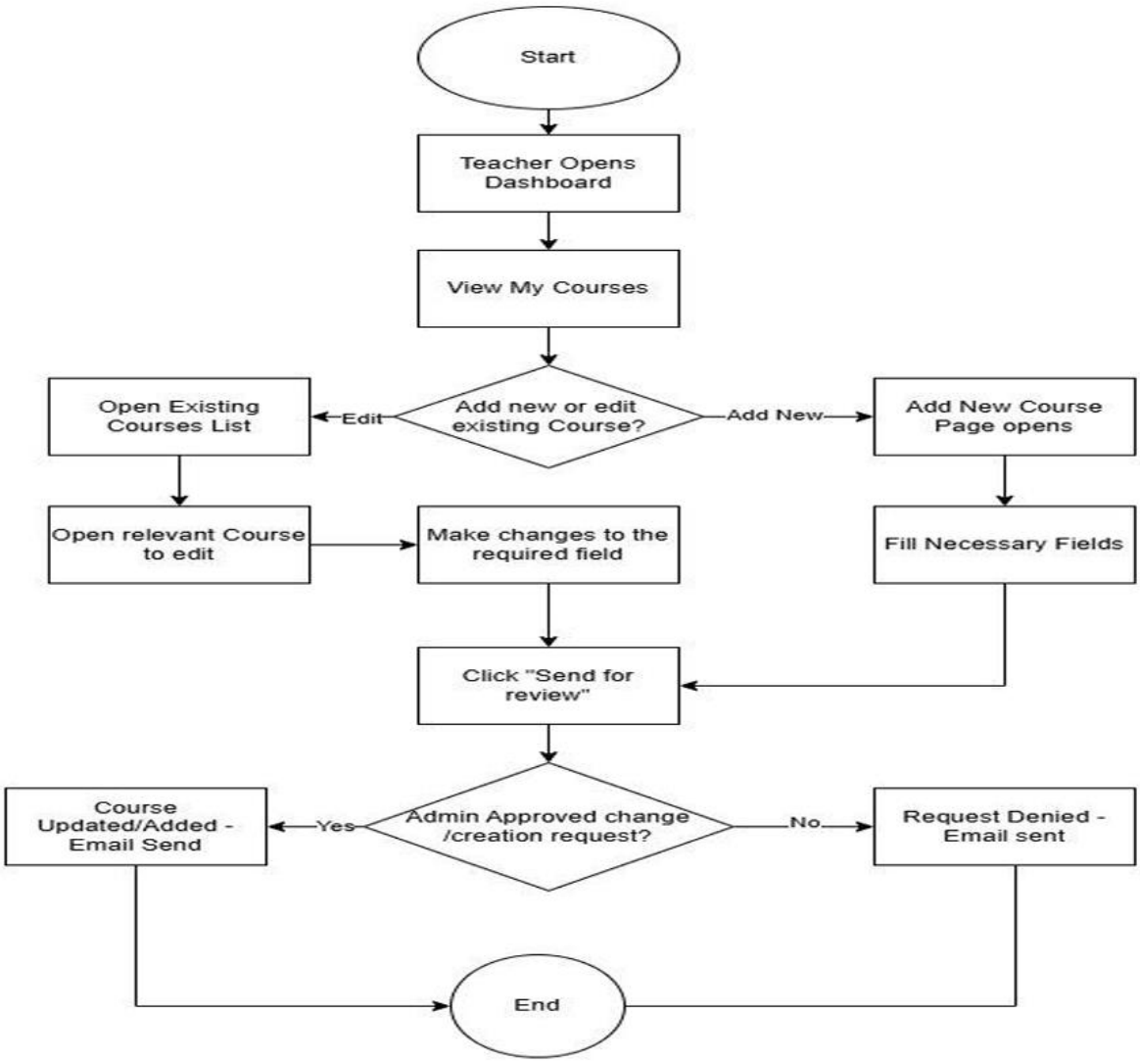


Figure 3.38: Course Enrolment Flow Diagram

3.5.3.3 Course Management Flow (Teacher)

The Course Management Flow describes the process by which a teacher creates a new course or modifies an existing one within the platform. The teacher accesses the dashboard and navigates to My Courses. At the primary decision point, the teacher elects to either add a new course or edit an existing one. The "Add New" path opens a course creation form in which all required fields must be completed. The "Edit" path opens the existing courses list, from which the relevant course is selected for modification. In both cases, changes are submitted for administrative review by clicking "Send for Review". The administrator then approves or denies the request. If approved, the course is updated or published and a confirmation email is dispatched. If denied, a rejection email is sent and the flow concludes.

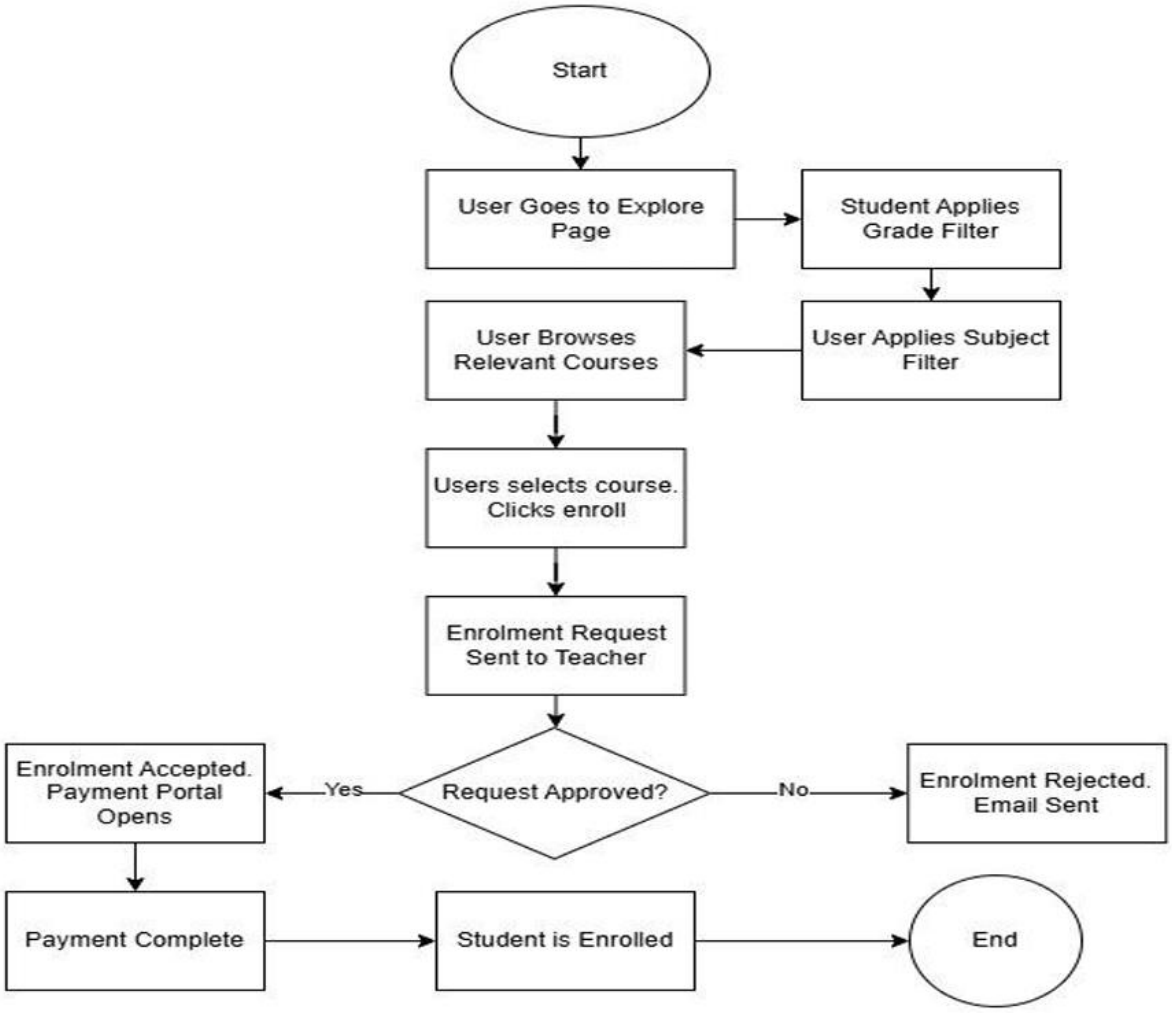


Figure 3.39: Course Management Flow

3.5.3.4 Forum and Group Messaging Access Flow

The Forum and Group Messaging Access Flow governs the process by which a student gains access to and participates in a forum or group channel. The flow begins with a check on the student's authentication status; if not logged in, the student is redirected to the Login Process. Upon successful authentication, the student navigates to the Forums section and selects the relevant forum. A participation check determines whether the student is already a registered participant. If so, the student may immediately send messages and read unread content. If not, the student must submit a request for view or participant access. If the access request is approved, the student is granted participant status and may engage with the forum. If the request is denied, an email notification is dispatched and the flow terminates.

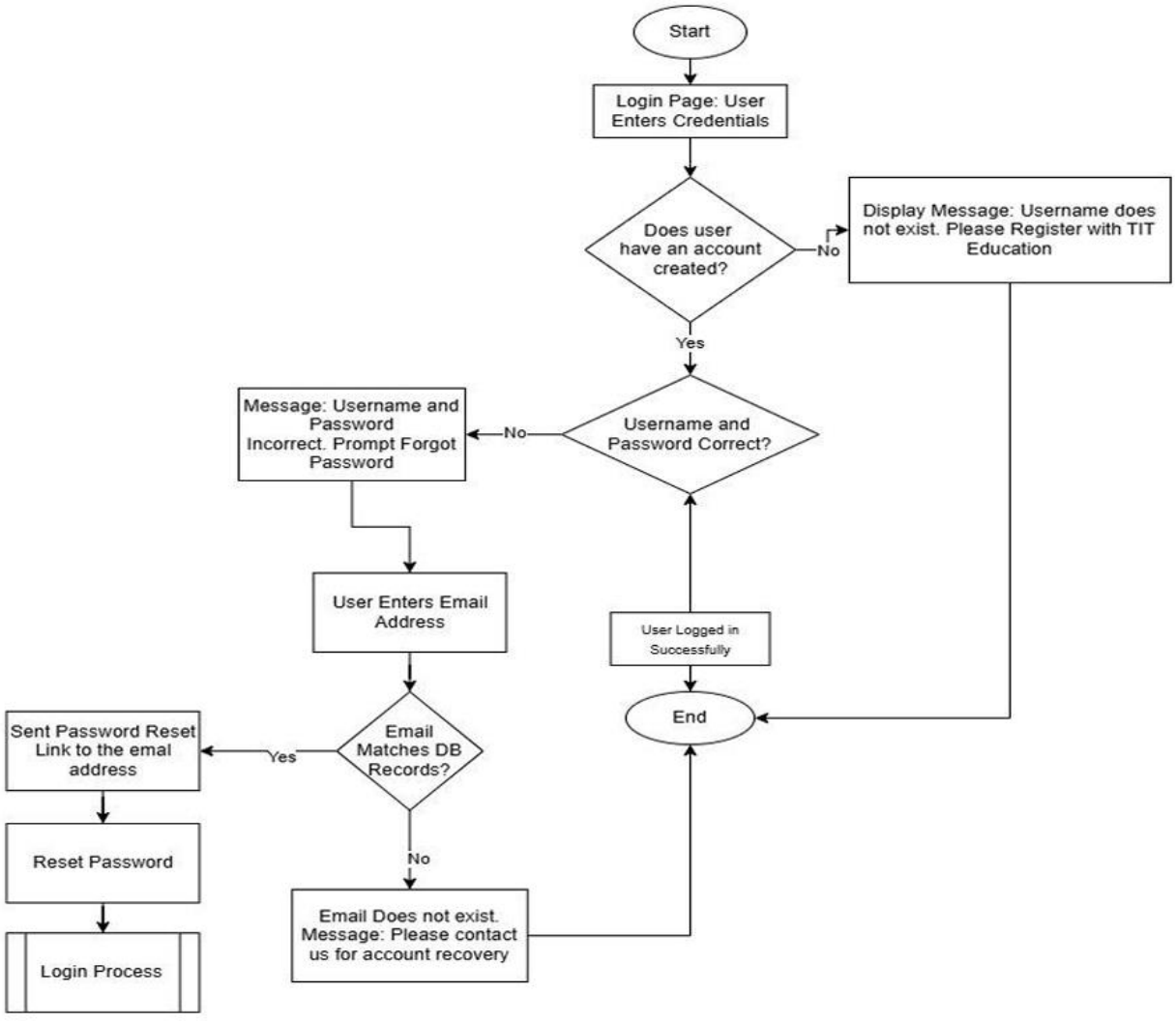


Figure 3.40: Forum and Group Messaging

3.5.4 Accessibility Considerations

The Mentify AI platform is designed to be inclusive and accessible to all users, including those with visual, auditory, motor, and cognitive impairments. The accessibility strategy is aligned with the Web Content Accessibility Guidelines (WCAG) 2.1 published by the World Wide Web Consortium (W3C). Conformance is targeted at Level AA, which represents the standard required by most international accessibility legislation, including the European Accessibility Act and relevant provisions of the Rehabilitation Act Section 508. The WCAG 2.1 guidelines are organised under four core principles, commonly referred to by the acronym PERCEIVABLE, OPERABLE, UNDERSTANDABLE, and ROBUST (POUR). The following subsections describe how each principle is addressed within the platform design.

3.5.4.1 Perceivable

All non-text content, including UI icons, course imagery, and AI-generated visualisations, is accompanied by descriptive alternative text attributes to ensure compatibility with screen reader technologies. The platform's colour palette is selected to maintain the minimum contrast ratios mandated by WCAG 2.1 criterion 1.4.3 (4.5:1 for normal text; 3:1 for large text). Status information — such as enrolment status, active user indicators, and assignment grading states — is conveyed through both colour and text labels, ensuring that users who cannot distinguish colour differences are not disadvantaged. Text elements are specified in relative units (rem/em) and remain legible at up to 200% zoom without horizontal overflow.

3.5.4.2 Operable

All interactive components — including navigation links, form controls, modal dialogs, and action buttons — are fully operable via keyboard alone. Tab order follows the natural document flow and is supplemented by explicit tabindex attributes where necessary to manage focus within complex components such as quiz answer grids and the AI chat input. Focus indicators are clearly visible and meet the minimum size and contrast requirements introduced in WCAG 2.1 criterion 2.4.7. Timed elements, such as the quiz countdown timer, provide configurable extensions for users who require additional processing time. No content is designed to flash at frequencies known to induce photosensitive responses.

3.5.4.3 Understandable

The page language is programmatically declared in all HTML documents to support language-aware assistive technologies. All form inputs are associated with explicit label elements. Inline

validation messages are presented in plain, descriptive language immediately adjacent to the relevant input field, and error recovery instructions are provided. The platform's navigation structure is consistent across all screens, reducing cognitive load associated with wayfinding. Instructional text is written at an appropriate reading level for the target educational audience, and technical AI-generated content is accompanied by simplified explanations where applicable.

3.5.4.4 Robust

The platform's front-end codebase conforms to valid HTML5 and ARIA (Accessible Rich Internet Applications) specification standards. Custom UI components — including collapsible course modules, tabbed navigation panels, and dynamic quiz widgets — are annotated with the appropriate ARIA roles, states, and properties (e.g., aria-expanded, aria-selected, aria-live) to ensure compatibility with a wide range of assistive technologies and browser combinations. Automated accessibility audits are integrated into the CI/CD pipeline using tooling such as axe-core, and manual testing with screen readers (NVDA, VoiceOver) is conducted as part of the user acceptance testing phase. The following table summarises the specific WCAG 2.1 criteria addressed within the platform design, the associated conformance level, and the implementation approach adopted for each.

WCAG 2.1 Criterion	Conformance Level	Implementation Approach
1.4.3 Contrast (Minimum)	Level AA	A minimum contrast ratio of 4.5:1 is maintained for normal text and 3:1 for large text throughout the interface.
1.4.4 Resize Text	Level AA	All text is resizable up to 200% without loss of content or functionality.
1.4.10 Reflow	Level AA	Content reflows without horizontal scrolling at 320px viewport width.
2.1.1 Keyboard	Level A	All interactive elements are fully operable via keyboard. Focus order is logical and sequential.
2.4.3 Focus Order	Level A	Keyboard focus follows a meaningful sequence consistent with visual layout.

2.4.4 Link Purpose	Level A	Link text is descriptive and unambiguous out of context.
2.4.7 Focus Visible	Level AA	A clearly visible focus indicator is provided for all keyboard-navigable elements.
3.1.1 Language of Page	Level A	The primary language of the document is programmatically identified.
3.3.1 Error Identification	Level A	Input errors are identified and described to the user in text.
3.3.2 Labels or Instructions	Level A	Labels are provided for all user input fields.
4.1.2 Name, Role, Value	Level A	ARIA attributes are applied to all custom UI components to expose name, role, and state.

3.34: WCAG 2.1 Accessibility Compliance Summary

In addition to the criteria outlined above, the platform undergoes periodic accessibility audits against the full WCAG 2.1 Level AA checklist. User testing sessions are conducted with participants who rely on assistive technologies to validate the efficacy of the accessibility implementation in real-world usage conditions. Findings from these sessions are logged and tracked in the project issue management system, and remediation is prioritised in subsequent development sprints.

3.6 Security Design

This chapter presents the security design of the system, encompassing the authentication mechanism, authorisation framework, data protection strategy, input validation and sanitisation approach, and the encryption standards applied to data at rest and in transit. The security architecture has been designed in alignment with industry best practices and relevant standards, including OWASP Top 10, TLS 1.2+, and AES-256 encryption. The overall security architecture is illustrated in Figure 6.1, which depicts the flow of authentication and authorisation for all client requests, the role of the API Gateway as the security perimeter, and the integration of external identity providers.

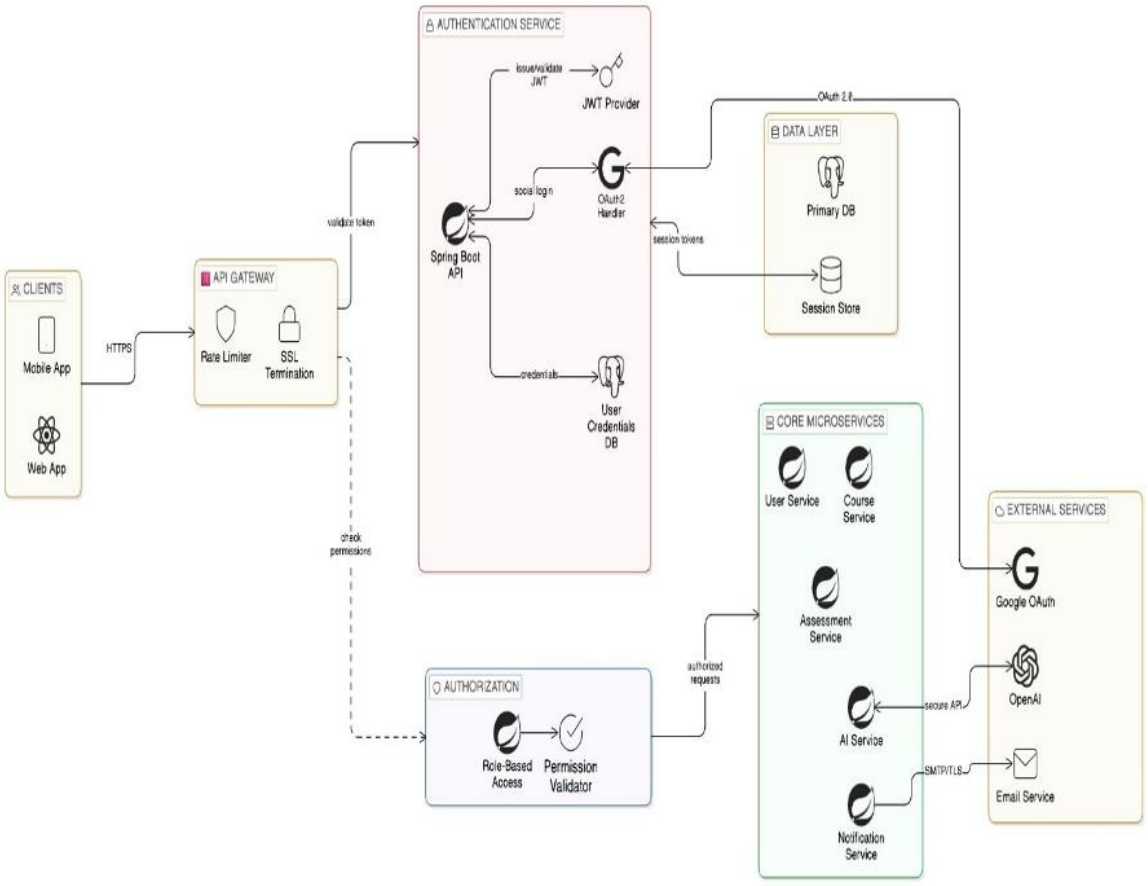


Figure 3.41: Security Architecture Diagram

3.6.1 Authentication Mechanism

Authentication is the process by which the identity of a user or system is verified prior to granting access to protected resources. The system implements a dual-mode authentication strategy

supporting both credential-based login and social login via OAuth 2.0, managed through a dedicated Authentication Service within the microservices architecture.

3.6.1.1 JSON Web Token (JWT) Authentication

Upon successful credential verification, the Authentication Service issues a JSON Web Token (JWT) to the requesting client. JWTs are self-contained, digitally signed tokens that encode user identity and role claims. The following lifecycle governs JWT usage within the system:

- The client submits credentials (username and password) over an HTTPS-encrypted connection to the API Gateway.
- The API Gateway forwards the request to the Authentication Service's Spring Boot API.
- The Spring Boot API validates the submitted credentials against the User Credentials Database.
- Upon successful validation, the JWT Provider issues a signed JWT using the RS256 algorithm (RSA Signature with SHA-256), embedding the user's identity, roles, and token expiry timestamp.
- The issued JWT is returned to the client and must be included in the Authorization header of all subsequent API requests.

The API Gateway validates the JWT signature and expiry on every inbound request before routing it to the target microservice. Access tokens are issued with a short expiry window (15 minutes) to limit the exposure window in the event of token compromise. Refresh tokens, stored securely in the Session Store (Redis), are issued alongside access tokens and are used to obtain new access tokens without requiring re-authentication, provided the refresh token remains valid.

3.6.1.2 OAuth 2.0 Social Login

The system supports social login via Google OAuth 2.0, implemented through the OAuth2 Handler component of the Authentication Service. This mechanism delegates identity verification to Google's identity platform, eliminating the need for users to create and manage separate credentials within the system. The OAuth 2.0 authorisation code flow is employed, which ensures that access tokens are never exposed to the client browser.

The OAuth 2.0 authentication flow proceeds as follows:

- The client initiates a social login request, which is redirected to the Google OAuth authorisation endpoint.
- The user authenticates with Google and grants the requested permissions.
- Google returns an authorisation code to the system's registered callback URI.
- The OAuth2 Handler exchanges the authorisation code for an access token and identity token via a server-to-server call to Google's token endpoint.
- The identity token is validated, and a system-internal JWT is issued to the client for use in subsequent API calls.

Authentication	Token Type	Storage	Expiry
Credential-Based Login	JWT (Access Token)	Client memory (not localStorage)	15 minutes
Credential-Based Login	Refresh Token	Redis Session Store (server-side)	7 days
Google OAuth 2.0	OAuth Identity Token	Sever-side only	Per google policy
Google OAuth 2.0	System JWT	Client Memory	15 minutes

3.35: Authentication Token Summary

3.6.2 Authorisation Levels

Authorisation determines the set of actions that an authenticated user is permitted to perform. The system implements Role-Based Access Control (RBAC), enforced by the Authorisation component which consists of a Role-Based Access module and a Permission Validator. The API Gateway delegates permission checks to this component prior to forwarding any authenticated request to a core microservice.

3.6.2.1 Role Definitions

The system defines the following user roles, each mapped to a specific set of permissions. Role assignments are encoded within the JWT claims and are validated by the Permission Validator on each request.

Role	Description	Permissions
Administrator	System-level administrative users responsible for platform management.	Full access to all system resources; user management; system configuration; audit log access.

Instructor	Academic Staff responsible for creating and managing course content	Create/edit/delete courses and assessments; view enrolled student data; access analytics for own courses.
Student	Enrolled learners accessing educational content	View enrolled course materials; submit assessments; view own grades and progress; access communication features.
Guest	Unauthenticated or unregistered users with limited read-only access.	View publicly listed courses; access registration and login endpoints only.

3.36: Role-Based Access Control Definitions

3.6.2.2 Authorisation Enforcement

Authorisation is enforced at two levels within the system architecture:

- Gateway Level: The API Gateway evaluates the JWT claims of every inbound request and forwards only those bearing a valid, non-expired token to the Permission Validator. Requests without a valid token are rejected with an HTTP 401 Unauthorized response.
- Service Level: Each microservice applies method-level security using Spring Security annotations (e.g., @PreAuthorize). This provides a second layer of access control, ensuring that even requests that bypass the gateway layer are subject to authorisation enforcement.

The principle of least privilege is applied throughout, such that each role is granted only the minimum permissions necessary to perform its designated functions. Privilege escalation between roles is not permitted without explicit administrative approval.

3.6.3 Data Protection Approach

Data protection encompasses the measures taken to safeguard the confidentiality, integrity, and availability of all data handled by the system. The following strategies are applied at each tier of the architecture.

3.6.3.1 Password and Credential Management

- User passwords are never stored in plaintext. The BCrypt hashing algorithm with a minimum work factor of 12 is applied to all passwords prior to persistence in the User Credentials Database.
- Password reset tokens are cryptographically random, single-use, and expire after 30 minutes.
- Failed authentication attempts are rate-limited and logged. Accounts are temporarily locked following five consecutive failed login attempts within a 10-minute window.

3.6.3.2 Session Management

- Session tokens and refresh tokens are stored server-side in the Redis Session Store, not in client side browser storage, to prevent cross-site scripting (XSS) token theft.
- Sessions are invalidated immediately upon user logout or detection of anomalous activity.
- Concurrent session limits are enforced per user account to mitigate credential-sharing risks.

3.6.3.3 Sensitive Data Classification

Data handled by the system is classified into three sensitivity tiers, each subject to a corresponding set of protection controls:

Classification	Examples	Protection Controls
Highly Sensitive.	Passwords, OAuth tokens, private keys	BCrypt hashing; AES-256 encryption at rest; never logged or transmitted in plaintext
Sensitive	User PII, assessment results, grades	AES-256 encryption at rest; TLS 1.2+ in transit; access restricted by RBAC.
General	Course metadata, public content, system logs	TLS 1.2+ in transit; integrity checks; audit trail maintained.

3.37: Data Sensitivity Classification and Controls

3.6.4 Input Validation and Sanitisation Strategy

All data entering the system from external sources is treated as untrusted and is subjected to a multi layered validation and sanitisation pipeline before being processed or persisted. This strategy

mitigates the risk of injection attacks, cross-site scripting (XSS), and other input-borne vulnerabilities identified in the OWASP Top 10.

3.6.4.1 Validation Layer

Input validation is enforced at the API Gateway level and again at the service level using Spring Boot's Bean Validation framework (JSR 380). The following validation rules are applied:

- Type Validation: All incoming request fields are validated against their declared data types. Requests containing type mismatches are rejected with an HTTP 400 Bad Request response.
- Length Constraints: Maximum and minimum length constraints are applied to all string fields to prevent buffer overflow and database field overflow conditions.
- Format Validation: Fields with defined formats (e.g., email addresses, dates, UUIDs) are validated against their respective regular expression patterns.
- Whitelist Validation: Where applicable, field values are validated against a defined whitelist of permitted values (e.g., role names, status codes).
- Request Size Limits: The maximum permitted HTTP request body size is enforced at the API Gateway to prevent denial-of-service attacks via excessively large payloads.

3.6.4.2 Sanitisation Layer

- SQL Injection Prevention: All database interactions are performed using parameterised queries via Spring Data JPA / Hibernate. Raw SQL concatenation with user-supplied input is prohibited.
- XSS Prevention: All user-supplied string input is HTML-encoded prior to storage and output encoding is applied before any user-supplied content is rendered. The Content-Security-Policy (CSP) HTTP header is configured to restrict script execution sources.
- File Upload Sanitisation: Uploaded files are validated for permitted MIME types and file extensions. File contents are scanned for malicious signatures before being stored in Amazon S3. Original filenames are replaced with system-generated UUIDs to prevent path traversal attacks.
- JSON Input Sanitisation: The API Gateway strips unrecognised fields from inbound JSON payloads (strict schema enforcement), preventing mass assignment vulnerabilities.

Attack Vector	Mitigation Mechanism	Implementation
SQL Injection	Parameterised queries	Spring Data JPA / Hibernate ORM
XSS	Output encoding + CSP header	Spring Security + CSP configuration
CSRF	CSRF token validation	Spring Security CSRF protection
Path Traversal	UUID-based filenames + MIME validation	Custom file upload service
Mass Assignment	Strict JSON schema enforcement	API Gateway request validation
DoS via Large Payloads	Request body size limits	API Gateway configuration

3.38: Input Validation and Sanitisation Mechanisms

3.6.5 Encryption Standards for Data at Rest and in Transit

The system applies cryptographic standards consistent with current industry guidance to protect data in all states. The following encryption standards govern data protection across the system.

3.6.5.1 Encryption in Transit

All data transmitted between clients and the system, and all inter-service communications, is encrypted using Transport Layer Security (TLS). SSL Termination is performed at the API Gateway, as depicted in Figure 6.1.

- TLS Version: TLS 1.2 is the minimum permitted version. TLS 1.3 is preferred and enabled where supported by the connecting client.
- Cipher Suites: Only cipher suites providing forward secrecy are permitted (e.g., ECDHE-RSA AES256-GCM-SHA384, ECDHE-RSA-CHACHA20-POLY1305). Weak cipher suites (e.g., RC4, 3DES) are explicitly disabled.
- HTTPS Enforcement: All HTTP traffic is redirected to HTTPS at the API Gateway. HTTP Strict Transport Security (HSTS) headers are applied with a minimum max-age of one year.
- Internal Service Communication: Communication between microservices within the EKS cluster is encrypted using mutual TLS (mTLS), implemented via a service mesh. This ensures that even intra-cluster traffic cannot be intercepted.

- External API Communication: Communication with external services (OpenAI API via secure API, Gmail SMTP via SMTP/TLS, Google OAuth) is conducted exclusively over TLS 1.2+.

3.6.5.2 Encryption at Rest

All persistent data stores are encrypted at rest using AES-256 (Advanced Encryption Standard with a 256-bit key), which is the current NIST-recommended standard for symmetric encryption.

- PostgreSQL Databases: AWS RDS instances are configured with AWS-managed AES-256 encryption for all database volumes, including automated backups and snapshots.
- MongoDB Databases: MongoDB Atlas cluster storage is encrypted at rest using AES-256. Encryption keys are managed through AWS Key Management Service (KMS).
- Amazon S3: All S3 buckets are configured with Server-Side Encryption using AWS KMS managed keys (SSE-KMS) with AES-256. Bucket policies enforce that unencrypted object uploads are rejected.
- Redis Session Store: The Redis instance is configured with encryption at rest. Sensitive session data stored in Redis is additionally encrypted at the application layer before being written, providing defence-in-depth.

3.6.5.3 Key Management

- All cryptographic keys are managed through AWS Key Management Service (KMS), which provides centralised key lifecycle management, automatic key rotation, and audit logging of all key usage.
- JWT signing keys (RS256 RSA private keys) are stored in AWS Secrets Manager and rotated on a 90-day schedule.
- Application secrets (database connection strings, API keys for external services) are stored in AWS Secrets Manager and injected into microservice pods as environment variables at runtime. Secrets are never embedded in container images or source code.

Data Category	Storage Location	Encryption Standard	Key Management
Relational data (User, Assessment, Course)	AWS RDS PostgreSQL	AES-256 (SSE)	AWS KMS

Document data (Communication, Analytics, AI)	MongoDB Atlas	AES-256 (SSE)	AWS KMS
Files and media assets	Amazon S3	AES-256 (SSE KMS)	AWS KMS
Session tokens	Redis	AES-256 (at-rest + app-layer)	AWS KMS
Application secrets and API keys	AWS Secrets Manager	AES-256	AWS KMS
Data in transit (client to gateway)	TLS 1.3 (min. TLS 1.2)	ECDHE cipher suites	AWS ACM certificates
Data in transit (inter-service)	Mutual TLS (mTLS)	ECDHE cipher suites	Service mesh PKI

3.39: Encryption Standards Summary

The combination of AES-256 encryption at rest, TLS 1.2+ in transit, and centralised key management through AWS KMS ensures that the system meets the cryptographic requirements of widely recognised security frameworks, including ISO/IEC 27001 and NIST SP 800-53.

3.7 Deployment Design

This chapter presents the deployment design for the system, detailing the hosting strategy, hardware specifications, network configuration, environment setup, and the continuous integration and continuous deployment (CI/CD) pipeline. The deployment architecture is designed to support scalability, high availability, and operational efficiency in a cloud-native environment hosted on Amazon Web Services (AWS).

3.7.1 Hosting Plan

The system is hosted on Amazon Web Services (AWS), leveraging a combination of managed cloud services and containerised microservices to achieve cost-effective scalability. The primary compute infrastructure is provisioned on AWS EC2 instances (T3 Micro / T3 Small), which are orchestrated using Docker Compose for local container management in the initial deployment phase. The following table summarises the key hosting components.

Component	Service	Tier	Purpose
Frontend Hosting	CloudFront CDN + S3 Static Hosting AWS	Free / Pay as-you-go	Serve the React SPA globally via CDN
Compute Layer	AWS EC2 T3 Micro / T3 Small	On-Demand / Reserved	Host all backend microservices via Docker Compose
API Gateway	AWS API Gateway	Pay-as-you-go	Manage, route, and secure all API traffic
Load Balancer	Nginx (Software LB)	Open Source / EC2-hosted	Distribute HTTP/S traffic across service containers
Message Broker	Apache Kafka (Port 9092)	Single Instance / Docker	Async event streaming and inter-service communication
Cache / Sessions	Redis (Port 6379)	Single Instance / Docker	High-speed caching and session storage

Primary Database	PostgreSQL / Atlas NB (Single Instance)	AWS RDS / Docker	Relational data persistence
Document Store	MongoDB (Single Instance)	AWS DocumentDB / Docker	Unstructured and semi structured data
File / Object Storage	AWS S3 (Nearby Storage)	Standard Storage Class	User uploads, media, and static assets
Service Discovery	Netflix Eureka (Port 8761)	Docker / Spring Cloud	Dynamic service registration and discovery
WAF (Future)	AWS WAF	Future Upgrade	Web Application Firewall for security hardening

3.40: Hosting Plan Summary

The architecture adopts a future-proof approach by identifying planned production upgrades, including AWS ECS/EKS for container orchestration at scale, AWS ALB (Application Load Balancer) for managed load balancing with multi-AZ support, and EventCourse for advanced event-driven workflows. Apache Kafka is adopted in the current development and staging environments as a single Docker container instance, ensuring architectural consistency across all environments prior to production scaling. These upgrades are designated as future enhancements with their approximate cost implications noted in the architecture diagram.

3.7.2 Hardware Requirements

The system is deployed in a cloud environment, and as such, traditional on-premises hardware is not applicable. The hardware requirements are expressed in terms of cloud instance specifications and resource allocations. The following table presents the minimum and recommended specifications for each tier.

Service Group	Instant Type	vCPU	RAM	Notes
API Gateway & Load Balancer	T3 Small	2	2GB	Managed AWS Gateway; Nginx on EC2

Core & Auth Microservices	T3 Small	2	2GB	User, Auth services; multi-port exposure
Learning & Assessment Services	T3 Micro	2	1 GB	Course, Content, Assignment, Quiz services
Communication Services	T3 Micro	2	1 GB	Ticket, Chat, Notify services
AI Services (Async)	T3 Small	2	2 GB	Quiz-gen, AI Grading, Recommend, Chatbot
Analytics Services	T3 Micro	2	1GB	Event-driven analytics processing
Databases (PostgreSQL/MongoDB)	T3 Small / RDS	2	2–4 GB	Persistent storage; single-instance initial phase
Kafka & Redis	T3 Micro (shared)	2	1 GB	Infrastructure containers on shared EC2

3.41: Cloud Hardware / Instance Specifications

All services are containerised via Docker and orchestrated by Docker Compose on the initial EC2 deployment. Storage is provided by AWS S3 for object storage and EBS volumes for database persistence. As traffic scales, a migration to ECS/EKS with auto-scaling groups is planned to dynamically adjust instance counts without manual intervention.

3.7.3 Network Diagram

The network diagram below illustrates the complete system architecture, encompassing the client layer, edge layer, API gateway layer, load balancer layer, application microservices, infrastructure

containers, data persistence layer, service discovery, external integrations, and planned future production upgrades. The diagram employs colour-coded groupings to distinguish between Spring Boot microservices, infrastructure containers, database/storage components, and future upgrade targets.

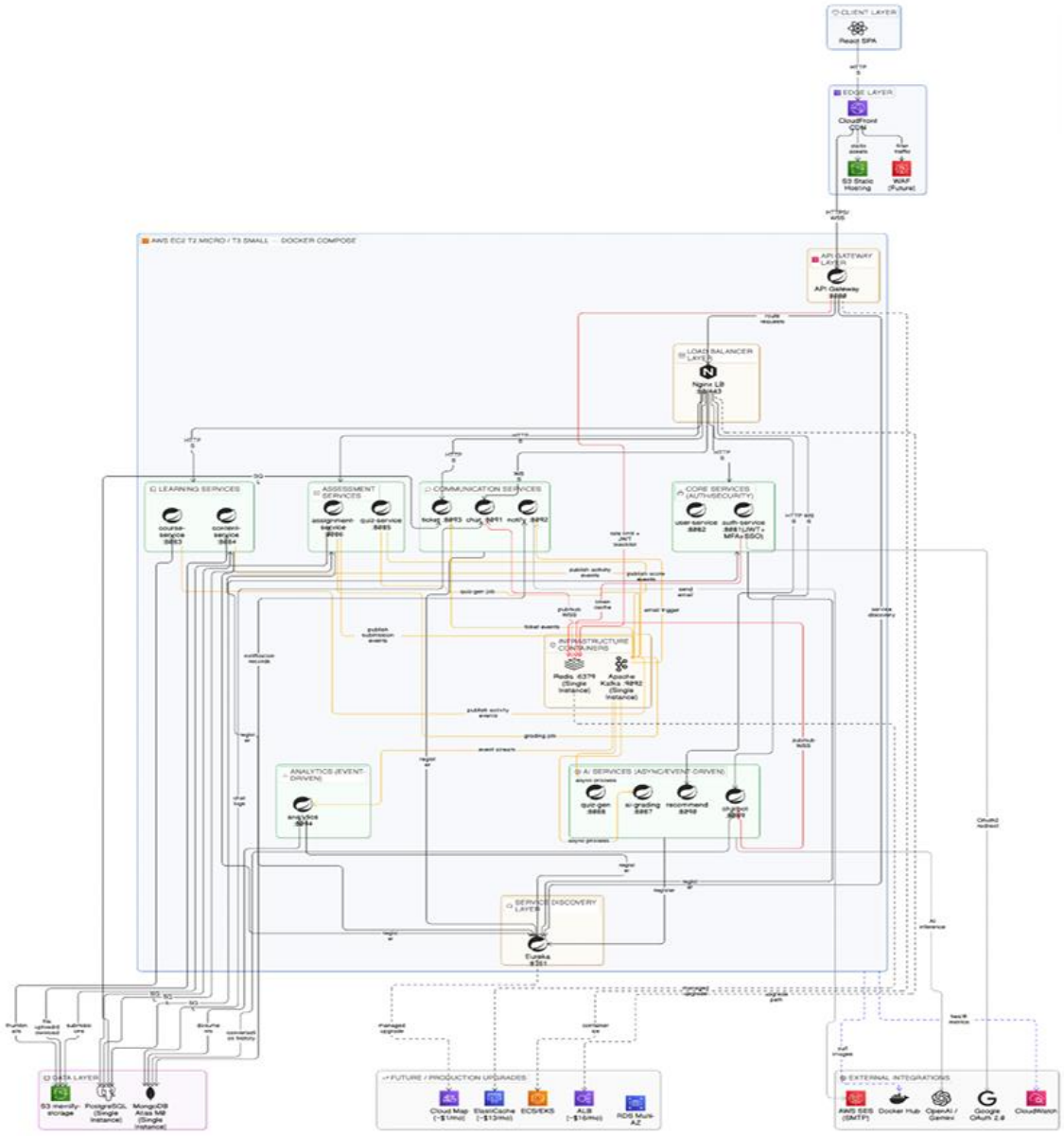


Figure 3.42: System Network and Deployment Architecture Diagram

3.7.3.1 Network Layer Description

The network is structured across distinct logical layers, each serving a specific role in the request lifecycle. Traffic originates from the Client Layer, where the React SPA is served via HTTPS/WSS from the CloudFront CDN. Static assets are stored in AWS S3, while a future WAF integration is planned at this edge layer for enhanced security. Inbound HTTPS/WebSocket requests are routed through the AWS API Gateway, which handles rate limiting, authentication token validation, and traffic management before forwarding requests to the Nginx Load Balancer. The load balancer distributes traffic across the internal microservice clusters using HTTP on port 80 and HTTPS on port 443. Internal service-to-service communication is conducted via synchronous HTTP/REST calls and asynchronous Apache Kafka event streaming. Kafka operates as a single Docker container instance in the development and staging environments, providing consistent event-driven behaviour across all environments prior to production scaling. Redis is utilised for caching and session management, while Netflix Eureka provides service registration and discovery for dynamic routing within the microservices mesh. The data persistence layer is isolated from the application tier through internal Docker network namespacing.

3.7.4 Environment Specification

Three distinct environments are maintained throughout the software delivery lifecycle: Development, Staging, and Production. Each environment is isolated to prevent cross-contamination of data and configuration.

Attribute	Development	Staging	Production
Purpose	Local development and unit testing	Integration and UAT testing	Live system serving real users
Infrastructure	Docker Compose (local machine)	AWS EC2 T3 Micro (shared)	AWS EC2 T3 Small / RDS (dedicated)
Database	Local Docker PostgreSQL & MongoDB	Shared staging DB (anonymised data)	RDS PostgreSQL + MongoDB Atlas
Domain	localhost / internal	staging.domain.com	www.domain.com
Secrets Management	Local .env files	AWS Secrets Manager (staging)	AWS Secrets Manager (production)

Monitoring	Console logs / local tools	CloudWatch (basic)	CloudWatch + alerts
CDN	Not Applicable	CloudFront (test distribution)	CloudFront (production distribution)
Data	Synthetic / mock data	Anonymised production snapshot	Live production data
Access Control	Open to dev team	Restricted to QA and dev leads	Restricted to operations team

3.42: Environment Specification Matrix

3.7.5 CI/CD Pipeline Overview

A Continuous Integration and Continuous Deployment (CI/CD) pipeline is implemented to automate the building, testing, and deployment of all microservices. The pipeline enforces code quality gates and ensures that only verified, tested artefacts are promoted to the Staging and Production environments. The pipeline is structured into the following sequential stages.

3.7.5.1 Source Control and Branching Strategy

All source code is managed in a Git-based version control repository (GitHub / GitLab). A trunk-based development model is adopted, with the following branch structure enforced:

- main — Production-ready code. Direct commits are prohibited; only merge requests from release branches are accepted.
- develop — Integration branch. All feature branches are merged here after peer review.
- feature/* — Short-lived branches for individual features or bug fixes. Naming convention: feature/-.
- release/* — Stabilisation branches created from develop prior to production deployment.
- hotfix/* — Emergency patches applied directly to main and back-merged to develop.

3.7.5.2 Pipeline Stages

Stage	Name	Description	Tooling
1	Code Commit & Trigger	A developer pushes to a feature or develop branch. The CI pipeline is	GitHub Actions / GitLab CI

		triggered automatically via a webhook.	
2	Build	Each microservice is compiled (Maven / Gradle for Spring Boot). Docker images are built for all services using their respective Dockerfiles.	Maven, Gradle, Docker
3	Unit Testing	Unit tests are executed for each service. A minimum code coverage threshold of 80% is enforced. Failing tests block the pipeline.	JUnit 5, Mockito
4	Static Code Analysis	Code quality and security vulnerability scanning are performed on the codebase. Results are published to the dashboard.	SonarQube / Checkstyle
5	Integration	Services are spun up using Docker Compose in an isolated test environment. API contract tests and integration tests are executed.	Testcontainers, Postman/Newman
6	Image Push	Successfully built and tested Docker images are tagged with the commit SHA and pushed to the container registry.	Docker Hub / AWS ECR
7	Deploy to Staging	Images are pulled and deployed to the Staging EC2 environment using Docker Compose. Smoke tests are executed post-deployment.	Docker Compose, SSH deploy scripts
8	UAT & Approval Gate	The QA team performs user acceptance testing on the Staging environment. A manual approval	Manual approval / JIRA

		gate is required before production promotion.	
9	Deploy to Production	Approved images are deployed to the Production EC2 environment using a rolling update strategy to ensure zero downtime deployment.	Docker Compose, Blue/Green strategy
10	Post-Deploy Monitoring	Health checks are executed against all service endpoints. CloudWatch metrics and alerts are reviewed to confirm system stability.	AWS CloudWatch, custom health endpoints

3.43: CI/CD Pipeline Stages

3.7.5.3 External Integrations in Deployment

The deployment pipeline interacts with several external services and registries that are integral to the system's operational readiness. The following external integrations are configured within the CI/CD and runtime environments:

Integration	Provider	Role in Deployment
Container Registry	Docker Hub	Stores and versions all microservice Docker images
AI Model API	OpenAI / Gemini	External AI inference for quiz generation, grading, and chatbot services
Version Control Hosting	Google GitHub	Source repository, CI trigger webhooks, and PR-based approvals
Email / SMTP	AWS SES (SMTP)	Transactional email delivery for notifications and alerts
Monitoring & Alerts	AWS CloudWatch	Infrastructure metrics, log aggregation, and operational alerting

3.44: External Integration Summary

3.7.5.4 Future Production Upgrade Path

As system load increases, the following infrastructure upgrades are planned and budgeted. These transitions are designed to be incremental and non-disruptive to the existing architecture:

- Apache Kafka is currently adopted in the development and staging environments as a single Docker container instance (Port 9092), replacing the previously planned RabbitMQ integration. In the production upgrade path, Kafka will be scaled to a fully managed, multi-broker cluster to support high-throughput, fault-tolerant event streaming across microservices.
- AWS ECS/EKS: Migration from Docker Compose to a fully managed container orchestration platform supporting auto-scaling, rolling deployments, and self-healing.
- AWS ALB (Application Load Balancer) with Multi-AZ: Replacement of the Nginx software load balancer with a managed, highly available ALB distributed across multiple Availability Zones.
- EventCourse: Integration of an advanced event-driven course processing pipeline to support large scale content ingestion and analytics workflows.
- AWS RDS Multi-AZ: Transition from single-instance PostgreSQL to a managed RDS instance with Multi-AZ failover for database high availability.

These upgrades are estimated to increase the monthly infrastructure cost from approximately USD 12–15 (current EC2-based Docker Compose setup) to a range of USD 15–19 per month for ECS/EKS and USD 12–15 for EventCourse, as indicated in the architecture diagram annotations.

4 PROTOYPE DEVELOPMENT AND CLIENT VALIDATION

4.1 UI/UX Design

The UI/UX design of Mentify focuses on creating a clean, intuitive, and user-centered interface that supports seamless interaction for students, educators, and administrators. The design emphasizes clear navigation, consistent layouts, and efficient access to key features such as classes, assessments, communication tools, and analytics, while minimizing complexity and cognitive load. Responsive and accessible design principles are applied to ensure usability across different devices. The following section presents the system prototypes, illustrating how these design concepts are implemented in the interface.

4.1.1 Students' View

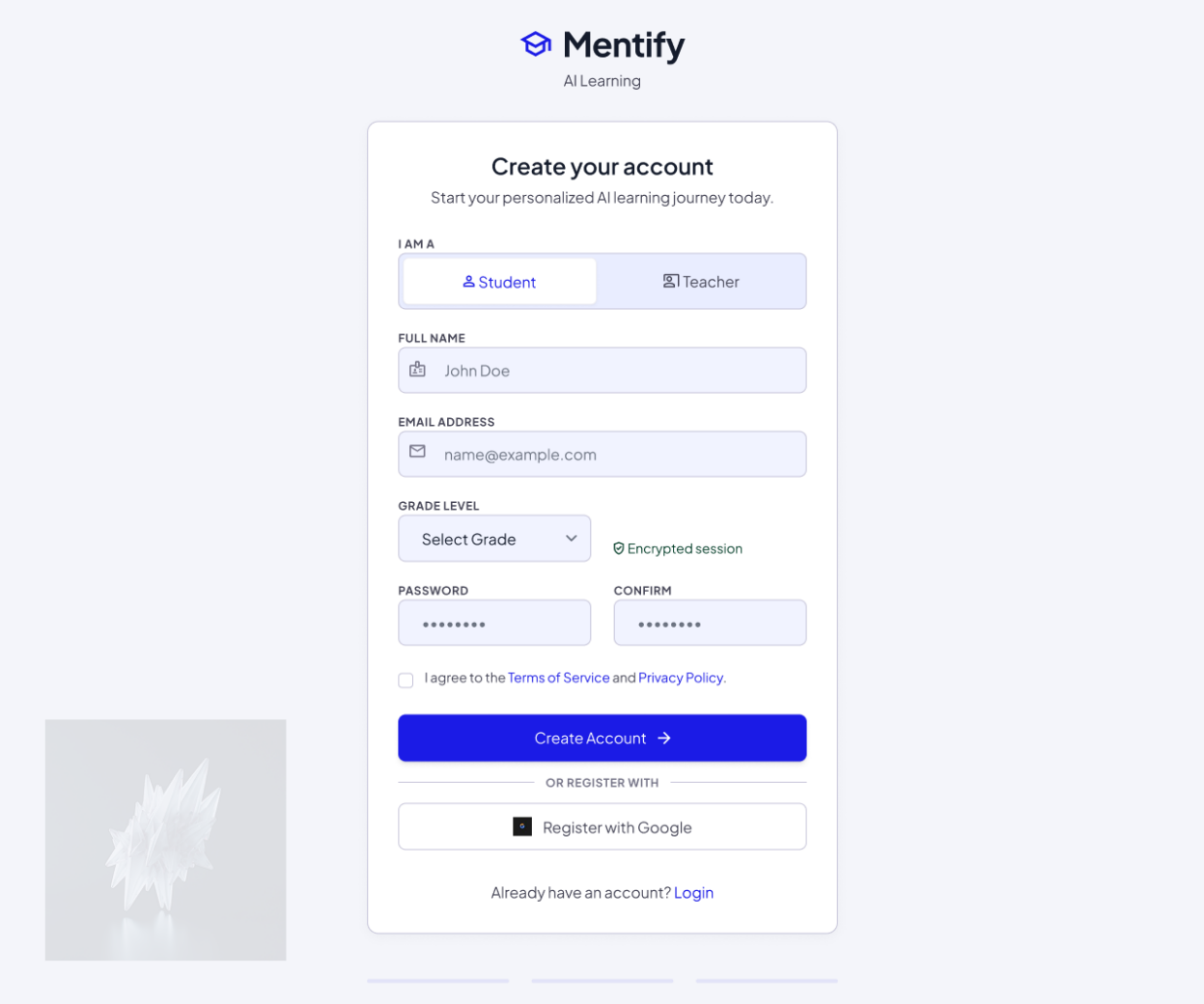


Figure 4.1: Student Registration Page

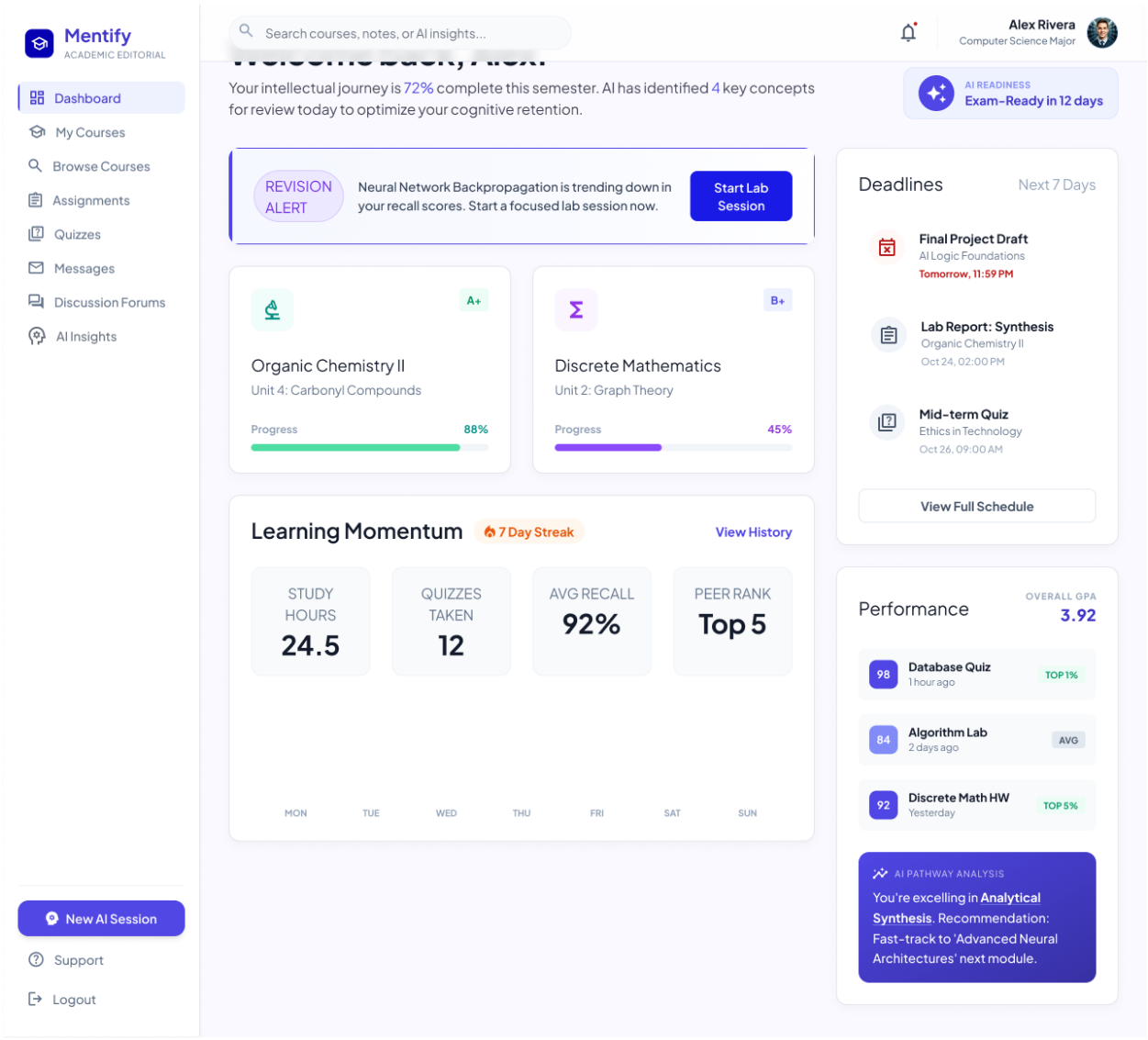


Figure 4.2: Student Dashboard

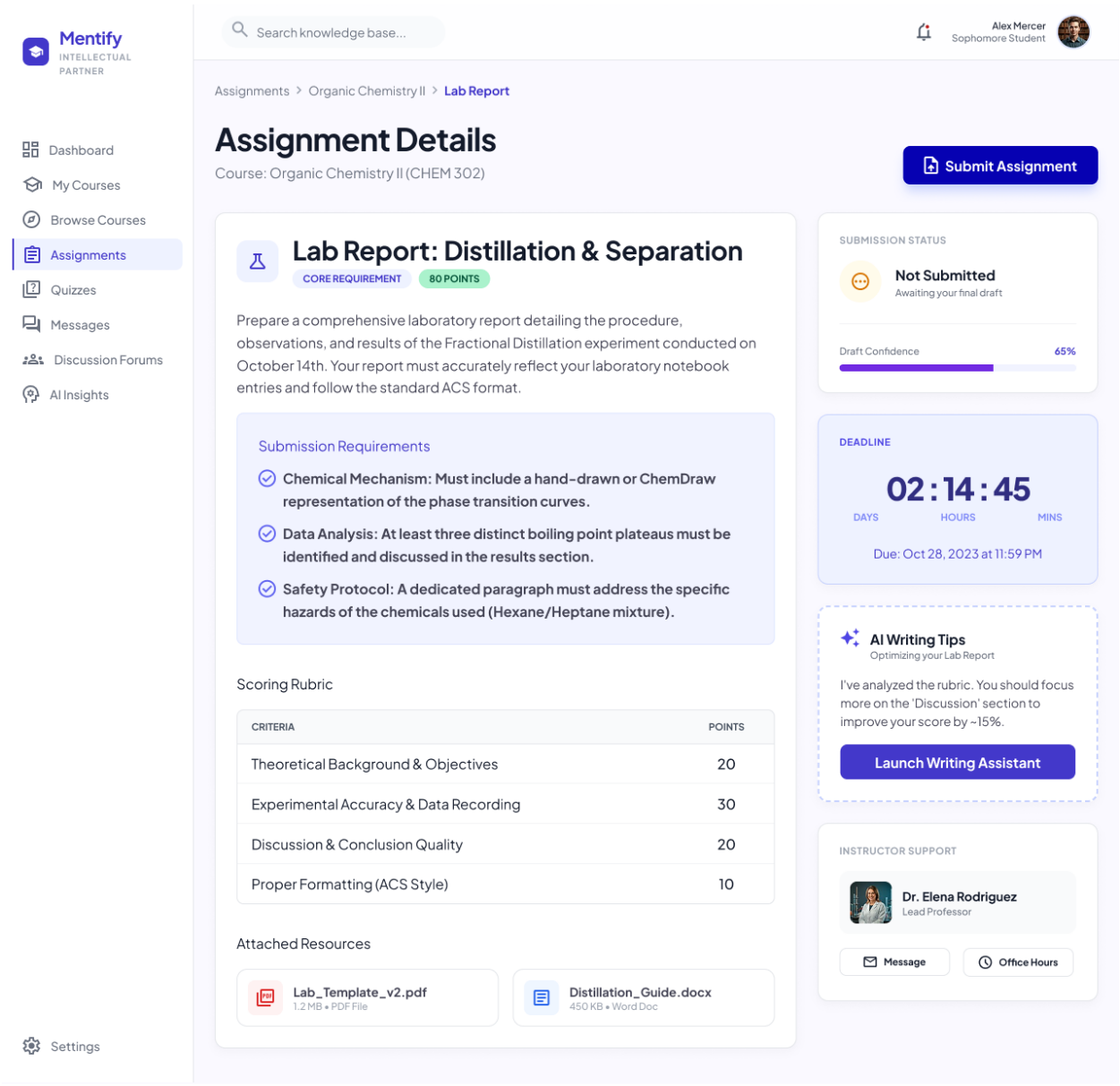


Figure 4.3: Assignment Details

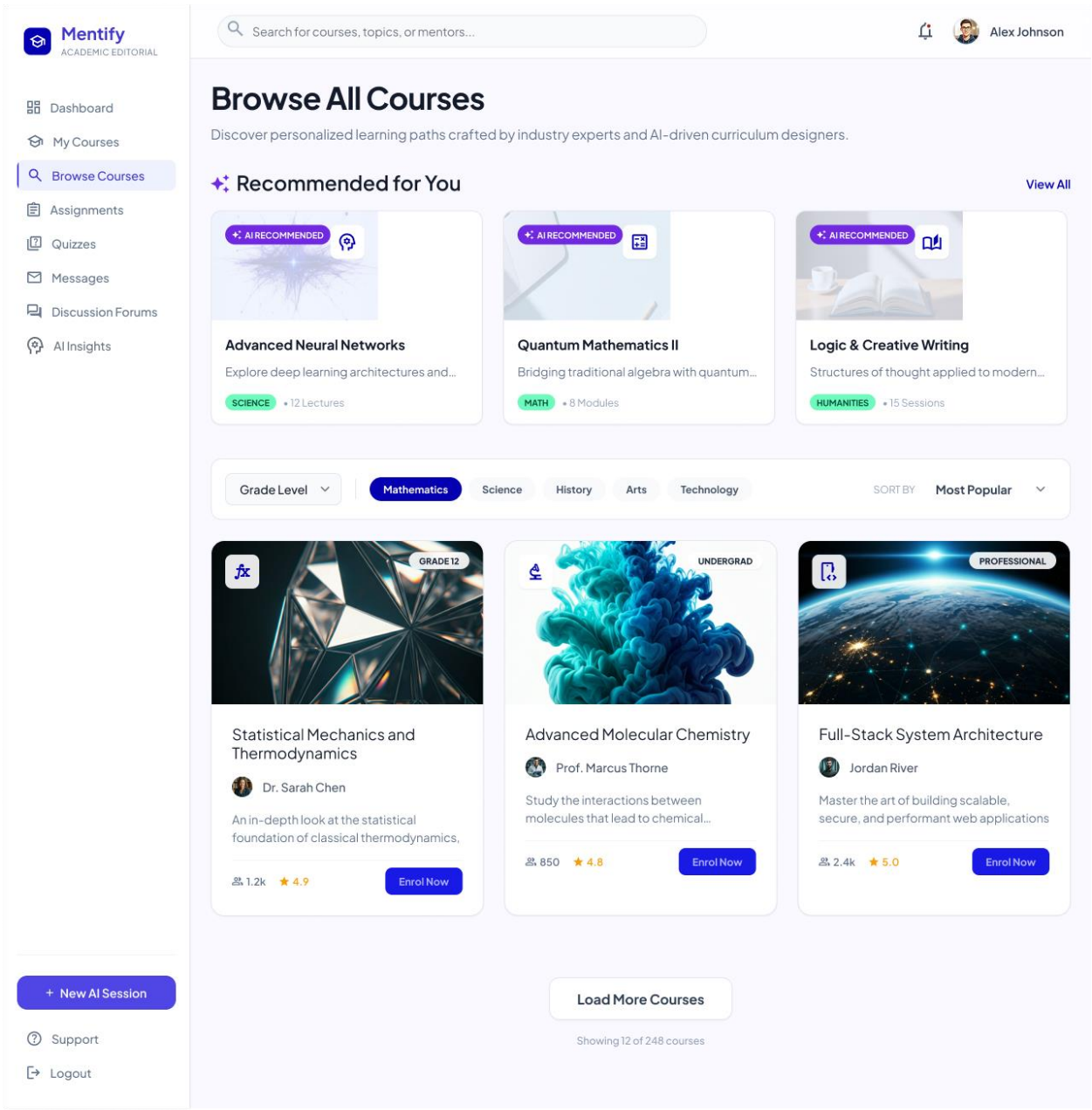


Figure 4.4: Browse Courses

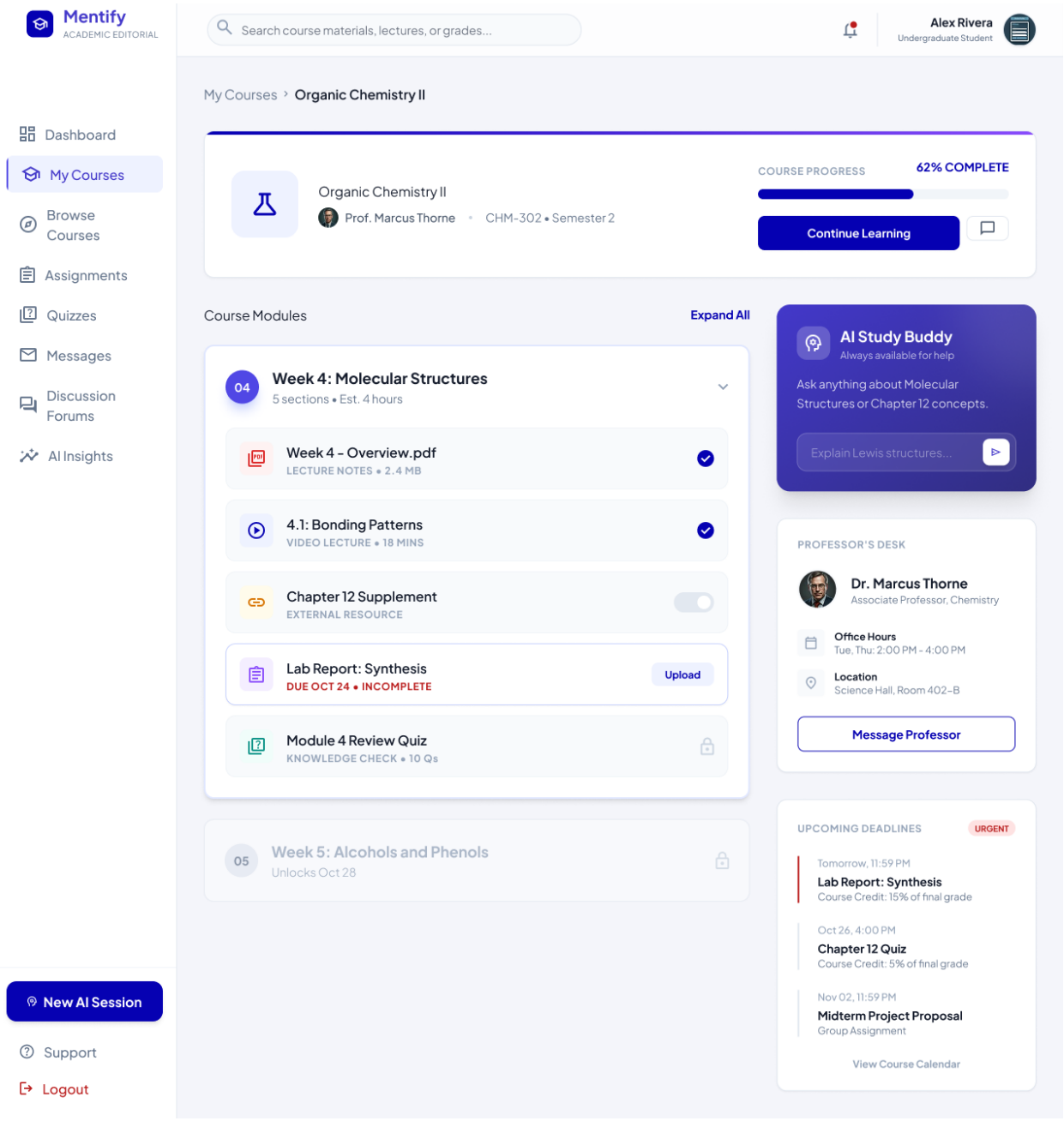


Figure 4.5: Course Content

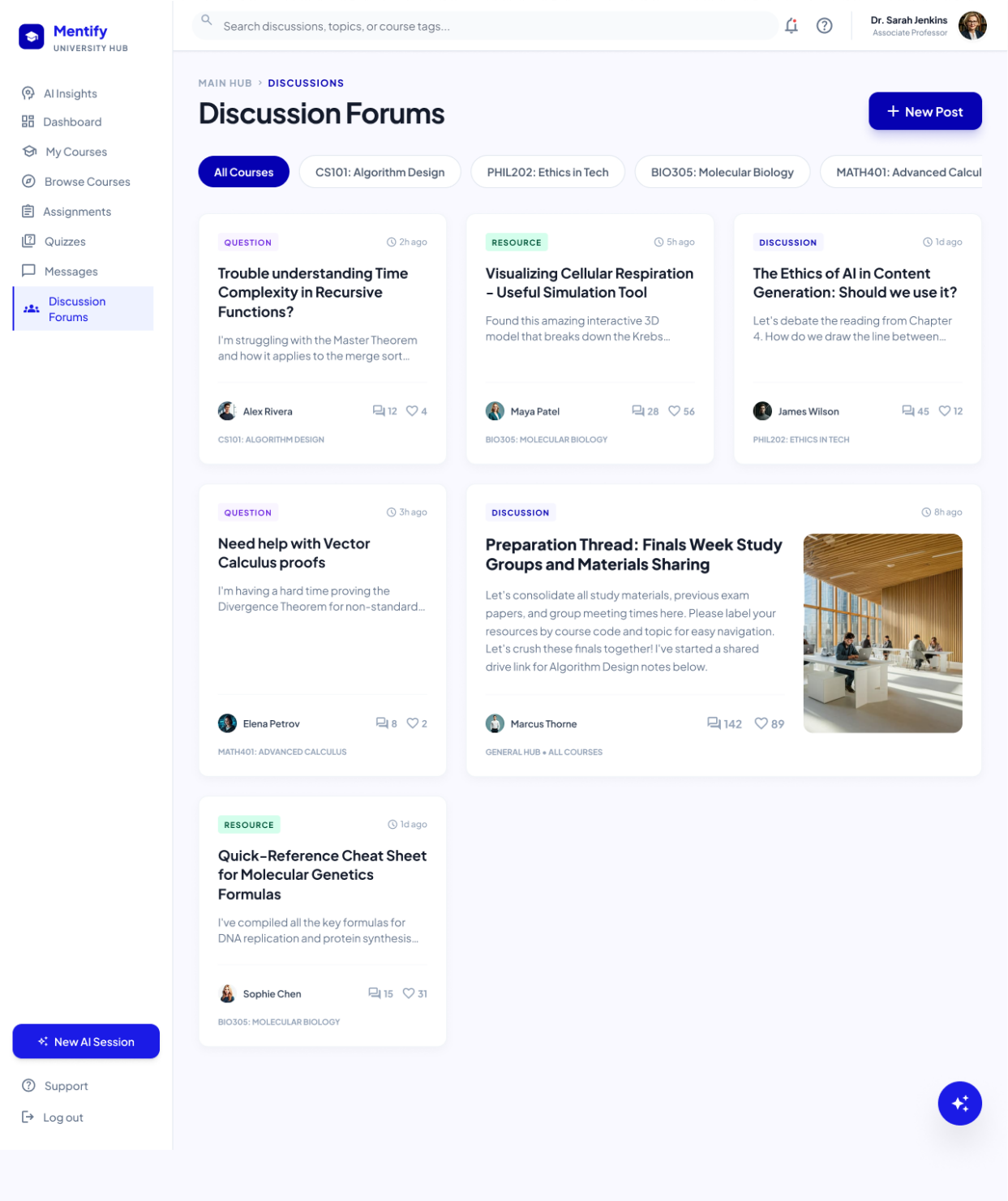


Figure 4.6: Discussion Forum

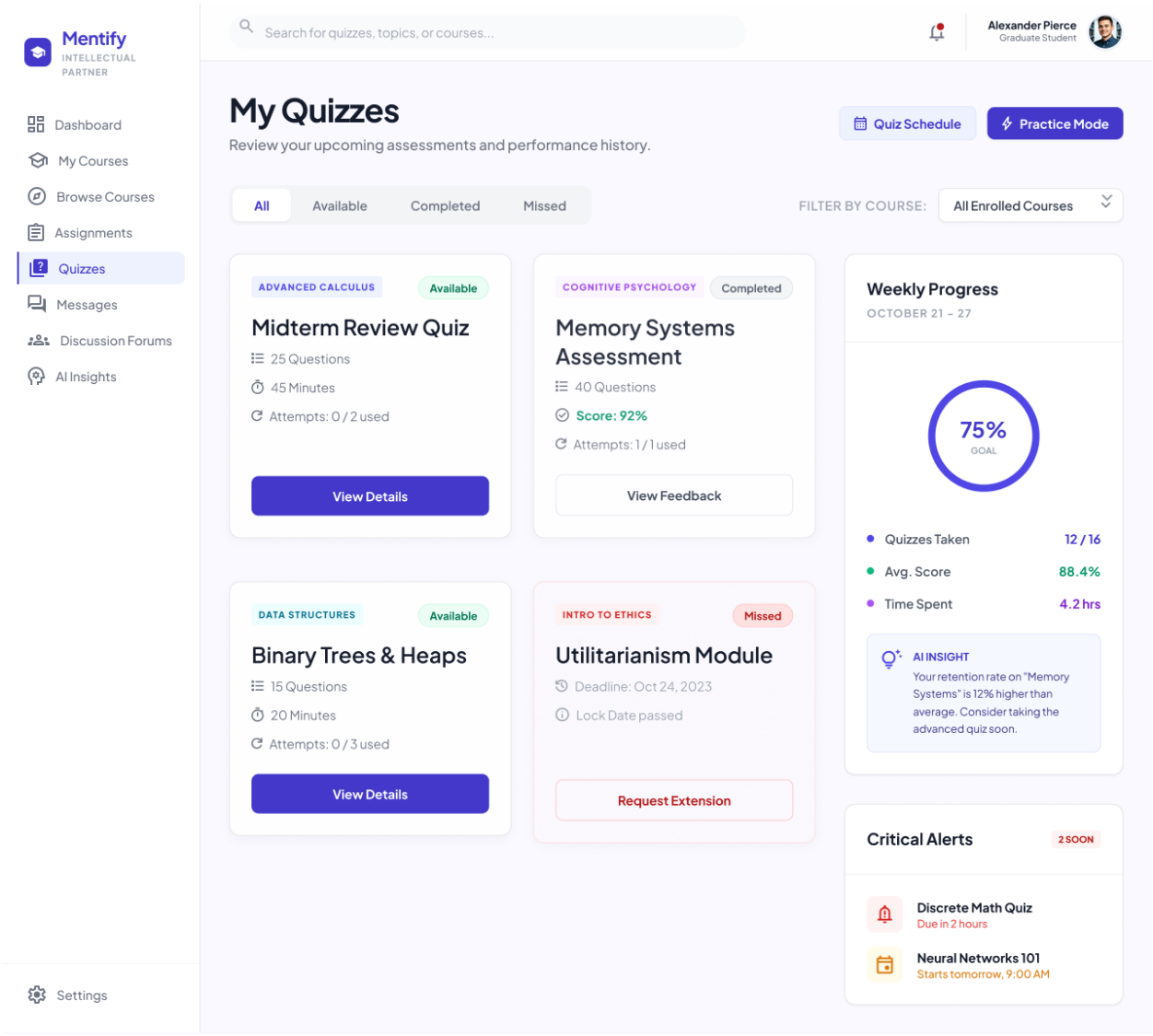


Figure 4.7: All Quizzes

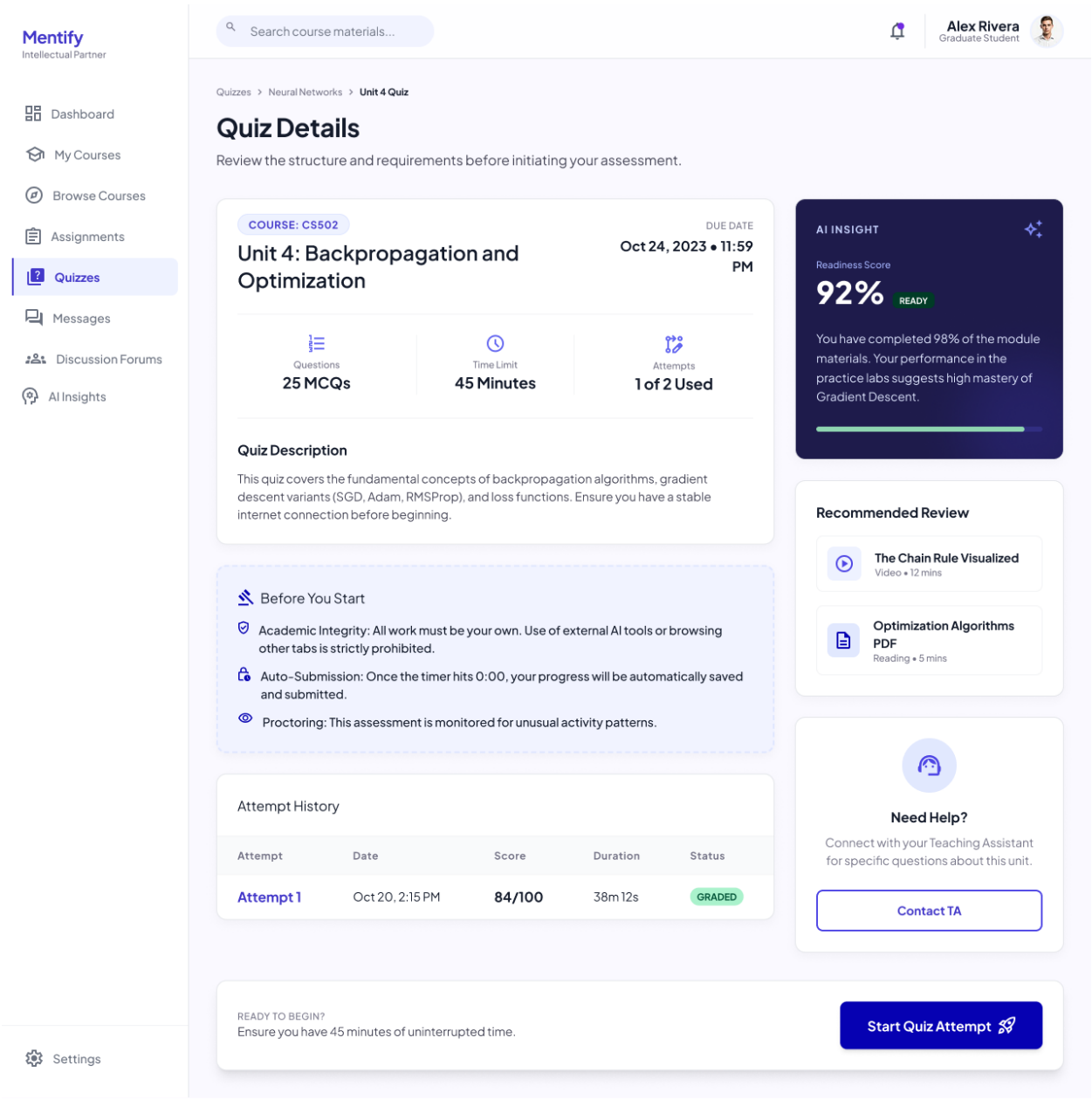


Figure 4.8: Quiz View

4.1.2 Teachers' View

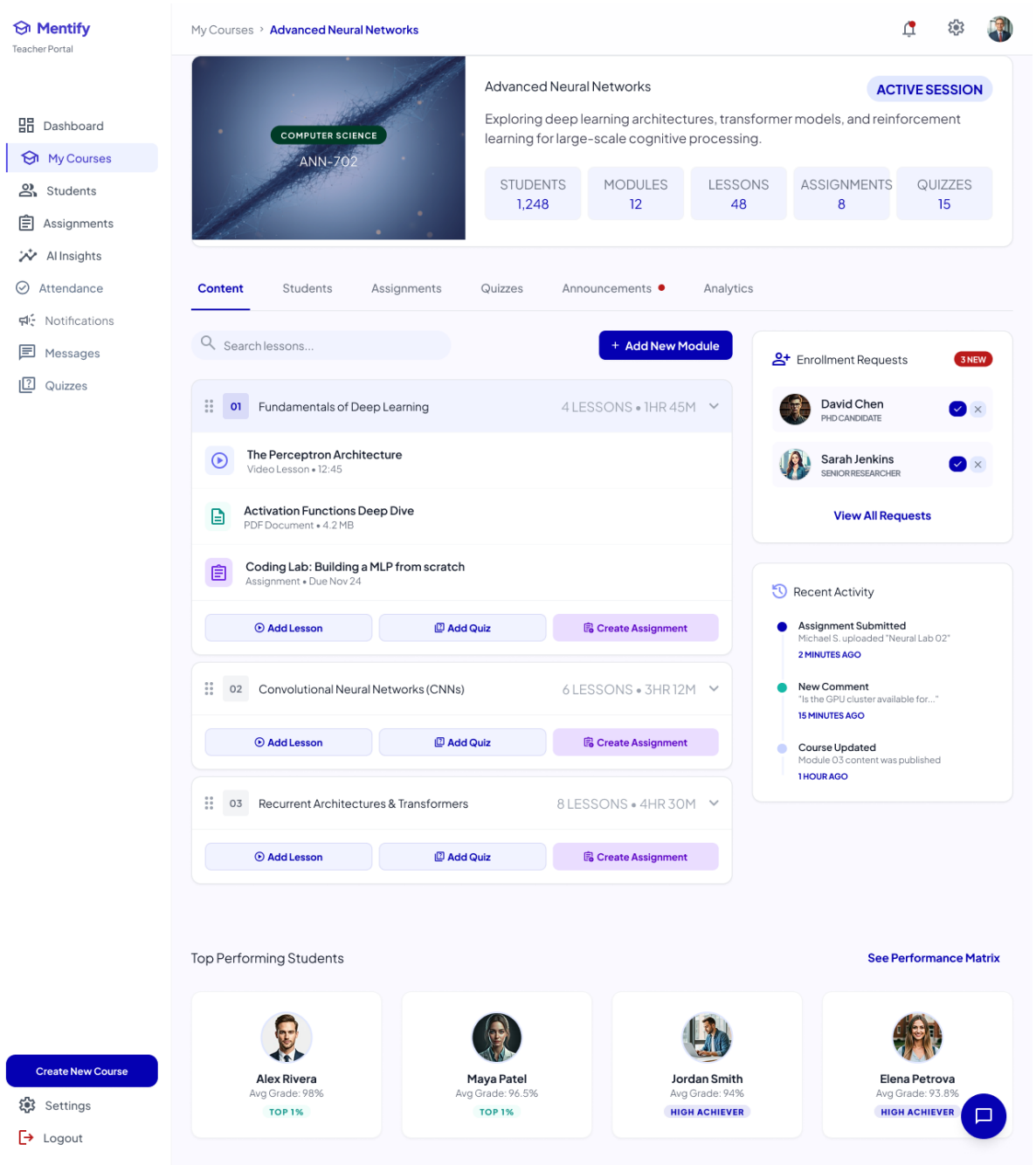


Figure 4.9: Course Management

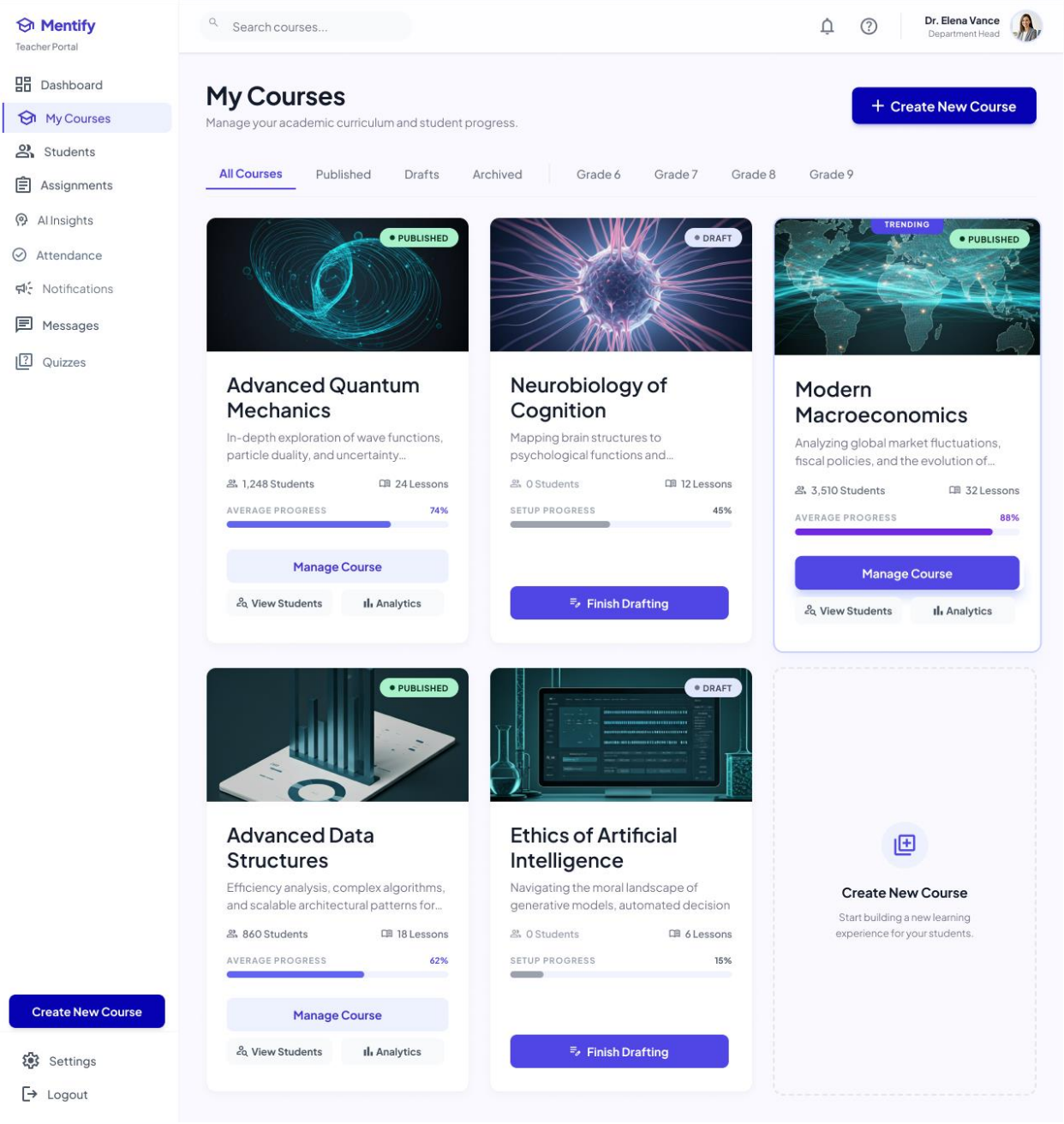


Figure 4.10: Courses View

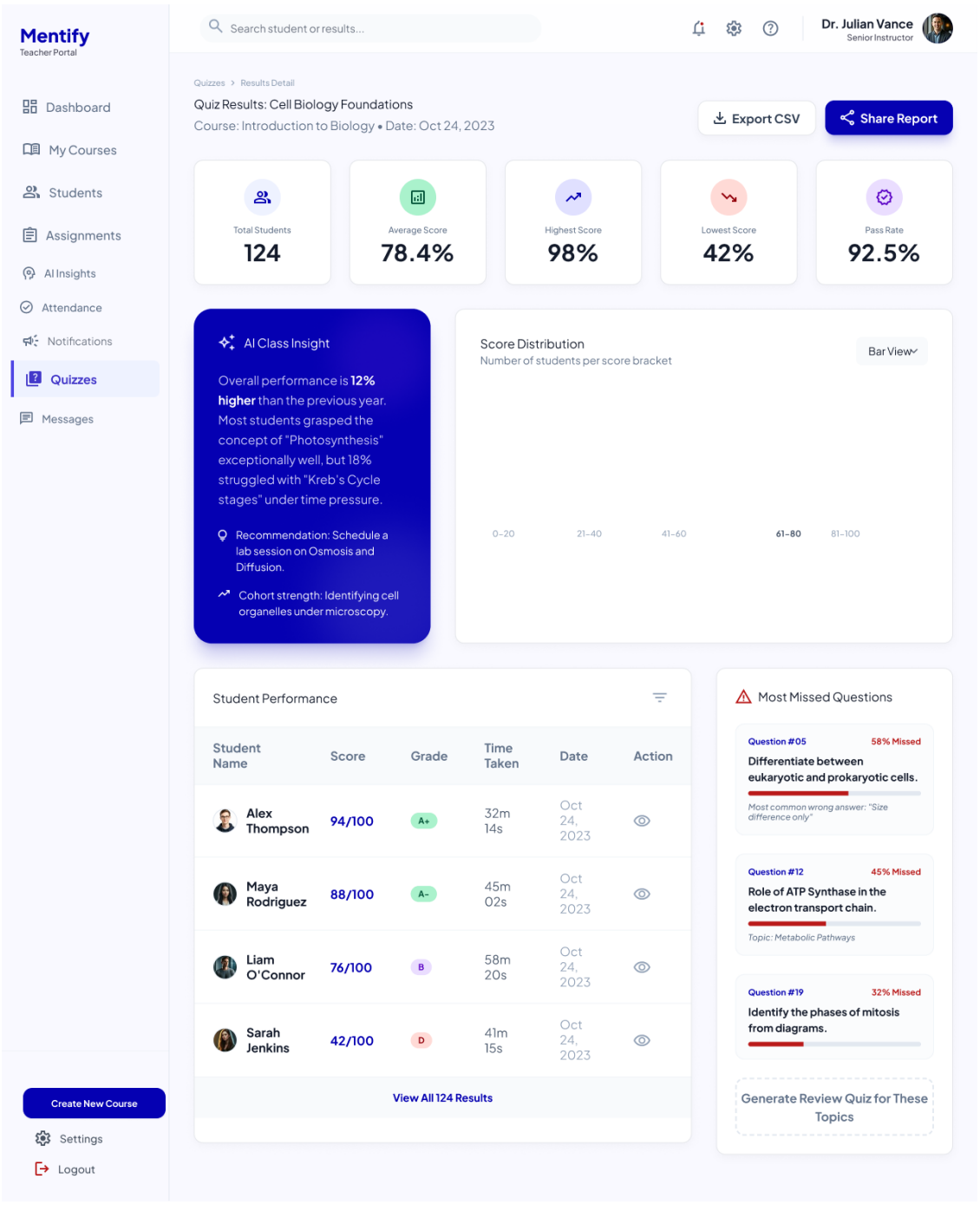


Figure 4.11: Quiz Analytics

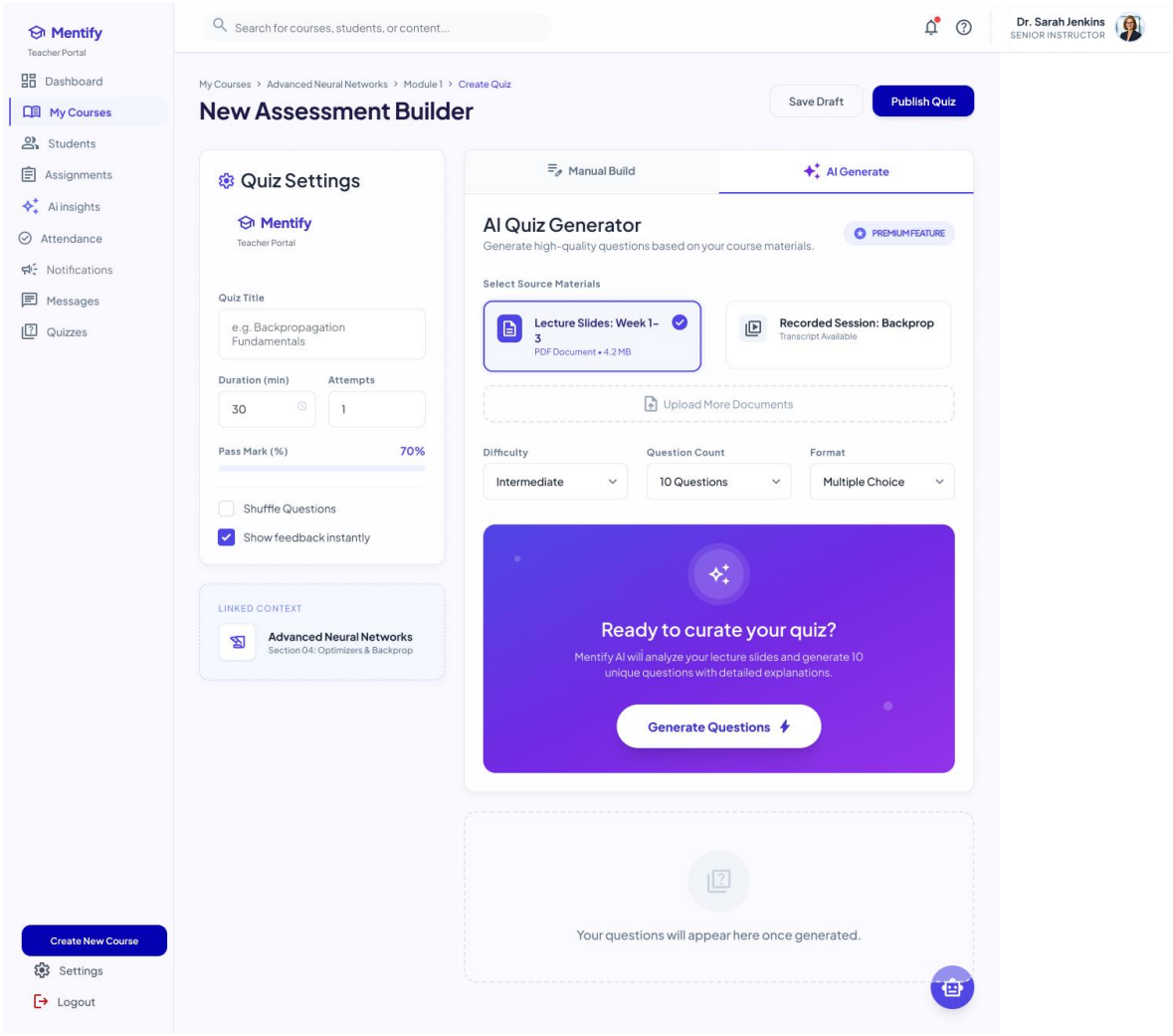


Figure 4.12: Quiz View

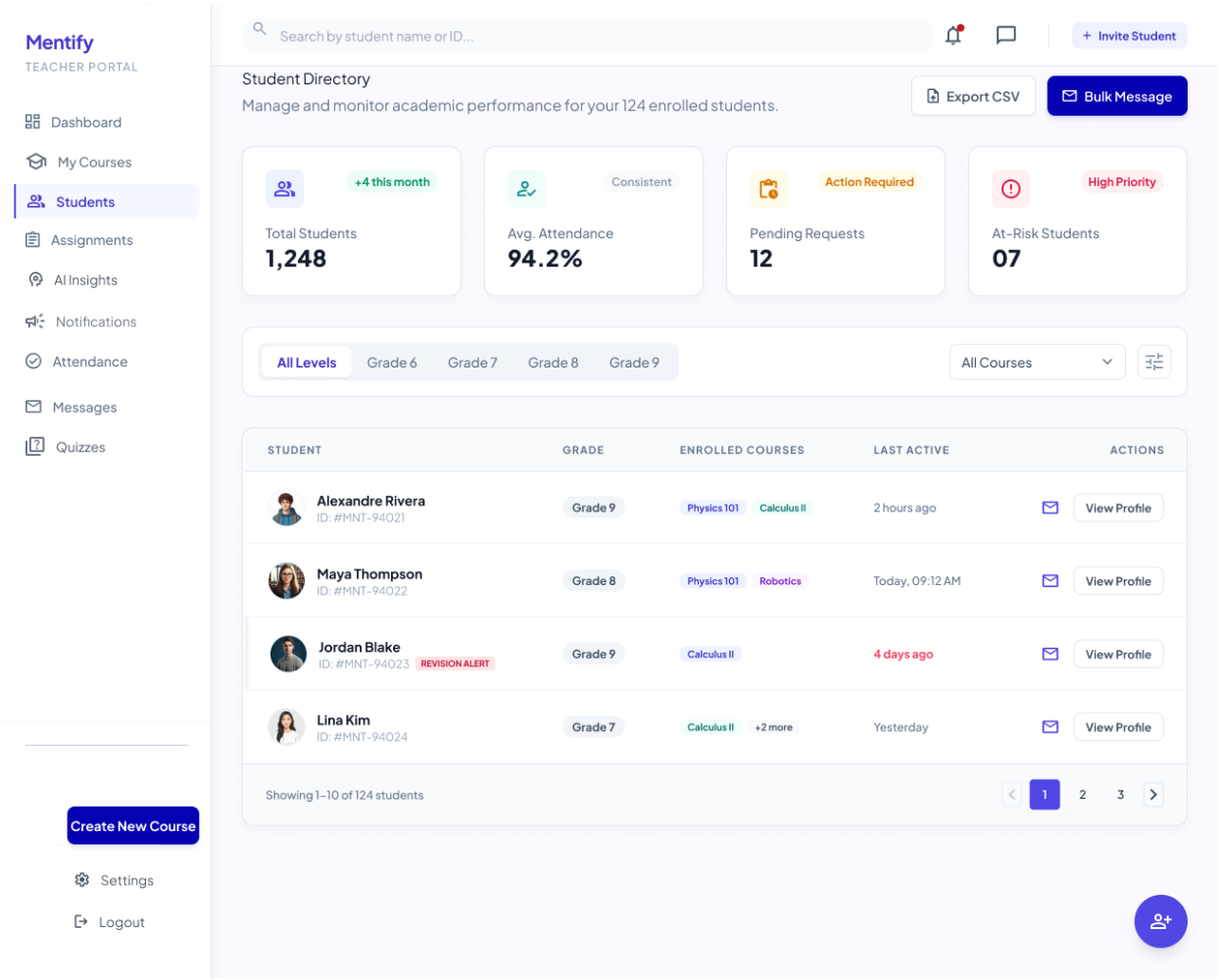


Figure 4.13: Student Management

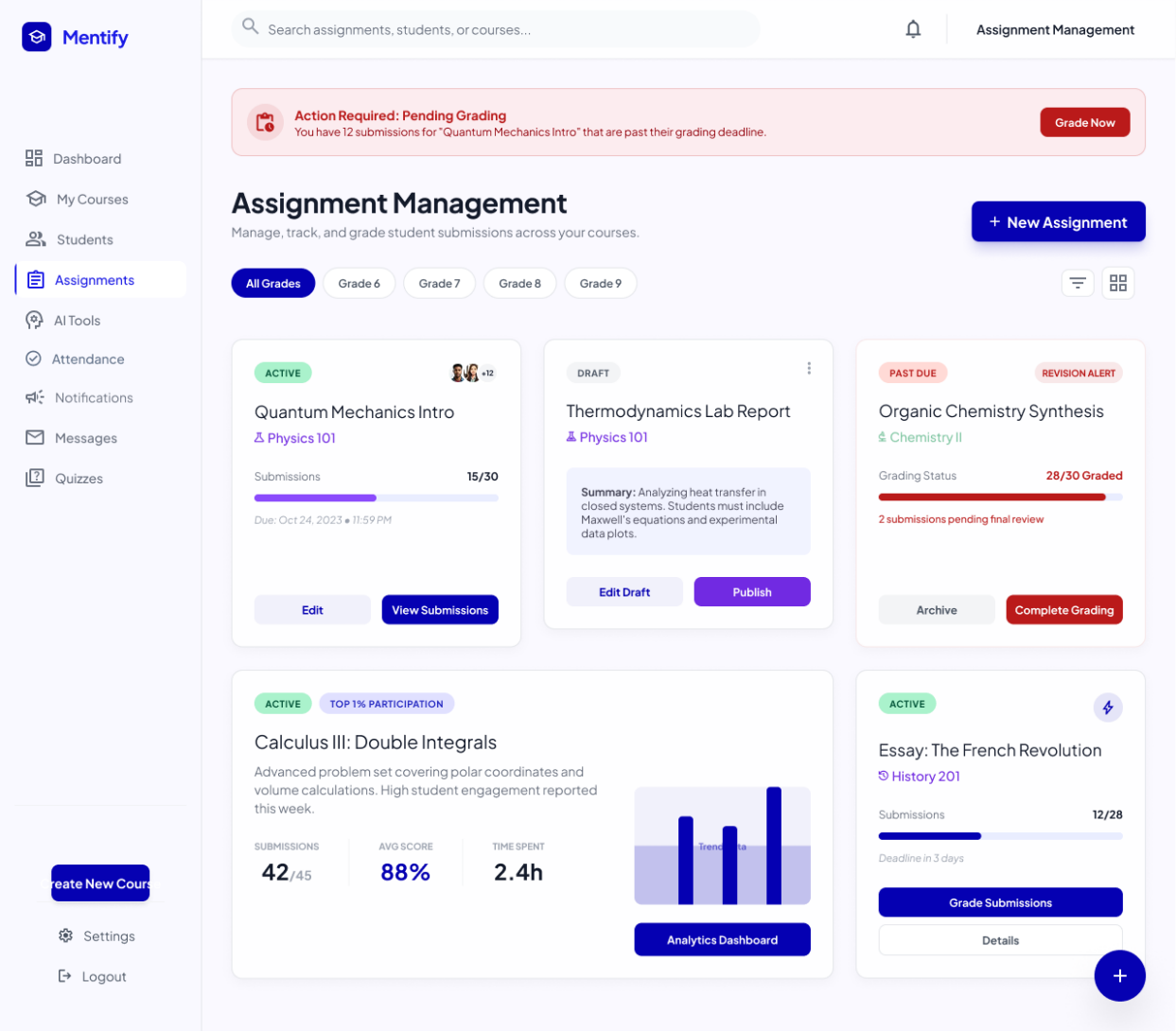


Figure 4.14: Assignment Management

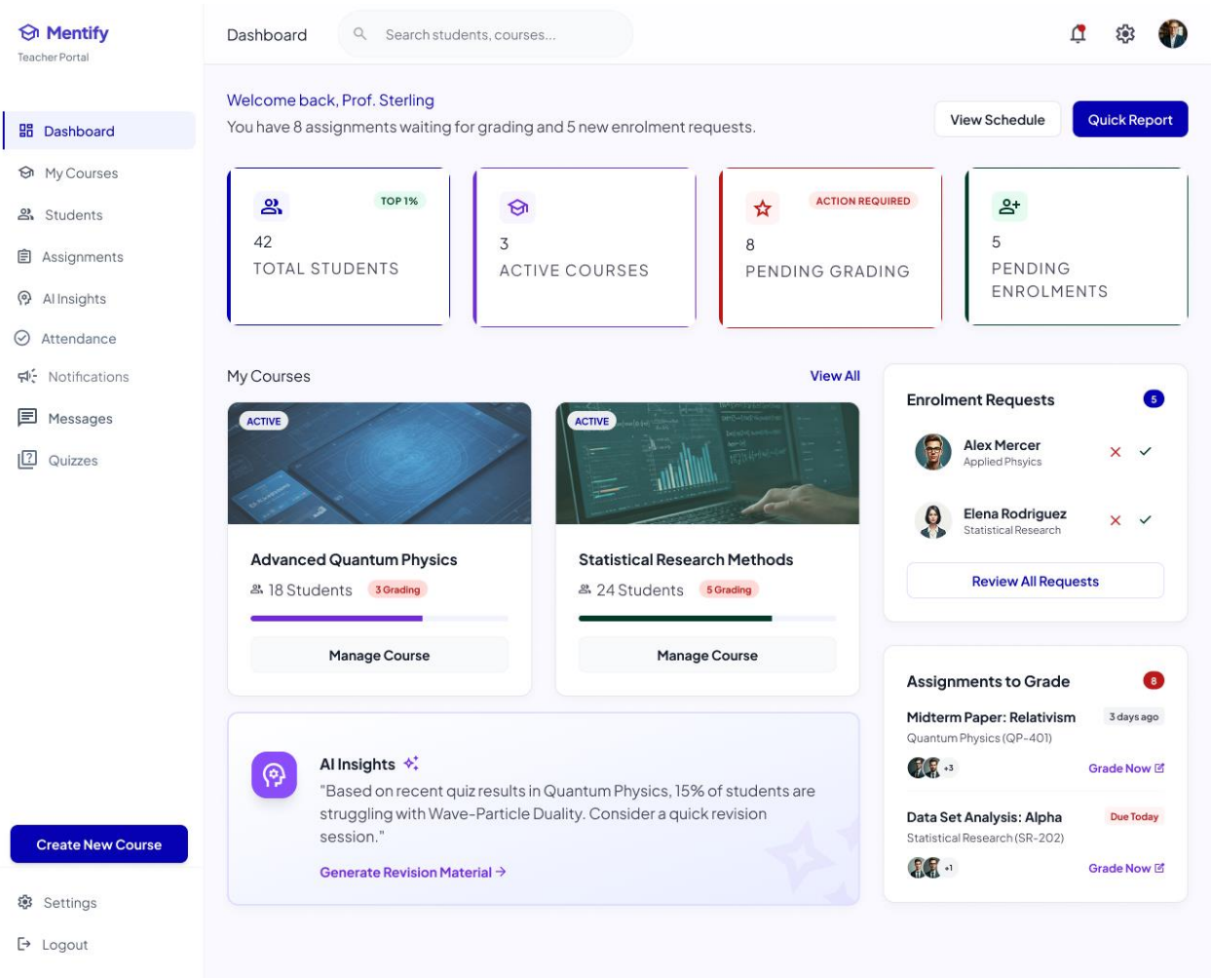


Figure 4.15: Teacher Dashboard

4.2 Client Involvement

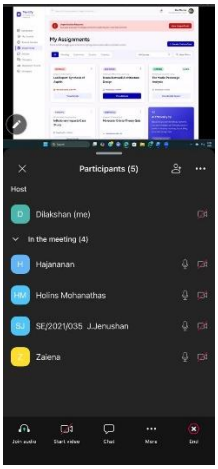
4.2.1 Client Demonstration

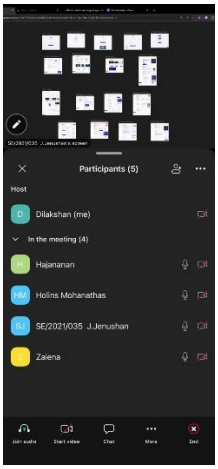
A client demonstration was conducted to present the Mentify system prototype to representatives of TiT Education. The session aimed to showcase the key features and overall functionality of the platform. The demonstration provided stakeholders with a clear understanding of how the system addresses existing challenges and improves the learning experience.

During the session, stakeholders were guided through the main user workflows, such as attending classes, attempting quizzes, reviewing feedback, and interacting through forums and chatbots. The demonstration also highlighted the system's user interface design, emphasizing its ease of use and accessibility.

Feedback collected from the client was generally positive, with appreciation for the integration of AI features, centralized communication, and performance tracking capabilities. However, some suggestions for improvement were identified, including enhancing certain UI elements, refining navigation, and adding more customization options for educators.

Overall, the client demonstration was successful in validating the system concept and gathering valuable feedback, which can be used to further improve and refine Mentify before full implementation.





4.2.2 Client Feedback

The client expressed appreciation for the overall user interface, describing it as visually clean and easy to navigate. No major structural concerns were raised regarding the general layout or design direction.

- CR-01 — Remove support agent role (v1)
  Request: The support agent role should not be included in the initial version of the system.
- CR-02 — Internal community use only; defer payment integration
  Request: The system is intended for internal community use only at this stage. Stripe payment integration should only be introduced after the system is proven to work successfully internally, before any global roll-out.
- CR-03 — Remove self-registration for teachers; admin-managed instead
  Request: Teachers should not have a self-registration flow. Instead, teacher accounts should be created and managed exclusively by the system administrator.

4.3 Ethical Considerations

The design and implementation of Mentify consider several important ethical considerations to ensure responsible use of technology in education. Ethics are considered in the following areas.

4.3.1 Data Privacy

A primary concern in the system is data privacy and confidentiality, as it handles sensitive student information, academic records, and performance data. Appropriate measures such as secure data storage, controlled access, and encryption must be implemented to protect user information and comply with relevant data protection standards.

4.3.2 AI-Driven features

Another key consideration is fairness and transparency in AI-driven features. Since Mentify utilizes AI for automated grading and feedback, it is essential to minimize bias and ensure that evaluations are accurate, consistent, and explainable. Students and educators should be aware of how AI-generated results are produced, and mechanisms should be in place to allow human review when necessary.

In addition, the use of AI-powered chatbots and analytics should be guided by ethical principles, ensuring that recommendations and interactions support learning without manipulating user behaviour or over-relying on automated decision-making.

4.3.3 Academic Integrity

Academic integrity is also a critical factor when ethics are taken into consideration. The platform must discourage dishonest practices such as cheating or plagiarism by incorporating monitoring tools, secure assessment methods, and clear guidelines for acceptable use.

Overall, Mentify aims to balance technological innovation with ethical responsibility, ensuring a safe, fair, and inclusive digital learning environment.

4.4 Reflection on Stakeholder Communication

Effective stakeholder communication played a crucial role throughout the design and development of Mentify. Continuous interaction with stakeholders helped ensure that the system requirements were accurately identified and aligned with real user needs. Discussions, feedback sessions, and requirement validation activities contributed to refining system features and improving overall design decisions.

However, time constraints and varying levels of stakeholder availability sometimes limited the frequency and depth of feedback. Despite these challenges, iterative communication and feedback loops helped minimize misunderstandings and ensured gradual improvement of the system design.

Overall, stakeholder communication was instrumental in guiding the development of Mentify. While there is room for improvement in terms of structured communication strategies and documentation, the process successfully contributed to creating a system that reflects stakeholder needs and expectations

4.5 Formal Sign Off from the Client for the Development

Following the successful presentation and evaluation of the Mentify system prototype, formal approval was obtained from the client, to proceed with the development phase. The sign-off indicates that the proposed system design, features, and overall approach meet the client's expectations and align with their institutional requirements.

During the review process, the client assessed key aspects of the system, including functionality, usability, scalability, and the integration of AI-driven features. Any minor feedback and suggested improvements identified during the client demonstration were acknowledged and documented for consideration in the development phase.

With this approval, the project transitions from the design phase to the implementation phase, where the Mentify system will be developed, tested, and deployed according to the agreed requirements.

5 CONCLUSION

5.1 Degree of Objectives Met

Mentify, an AI-enhanced learning management system developed for TiT Education, successfully achieves its primary design objectives by delivering a centralized, intelligent, and scalable platform that enhances both teaching and learning experiences. The system integrates core LMS functionalities with advanced AI capabilities, enabling efficient academic management, improved student engagement, and data-driven insights. Key objectives achieved include:

- **User-Friendly Web Application:** The platform is designed with an intuitive and accessible interface, allowing students, educators, and administrators to navigate seamlessly. Clear workflows for attending classes, accessing materials, and completing assessments improve overall usability and user satisfaction.
- **Centralized Learning Environment:** Mentify consolidates all academic activities—including course content delivery, assessments, communication, and collaboration—into a single platform. This eliminates dependency on multiple external tools and ensures a consistent and organized learning experience.
- **AI-Powered Assessment and Feedback:** The system incorporates intelligent quiz grading mechanisms that support both automated and teacher-evaluated assessments. AI-generated feedback provides immediate insights, helping students understand mistakes and improve learning outcomes efficiently.
- **Interactive Learning and Collaboration:** Features such as discussion forums, chat systems, and AI-powered chatbots promote continuous interaction between students and educators. These tools enhance collaboration, encourage knowledge sharing, and support learning beyond traditional classroom boundaries.
- **Personalized Learning Experience:** Mentify introduces adaptive learning capabilities that tailor content and recommendations based on individual student performance. This ensures a more student-centered approach compared to traditional one-size-fits-all systems.
- **Performance Tracking and Analytics:** The platform includes comprehensive dashboards that allow educators to monitor student progress, analyze performance trends, and identify

areas requiring intervention. This supports informed decision-making and improves academic outcomes.

- **Efficiency in Academic Workflows:** By automating tasks such as grading, feedback generation, and content management, the system significantly reduces the manual workload on educators, enabling them to focus more on teaching quality and student engagement.
- **Scalability and Future Readiness:** The system architecture is designed to be scalable and adaptable, allowing for the integration of future enhancements such as advanced AI models, additional learning tools, and increased user capacity.

Overall, Mentify effectively meets its intended objectives by transforming traditional educational processes into a modern, intelligent, and collaborative digital learning ecosystem.

5.2 Usability, Accessibility, Reliability and Friendliness

Mentify is designed with a strong emphasis on usability, accessibility, reliability, and overall user friendliness to ensure an effective and inclusive learning experience for all users, including students, educators, and administrators.

Usability: The system provides an intuitive and well-structured user interface that enables users to perform tasks efficiently with minimal learning effort. Clear navigation menus, consistent design patterns, and organized dashboards allow users to easily access courses, attend classes, attempt quizzes, and communicate with others. The platform minimizes complexity by streamlining workflows such as assignment submission, grading, and feedback review, ensuring a smooth user experience.

Accessibility: Mentify is developed to be accessible across a wide range of devices, including desktops, tablets, and smartphones, enabling users to engage with the platform anytime and anywhere. The design considers inclusive practices such as readable typography, proper color contrast, and support for assistive technologies. This ensures that users with varying abilities and technical backgrounds can effectively interact with the system.

Reliability: The system is built using robust and scalable technologies to ensure consistent performance and minimal downtime. Reliable data management mechanisms are implemented to securely store academic records, assessment results, and user information. Features such as automated backups, error handling, and system monitoring help maintain data integrity and system stability, ensuring uninterrupted access to learning resources and services.

User Friendliness: Mentify prioritizes a user-centered design approach, focusing on creating a positive and engaging experience. Features such as AI-powered chatbots provide instant assistance, while dashboards present information in a clear and meaningful way. The system reduces cognitive load by presenting only relevant information and guiding users through tasks with prompts and feedback. This makes the platform approachable even for users with limited technical expertise.

Overall, Mentify ensures that all users can interact with the system efficiently, comfortably, and confidently, thereby enhancing the overall effectiveness of the digital learning environment.

5.3 Limitations and Drawbacks

While Mentify offers a robust and intelligent learning management solution, several limitations and drawbacks exist from both a design and system execution perspective. Recognizing these constraints is important for guiding future enhancements and ensuring realistic system expectations.

From a functional standpoint, the system's reliance on AI for automated grading and feedback introduces certain limitations. Although AI significantly improves efficiency, it may struggle to accurately evaluate subjective, analytical, or creative responses. This can lead to inconsistencies in assessment quality, requiring continued human oversight to ensure fairness and accuracy.

In terms of **system design**, the integration of multiple advanced features—such as AI modules, real-time communication tools, analytics dashboards, and collaborative environments—adds considerable architectural complexity. Designing a system that maintains consistency, responsiveness, and seamless interaction across all these components can be challenging. There is

also a risk of feature overload, where the inclusion of numerous functionalities may overwhelm users if not carefully structured through thoughtful UI/UX design.

From a **usability design perspective**, accommodating diverse user groups (students, educators, administrators) within a single interface can create design trade-offs. Ensuring that the platform remains simple for beginners while still offering advanced features for experienced users requires careful balance. Poorly optimized workflows or cluttered interfaces could reduce overall usability and adoption.

Regarding **system execution**, performance and scalability present notable challenges. Real-time features such as live classes, chat systems, and concurrent quiz attempts require efficient resource management. Under high user loads, the system may face latency issues or reduced responsiveness if infrastructure is not adequately scaled. Additionally, AI processing tasks can be computationally intensive, potentially affecting system performance.

The platform's dependency on stable internet connectivity is another limitation. Since Mentify operates as a web-based system with real-time functionalities, users in low-bandwidth environments may experience disruptions, affecting accessibility and consistency of use.

Implementation and maintenance also introduce constraints. The development and deployment of such a feature-rich system require significant technical expertise, time, and financial resources. Ongoing maintenance—including updates, bug fixes, and AI model improvements—adds to operational overhead.

Data privacy and security remain critical concerns in both design and execution. Handling sensitive academic and personal data requires strong security architecture, encryption mechanisms, and compliance with data protection standards. Any weaknesses in these areas could compromise user trust.

Finally, the effectiveness of personalization and analytics depends on the availability of sufficient and high-quality data. During initial deployment phases or in cases of limited user interaction, the system may not deliver highly accurate insights or recommendations.

In conclusion, while Mentify demonstrates strong potential in transforming digital education, addressing these design and execution-related limitations is essential to ensure scalability, usability, performance, and long-term sustainability.

5.4 Timeline

Stage	Deliverable	Completion Date
Stage 1	Client Agreement and Problem Validation (Idea Pitch)	March 2026
Stage 2	Software Requirement Specification (SRS)	March 2026
Stage 3	System Design Specification (SDS)	April 2026
Stage 4	Final Design Report	April 2026
Future	Prototype Refinement and Client Validation	May 2026 (Planned)
Future	System Development (SENG 34213)	2026–2027 (Planned)

5.5 Future modifications, improvements and extensions possible

While Mentify provides a comprehensive and intelligent learning management solution, several future enhancements can be implemented to further improve its functionality, scalability, and overall user experience.

One key area for improvement is the advancement of **AI capabilities**. Future versions of the system could incorporate more sophisticated models for personalized learning, predictive analytics, and intelligent tutoring, enabling more accurate recommendations and deeper insights into student performance. Enhancements to AI-based grading could also improve the evaluation of subjective and descriptive answers.

The platform can also be extended to support **mobile applications** for Android and iOS, providing a more optimized and accessible experience for users who primarily rely on smartphones. This would enhance flexibility and increase user engagement.

Another potential improvement is the integration of **third-party tools and services**, such as video conferencing platforms, plagiarism detection systems, and external educational resources. This would expand the system's capabilities and provide a more enriched learning environment.

In terms of **collaboration features**, Mentify could introduce real-time collaborative tools such as shared whiteboards, group project workspaces, and peer review systems. These additions would further promote interactive and cooperative learning.

The system can also be enhanced with **gamification elements**, such as badges, leaderboards, and achievement tracking, to increase student motivation and engagement. Additionally, more advanced **analytics and reporting features** could be developed to provide deeper insights for educators and administrators, including predictive alerts for at-risk students.

From a technical perspective, improvements in **scalability and performance optimization** can be implemented to support a growing number of users and ensure smooth system operation under high demand. Strengthening **security measures** and ensuring compliance with evolving data protection standards will also remain a priority.

Finally, Mentify could be extended to support **multi-institution usage**, allowing it to be deployed across different educational organizations with customizable configurations to meet varying requirements.table

Overall, these future modifications and extensions will enable Mentify to evolve into a more powerful, flexible, and intelligent educational platform capable of adapting to emerging technological trends and user needs.

APPENDIX A – DOCUMENT REVIEW

This document has been reviewed and proofread to ensure clarity, consistency, and accuracy. All sections have been checked for grammatical correctness, proper formatting, and alignment with the specified requirements. Technical content has been verified to ensure coherence and completeness, and any identified errors or inconsistencies have been corrected.

The document is hereby considered finalized and ready for submission.



Nirojan Yoharajah

Software Engineer

3axislabs

APPENDIX B – UI PROTOTYPES

Figma Link

APPENDIX C – REFERENCES FROM THE SDS DOCUMENT

CHAPTER 08: REFERENCES

The following references are cited across all chapters of this System Design Specification (SDS), covering Architectural Design, Database Design, Class Diagrams, Sequence Diagrams, Security Design, and Deployment Design. All references are formatted in accordance with the IEEE referencing standard.

Chapter 1 — Architectural Design

[1] VMware, "What Is Microservices Architecture?," *VMware*, 2023. [Online]. Available: https://www.vmware.com/topics/glossary/content/microservices-architecture.html. [Accessed: Apr. 2026].

[2] Pivotal Software, "Spring Boot Reference Documentation," *Spring*, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/htmlsingle/. [Accessed: Apr. 2026].

[3] Amazon Web Services, "Amazon Elastic Kubernetes Service (EKS)," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/eks/latest/userguide/what-is-eks.html. [Accessed: Apr. 2026].

[4] Amazon Web Services, "Amazon Elastic Container Registry (ECR)," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonECR/latest/userguide/what-is-ecr.html. [Accessed: Apr. 2026].

[5] Amazon Web Services, "Amazon Simple Storage Service (S3)," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonS3/latest/userguide/Welcome.html. [Accessed: Apr. 2026].

[6] Amazon Web Services, "Amazon Relational Database Service (RDS)," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Welcome.html. [Accessed: Apr. 2026].

[7] Amazon Web Services, "Elastic Load Balancing — Application Load Balancer," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html. [Accessed: Apr. 2026].

[8] MongoDB, Inc., "MongoDB Atlas Documentation," *MongoDB*, 2024. [Online]. Available: https://www.mongodb.com/docs/atlas/. [Accessed: Apr. 2026].

[9] Redis Ltd., "Redis Documentation," *Redis*, 2024. [Online]. Available: https://redis.io/docs/. [Accessed: Apr. 2026].

[10] Apache Software Foundation, "Apache Kafka Documentation," *Apache Kafka*, 2024. [Online]. Available: https://kafka.apache.org/documentation/. [Accessed: Apr. 2026].

[11] Pivotal Software, "Spring Cloud Config Reference Documentation," *Spring*, 2024. [Online]. Available: https://docs.spring.io/spring-cloud-config/docs/current/reference/html/. [Accessed: Apr. 2026].

[12] GitHub, "GitHub Actions Documentation," *GitHub*, 2024. [Online]. Available: https://docs.github.com/en/actions. [Accessed: Apr. 2026].

[13] Docker Inc., "Docker Documentation," *Docker*, 2024. [Online]. Available: https://docs.docker.com/. [Accessed: Apr. 2026].

[14] Google LLC, "Google Identity: OAuth 2.0 for Web Server Applications," *Google Developers*, 2024. [Online]. Available: https://developers.google.com/identity/protocols/oauth2. [Accessed: Apr. 2026].

[15] OpenAI, "OpenAI API Reference," *OpenAI*, 2024. [Online]. Available: https://platform.openai.com/docs/api-reference. [Accessed: Apr. 2026].

[16] Kubernetes Authors, "Kubernetes Documentation," *Kubernetes*, 2024. [Online]. Available: https://kubernetes.io/docs/home/. [Accessed: Apr. 2026].

[17] Pivotal Software, "Spring Boot Actuator: Production-ready Features," *Spring*, 2024. [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/actuator.html. [Accessed: Apr. 2026].

[18] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. Sebastopol, CA: O'Reilly Media, 2021.

[19] M. Fowler and J. Lewis, "Microservices," *martinfowler.com*, 2014. [Online]. Available: https://martinfowler.com/articles/microservices.html. [Accessed: Apr. 2026].

Chapter 2 — Database Design

[20] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Harlow: Pearson, 2016.

[21] C. J. Date, *An Introduction to Database Systems*, 8th ed. Reading, MA: Addison-Wesley, 2004.

[22] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/16/. [Accessed: Apr. 2026].

[23] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[24] R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd ed. New York, NY: McGraw-Hill, 2003.

Chapter 3 — Class Diagram

[25] Object Management Group, "Unified Modeling Language (UML) Specification, Version 2.5.1," OMG, Dec. 2017. [Online]. Available: https://www.omg.org/spec/UML/2.5.1/. [Accessed: Apr. 2026].

[26] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*, 2nd ed. Reading, MA: Addison-Wesley, 2005.

[27] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed. Reading, MA: Addison-Wesley, 2003.

[28] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Upper Saddle River, NJ: Prentice Hall, 2017.

[29] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Boston, MA: Addison-Wesley, 2003.

Chapter 4 — Sequence Diagrams

[30] Internet Engineering Task Force, "RFC 7519: JSON Web Token (JWT)," IETF, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519. [Accessed: Apr. 2026].

[31] Internet Engineering Task Force, "RFC 7517: JSON Web Key (JWK)," IETF, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7517. [Accessed: Apr. 2026].

[32] Internet Engineering Task Force, "RFC 6455: The WebSocket Protocol," IETF, Dec. 2011. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6455. [Accessed: Apr. 2026].

[33] Pivotal Software, "Spring Security Reference — Method Security," *Spring*, 2024. [Online]. Available: https://docs.spring.io/spring-security/reference/servlet/authorization/method-security.html. [Accessed: Apr. 2026].

[34] P. Upadhyay, "Quartz Scheduler Documentation," *Quartz Scheduler*, 2023. [Online]. Available: http://www.quartz-scheduler.org/documentation/. [Accessed: Apr. 2026].

[35] Amazon Web Services, "Amazon CloudFront Developer Guide," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html. [Accessed: Apr. 2026].

[36] Pinecone Systems, Inc., "Pinecone Documentation," *Pinecone*, 2024. [Online]. Available: https://docs.pinecone.io/. [Accessed: Apr. 2026].

Chapter 5 — UI/UX Design

[37] World Wide Web Consortium, "Web Content Accessibility Guidelines (WCAG) 2.1," W3C Recommendation, Jun. 2018. [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: Apr. 2026].

[38] World Wide Web Consortium, "Accessible Rich Internet Applications (WAI-ARIA) 1.2," W3C Recommendation, Jun. 2023. [Online]. Available: https://www.w3.org/TR/wai-aria-1.2/. [Accessed: Apr. 2026].

[39] Deque Systems, "axe-core: Accessibility Engine for Automated Web UI Testing," *GitHub*, 2024. [Online]. Available: https://github.com/dequelabs/axe-core. [Accessed: Apr. 2026].

[40] NV Access, "NVDA Screen Reader," *NV Access*, 2024. [Online]. Available: https://www.nvaccess.org/. [Accessed: Apr. 2026].

[41] Apple Inc., "VoiceOver User Guide," *Apple Support*, 2024. [Online]. Available: https://support.apple.com/guide/voiceover/welcome/mac. [Accessed: Apr. 2026].

[42] S. Krug, *Don't Make Me Think, Revisited: A Common Sense Approach to Web Usability*, 3rd ed. San Francisco, CA: New Riders, 2014.

Chapter 6 — Security Design

[43] OWASP Foundation, "OWASP Top Ten," OWASP, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 2026].

[44] Internet Engineering Task Force, "RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3," IETF, Aug. 2018. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc8446. [Accessed: Apr. 2026].

[45] National Institute of Standards and Technology, "FIPS PUB 197: Advanced Encryption Standard (AES)," NIST, Nov. 2001. [Online]. Available: https://csrc.nist.gov/publications/detail/fips/197/final. [Accessed: Apr. 2026].

[46] N. Provos and D. Mazieres, "A Future-Adaptable Password Scheme," in *Proc. USENIX Annual Technical Conference*, Jun. 1999. [Online]. Available: https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf. [Accessed: Apr. 2026].

[47] Internet Engineering Task Force, "RFC 6749: The OAuth 2.0 Authorization Framework," IETF, Oct. 2012. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc6749. [Accessed: Apr. 2026].

[48] Amazon Web Services, "AWS Key Management Service (KMS) Developer Guide," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/kms/latest/developerguide/overview.html. [Accessed: Apr. 2026].

[49] Amazon Web Services, "AWS Secrets Manager User Guide," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html. [Accessed: Apr. 2026].

[50] OWASP Foundation, "OWASP SQL Injection Prevention Cheat Sheet," OWASP, 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html. [Accessed: Apr. 2026].

[51] OWASP Foundation, "OWASP Cross Site Scripting Prevention Cheat Sheet," OWASP, 2023. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html. [Accessed: Apr. 2026].

[52] Jakarta EE, "Bean Validation 3.0 (JSR 380) Specification," Eclipse Foundation, 2020. [Online]. Available: https://beanvalidation.org/3.0/. [Accessed: Apr. 2026].

[53] Pivotal Software, "Spring Security Reference Documentation," *Spring*, 2024. [Online]. Available: https://docs.spring.io/spring-security/reference/. [Accessed: Apr. 2026].

[54] National Institute of Standards and Technology, "NIST SP 800-63B: Digital Identity Guidelines — Authentication and Lifecycle Management," NIST, Jun. 2017. [Online]. Available: https://pages.nist.gov/800-63-3/sp800-63b.html. [Accessed: Apr. 2026].

Chapter 7 — Deployment Design

[55] Amazon Web Services, "Amazon EC2 Instance Types," *AWS Documentation*, 2024. [Online]. Available: https://aws.amazon.com/ec2/instance-types/. [Accessed: Apr. 2026].

[56] Amazon Web Services, "AWS API Gateway Developer Guide," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html. [Accessed: Apr. 2026].

[57] Nginx Inc., "NGINX Documentation," *NGINX*, 2024. [Online]. Available: https://nginx.org/en/docs/. [Accessed: Apr. 2026].

[58] Netflix OSS, "Netflix Eureka — Service Discovery," *GitHub*, 2023. [Online]. Available: https://github.com/Netflix/eureka. [Accessed: Apr. 2026].

[59] Docker Inc., "Docker Compose Documentation," *Docker*, 2024. [Online]. Available: https://docs.docker.com/compose/. [Accessed: Apr. 2026].

[60] Amazon Web Services, "Amazon CloudWatch User Guide," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/WhatIsCloudWatch.html. [Accessed: Apr. 2026].

[61] Amazon Web Services, "Amazon Simple Email Service (SES) Developer Guide," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/ses/latest/dg/Welcome.html. [Accessed: Apr. 2026].

[62] Amazon Web Services, "AWS WAF Developer Guide," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/waf/latest/developerguide/waf-chapter.html. [Accessed: Apr. 2026].

[63] Amazon Web Services, "Amazon Elastic Block Store (EBS) User Guide," *AWS Documentation*, 2024. [Online]. Available: https://docs.aws.amazon.com/ebs/latest/userguide/what-is-ebs.html. [Accessed: Apr. 2026].

[64] SonarSource, "SonarQube Documentation," *SonarSource*, 2024. [Online]. Available: https://docs.sonarsource.com/sonarqube/latest/. [Accessed: Apr. 2026].

[65] JUnit Team, "JUnit 5 User Guide," *JUnit*, 2024. [Online]. Available: https://junit.org/junit5/docs/current/user-guide/. [Accessed: Apr. 2026].

[66] Testcontainers, "Testcontainers for Java Documentation," *Testcontainers*, 2024. [Online]. Available: https://java.testcontainers.org/. [Accessed: Apr. 2026].

[67] Postman Inc., "Newman: Command-line Collection Runner for Postman," *GitHub*, 2024. [Online]. Available: https://github.com/postmanlabs/newman. [Accessed: Apr. 2026].

[68] Google LLC, "Gemini API Documentation," *Google AI for Developers*, 2024. [Online]. Available: https://ai.google.dev/gemini-api/docs. [Accessed: Apr. 2026].